

Binary Search and Variants

- Applicable whenever it is possible to reduce the search space by **half** using one query
- Search space size **N**
number of queries = **$O(\log N)$**
- Ubiquitous in algorithm design. Almost every complex enough algorithm will have binary search somewhere.

Classic example

- Given a sorted array A of integers, find the location of a target number x (or say that it is not present)

Pseudocode:

Initialize $start \leftarrow 0$, $end \leftarrow n$;

Locate(x , $start$, end){

 if ($end < start$) return not found;

$mid \leftarrow (start + end) / 2$;

 if ($A[mid] = x$) return mid ;

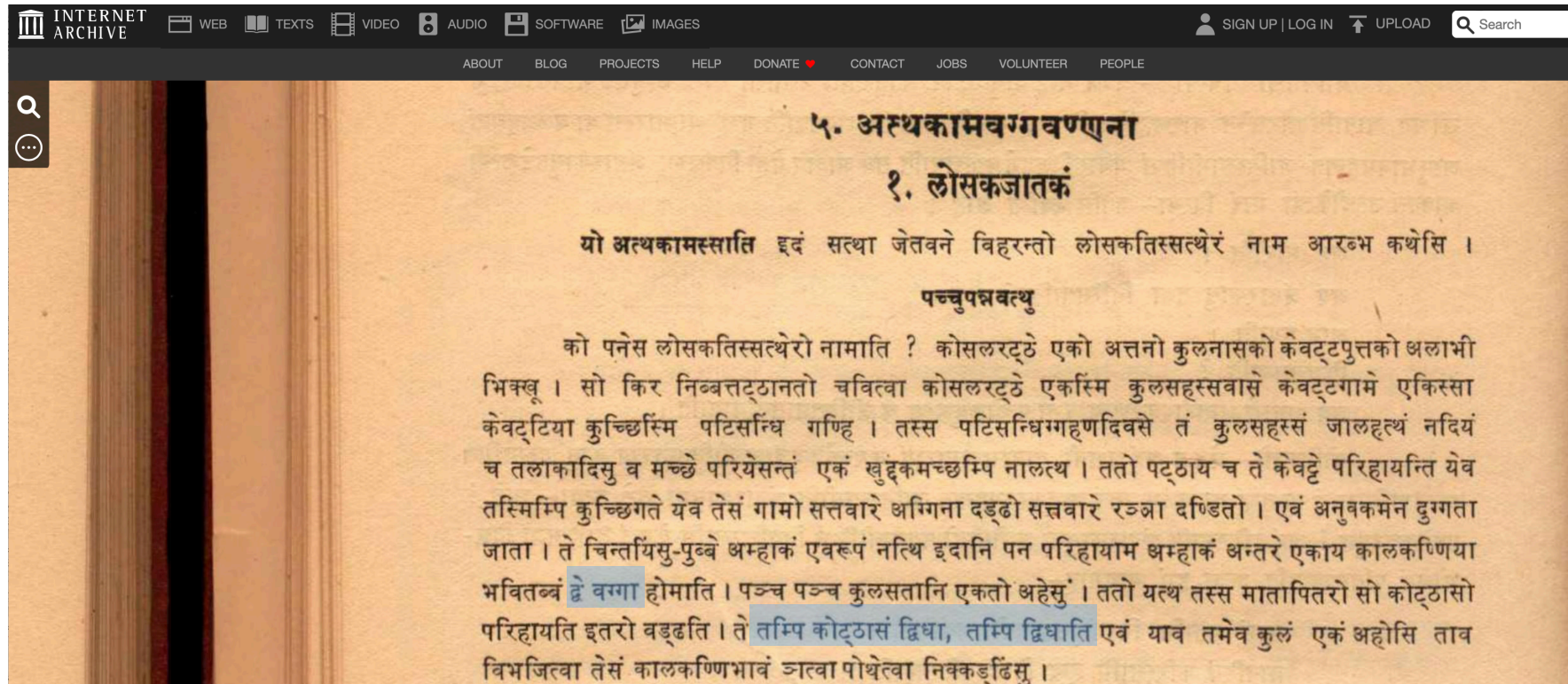
 if ($A[mid] < x$) return Locate(x , $mid + 1$, end);

 if ($A[mid] > x$) return Locate(x , $start$, mid);

}

History

- Binary search was first first mentioned by John Mauchly (1946)
- The art of computer programming, vol 3, p 422
- Mention in Buddhist Jataka Tales



Other examples

- Looking for a word in the dictionary
- Finding a scene in a movie
- Debugging a linear piece of code
- Cooking
- Science/Engineering: Finding the right value of any resource
 - length of youtube ads
 - pricing of a service
- Twenty questions

Egg drop problem

- In a building with n floors,
find the highest floor from where egg can be dropped without breaking.
- $O(\log n)$ egg drops are sufficient.
- Binary search for the answer.
- Drop an egg from floor x
 - if the egg breaks, answer is less than x
 - if the egg doesn't break, the answer is at least x
- Using the standard binary search idea, start with $x = n/2$.
If egg breaks, then go to $x = n/4$ and if it doesn't then go to $x = 3n/4$.
And so on

Egg drop: unknown range

- In a building with *unknown number* of floors, find the highest floor *h* from where egg can be dropped without breaking.
- $O(\log h)$ egg drops sufficient?
- Exponential search: Try floors, 1, 2, 4, ..., till the egg breaks.
- The egg will break at floor 2^{k+1} , where $2^k \leq h < 2^{k+1}$
- Then binary search in the range $[2^k, 2^{k+1}]$
- Total number of egg drops $\leq k+1+k = 2k+1 \leq 2 \log h + 1$.
- Is there a better way?

Lower bound

- Search space size: N
Are $\log N$ queries necessary?
- Yes. When each query is a yes/no type, then the search space gets divided into two parts with each query (some solutions correspond to yes and others to no).
- One of the parts will be at least $N/2$.
- In worst case, with each query, we get the larger of the two parts.
- To reduce the search space size to 1, we need $\log N$ queries

Lower bound

- Search space size: N
Are $\log N$ queries necessary? Another argument.
- Suppose you make k queries.
- There are 2^k possible outcomes.
- If $2^k < N$, then two different elements must lead to same outcomes for all queries.
- We won't be able to say which of the two is correct.

Lower bound

- Search space size: N
Are $\log N$ queries necessary?
- Another argument based on information theory.
- Each yes/no query gives us 1 bit of information.
- The final answer is a number between 1 and N , and thus, requires $\log N$ bits of information.
- Hence, $\log N$ queries are necessary.
- Ignore this argument if it is hard to digest.

Exercise: subarray sum

- Given an array with n positive integers, and a number S , find the minimum length subarray whose sum is at least S ?
- Subarray is a contiguous subset, i.e., $A[i], A[i+1], A[i+2], \dots, A[j-1], A[j]$
- $[10, 12, 4, 9, 3, 7, 14, 8, 2, 11, 6]$

$$S = 27$$

- Can you design an $O(n \log n)$ algorithm?

Exercise: subarray sum

$O(n^3)$ algorithm:

```
for ( $l \leftarrow 1$  to  $n$ ) { // looping over all possible lengths
    for ( $j \leftarrow 0$  to  $n-l-1$ ) { // looping over all possible starting points
         $T \leftarrow \text{sum}(A[j], A[j+1], \dots, A[j+l-1])$ 
        if  $T \geq S$ 
            then return  $l$  and the subarray  $(j, j+l-1)$ ;
    }
}
```

- Two for loops one inside the other, each making at most n iterations.
- Sum function is another for loop with at most n iterations.

Exercise: subarray sum

$O(n^2)$ algorithm:

```
for ( $l \leftarrow 1$  to  $n$ ){ // looping over all possible lengths
     $T \leftarrow$  sum of first  $l$  numbers.
    for ( $j \leftarrow 0$  to  $n-l-1$ ){ // looping over all possible starting points
        if  $T \geq S$  then return  $l$  and the subarray  $(j, j+l-1)$ ;
         $T \leftarrow T - A[j] + A[l+j]$ ;
    }
}
```

- Two for loops one inside the other. Each makes at most n iterations. Hence, $O(n^2)$ time.

Exercise: subarray sum

$O(n \log n)$ algorithm. Approach 1:

Binary search for the minimum length l .

For the current value of l :

- Check if there is a subarray of length l with sum at least S .
- This check can be done in $O(n)$ time. See the inner loop on previous slide.
- If YES then try a smaller value of l
- If NO then try a larger value of l

Exercise: subarray sum

- $O(n \log n)$ algorithm. Approach 2 (suggested by students):
- First compute all the prefix sums and store in an array.
 $O(n)$ time.
- $\text{prefix_sum}[0] \leftarrow A[0];$
for ($i \leftarrow 1$ to $n-1$)
 $\text{prefix_sum}[i] = \text{prefix_sum}[i-1] + A[i];$
- Any subarray sum from i to j can now be computed in $O(1)$ time as
 $\text{prefix_sum}[j] - \text{prefix_sum}[i-1].$
- Now, for each choice of starting point, do a binary search for the minimum end point such that the subarray sum is at least S .
 - If sum of a subarray $< S$, then choose a larger end point, otherwise smaller.

Exercises

- Given two sorted arrays of size n , find the median of the union of the two arrays.
 $O(\log n)$ time?
- Given a convex function $f(x)$ oracle, find an integer x which minimizes $f(x)$.
- Land redistribution:
given list of landholdings $a_1, a_2, \dots, a_n$,
given a floor value f ,
find the right ceiling value c

Division algorithm

- As we find the next digit of the quotient, the search space of the quotient goes down by a factor of 10.
- This could be called a denary search.
- For binary representation of numbers, the division algorithm will be a binary search.

Finding square root

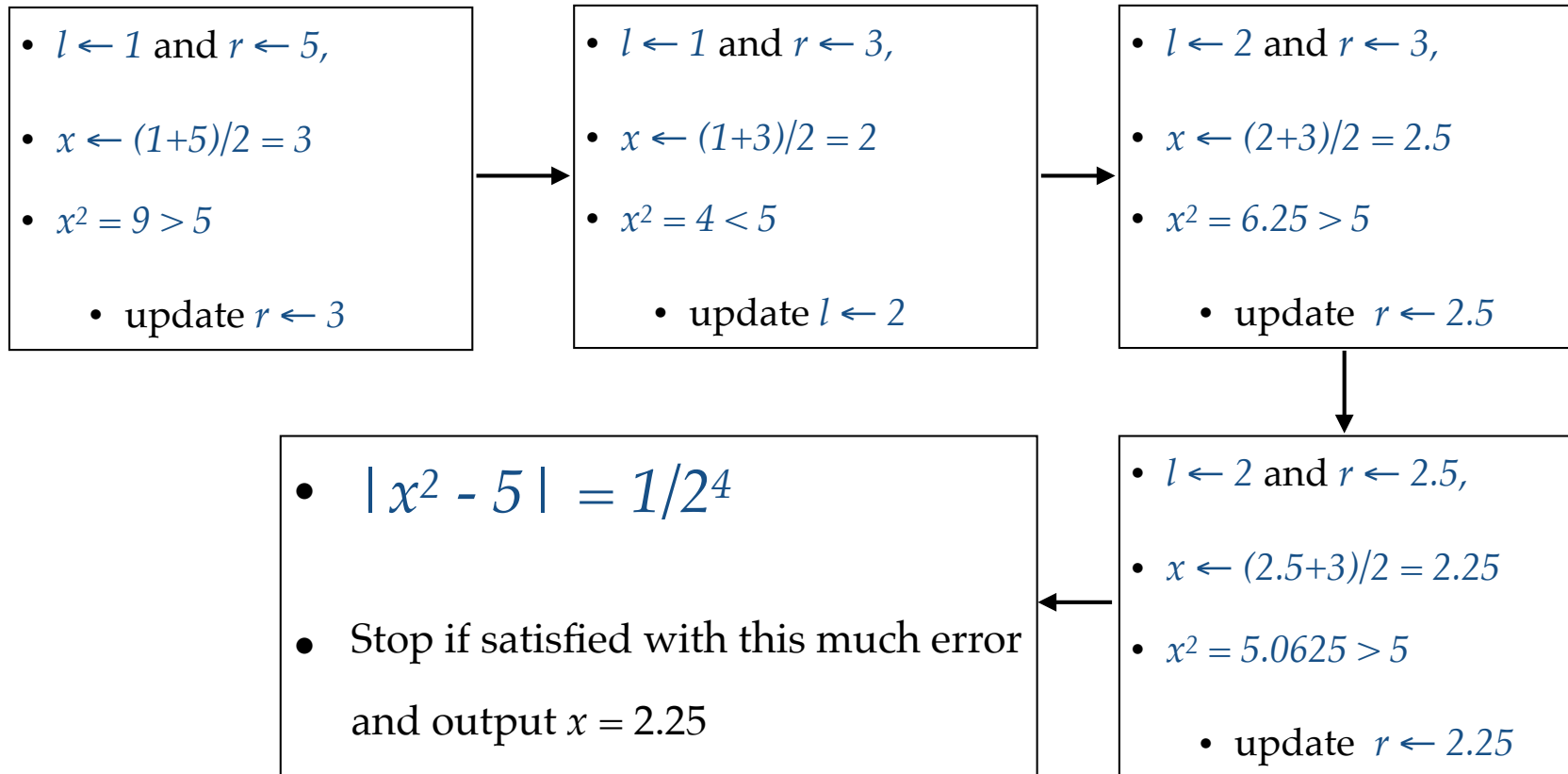
- Given an integer a , find $\sqrt{a}$
- Starting with $l \leftarrow 1$ and $r \leftarrow a$,
 - i.e., starting range is $[1, a]$
- $x \leftarrow (l+r)/2$ // Guess for $\sqrt{a}$,
- If $x^2 > a$, then the answer is less than x .
 - update $r \leftarrow x$
- Else the answer is at least x .
 - update $l \leftarrow x$
- Repeat till we get $x^2 = a$ (or till we get desired precision e.g., $|x^2 - a| < 1/2^{10}$)
- **Additional exercise:** find a division like algorithm.

Finding square root

- Given an integer a , find $\sqrt{a}$ up to l digits after decimal.
- Can we compute it in $O(l)$ iterations?
- Yes. See example on next slide.

Finding square root

- Computing $\text{sqrt}(5)$.



More applications

- Finding inverse of an increasing function $f(x)$?
- Finding root of an odd-degree polynomial?
- Finding the smallest prime dividing N ?
- Is sorting a kind of binary search?
 $O(n \log n)$ comparisons necessary?

Inverse of an increasing function

- Finding inverse of an increasing function?
- For any given x , we have a method to compute $f(x)$.
For a given y , we want to compute $f^{-1}(y)$.
- Make a guess x and compare it $f(x)$ with y
- If $f(x) < y$ then the answer is larger than x
- Else the answer is at least x .

Root of a polynomial

- Finding a root of polynomial $f(x)$ with odd degree?
- Always maintain two points a and b such that $f(a) > 0$ and $f(b) < 0$.
- To find the starting points one can do exponential search.
- Check whether $f((a+b)/2) > 0$.
- If yes, then there is a root between $(a+b)/2$ and b .
- Else, there is a root between $(a+b)/2$ and a .

Smallest dividing prime

- Finding the smallest prime dividing N ?
- No.
- We can make a guess x . If x does not divide N , then we cannot say anything about where should be the smallest prime dividing N .

Sorting as binary search

- Is sorting a kind of binary search?
 $n \log n$ comparisons necessary?
- Yes.
- When we have not made any comparisons, then any of the $n!$ rearrangements is a possible answer.
- So the search space size is $n!$.
- Each comparison will reduce the search space size by only $1/2$ (in worst case).
- Hence, $\log(n!) \geq n \log n - n \log e$ comparisons are necessary.

Analyzing algorithms

- Comparison between various candidate algorithms
- Why not implement and test?
 - too many algorithms
 - depends on input size, how inputs are chosen
- Will count the number of basic operations like addition, comparison etc.
- And see how this number grows as a function of the input size. This measure is independent of the choice of the machine.

$O(\cdot)$ notation

- For input size n , running time $f(n)$
- We say $f(n)$ is $O(T(n))$
if for all large enough n and some constant c ,
 $f(n) \leq c \cdot T(n)$

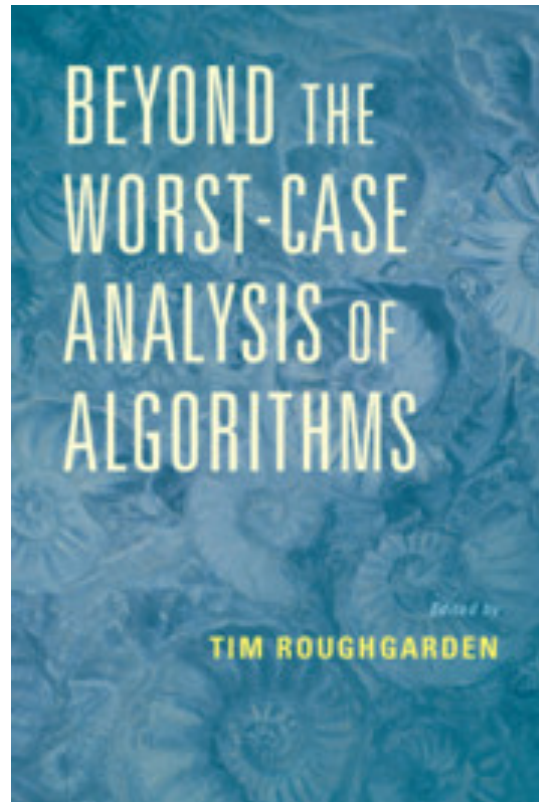
$O(\cdot)$ notation

- Why do we ignore constant factors?
- Because it's not always possible to find the precise constant factor. Various basic operations do not take the same amount of time.
- Is $O(n)$ always better than $O(n \log n)$?
- For large enough inputs, yes. But, depending on the hidden constant factors, it's possible that $O(n \log n)$ algorithm is faster on reasonable size inputs.

Worst case analysis

- **Worst case bound**: running time guarantee for all possible inputs of a size.
- There could be algorithms which are slow on a few pathological instances, but otherwise quite fast.
- Why not analyze only for “real world inputs”?
- It’s not clear how to model “real world inputs”.
- For many algorithms, we are able to give worst case bounds, so why not do it.

Worst case analysis



Out of scope of this course

Describing algorithms

- Find the maximum sum subarray of length k

```
s = 0;
for (i=0, i < k, i++) s=s+A[i];
m = s;
for (i=0, i < n-k, i++){
    s = s - A[i] + A[k+i];
    if (s > m) m = s;
}
return m;
```

Compute the sum of first k numbers. We will go over all length k subarrays from left to right. In an iteration, update the sum by subtracting the first number of the current array and adding the number following the last one. If the sum is larger than the maximum seen so far, we update the maximum.

```
s ← sum of first  $k$  numbers;
for ( $i \leftarrow 0$  to  $n-k-1$ ){
     $s \leftarrow s - A[i] + A[k+i]$ ;
     $m \leftarrow \max(m, s)$ ;
}
return  $m$ ;
```

Precise, but hard to understand. Error prone.

Not precise. Open to multiple interpretations.

Somewhere in the middle.

Describing algorithms

- Two sorted (increasing) arrays A and B of length n .
Count pairs (a,b) such that $a \in A$ and $b \in B$
and $a < b$

```
j=0; count = 0;  
for (i=0, i < n, i++){  
    while (A[i] >= B[j]) j++;  
    count=count + n - j;  
}  
return count;
```

```
j ← 0; count ← 0;  
for (i ← 0 to n-1){  
    keep increasing  $j$  until we get  $B[j] > A[i]$ ;  
    count ← count + n - j;  
}  
return count;
```

Describing algorithms

- Two sorted (increasing) arrays A and B of length n .
Count pairs (a,b) such that $a \in A$ and $b \in B$
and $a < b$

```
j=0; count = 0;  
for (i=0, i < n, i++){  
    while (A[i] >= B[j]) j++;  
    count=count + n - j;  
}  
return count;
```

```
j ← 0; count ← 0;  
for (i ← 0 to n-1){  
    Find the first index  $j$  such that  $B[j] > A[i]$ ;  
    count ← count + n - j;  
}  
return count;
```

$O(\cdot)$ notation

- True or False?
- $2n+3$ is $O(n^2)$
 - True
- $1^2 + 2^2 + \dots + (n-1)^2 + n^2$ is $O(n^2)$
 - False (it is $\Theta(n^3)$)
- $1 + 1/2 + 1/3 + \dots + 1/n$ is $O(?)$
 - $O(\log n)$
- n^n is $O(2^n)$
 - False

$O(\cdot)$ notation

- True or False?
- 2^{3n} is $O(2^n)$
 - False
- $(n+1)^3$ is $O(n^3)$
 - True
- $(n + \sqrt{n})^2$ is $O(n^2)$
 - True
- $\log(n^3)$ is $O(\log n)$
 - True

Principles of algorithm design

First principle: reducing to a subproblem

- Subproblem: same problem on a smaller input
- Assume that you have already built the solution for the subproblem and using that try to build the solution for the original problem.
- Subproblem will be solved using the same strategy.
- Implementation: recursive or iterative

First principle: reducing to a subproblem

- Example 1: finding minimum value in an array.
- Suppose we have already found minimum among first $n-1$ numbers, say min_{n-1}
- $min_n = \text{minimum}(min_{n-1}, A[n])$
- **Iterative implementation:**
Go over the array from 1 to n and maintain a variable min
- **Invariant:** after seeing i numbers, min will be the minimum among first i numbers.
- $min_i = \text{minimum}(min_{i-1}, A[i])$

First principle: reducing to a subproblem

- $min_i = \text{minimum}(min_{i-1}, A[i])$

- Iterative implementation

```
min  $\leftarrow$  A[1];  
for (i  $\leftarrow$  2 to n)  
    min  $\leftarrow$  minimum(min, A[i])
```

- Recursive implementation

```
f(A, i):  
    if i=1 return A[1];  
    else return minimum(f(A, i-1), A[i]);  
Compute f(A, n);
```

Maximum subarray sum

- Given an integer array with positive / negative numbers.
Find the subarray with maximum possible sum.
- 1, 2, -5, 4, -6, 8, 7, -3, 2, 10, 3, -7, 4, 2
- $O(n^2)$ algorithm
- For every choice of starting point,
go over all choices of end points and maintain the sum
between starting and end points.
- Maintain the *max_sum* value by comparing with the current
sum

Maximum subarray sum

$O(n^2)$ algorithm:

```
max_sum  $\leftarrow$  0;
```

```
for (start  $\leftarrow$  1 to n){
```

```
    curr_sum  $\leftarrow$  0;
```

```
    for (end  $\leftarrow$  start to n){
```

```
        curr_sum  $\leftarrow$  curr_sum + A[end]; // update the current sum
```

```
        max_sum  $\leftarrow$  maximum(curr_sum, max_sum);
```

```
    }
```

```
}
```

Maximum subarray sum

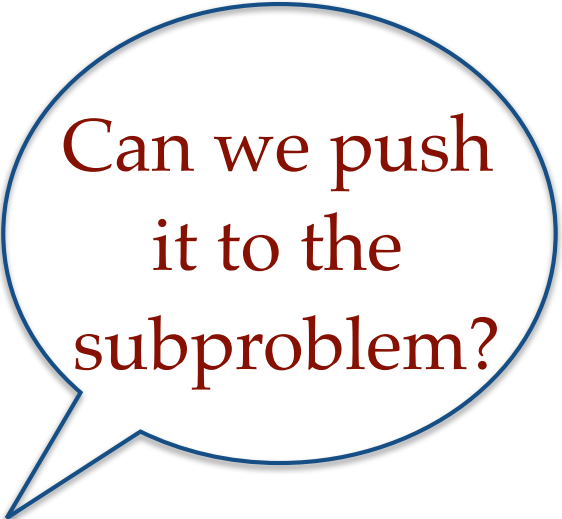
$O(n^2)$ algorithm:

- Let *max_sum* store the maximum subarray sum seen so far. Initialize as 0.
- Go over all choices of starting point *start* (from 1 to *n*). For every choice of *start*:
 - Let *curr_sum* maintain the sum of the current subarray. Initialize as 0.
 - Go over all choices of ending point *end* (from *start* to *n*). For every choice of *end*:
 - Update the *curr_sum* by adding $A[end]$ to it.
 - Compare *max_sum* with *curr_sum*.
 - If *curr_sum* is larger then update *max_sum* with the value of *curr_sum*.

Max subarray sum: subproblem

- Suppose we have already found the maximum subarray sum for $A[1 \dots n-1]$, say max_sum_{n-1}
- How do we compute max_sum_n
- Two kinds of subarrays of A :
 1. subarrays of $A[1 \dots n-1]$
 2. subarrays of A which end at n
- $max_sum_n = \text{Maximum}($

$$\left. \begin{array}{l} max_sum_{n-1}, \\ Sum(1 \dots n), \\ Sum(2 \dots n), \\ \vdots \\ Sum(n \dots n), \end{array} \right\} O(n)$$



Can we push
it to the
subproblem?

$$T(n) = T(n-1) + O(n) \quad \Rightarrow \quad T(n) = O(n^2)$$

Max subarray sum: subproblem

- Improvement ?
- Observation:
 - $Sum(1 \dots n) = Sum(1 \dots n-1) + A[n],$
 - $Sum(2 \dots n) = Sum(2 \dots n-1) + A[n],$
 $\vdots$
 - $Sum(n-1 \dots n) = Sum(n-1 \dots n-1) + A[n],$

$$\text{Maximum} \begin{pmatrix} Sum(1 \dots n), \\ Sum(2 \dots n), \\ \vdots \\ Sum(n-1 \dots n) \end{pmatrix} = \text{Maximum} \begin{pmatrix} Sum(1 \dots n-1), \\ Sum(2 \dots n-1), \\ \vdots \\ Sum(n-1 \dots n-1) \end{pmatrix} + A[n]$$

- We have converted it to another problem on first $n-1$ numbers

Max subarray sum: subproblem

- Improvement ?

- Observation:

- $Sum(1 \dots n) = Sum(1 \dots n-1) + A[n],$
- $Sum(2 \dots n) = Sum(2 \dots n-1) + A[n],$
 $\vdots$
- $Sum(n-1 \dots n) = Sum(n-1 \dots n-1) + A[n],$

$$\text{Maximum} \begin{pmatrix} Sum(1 \dots n), \\ Sum(2 \dots n), \\ \vdots \\ Sum(n-1 \dots n) \end{pmatrix} = \text{Maximum} \begin{pmatrix} Sum(1 \dots n-1), \\ Sum(2 \dots n-1), \\ \vdots \\ Sum(n-1 \dots n-1) \end{pmatrix} + A[n]$$

Push it to the subproblem

- We have converted it to another problem on first $n-1$ numbers

$max_suffix_sum_{n-1}$

Asking subproblem to do more

- Subproblem: max_sum_{n-1} and $max_suffix_sum_{n-1}$
- $max_suffix_sum_{n-1}$ is defined as
Maximum($Sum(1 \dots n-1)$, $Sum(2 \dots n-1)$, ... $Sum(n-1 \dots n-1)$)

- $max_sum_n = \text{Maximum} \left(\begin{array}{l} max_sum_{n-1}, \\ Sum(1 \dots n-1) + A[n], \\ Sum(2 \dots n-1) + A[n], \\ \vdots \\ Sum(n-1 \dots n-1) + A[n], \\ A[n] \end{array} \right)$

- $= \text{Maximum} \left(\begin{array}{l} max_sum_{n-1}, \\ max_suffix_sum_{n-1} + A[n], \\ A[n] \end{array} \right)$

Asking subproblem to do more

- Subproblem: max_sum_{n-1} and $max_suffix_sum_{n-1}$

- $max_sum_n = \text{Maximum} \left(\begin{array}{l} max_sum_{n-1}, \\ max_suffix_sum_{n-1} + A[n], \\ A[n] \end{array} \right)$

- We are asking the subproblem to compute max_suffix_sum for size $n-1$
- So, we also need to compute max_suffix_sum for size

- $max_suffix_sum_n = \text{Maximum} \left(\begin{array}{l} max_suffix_sum_{n-1} + A[n], \\ A[n] \end{array} \right)$

$$T(n) = T(n-1) + O(1) \Rightarrow T(n) = O(n)$$

$O(n)$ algorithm

- Go over i from 1 to n
- For each i ,
 - Maintain max_sum — maximum subarray sum seen in $A[1...i]$
 - Maintain max_suffix_sum — maximum suffix sum seen in $A[1...i]$
- Update the two variables as mentioned earlier

Maximum subarray sum

$O(n)$ algorithm:

$max_sum \leftarrow 0$; $max_suffix_sum \leftarrow 0$;

for ($i \leftarrow 1$ to n){

$max_sum \leftarrow \text{maximum}(max_sum,$
 $max_suffix_sum + A[i],$
 $A[i]);$

$max_suffix_sum \leftarrow \text{maximum}(max_suffix_sum + A[i], A[i]);$

}

Alternate implementation

max_sum $\leftarrow$ 0; *max_suffix_sum* $\leftarrow$ 0;

for (*i* $\leftarrow$ 1 to *n*){

max_suffix_sum $\leftarrow$ maximum(*A*[*i*], *max_suffix_sum* + *A*[*i*]);

max_sum $\leftarrow$ maximum(*max_suffix_sum*, *max_sum*);

}

- Here we are updating the two variables in a different order.

Reducing to a subproblem

- When solving a problem recursively / inductively, it is sometimes useful to solve a more general problem
- Stronger induction hypothesis

Exercises

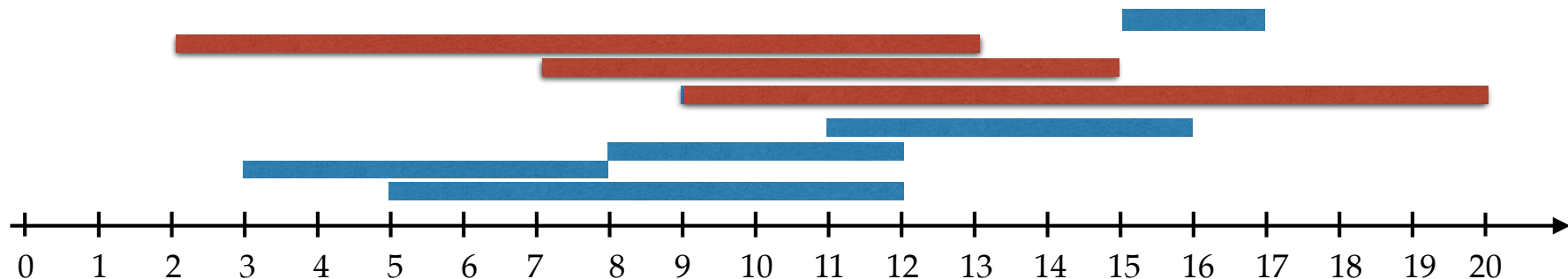
- Given share prices for n days
 $p_1, p_2, \dots, p_n$
- You have to buy it on one of the days and sell it on a later day.
- Maximum profit possible in $O(n)$?
- $\max_{\{j > i\}} (p_j - p_i)$

Exercises

- There is a party with n people, among them there is 1 celebrity.
- A **celebrity** is someone who is known to everyone, but she does not know anyone.
- you ask the any person i if they know person j .
- Can you do find the celebrity in $O(n)$ queries?

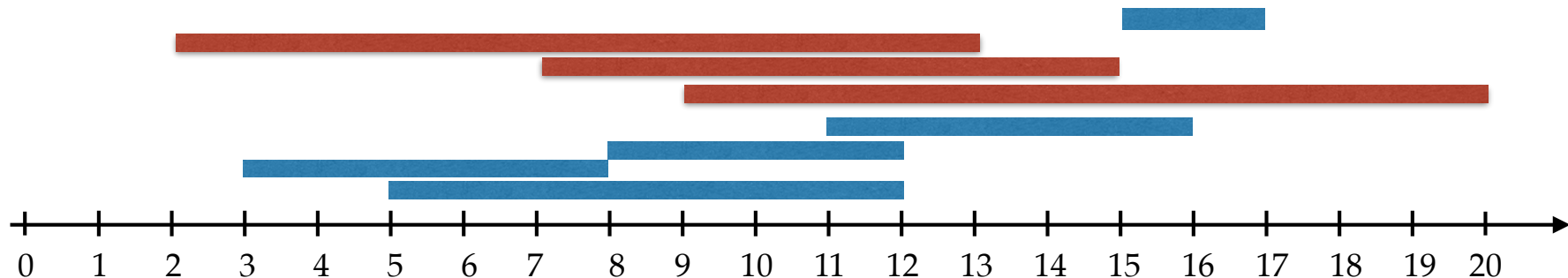
Interval containment

- Given a set of intervals
count the number of intervals which are not contained in any other interval.
- $(5, 12)$, $(3, 8)$, $(8, 12)$, $(11, 16)$, $(9, 20)$, $(15, 17)$, $(7, 15)$, $(2, 13)$
- $(9, 20)$, $(7, 15)$, $(2, 13)$



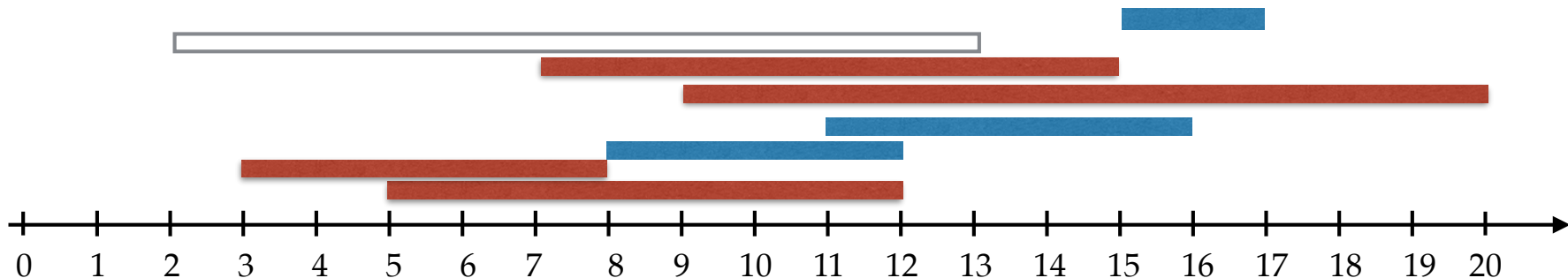
Interval containment

- Given a set of intervals
count the number of intervals which are not contained in any other interval.
- Naive solution: for every interval, check every other interval
- $O(n^2)$



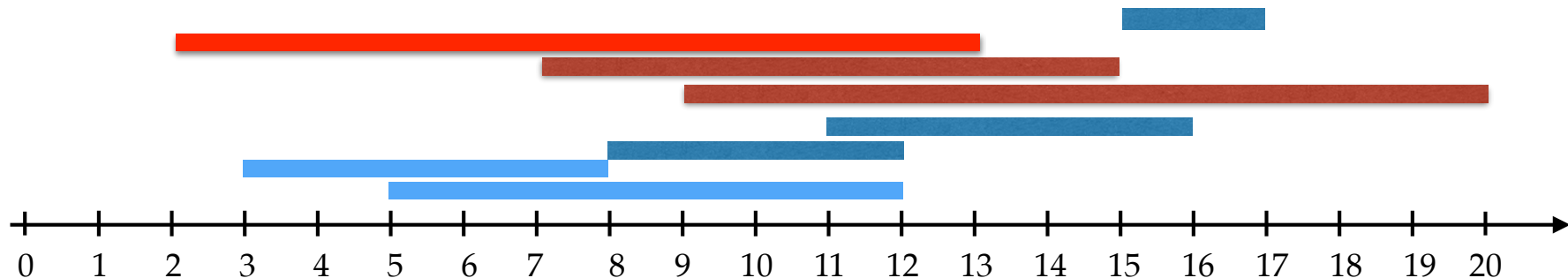
Subproblem

- Suppose we have a solution for first $n-1$ intervals.
- $(5, 12), (3, 8), (8, 12), (11, 16), (9, 20), (15, 17), (7, 15), (2, 13)$
- $(3,8), (5, 12), (9, 20), (7, 15)$



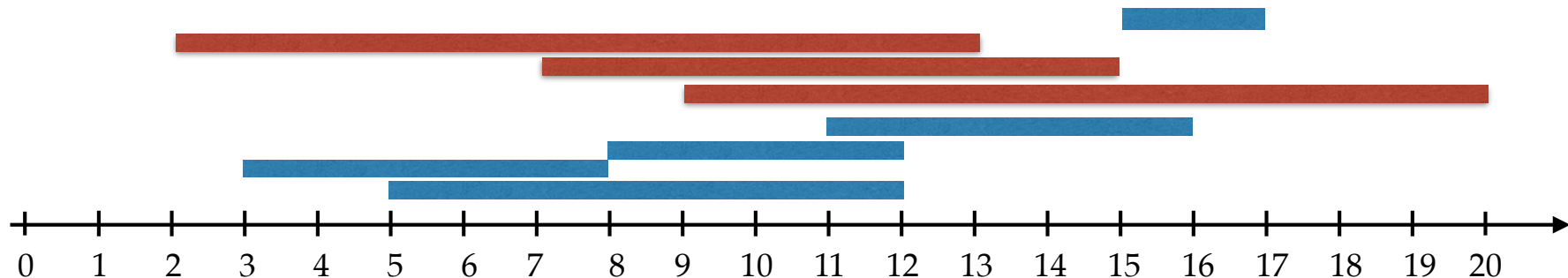
Interval containment

- When we introduce the n th interval need to check if it is contained in any other interval or if it contains other intervals. Seems to take $O(n)$ time.
- (5, 12), (3, 8), (8, 12), (11, 16), (9, 20), (15, 17), (7, 15), (2, 13)
- ~~(3,8)~~, ~~(5,12)~~, (9, 20), (7, 15), (2, 13)



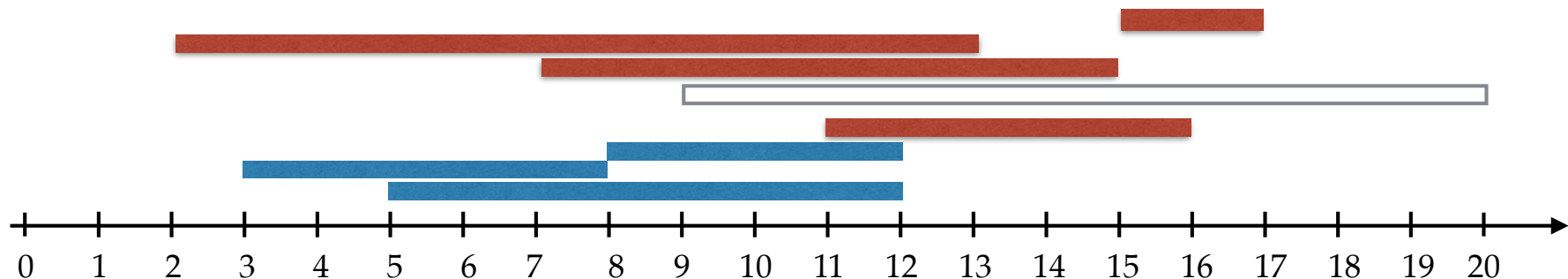
Reordering input

- Can we reorder the intervals so that the work at the last step reduces?
- Two possible orders: increasing start time, increasing finish time
 - (2, 13), (3, 8), (5, 12), (7, 15), (8, 12), (9, 20), (11, 16), (15, 17),
 - (3, 8), (5, 12), (8, 12), (2, 13), (7, 15), (11, 16), (15, 17), (9, 20),



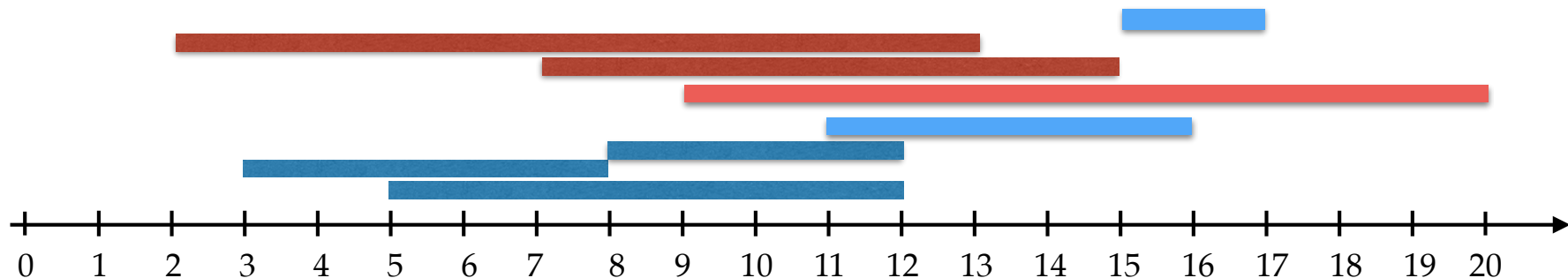
Increasing finish time

- Can we reorder the intervals so that the work at the last step reduces?
- Consider increasing finish time
 - $(3, 8)$, $(5, 12)$, $(8, 12)$, $(2, 13)$, $(7, 15)$, $(11, 16)$, $(15, 17)$, $(9, 20)$,
 - $(2, 13)$, $(7, 15)$, $(11, 16)$, $(15, 17)$,



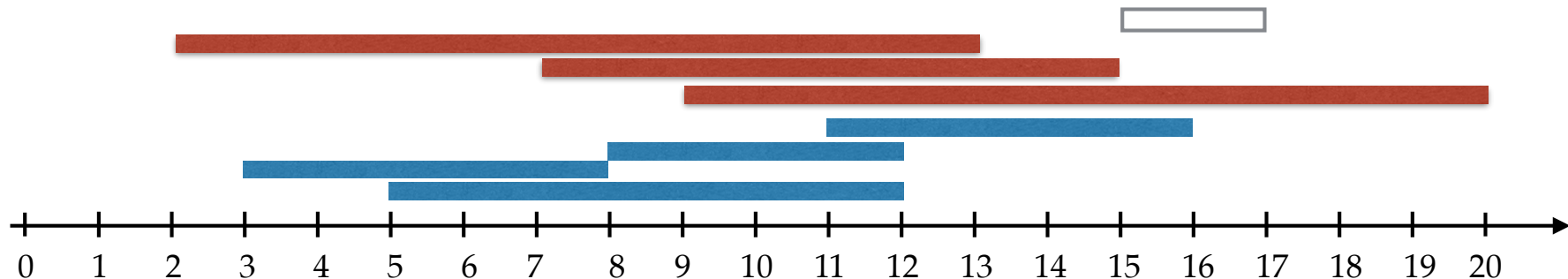
Increasing finish time

- Can we reorder the intervals so that the work at the last step reduces?
- Consider increasing finish time. Same $O(n)$.
 - $(3, 8), (5, 12), (8, 12), (2, 13), (7, 15), (11, 16), (15, 17), (9, 20),$
 - $(2, 13), (7, 15), \textcolor{brown}{(11, 16)}, \textcolor{brown}{(15, 17)}, (9, 20),$



Increasing start time

- Consider increasing start time
 - (2, 13), (3, 8), (5, 12), (7, 15), (8, 12), (9, 20), (11, 16), (15, 17),
 - The last interval cannot contain any other
 - Need to check whether the last interval is contained in any other
 - Same as whether any previous interval finishes after the last one.
 - Takes $O(n)$ time?



Algorithm

- Maintain the highest finish time among interval seen so far
- Go over all intervals in increasing order of start times.
- For an interval (s_i, f_i) :
 - if $f_i > \text{largest_finish_time}$ then
 - number_of_maximal_intervals ++ ;
 - $\text{largest_finish_time} \leftarrow f_i$
- $O(n \log n)$ time

Questions

- Does this implementation work if some intervals have same start times or same finish times?
- If not, how would you change the implementation?
- Can you also design an algorithm which works with increasing order of finish times?
- Is it possible to get an algorithm with $O(n)$ time?
- Or can you show a lower bound of $\Omega(n \log n)$?

Ideas so far

- Reducing to a subproblem
- **Stronger induction hypothesis:** Sometimes may need to solve more than what is asked for
- Reordering the input can be helpful.

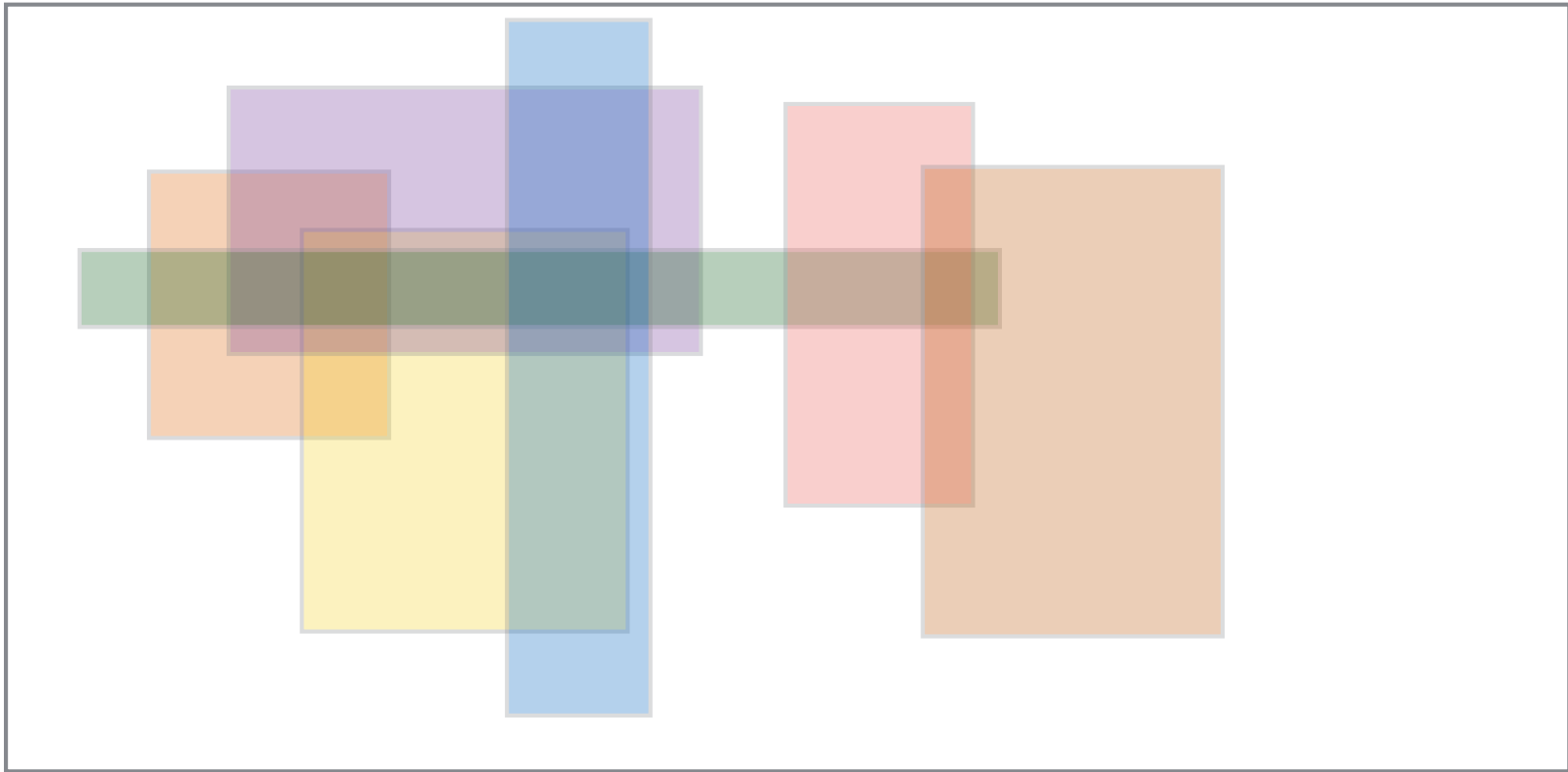
Other problems on intervals

- Given n intervals, find the total length of their union.
- Given n intervals, find the number of intervals which do not overlap with any other interval.
- Given n intervals, find the maximum number of mutually intersecting intervals.

Divide and Conquer

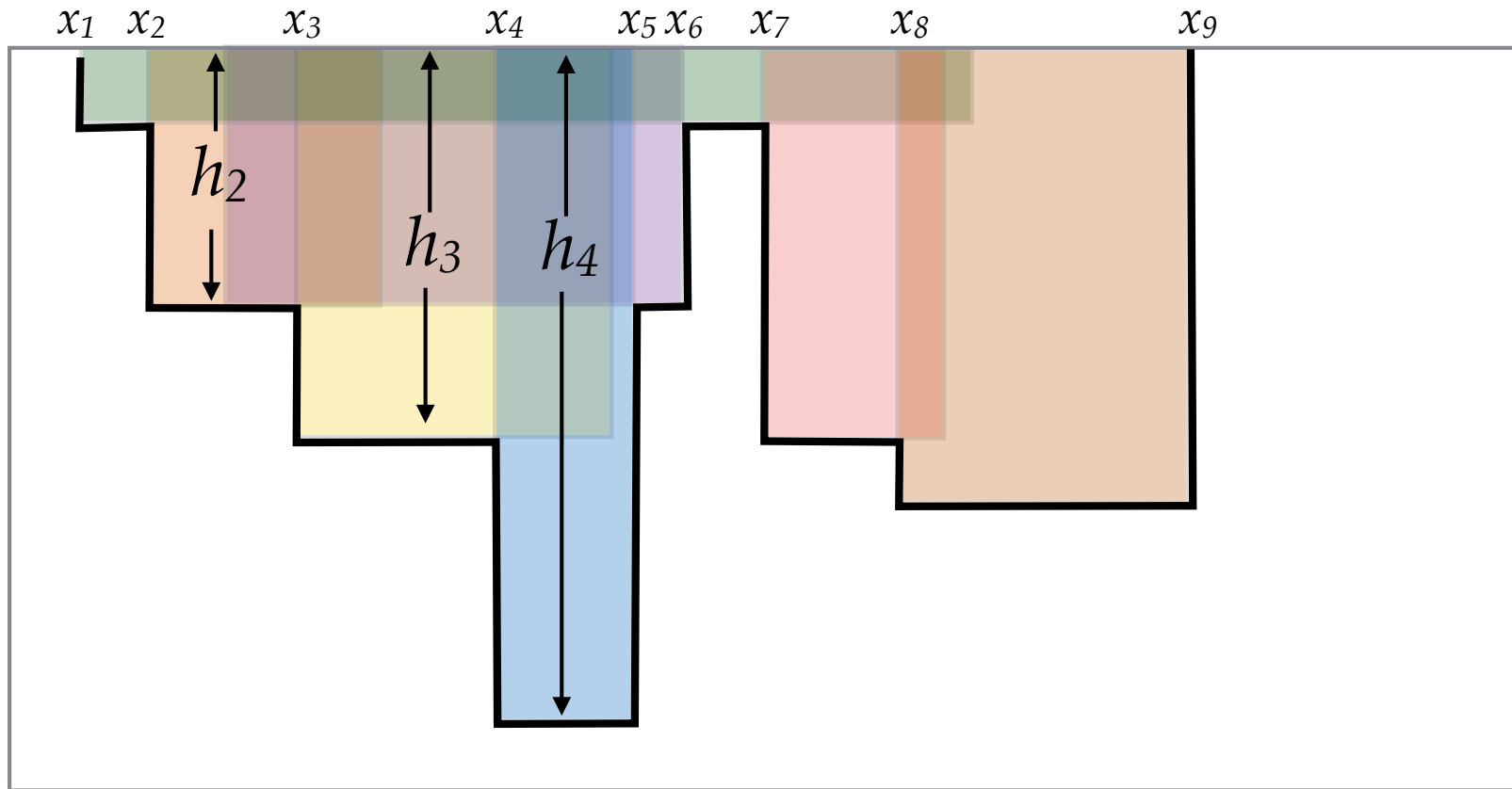
- Classic example: merge sort
- Divide the problem into two (or more) subproblems of size $n/2$
- Combine the solutions of the subproblems and build a solution for the original problem
- $T(n) = a T(n/2) + f(n)$
- Divide and conquer might give a running time improvement e.g., from $O(n^2)$ to $O(n \log n)$.

Area coverage



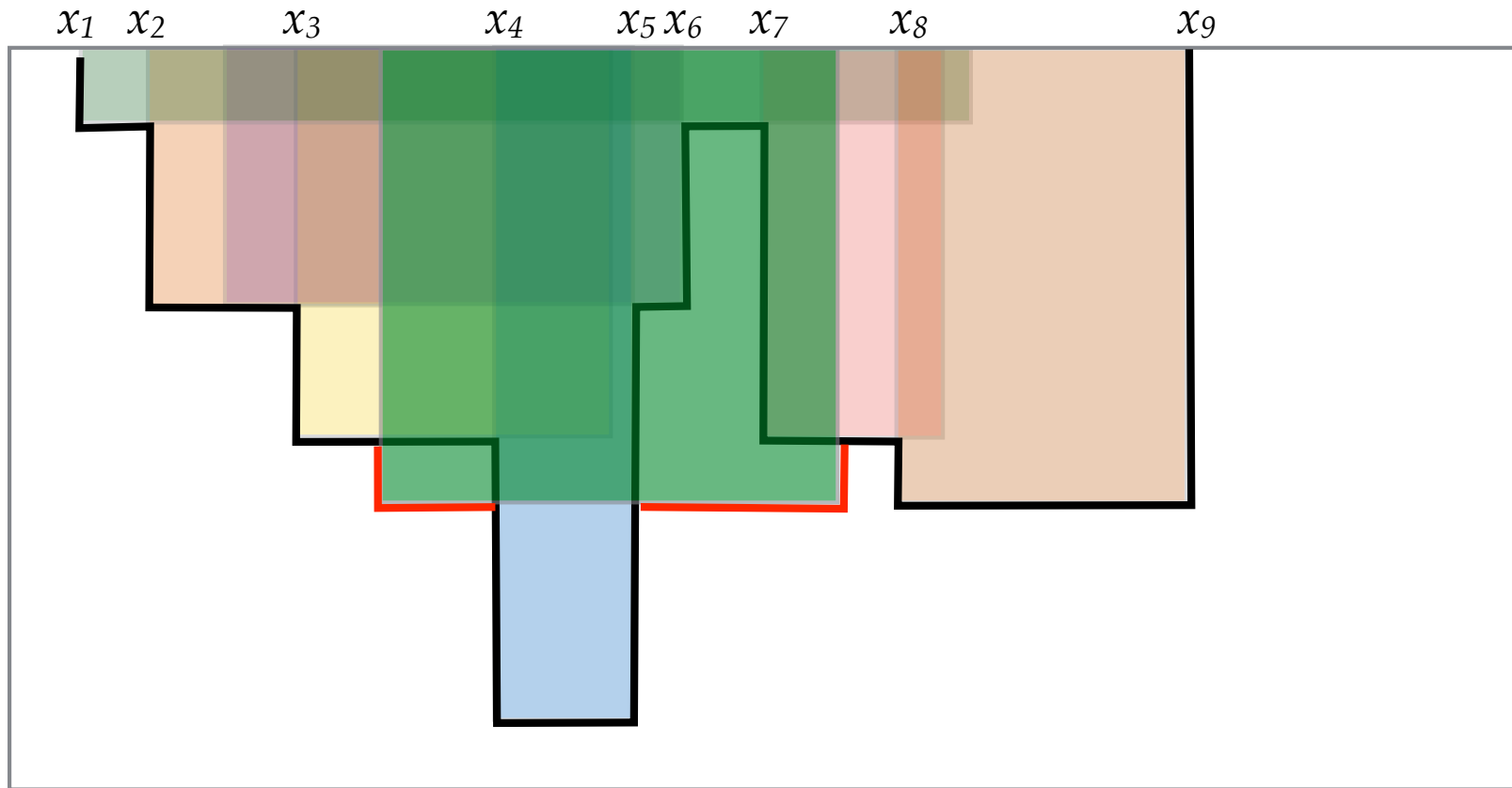
- Find the total area covered

Simpler version



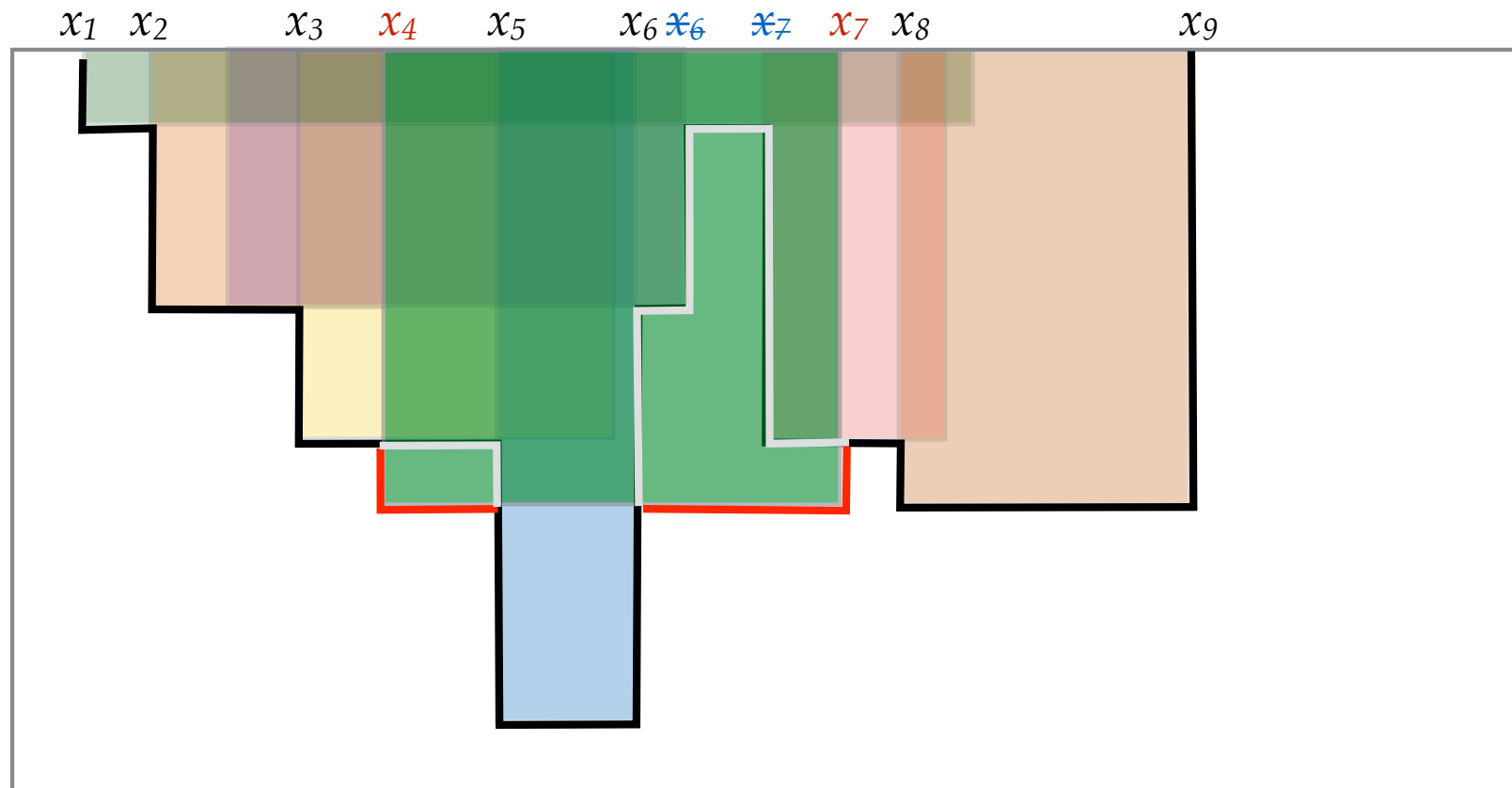
- Also known as skyline problem
- Input: For each rectangle (l_i, r_i, d_i) .
- Compute outline: $(x_1, h_1), (x_2, h_2), (x_3, h_3), (x_4, h_4), (x_5, h_5), (x_6, h_6), (x_7, h_7), (x_8, h_8), (x_9, 0)$,

$O(n^2)$ algorithm



- First solution: introduce rectangles one by one, and update the outline.
-

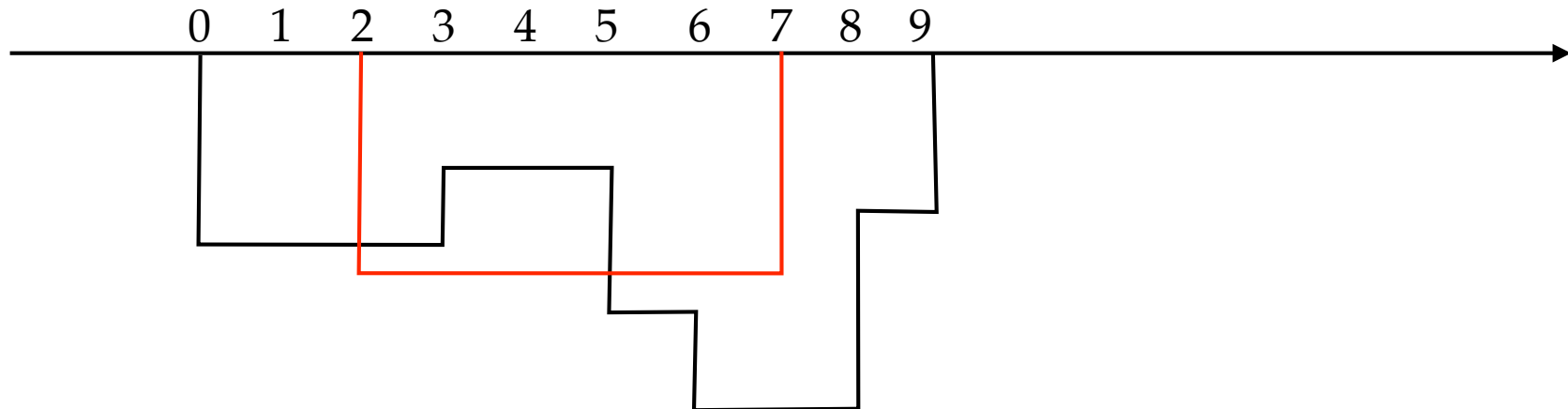
$O(n^2)$ algorithm



- First solution: introduce rectangles one by one, and update the outline.
- Time: $O(n)$ per update.

Update example

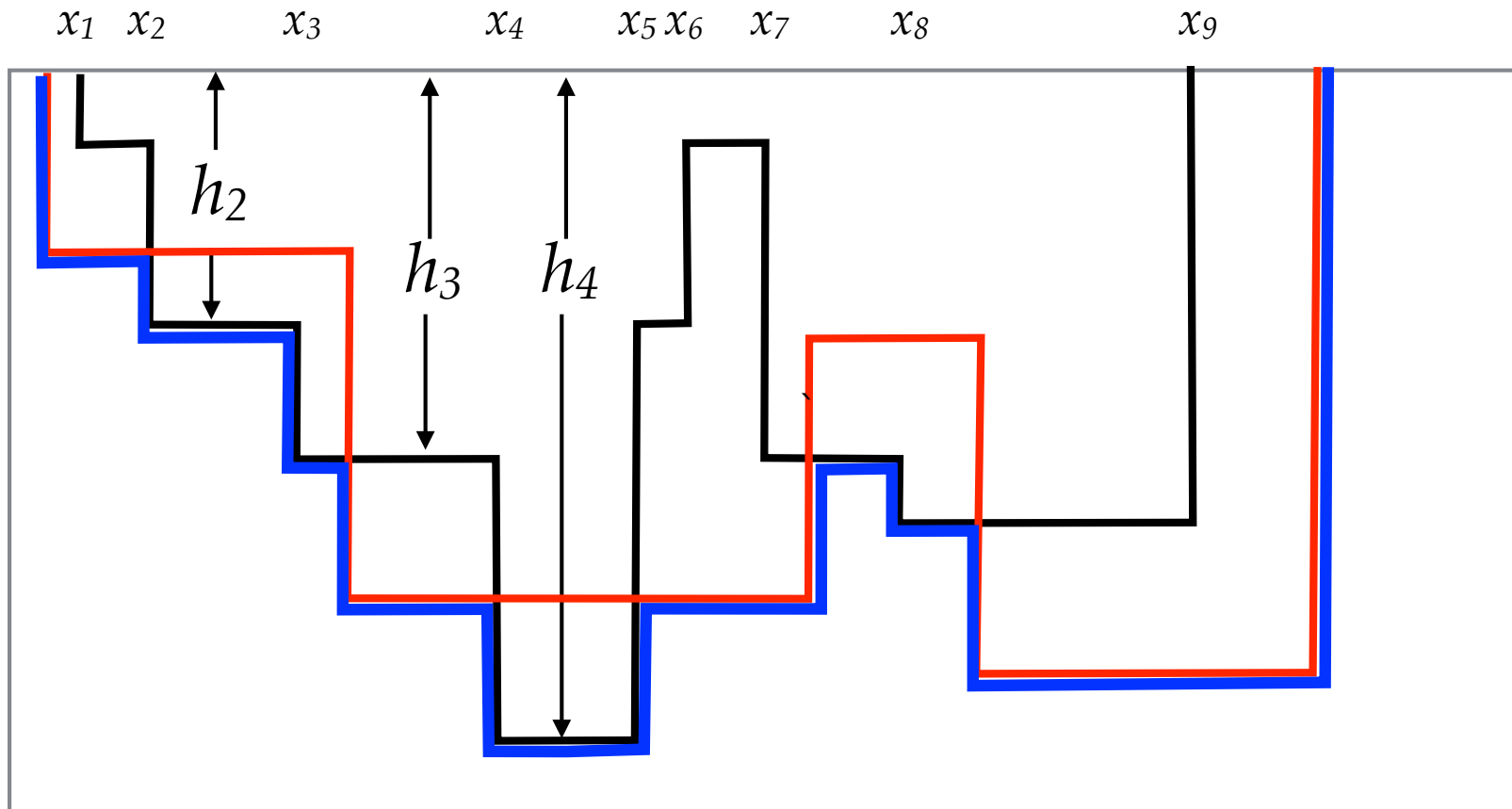
- Current outline: (0, 5), (3, 3), (5, 7), (6, 9), (8, 4), (9, 0)
- Incoming rectangle: (2, 7, 6)
- Updated outline: (0, 5), (2, 6), (~~3, 3~~), (5, 7), (6, 9), (8, 4), (9, 0)
- Update can be done in $O(n)$ time?



Divide and conquer approach

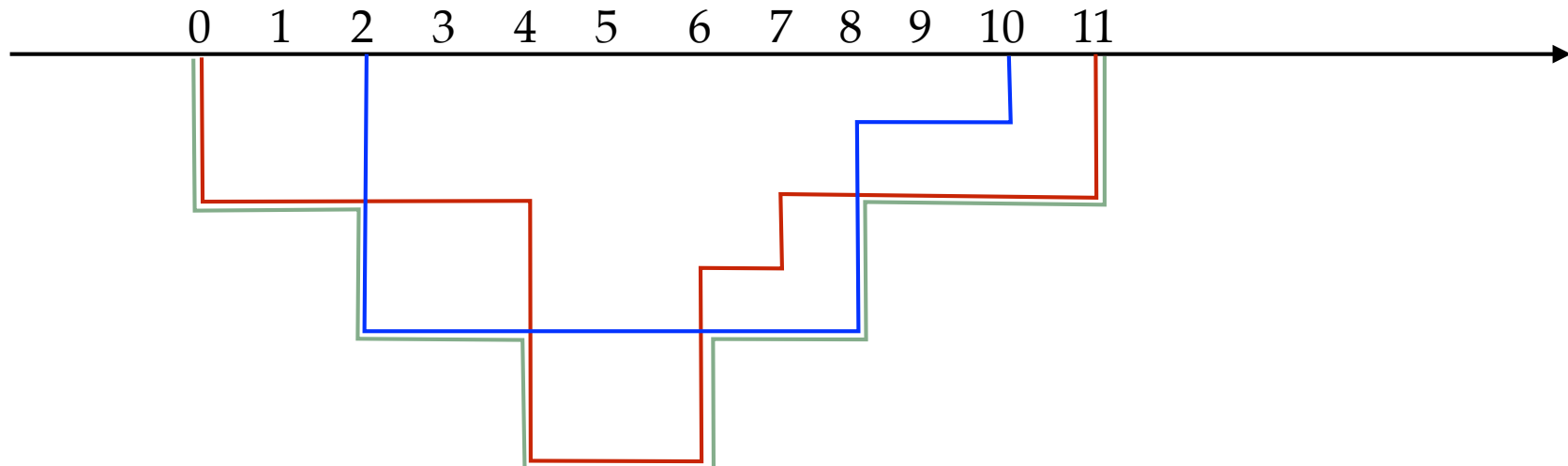
- Divide the set of rectangles into two parts with $n/2$ rectangles each.
- Suppose we have computed the outline for each set of $n/2$ rectangles.
- Outline 1: $(x_1, h_1), (x_2, h_2), (x_3, h_3), \dots, (x_m, 0)$
- Outline 2: $(a_1, p_1), (a_2, p_2), (a_3, p_3), \dots, (a_k, 0)$
- “merge” the two outlines to compute a new outline.

Merge two outlines



Merge two outlines

- Outline 1: (0, 2), (4, 6), (6, 3), (7, 2), (11, 0)
- Outline 2: (2, 4), (8, 1), (10, 0)
- Merged: (0, 2), (2, 4), (4, 6), (6, 4), (8, 2), (11, 0)



Merge two outlines

- Outline 1: $(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, 0)$
- Outline 2: $(a_1, b_1), (a_2, b_2), (a_3, b_3), \dots, (a_k, 0)$
- Pointers for two queues i and j .
- Maintain the current height in each outline
 $height_1, height_2$
- If $x_i < a_j$ then
 - $height_1 \leftarrow y_i$
 - Push $(x_i, \max (height_1, height_2))$
 - $i \leftarrow i + 1$
- Else is similar
- Final clean up: remove consecutively repeating heights in the merged outline

Alternative implementation

- Maintain the current height in each outline
height_1, height_2
- If $x_i < a_j$ then
 - $height_1 \leftarrow y_i$
 - $height_to_push \leftarrow \max (height_1, height_2)$
 - If $(last_pushed_height \neq height_to_push)$
 - Push $(x_i, height_to_push)$
 - $last_pushed_height \leftarrow height_to_push$
 - $i \leftarrow i + 1$
- Else is similar

Other approaches

- Divide and conquer with respect to heights?
- Approaches without divide and conquer
- Reordering the input
 - Increasing order of left end points: $O(n \log n)$ time implementation possible using a data structure like balanced binary tree.
 - Decreasing order of heights: $O(n \log n)$ time implementation possible using a data structure like balanced binary tree or heap.
- $O(n \log n)$ necessary ?

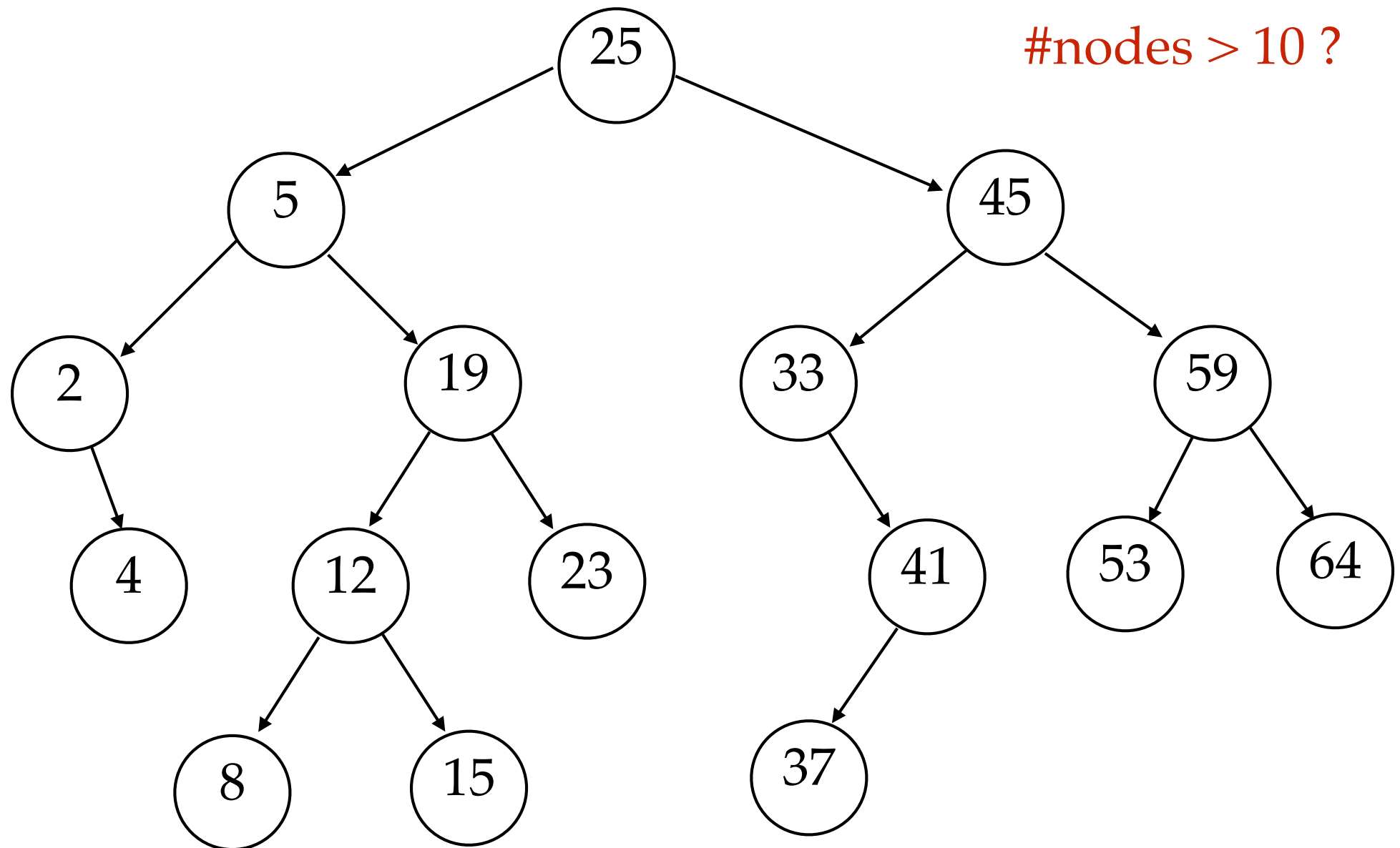
Significant inversion

- Given an array A of integers
- a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$.
- $O(n \log n)$ time algorithm to find the **number** of significant inversions.
- Divide and conquer will work (*Homework*).
- Alternate approach.

Significant inversion

- a pair (i, j) is called a significant inversion if $i < j$ and $A[i] > 2A[j]$.
- For any j , count how many are $> 2A[j]$ among first $j-1$ numbers.
- Easy to count if first $j-1$ numbers are in a sorted array
- But to maintain sorted array we will need $O(n)$ per insertion.
- What other data structure can be used?

Binary search tree



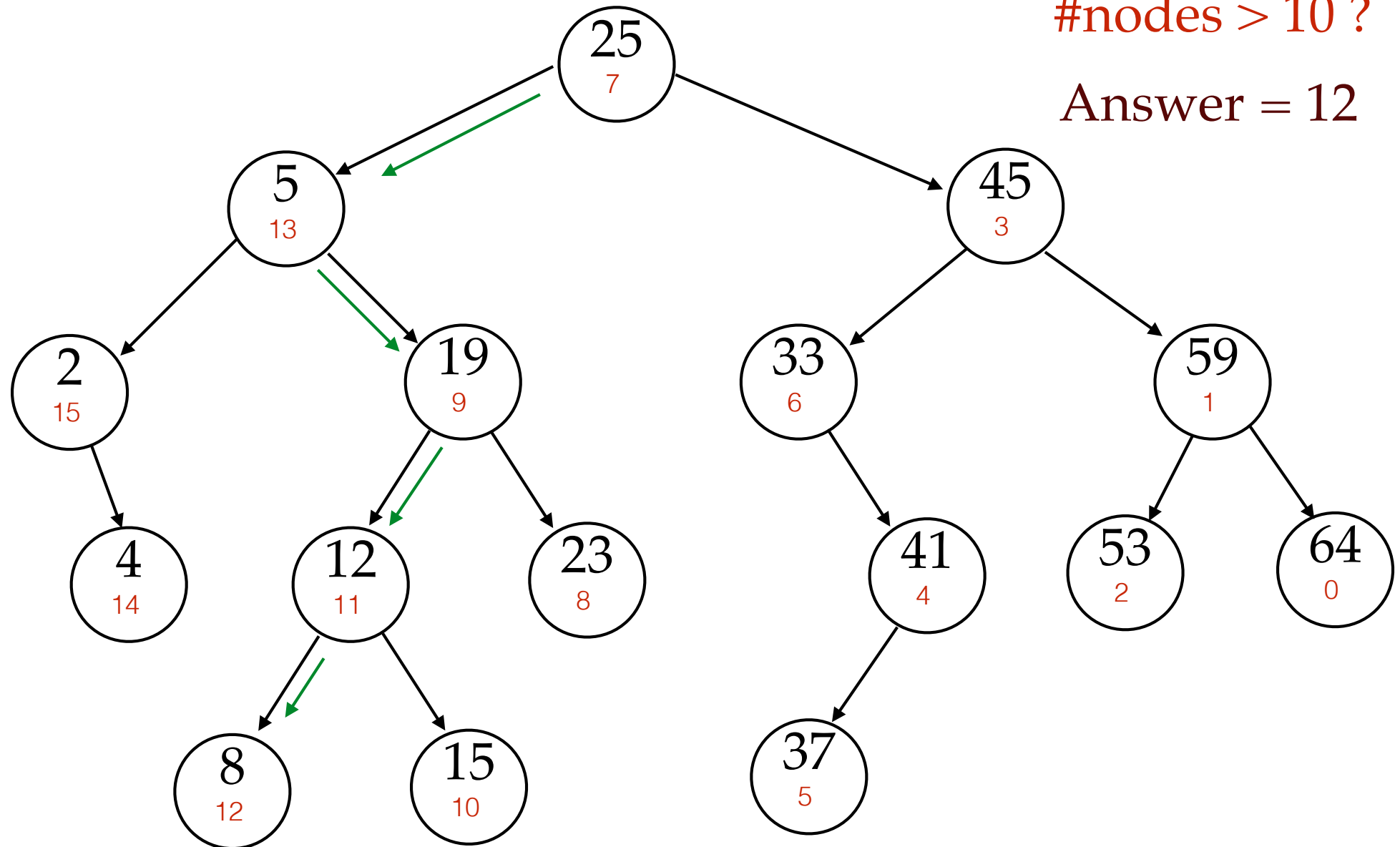
Counting in BST

- Counting number of *nodes* $> x$ seems to take $O(n)$ time in the standard BST.
- What if store for each node, the number of nodes greater than itself ?

Each node with number of nodes greater than
itself

#nodes > 10 ?

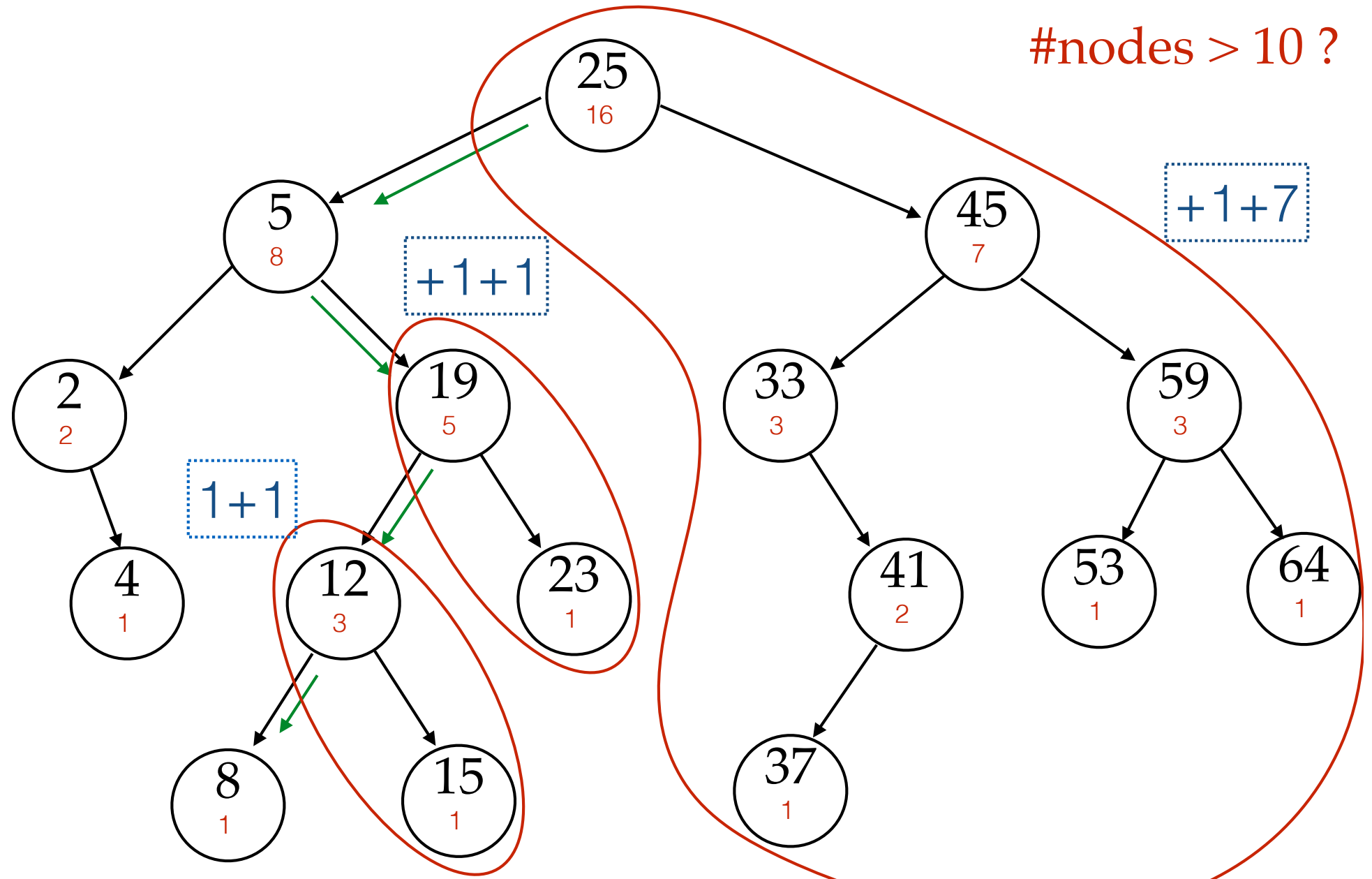
Answer = 12



Counting in BST

- What if store for each node, the number of nodes greater than itself ?
- Too much time to update these numbers on insertion of a new element.
- Better idea?
- Store for each node x , the size of the subtree rooted at x

Each node with subtree size



Counting in BST

- Store for each node x , the size of the subtree rooted at x
- Can we maintain this information on insertion of a new element?
- Need to update the counts for all nodes on the path from the root to the new node.
- $O(\log n)$ for balanced binary tree.
- Can counts be maintained in the re-balance step?
- Can you do range count: $\#nodes$ in range l to r
- Other data structures?

Algorithm: Significant inversion

- Maintain a balanced BST. Initially empty.
For each node x , we need to store size of the subtree rooted at x .
- $no_of_pairs \leftarrow 0$;
- **For** $j \leftarrow 1$ to n
 - Look for the right position p for $2A[j]$ in the BST (no insertion).
 - **For** every node x on the path from p to the root:
 - **If** ($x > 2A[j]$) *then*
Add $(1 + \text{size of the subtree rooted at right child of } x)$ to no_of_pairs
 - **Insert** $A[j]$ in the BST and increase the size count for every node on the path from root to $A[j]$ by 1

Exponentiation

- Given a and n , compute a^n .
- $a \times a \times \cdots \times a$ n times
- $n-1$ multiplications
- $a^{16} = (((a^2)^2)^2)^2$ only 4 multiplications
- $a^{11} = (((a^2)^2)^2 \times a^2 \times a)$ only 5 multiplications
- Repeated squaring technique

Repeated Squaring

- $\text{Exp}(a, n)$:
 - If n is even
 - return ($\text{Exp}(a, n/2)$)²
 - If n is odd
 - return ($\text{Exp}(a, (n-1)/2)$)² $\times a$
 - If n is 1
 - return a

Number of multiplications

$$T(n) \leq T(n/2) + 2$$

$$T(n) \leq 2 \log n$$

Another implementation

- $\text{Exp}(a, n)$:
 - If n is even
 - return ($\text{Exp}(a^2, n/2)$)
 - If n is odd
 - return ($\text{Exp}(a^2, (n-1)/2)$) $\times a$
 - If n is 1
 - return a

Iterative implementation

- Input: a, n
- Initialize
 - $a_power_n \leftarrow 1$; // this will be a^n at the end
 - $a_two_power \leftarrow a$; // this will be a^{2^i} after i iterations
- while ($n > 0$)
 - If (n is odd) then $a_power_n \leftarrow a_two_power \times a_power_n$;
 - $a_two_power \leftarrow a_two_power \times a_two_power$;
 - $n \leftarrow n/2$ //integer part after division by 2 //or right-shift by 1 bit

Repeated squaring

- Does it give the minimum number of multiplications?
- What about a^{15} ?
 - can be done in 5 multiplications
- Given n , what the minimum number of multiplications required for a^n ? No easy answer.
- Can apply repeated squaring for other operations like Matrix powering.
- Apparently proposed by Pingala (200 BC ?).

Fibonacci numbers

- $F(n) = F(n-1) + F(n-2)$
- Used by Pingala to count the number of patterns of short and long vowels.
- Here again we are repeating the same operation n times.
- Can repeated squaring be used?

Exponentiation

- Given a and n , compute a^n .
- $a \times a \times \cdots \times a$ n times
- $n-1$ multiplications
- $a^{16} = (((a^2)^2)^2)^2$ only 4 multiplications
- $a^{11} = (((a^2)^2)^2 \times a^2 \times a)$ only 5 multiplications
- Repeated squaring technique

Repeated Squaring

- $\text{Exp}(a, n)$:
 - If n is even
 - return ($\text{Exp}(a, n/2)$)²
 - If n is odd
 - return ($\text{Exp}(a, (n-1)/2)$)² $\times a$
 - If n is 1
 - return a

Number of multiplications

$$T(n) \leq T(n/2) + 2$$

$$T(n) \leq 2 \log n$$

Another implementation

- $\text{Exp}(a, n)$:
 - If n is even
 - return ($\text{Exp}(a^2, n/2)$)
 - If n is odd
 - return ($\text{Exp}(a^2, (n-1)/2)$) $\times a$
 - If n is 1
 - return a

Iterative implementation

- Input: a, n
- Initialize
 - $a_power_n \leftarrow 1$; // this will be a^n at the end
 - $a_two_power \leftarrow a$; // this will be a^{2^i} after i iterations
- while ($n > 0$)
 - If (n is odd) then $a_power_n \leftarrow a_two_power \times a_power_n$;
 - $a_two_power \leftarrow a_two_power \times a_two_power$;
 - $n \leftarrow n/2$ //integer part after division by 2 //or right-shift by 1 bit

Repeated squaring

- Does it give the minimum number of multiplications?
- What about a^{15} ?
 - can be done in 5 multiplications
- Given n , what the minimum number of multiplications required for a^n ? No easy answer.
- Can apply repeated squaring for other operations like Matrix powering.
- Apparently proposed by Pingala (200 BC ?).

Fibonacci numbers

- $F(n) = F(n-1) + F(n-2)$
- Used by Pingala to count the number of patterns of short and long vowels.
- Here again we are repeating the same operation n times.
- Can repeated squaring be used?
- Can we compute $F(n)$ in $O(\log n)$ arithmetic operations?

Fibonacci numbers

- $F(n) = F(n-1) + F(n-2)$
- if n is even
 - $$\begin{aligned} F(n) &= F(n/2) F(n/2-1) + F(n/2+1)F(n/2) \\ &= F(n/2) (2F(n/2+1) - F(n/2)) \end{aligned}$$
 - $F(n+1) = F(n/2) F(n/2) + F(n/2+1)F(n/2+1)$
- similarly, if n is odd

Fibonacci numbers

$$\begin{pmatrix} F_3 \\ F_2 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

$$\begin{pmatrix} F_5 \\ F_4 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} F_3 \\ F_2 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}^2 \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}^{n/2} \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}$$

Integer Multiplication

- **Addition:** Adding two n bit numbers
School method $O(n)$ time
- $O(n)$ time is necessary to write the output.
- **Multiplication:** multiplying two n bit numbers
School method $O(n^2)$ time
- is $O(n^2)$ time is necessary?

Integer Multiplication

- Kolmogorov (1960) conjectured that $O(n^2)$ time is necessary.
- In a week, Karatsuba found $O(n^{1.59})$ time algorithm.
- Karatsuba quoted in Jeff Erickson's Algorithms
- “After the seminar I told Kolmogorov about the new algorithm and about the disproof of the n^2 conjecture. Kolmogorov was very agitated because this contradicted his very plausible conjecture. At the next meeting of the seminar, Kolmogorov himself told the participants about my method, and at that point the seminar was terminated.”

Special case: Integer Squaring

- If we have a squaring algorithm, does it directly give us a multiplication algorithm?
- Input: n bit integer a
- Break it into two chunks $n-1$ bits and 1 bit
- $a = \varepsilon + 2b$
- $a^2 = \varepsilon^2 + 2b\varepsilon + b^2$
- $T(n) = T(n-1) + O(n) = O(n^2)$

Squaring: divide and conquer

- Input: n bit integer a
- Break it into two chunks: $n/2$ bits and $n/2$ bits
- $a = b + c 2^{n/2}$.
- $a^2 = b^2 + 2 b c 2^{n/2} + c^2 2^n$
- Need to compute the multiplication bc recursively.
- How to compute bc with the square function?
- $2 bc = (b+c)^2 - b^2 - c^2$

Squaring: divide and conquer

- Input: n bit integer a
- Break it into two chunks: $n/2$ bits and $n/2$ bits
- $a^2 = b^2 + (b^2 + c^2 - (b-c)^2) 2^{n/2} + c^2 2^n$
- Squaring an n bit number reduced to
 - squaring **three** $n/2$ bit numbers
 - and few additions and left shifts $O(n)$
- $T(n) = 3 T(n/2) + O(n)$

Running time analysis

- $T(n) = 3 T(n/2) + O(n)$
 - $T(n) \leq c n + 3 T(n/2)$ for some constant c
 - $T(n) \leq c n + 3cn/2 + 3^2 T(n/2^2)$
 - $T(n) \leq c n + 3cn/2 + 3^2 cn/2^2 + 3^3 T(n/2^3)$
 - $T(n) \leq c n + 3cn/2 + 3^2 cn/2^2 + \dots + 3^{k-1} cn/2^{k-1} + 3^k T(n/2^k)$
 - Assuming $n = 2^k$ and $T(1) = 1$
 - $T(n) \leq c n (1 + 3/2 + 3^2/2^2 + \dots + 3^{k-1}/2^{k-1}) + 3^k$
 - $T(n) \leq 2 c n (3^k/2^k - 1) + 3^k \leq (2c+1) 3^k = O(3^k) = O(3^{\log n}) = O(n^{\log 3})$

Karatsuba's multiplication

- Input: n bit integers a and b
- Break integers into two chunks: $n/2$ bits and $n/2$ bits
- $a = a_0 + a_1 2^{n/2}$.
- $b = b_0 + b_1 2^{n/2}$.
- $ab = a_0 b_0 + (a_0 b_1 + a_1 b_0) 2^{n/2} + a_1 b_1 2^n$
- Want to compute the three terms $a_0 b_0$, $(a_0 b_1 + a_1 b_0)$, $a_1 b_1$ with only **three** multiplications and a few additions
- $T(n) = 3 T(n/2) + O(n)$
- $T(n) = O(n^{\log 3}) = O(n^{1.585})$

Toom-Cook multiplication

- Break it into three chunks: $n/3$ bits each
- $a = a_0 + a_1 2^{n/3} + a_2 2^{2n/3}$
- $b = b_0 + b_1 2^{n/3} + b_2 2^{2n/3}$
- $ab = a_0 b_0 + (a_0 b_1 + a_1 b_0) 2^{n/3} + (a_0 b_2 + a_1 b_1 + a_2 b_0) 2^{2n/3}$
 $+ (a_1 b_2 + a_2 b_1) 2^{3n/3} + a_2 b_2 2^{4n/3}$
- In how many multiplications of $n/3$ bit numbers, can you find these five terms?

Toom-Cook multiplication

- $ab = a_0 b_0 + (a_0 b_1 + a_1 b_0) 2^{n/3} + (a_0 b_2 + a_1 b_1 + a_2 b_0) 2^{2n/3} + (a_1 b_2 + a_2 b_1) 2^{3n/3} + a_2 b_2 2^{4n/3}$
- In how many multiplications of $n/3$ bit numbers, can you find these **five** terms?
- If **six** multiplications
 - $T(n) = 6 T(n/3) + O(n) \Rightarrow T(n) = O(n^{(\log 6)/(\log 3)}) = O(n^{1.63})$
- If **five** multiplications
 - $T(n) = 5 T(n/3) + O(n) \Rightarrow T(n) = O(n^{(\log 5)/(\log 3)}) = O(n^{1.46})$

Toom-Cook multiplication

- Homework: find these five terms using **five** multiplications and a few additions
 - $a_0 b_0$
 - $a_0 b_1 + a_1 b_0$
 - $a_0 b_2 + a_1 b_1 + a_2 b_0$
 - $a_1 b_2 + a_2 b_1$
 - $a_2 b_2$

Toom-Cook multiplication

- Homework: find these five terms using **five** square operations and a few additions
 - P^2
 - PQ
 - $2PR + Q^2$
 - QR
 - R^2

Announcements

- Quiz: Wednesday 8:30-9:25 AM
- Venue: LA 001 (odd roll numbers)
LA 002 (even roll numbers).
- Syllabus: Lecture 1-7 (till repeated squaring)
- One handwritten A4 size page (both sides)
- **Make up class:** This Fri, 5:30 pm, LH 302.

Integer multiplication history

- Karatsuba: break integers into two parts: $O(n^{1.585})$
- Toom-Cook: break integers into three parts: $O(n^{1.46})$
- What if break into more parts?
 - can get better and better time complexity by increasing the number of parts
 - for k parts, we will get $O(k^2 n^{\log(2k-1)/\log k})$
 - we can get $O(n^{1+\varepsilon})$ for any constant $\varepsilon > 0$

Integer multiplication history

- for k parts, we will get $O(k^2 n^{\log(2k-1)/\log k})$
- we can get $O(n^{1+\varepsilon})$ for any constant $\varepsilon > 0$
- **Exercise:** how many parts to break into for $O(n^{1.1})$
- Theoretically faster, but not necessarily faster in practice due to large constants.
- For multiplying 64 bit integers, Karatsuba may not be faster than school method. A combination of the two may be faster.

Integer multiplication history

- [1960] Karatsuba $O(n^{1.585})$
- [1963, 1966] Toom-Cook: $O(n^{1.46})$
- [1981] Donald Knuth: $O(n 2^{\sqrt{2 \log n}} \log n)$
- [1971] Schönhage–Strassen: $O(n \log n \log \log n)$
- [2007, 2008] Fürer, De-Kurur-Saha-Saptharishi: $O(n \log n 2^{\log^* n})$
- [2019] Harvey-van der Hoeven: $O(n \log n)$
- **Conjecture:** $O(n \log n)$ can not be improved

Matrix multiplication

- Input: $n \times n$ matrices A and B
- Break matrices into four parts: $n/2 \times n/2$ each

$$A = \begin{pmatrix} A_0 & A_1 \\ A_2 & A_3 \end{pmatrix} \quad B = \begin{pmatrix} B_0 & B_1 \\ B_2 & B_3 \end{pmatrix}$$

$$AB = \begin{pmatrix} A_0B_0 + A_1B_2 & A_0B_1 + A_1B_3 \\ A_2B_0 + A_3B_2 & A_2B_1 + A_3B_3 \end{pmatrix}$$

- Want to compute the four matrices with only **seven** multiplications and a few additions

Polynomial multiplication

- Input: degree d polynomials P and Q
- $P(x) = p_0 + p_1 x + p_2 x^2 + \cdots + p_d x^d.$
- $Q(x) = q_0 + q_1 x + q_2 x^2 + \cdots + q_d x^d.$
- $P(x) Q(x) = p_0 + (p_0 q_1 + p_1 q_0)x + \cdots + p_d q_d x^{2d}.$
- Assume integer multiplication in unit time.
- School multiplication $O(d^2)$ time.
- Can we do better?

Polynomial multiplication

- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $B(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_{d-1} x^{d-1}$
- $A(x)B(x)$
 $= a_0 b_0 + (a_0 b_1 + a_1 b_0) x + (a_0 b_2 + a_1 b_1 + a_2 b_0) x^2$
 $+ \cdots + a_{d-1} b_{d-1} x^{2d-2}$
- $A(x)B(x) = \sum_{j=0}^{2d-2} x^j \left(\sum_{i=0}^j a_i b_{j-i} \right)$

Polynomial multiplication

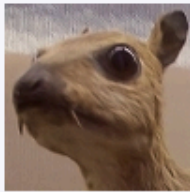
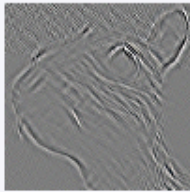
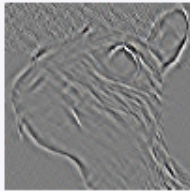
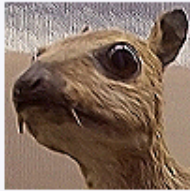
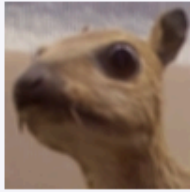
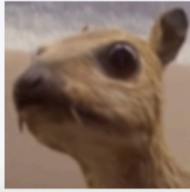
- $A(x)B(x)$
 $= a_0 b_0 + (a_0 b_1 + a_1 b_0) x + (a_0 b_2 + a_1 b_1 + a_2 b_0) x^2$
 $+ \cdots + a_{d-1} b_{d-1} x^{2d-2}$
- Assuming unit cost arithmetic operations, what is the time complexity?
- Naive algorithm $O(d^2)$
- Can we use Karatsuba's idea for polynomial multiplication?

Convolution

- $a = (a_0, a_1, a_2, \dots, a_{m-1}) \in R^m$
- $b = (b_0, b_1, b_2, \dots, b_{n-1}) \in R^n$
- $a * b = (a_0 b_0, (a_0 b_1 + a_1 b_0), (a_0 b_2 + a_1 b_1 + a_2 b_0),$
 $\dots, (\sum_i a_i b_{j-i}), \dots, a_{m-1} b_{n-1}) \in R^{m+n-1}$
- Dot product with a sliding window

Applications of Convolution

- Signal processing
 - Smoothing of noisy data
 - Covid cases per day: take seven day averages
- Image processing: 2D convolution (bivariate polynomial multiplication)
- Convolutional neural networks

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur 3 x 3 (approximation)	$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

- Source: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing))

Applications of Convolution

- Probability distribution of a sum of two random variables

- Dice with

Outcome	1	2	3	4	5	6
Probability	0.2	0.3	0.1	0.1	0.2	0.1

- Sum of two such dice

Outcome	1	2	3	4	5	6	7	...
Probability	0	0.04	0.12					

Puzzle

- Puzzle: secret sharing
- Share secret among n parties
If any k of them come together, the secret can be reconstructed.

Polynomial multiplication / Convolution

- Can we get faster than the naive $O(d^2)$ algorithm?
- Different representations of a polynomial
 - coefficients
 - roots
 - evaluations
- Claim: given d evaluations, there is a unique degree $d-1$ polynomial satisfying those evaluations.

Unique polynomial with d evaluations

- **Claim:** given d evaluations, there is a unique degree $d-1$ polynomial satisfying those evaluations.
- **Proof:**
 - Suppose there are two different degree $d-1$ polynomials $P(x)$ and $Q(x)$ which have the same d evaluations.
 - Define $R(x) = P(x) - Q(x)$.
 - Then $R(x)$ is zero at these d points
 - But, a degree $d-1$ polynomial cannot have d roots.
Contradiction.

Roots of a polynomial

- **Claim:** A (nonzero) degree $d-1$ polynomial can have at most $d-1$ roots
- **Proof:** We prove it by induction.
- Base case: degree 1 polynomial has exactly one root.
- Induction hypothesis: a degree $d-2$ polynomial has at most $d-2$ roots
- Induction step: Let $P(x)$ be a polynomial of degree $d-1$.
 - Let α be a root of $P(x)$. Then $x-\alpha$ divides $P(x)$.
 - Let $P(x) = (x-\alpha) Q(x)$. If $P(\beta) = 0$ for some $\beta \neq \alpha$, then $Q(\beta) = 0$.
 - Hence, all other roots of $P(x)$ are also roots of $Q(x)$.
 - By induction hypothesis, $Q(x)$ has at most $d-2$ roots.
 - Hence, $P(x)$ has at most $d-2+1 = d-1$ roots.

Polynomial representations

- Computations with different representations

	Addition	Multiplication
Coefficients	$O(d^2)$	$O(d^2)$
Roots	?	$O(d)$
Evaluations	$O(d)$	$O(d)$

- But, can we convert between different representations?

Polynomial Representations

- Given d coefficients, how much time needed to compute evaluations of the polynomial at d points?
- Naively, each evaluation will need $O(d)$ multiplications and additions
- Overall time $O(d^2)$
- Can evaluation at one point help with evaluating at another point?

Coefficients to evaluations

- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$

- $A(1) = a_0 + a_1 + a_2 + \cdots + a_{d-1}$

- $A(-1) = a_0 - a_1 + a_2 - \cdots - a_{d-1}$

- Total additions = $2d-2$

- First compute

$$\text{even-sum} = a_0 + a_2 + a_4 + \cdots + a_{d-2}$$

$$\text{odd-sum} = a_1 + a_3 + a_5 + \cdots + a_{d-1}$$

- $A(1) = \text{even-sum} + \text{odd-sum}$

- $A(-1) = \text{even-sum} - \text{odd-sum}$

- Total additions = d

Coefficients to evaluations

- Similar trick with $A(\alpha)$ and $A(-\alpha)$
- $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
- $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{d-2} x^{d/2-1}$
- $A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{d-1} x^{d/2-1}$
- $A(\alpha) = A_{\text{even}}(\alpha^2) + \alpha \cdot A_{\text{odd}}(\alpha^2)$
- $A(-\alpha) = A_{\text{even}}(\alpha^2) - \alpha \cdot A_{\text{odd}}(\alpha^2)$
- **Two** evaluations of degree $d-1$ polynomial
→ **Two** evaluations degree $d/2-1$ polynomials

Coefficients to evaluations

- $A(\alpha) = A_{\text{even}}(\alpha^2) + \alpha \cdot A_{\text{odd}}(\alpha^2)$
- $A(-\alpha) = A_{\text{even}}(\alpha^2) - \alpha \cdot A_{\text{odd}}(\alpha^2)$
- Evaluations of $A(x)$ at $\alpha, -\alpha, \beta, -\beta, \gamma, -\gamma, \dots$
- $\rightarrow$ Evaluations of $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$ at $\alpha^2, \beta^2, \gamma^2, \dots$
- Total work reduced by half
- Can we further apply this trick?
- We will need $\alpha^2 = -\beta^2$?

Roots of unity

- Evaluations of $A(x)$ at $1, -1, i, -i$
- $\rightarrow$ Evaluations of $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$ at $1, i^2 = -1$
- $\rightarrow$ Evaluations of $A_{\text{even-even}}(x)$, $A_{\text{even-odd}}(x)$, $A_{\text{odd-even}}(x)$, $A_{\text{odd-odd}}(x)$ at 1
- To generalize assume $d = 2^t$
- We will start with d -th roots of unity.

Roots of unity

- $a + i b = r e^{i\theta}$
- $r = \sqrt{a^2 + b^2}$ is the absolute value
- $\theta = \tan^{-1}(b/a)$ is the angle with real axis
- $e^{i\pi} = -1$
- Primitive d -th root of unity = $\omega = e^{2\pi i / d}$
 - All d -th roots of unity = $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$

Properties of roots of unity

- Primitive d -th root of unity = $\omega = e^{2\pi i / d}$
 - $\omega^{d/2} = e^{2\pi i / 2} = -1$
 - All d -th roots of unity = $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$
 - All $(d/2)$ -th roots of unity = $1, \omega^2, \omega^4, \omega^6, \dots, \omega^{2d-2}$
 - $1 + \omega + \omega^2 + \omega^3 + \dots + \omega^{d-1} = 0$
 - $1 + \omega^j + \omega^{2j} + \omega^{3j} + \dots + \omega^{(d-1)j} = 0$ for $0 < j < d$

Discrete Fourier Transform

- Evaluate a degree $d-1$ polynomial at d -th roots of unity
- Evaluations of $A(x)$ at
 $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$ (d -th roots)
- Given $a_0, a_1, a_2, \dots, a_{d-1}$
- Output

$$a_0 + a_1 \cdot 1 + a_2 \cdot 1^2 + \dots + a_{d-1} \cdot 1^{d-1}$$

$$a_0 + a_1 \omega + a_2 \omega^2 + \dots + a_{d-1} \omega^{d-1}$$

$$a_0 + a_1 \omega^2 + a_2 \omega^4 + \dots + a_{d-1} \omega^{2d-2}$$

$$a_0 + a_1 \omega^{d-1} + a_2 \omega^{2d-2} + \dots + a_{d-1} \omega^{2d-1}$$

Fourier Transform

- $F(k) = \int_{-\infty}^{\infty} f(x) \sin(2\pi kx) dx$

- $G(k) = \int_{-\infty}^{\infty} f(x) \cos(2\pi kx) dx$

- $G(k) + iF(k) = \int_{-\infty}^{\infty} f(x) e^{2\pi kx} dx$

Fast Fourier Transform

- Evaluations of $A(x)$ at $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$

- Assume d is a power of 2

- $A_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \dots + a_{d-2} x^{d/2-1}$

- $A_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \dots + a_{d-1} x^{d/2-1}$

- $-1 = \omega^{d/2}$

- $-\omega = \omega^{1+d/2}$

.....

- $A(\alpha) = A_{\text{even}}(\alpha^2) + \alpha \cdot A_{\text{odd}}(\alpha^2)$

- $A(-\alpha) = A_{\text{even}}(\alpha^2) - \alpha \cdot A_{\text{odd}}(\alpha^2)$

- $-\omega^{d/2-1} = \omega^{d/2+d/2-1} = \omega^{d-1}$

- Squares of the d -th roots : $1, \omega^2, \omega^4, \dots, \omega^{2(d/2-1)}$ ($d/2$ -th roots)

Fast Fourier Transform

Cooley-Tukey 1965

- Evaluations of $A(x)$
at $1, \omega, \omega^2, \omega^3, \dots, \omega^{d-1}$ (d -th roots)
- Recursively evaluate $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$
at $1, \omega^2, \omega^4, \omega^6, \dots, \omega^{d-2}$ ($d/2$ -th roots)
- For $0 \leq k \leq d/2-1$
 - $A(\omega^k) = A_{\text{even}}(\omega^{2k}) + \omega^k \cdot A_{\text{odd}}(\omega^{2k})$
 - $A(\omega^{k+d/2}) = A_{\text{even}}(\omega^{2k}) - \omega^k \cdot A_{\text{odd}}(\omega^{2k})$

Fast Fourier Transform

- Let $T(d)$ be the time to evaluate a degree $d-1$ polynomial at d -th roots of unity.
- For $0 \leq k \leq d/2-1$
 - $A(\omega^k) = A_{\text{even}}(\omega^{2k}) + \omega^k \cdot A_{\text{odd}}(\omega^{2k})$
 - $A(\omega^{k+d/2}) = A_{\text{even}}(\omega^{2k}) - \omega^k \cdot A_{\text{odd}}(\omega^{2k})$
- $T(d) = 2 T(d/2) + O(d)$
- $T(d) = O(d \log d)$

Polynomial multiplication / convolution

- Coefficients $\rightarrow$ Evaluations (FFT)
 - $O(d \log d)$
- Pointwise multiply evaluations of the two polynomials
 - $O(d)$
- Evaluations $\rightarrow$ Coefficients (inverse FFT)
 - Inverse FFT is just FFT with ω replaced by ω^{-1}
 - $O(d \log d)$

Inverse FFT

- Inverse FFT is just FFT with ω replaced by $\omega^{-1} = \omega^{d-1}$

$$e_0 = a_0 + a_1 1 + a_2 1^2 + \cdots + a_{d-1} 1^{d-1}$$

$$e_1 = a_0 + a_1 \omega + a_2 \omega^2 + \cdots + a_{d-1} \omega^{d-1}$$

.....

$$e_{d-1} = a_0 + a_1 \omega^{d-1} + a_2 \omega^{2d-2} + \cdots + a_{d-1} \omega^{2d-1}$$

$$d a_0 = e_0 + e_1 1 + e_2 1^2 + \cdots + e_{d-1} 1^{d-1}$$

$$d a_1 = e_0 + e_1 \omega^{-1} + e_2 \omega^{-2} + \cdots + e_{d-1} \omega^{-d+1}$$

.....

$$d a_{d-1} = e_0 + e_1 \omega^{-d+1} + e_2 \omega^{-2d+2} + \cdots + e_{d-1} \omega^{-2d+1}$$

FFT Implementation issues

- Can we really do additions / multiplications with roots of unity in constant time?
 - approximations: how many bits sufficient?
 - Requires careful implementation.
 - modular arithmetic: work modulo a large enough prime, such that d -th roots of unity exist.
- Is it fast in practice?
 - Iterative implementation, which is memory efficient
 - FFT convolution faster than direct convolution for $d=128$

Fast integer multiplication

- Split the integers into d parts each having t bits
- $a = a_0 + a_1 2^t + a_2 2^{2t} + \cdots + a_{d-1} 2^{(d-1)t}$
- $b = b_0 + b_1 2^t + b_2 2^{2t} + \cdots + b_{d-1} 2^{(d-1)t}$
- Define
 - $A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{d-1} x^{d-1}$
 - $B(x) = b_0 + b_1 x + b_2 x^2 + \cdots + b_{d-1} x^{d-1}$
- Multiply as polynomials (using FFT) and put $x = 2^t$

Greedy Algorithms and Dynamic Programming.

sep 17, 2020

Till now, we have seen problems that have some immediate polynomial time (eg, $O(n^3)$, $O(n^4)$) algorithms and we saw some tools like divide and conquer to make them more efficient.

for example $O(n^2) \rightarrow O(n \log n)$
 $O(n^3) \rightarrow O(n)$

Now, we will see more examples where the obvious algorithm takes exponential time and the challenge is to first design some polynomial time algorithm.

Towards this, the two paradigms Greedy & DP are often helpful.

They will involve more clever and subtle ideas than what we have seen till now.

We will study Greedy & DP simultaneously.

pick up a problem and see whether Greedy works or not, whether DP works or not.

Greedy Algorithm

→ local optimality

→ near-sighted

→ don't care about long-run

Proof of correctness is important.

because correctness is not obvious in most cases.

we will see some basic format of how to argue correctness.

Dynamic Programming

→ Recursion with a better memory management.

or → a systematic approach to search through all possible solutions.

Problem 1

You are allowed to choose five apples from a basket. You want to maximize the total weight of your apples.

Greedy approach works here.

→ Choose heaviest, 2nd heaviest, ..., 5th heaviest

Problem 2 Total weight of the apples ≤ 2 kg.
Basket has apples of weight

→ 450 gms, 400 gms.

Greedy $4 \times 450 = 1800$ gms.

Alternate $5 \times 400 = 2000$ gms.

Problem 3

Subsequence Problem.

$S_1 = a c b g a b a g b c a$ sequence

$S_2 = c a b g c$ (subsequence)

Que Given two sequences S_1 & S_2 , whether S_2 is a subsequence of S_1

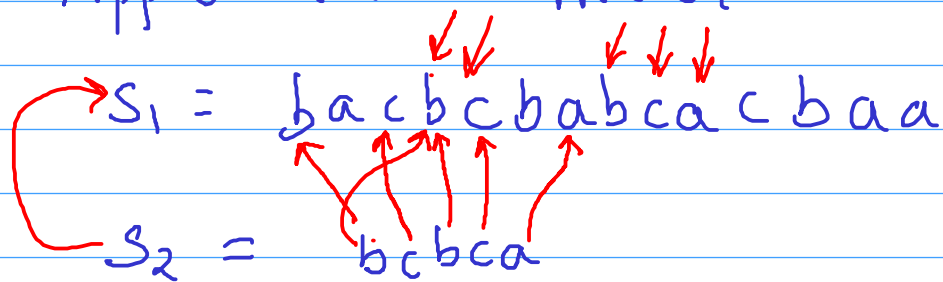
$|S_1| = n$ $|S_2| = p$

Approach 1 → search through all subsequences of length p and check if one of them is equal to S_2 .

no. of such subsequences = $\binom{n}{p}$

Approach 2: match letter by letter

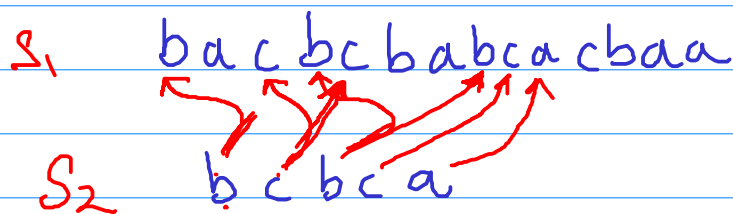
$S_1 =$ b a c b c b a b c a c b a a
 $S_2 =$ b c b c a



Match the current letter in S_2 with its first occurrence you see in S_1 after the previous matching.

Argument for correctness

S_1 b a c b c b a b c a c b a a
 S_2 b c b c a



We want to argue that the greedy approach works. That is, if S_2 is a subsequence of S_1 then we should be able to see it by matching each subsequent letter of S_2 with its first occurrence in S_1 after the previous matching.

To show that this is indeed true, start with an arbitrary subsequence of S_1 that matches with S_2 . Now, move the matching of first letter of S_2 to its first occurrence in S_1 . You still have the valid subsequence. Repeat the argument for every letter of S_2 one by one.

Dynamic Programming.

Simple Example:

$$F_n = F_{n-1} + F_{n-2}$$

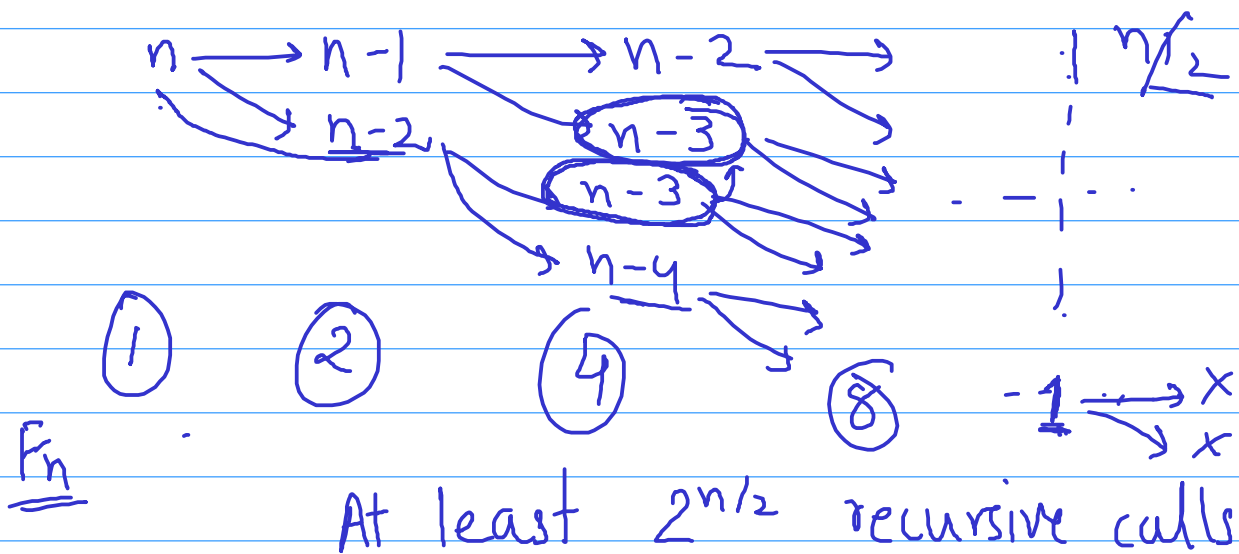
$$F_0 = F_1 = 1$$

Fibonacci (n):

if $n=0$ return 1

if $n=1$ return 1

else return Fibonacci (n-1) + Fibonacci (n-2)



Bad Implementation.

Better Implementation

Array F of length $n+1$.

$$F[0] = 1$$

$$F[1] = 1$$

Bottom-up

for $i=2$ to n

$$F[i] = F[i-1] + F[i-2]$$

end for.

Memoization.

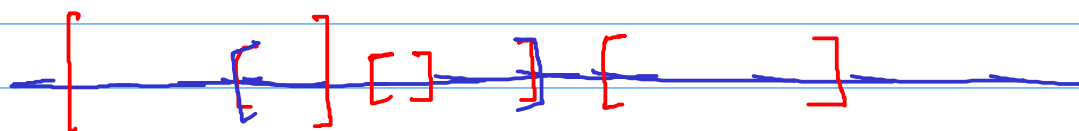
already stored in array.

In dynamic programming, you reduce your problem to one or many subproblems. And if the same subproblem instance is used multiple times then you solve it only once and afterwards, keep using its stored solution. If the total number of **distinct subproblem instances** needed during the course of your algorithm is polynomially bounded then your algorithm is efficient.

For example, in the Fibonacci problem we had n distinct instances of the subproblem $\text{Fibonacci}(n), \text{Fibonacci}(n-1), \dots, \text{Fibonacci}(1)$

Interval Scheduling. [Kleinberg Tardos]
Chapter 4

Resource / server doing computation.



n requests.

$(s_1, t_1) (s_2, t_2), \dots, (s_n, t_n)$

Maximize the number requests you can cater to.
Constraints:

- If you accept a request then you have to allocate the resource for the whole duration of desired interval (No partial allocation)
- The resource can be allocated to only one person at a time.

Greedy Algorithms

Aug 30

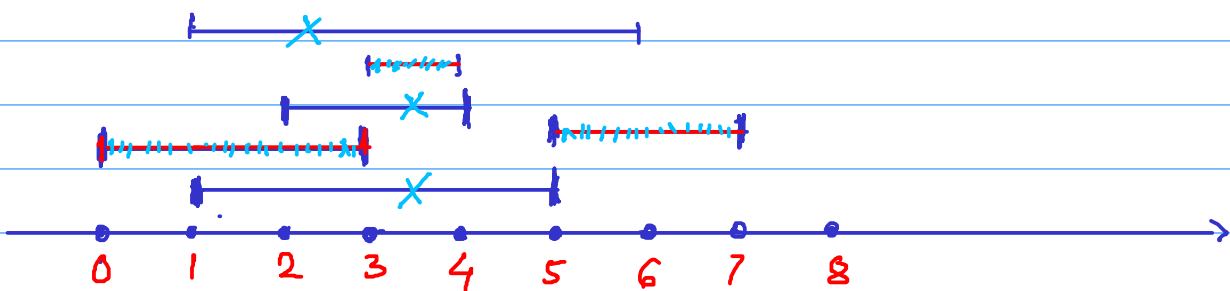
- locally optimal, near-sighted, immediate benefit
- sometimes intuitively clear, sometimes not.
- Correctness of the algorithm is most often not obvious, and thus requires a concrete argument

Classic Example: minimum spanning tree

Interval Scheduling: [Kleinberg Tardos Chapter 4]

Given a set of intervals (on real number line)
find the **largest** subset of **disjoint** intervals.

Example: → (1, 5), (0, 3), (2, 4), (1, 6), (3, 4), (5, 7)

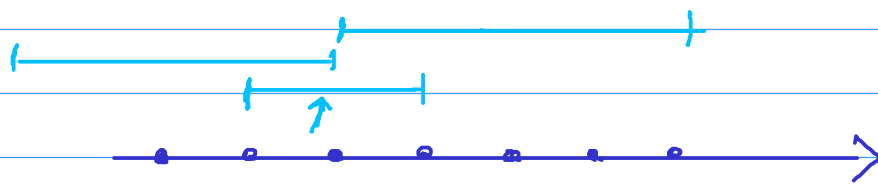


Application: resource allocation requests for fixed intervals of time

- resource can be allocated to only one request at a time
- no partial allocation
- cater to maximum number of requests.

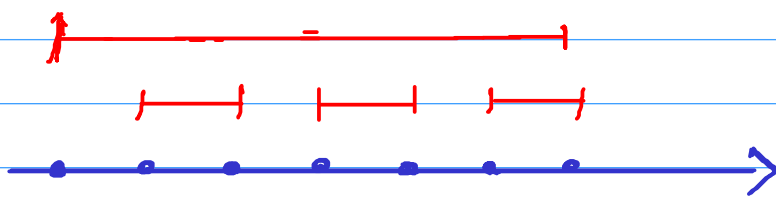
Searching through all possible solutions $\approx 2^n$ solutions

Greedy Strategy 1: keep choosing the smallest interval
(and removing those which intersect with chosen ones)



Greedy 1
Optimal 2.

Greedy Strategy 2: earliest starting time



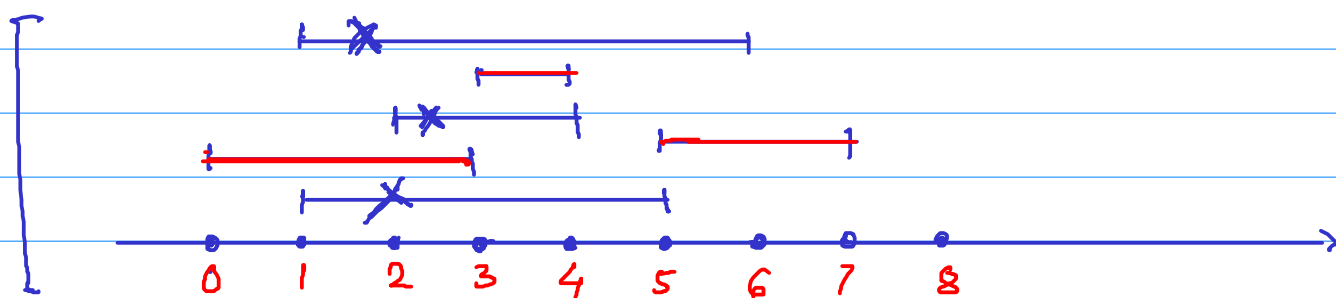
Greedy 1
Optimal 3

Greedy Strategy 3: keep choosing the interval with smallest no. of overlaps.

HW



Greedy Strategy 4: keep choosing the interval with earliest ending time

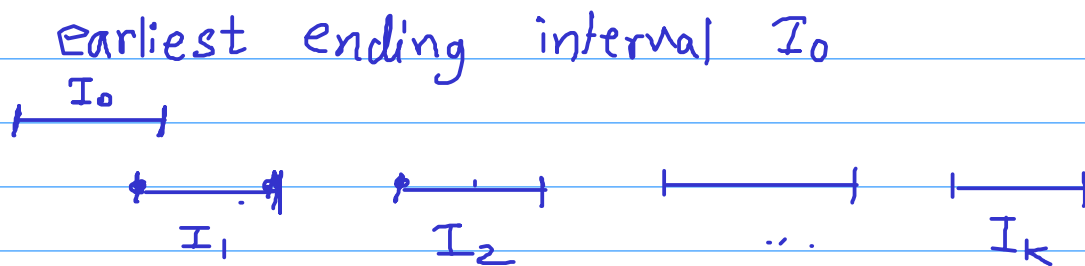


Intuitively, you are maximizing the remaining time, thus maximizing the possible number of requests catered afterwards.

Claim: Greedy strategy 4 always gives an optimal solution

General Framework of argument:

- ① show that there exists an optimal solution that agrees with the first greedy step.
 - ② The rest of the argument will work inductively.
- consider an optimal solution $I_1, I_2, I_3, \dots, I_k$.



swap I_0 with I_1 .

Consider a new solution $I_0, I_2, I_3, \dots, I_k$

Claim It is valid solution.

$$\text{endtime}(I_0) < \text{endtime}(I_1) \leq \text{starttime}(I_2)$$

$\Rightarrow I_0$ does not overlap with $I_2, I_3, \dots, I_k$

$I_0, I_2, I_3, \dots, I_k$ is an optimal solution agreeing with first greedy step.

Inductive proof based on the number of intervals.

Inductive Hypothesis: Greedy algo works for any instance with up to $n-1$ intervals

Inductive Step: It works for all instances with n intervals

Base case: $n=1$. Obvious.

Input: $\mathcal{I} = \{I_1, I_2, I_3, \dots, I_n\}$

for convenience

assume $\text{endtime}(I_1) \leq \text{endtime}(I_2) \leq \dots$

Algorithm chooses I_1 .

then removes all intervals overlapping with I_1

$\rightarrow \mathcal{I}' = \mathcal{I} - \{\text{intervals overlapping with } I_1\}$

Recursively applying the same algorithm on $\mathcal{I}'$

By the inductive hypothesis, we know that the algorithm gives optimal solution for $\mathcal{I}'$

That is,

$$\boxed{\text{Algo}(\mathcal{I}') = \text{OPT}(\mathcal{I}')}_{--}$$

Output: $I_1 + \text{Algo}(\mathcal{I}') = I_1 + \text{OPT}(\mathcal{I}')$

Claim: $(I_1 + \text{any optimal solution for } \mathcal{I}')$
is an optimal solution for $\mathcal{I}$.

Claim $I_1 + \text{OPT}(\mathcal{I}')$ is an optimal solution for $\mathcal{I}$.

Proof: Recall that we showed that there exist an optimal solution for $\mathcal{I}$ that contains I_1 .

Let it be $I_1, J_1, J_2, \dots, J_k$

Clearly $J_1, J_2, \dots, J_k$ are disjoint from I_1 .

Hence, $\{J_1, J_2, \dots, J_k\}$ is a valid solution for $\mathcal{I}'$.

$$\rightarrow |\text{OPT}(\mathcal{I}')| \geq |\{J_1, J_2, \dots, J_k\}|$$

$$\Rightarrow |I_1 + \text{OPT}(\mathcal{I}')| \geq |\{I_1, J_1, J_2, \dots, J_k\}|$$

$\downarrow$
 optimal for $\mathcal{I}$.

$$= |\text{OPT}(\mathcal{I})|$$

Pseudocode

Input: $(s_1, f_1), (s_2, f_2), \dots, (s_n, f_n)$

- Sort according to f_j .

$\rightarrow f = -\infty$ (finish time of latest interval selected so far)

```

for (i = 1 to n)
  → if ( $s_i \geq f$ ) then
    select  $(s_i, f_i)$ 
     $f \leftarrow f_i$ 
  
```

HW Assignments

deadlines $d_1, d_2, \dots, d_n$

time required $t_1, t_2, \dots, t_n$

lateness of i -th assignment = $(t_i - d_i)$ if $t_i > d_i$
 $\uparrow$ actual submission

Minimize maximum lateness over all assignments.

→ smallest length first?

→ earliest deadline first?

→ minimum $d_i - l_j$ first?

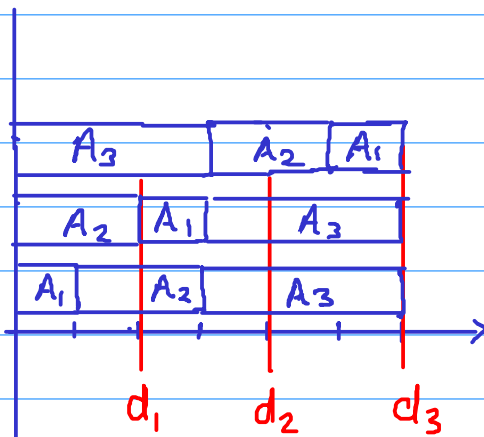
Example :

	A_1	A_2	A_3
length	1	2	3
deadlines	2	4	6

	A_1	A_2	A_3	
Lateness	4	1	0	← $A_3 A_2 A_1$
Latness	1	0	0	← $A_2 A_1 A_3$
Latness	0	0	0	← $A_1 A_2 A_3$

Red shows maximum lateness.

Best order is $A_1A_2A_3$.

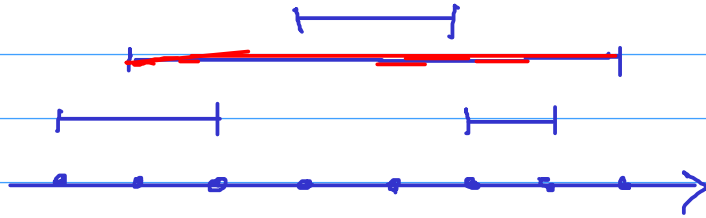


② Interval scheduling: cater to all requests with using minimum number of servers.
(one server can cater to one request at a time)

Equivalently, finding the minimum number of platforms required for a set of trains stopping at a station.

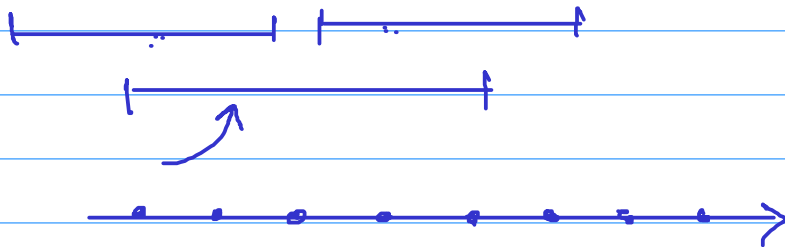
Interval Scheduling : another variant

- select a set of disjoint intervals to maximize the total length of selected intervals.

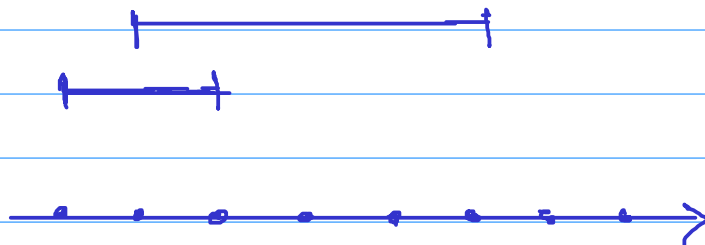


For example, there might be a profit proportional to the duration of use.

Greedy Strategy 1 keep picking the longest length intervals



Greedy Strategy 2 : earliest starting time

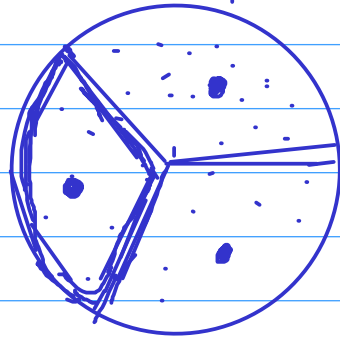


Dynamic Programming:

Recursion with better memory management.

General Idea: try to categorize the possible solutions into different types.

For each type, find the optimal solution via recursion.
Then compare the various types of optimal solutions with each other.



Interval Scheduling with maximum total length.

$$\mathcal{I} = \{I_1, I_2, I_3, \dots, I_n\}$$

Two kinds of solutions

→ contain I_1

→ don't contain I_1

Can we find optimal from both the kinds recursively?

$$\mathcal{I}' = \mathcal{I} - \underline{\text{Overlap}(I_1)}$$

$$\underline{\text{OPT}(\mathcal{I}') + I_1}$$

$$\mathcal{I}'' = \mathcal{I} - I_1 \quad \underline{\text{OPT}(\mathcal{I}'')}$$

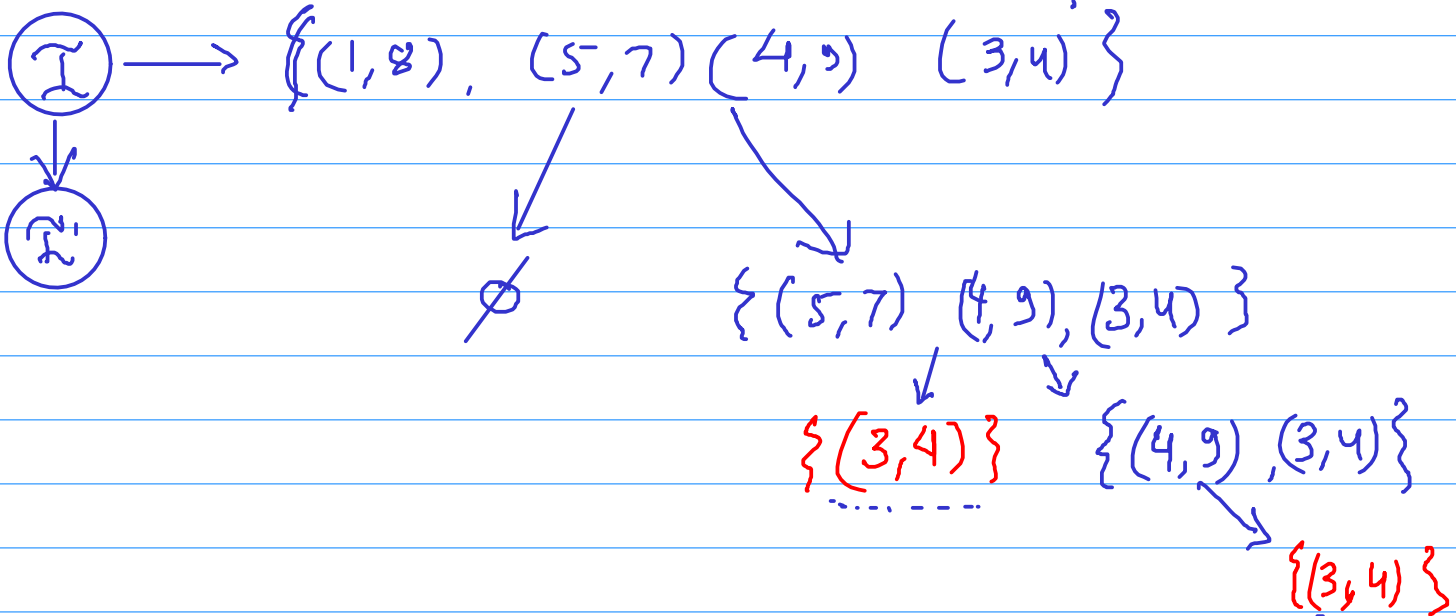
$$\text{OPT}(\mathcal{I}) = \max \{ \underline{I_1} + \underline{\text{OPT}(\mathcal{I}')} , \underline{\text{OPT}(\mathcal{I}'')} \}$$

Example $\mathcal{I} = \{ (0,5) (1,8), (5,7) (4,9), (3,4) \}$

$$I_1 = (0,5)$$

$$\mathcal{I}' = \{ (5,7) \}$$

$$\mathcal{I}'' = \{ (1,8), (5,7), (4,9), (3,4) \}$$



Nothing clever here. We are simply trying to go over all possible solutions recursively.

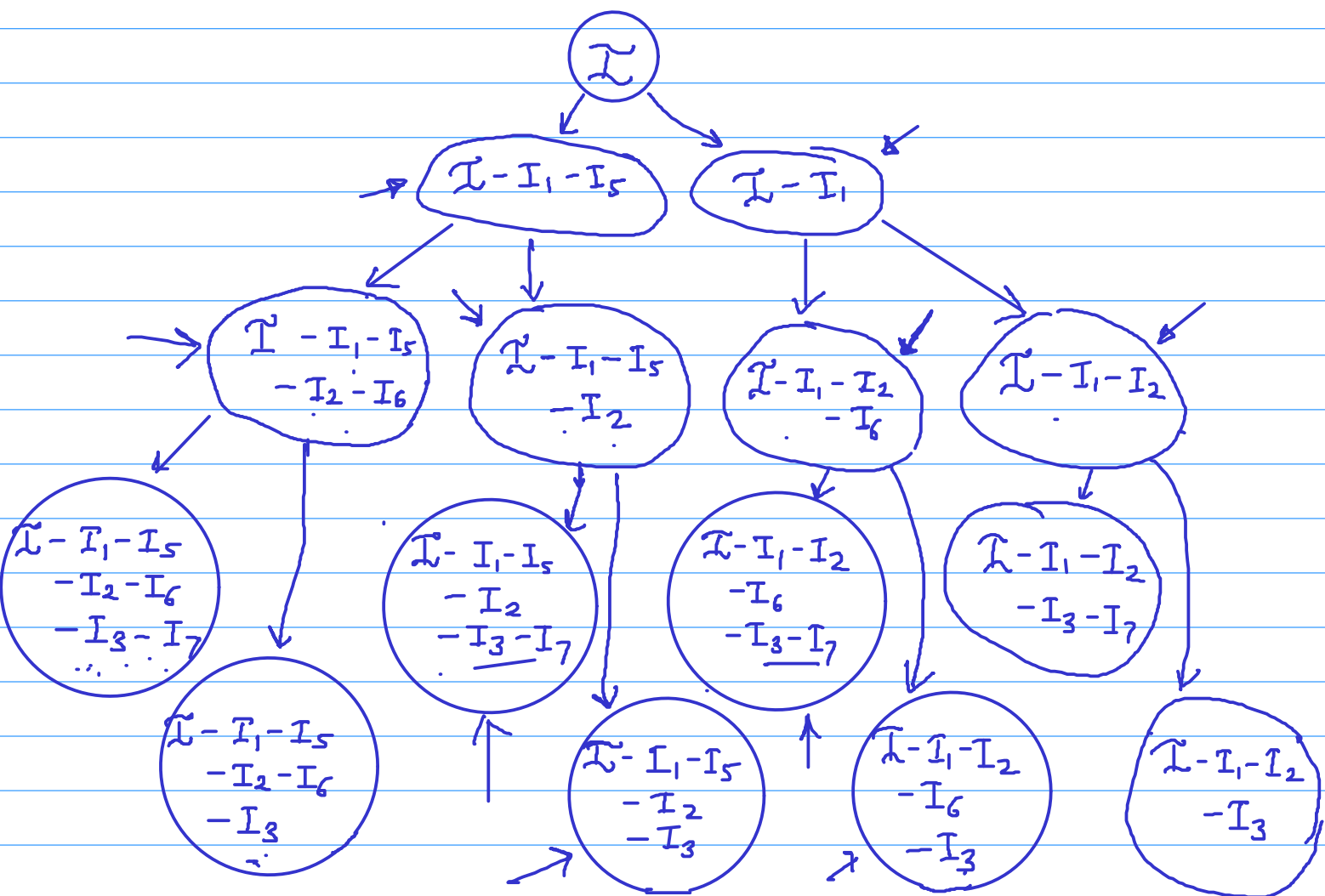
No. of recursive calls seems to be growing exponentially. because each call makes two new recursive calls.

Efficient implementation possible if no. of total distinct recursive calls is small.

$$\mathcal{I} = \{ \underset{I_1}{(1,3)}, \underset{I_2}{(11,13)}, \underset{I_3}{(21,23)}, \underset{I_4}{(31,33)}, \underset{I_5}{(2,4)}, \underset{I_6}{(12,14)}, \underset{I_7}{(22,24)}, \underset{I_8}{(32,34)} \}$$

HW work out the recursion tree and Sep 2
figure out whether the number of distinct
recursive calls is growing exponentially or polynomially

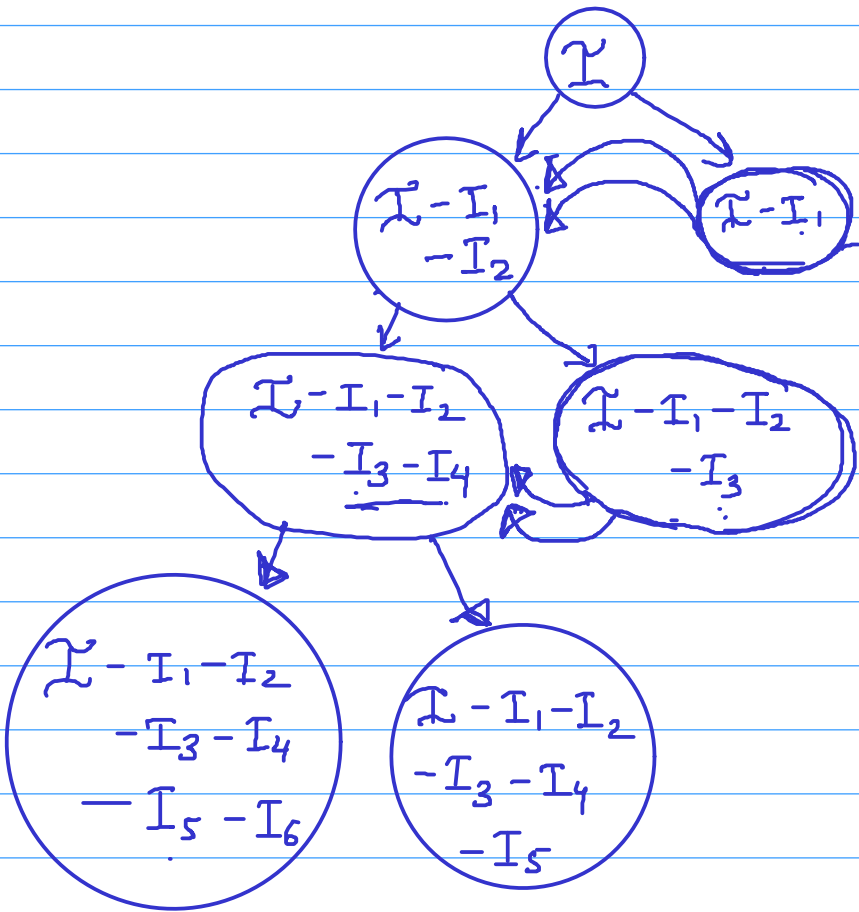
$$\mathcal{I} = \left\{ \begin{array}{cccc} (1, 3) & (11, 13) & (21, 23) & (31, 33) \\ I_1 & I_2 & I_3 & I_4 \\ (2, 4) & (12, 14) & (22, 24) & (32, 34) \\ I_5 & I_6 & I_7 & I_8 \end{array} \right\}$$



If we have $2n$ intervals, we will have

at least 2^n distinct recursive calls or subproblems

$$\mathcal{I} = \left\{ \begin{array}{cccccc} (1,3), (2,4), & (11,13), (12,14), & (21,23), (22,24), \\ I_1 & I_2 & I_3 & I_4 & I_5 & I_6 \\ \\ (31,33), (32,34) \\ I_7 & I_8 \end{array} \right\}$$



Only $2n$ distinct recursive calls or subproblems.

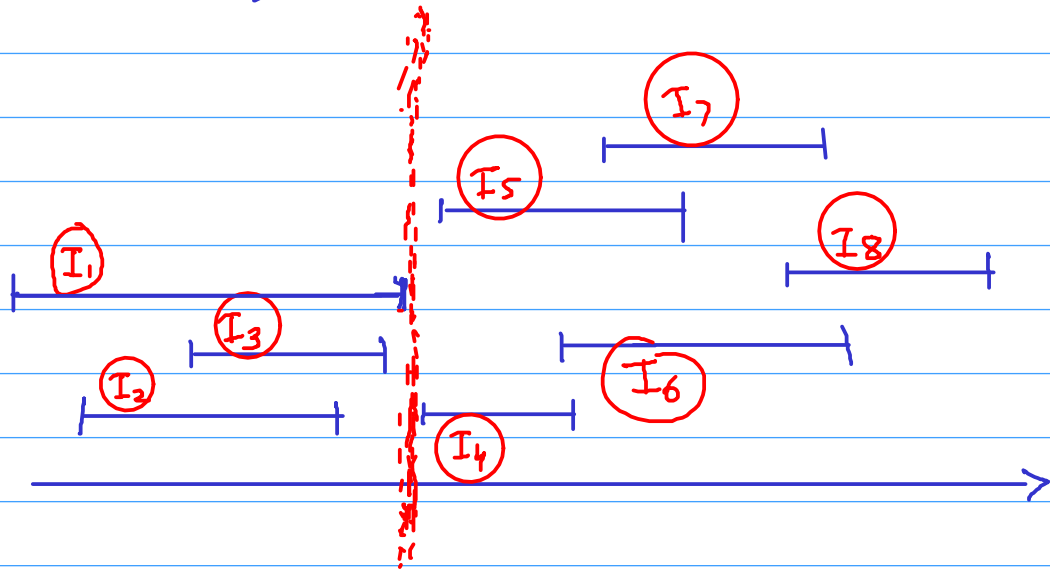
It seems if the intervals are arranged in a particular order, the number of distinct subproblems will be small.

Possible orders:

- ① starting time
- ② ending time

Sort the intervals in increasing order of starting time

$I_1, I_2, I_3, \dots, I_n$



Claim: The input set of intervals for any recursive call will look like
 $\rightarrow \{I_j, I_{j+1}, I_{j+2}, \dots, I_n\}$

(as opposed to an arbitrary subset of intervals)

This is because when we remove intervals overlapping with I_k , the remaining set is simply

all intervals with starting time $>$ end-time(I_k).

No. of distinct subproblems $\leq n$.

Recursive Implementation

$\text{Opt} \leftarrow$ array with all zeros.

$\text{Opt}[n] \leftarrow \text{length}(I_n)$

Output $\text{ALG}(1)$;

$\text{ALG}(j)$: // $\text{ALG}(j)$ computes optimal solution for $\rightarrow \{I_j, I_{j+1}, \dots, I_n\}$

if $\text{Opt}[j] > 0$ return $\text{Opt}[j]$

// means already solved and stored.

else

$\text{Opt}[j] \leftarrow \max \left\{ \begin{array}{l} \text{ALG}(j+1) \\ \text{length}(I_j) + \text{ALG}(P(j)) \end{array} \right.$

return $\text{Opt}[j]$

least index k
such that

$\text{start}(I_k) > \text{end}(I_j)$

Iterative Implementation:

$\text{Opt} \leftarrow$ array with all zeros.

$\text{Opt}[n] \leftarrow \text{length}(I_n)$

for ($j = n-1$ to 1)

$\text{Opt}[j] \leftarrow \max \left\{ \begin{array}{l} \text{Opt}[j+1] \\ \text{length}(I_j) + \text{Opt}[P(j)] \end{array} \right.$

HW

Add code to compute the optimal set of intervals.

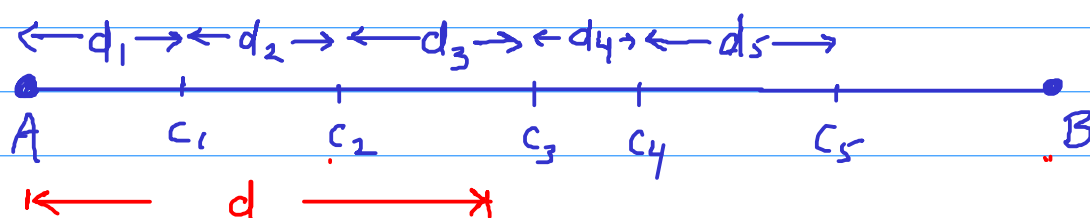
Conclusion: Order of processing the input is important.

Intervals: Order of starting time / ending time

Sequences: left to right

Try to ensure that no. of distinct subproblems is small.

HW 1



Maximum travel in a day - d .

Night stay prices - $P_1, P_2, \dots$

Minimize total stay cost during the journey.

$O(n)$

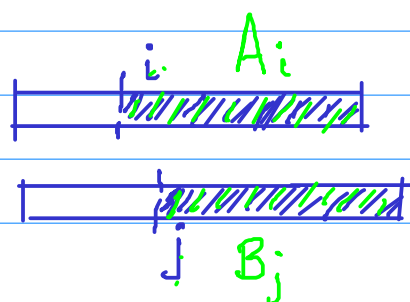
no. of distinct subproblems = n .

HW 2

$\rightarrow A = \overset{2}{b} \overset{5}{a} \overset{3}{c} \overset{1}{b} \overset{2}{c} \overset{5}{b} \overset{3}{a} \overset{1}{b} \overset{3}{a} \overset{1}{c} \overset{2}{b} \overset{4}{a} \overset{1}{a}$

$\rightarrow B = \overset{1}{b} \overset{2}{c} \overset{3}{b} \overset{4}{c} \overset{5}{a}$

Match B inside A with minimum cost



no. of distinct subproblems $O(mn)$

Subset Sum problem

Given set of integers $A = \{a_1, a_2, a_3, \dots, a_n\}$

is there a subset with sum zero?

Example: ↓ ↓ ↓ ↓ ↓

2, -5, 1, 7, -4, -6

$$\{2, 7, -5, -4\}$$

Solutions $\begin{cases} \rightarrow \text{containing an } \underline{\hspace{1cm}} \\ \rightarrow \text{not containing an } \end{cases}$

is there a subset of $\{2, -5, 1, 7, -4\}$ with sum zero?

is there a subset of $\{2, -5, 1, 7, -4\}$
with sum six?

Subproblem $A_j = \{a_1, \dots, a_j\}$, number N

is there a subset of A_j with sum N .

No. of distinct subproblems = $n \times \sum_i |a_i|$

ALG (j, N):

→ $ALG(\underbrace{j-1}, \underbrace{N})$ OR $ALG(\underbrace{j-1}, \underbrace{N-a_j})$

Pseudo polynomial time.

A polynomial time algorithm is supposed to take time $\text{poly}(n, \text{no. of bits in } a_1, a_2, \dots)$.

Subset Sum Pseudocode

$S \leftarrow$ Boolean two-dim array of size $n \times (\sum |a_i| + 1)$

$S[j, k]$ will denote whether there is a subset of first j numbers with sum = k .

say, range of j $1 \leq j \leq n$.

and range of k $\text{neg} \leq k \leq \text{pos}$

$\downarrow$ $\searrow$

sum of all sum of all

negative numbers positive numbers

Initialization:

$$S[1, k] \leftarrow \begin{cases} \text{True} & \text{for } k = a_1 \\ \text{False} & \text{for all other value of } k. \end{cases}$$

for (j = 2 to n)

for (neg ≤ k ≤ pos)

$$S[j, k] \leftarrow S[j-1, k] \text{ OR } S[j-1, k-a_j]$$

To construct the desired set

Assume $S[n, 0]$ is true.

sum $\leftarrow$ 0

```
for ( j= n to 1)
```

if $s[j-1, \text{sum}] = \text{true}$, don't take a_j

elseif $S[j-1, \text{sum}-a_j] = \text{true}$, take a_j and
 $\text{sum} \leftarrow \text{sum} - a_j$

HW Knapsack problem

n objects weights $w_1, w_2, \dots, w_n$
 values $v_1, v_2, \dots, v_n$

Select a subset S s.t.

$$\sum_{i \in S} w_i \leq W$$

and the total value $\sum_{i \in S} v_i$ is maximized.

Sep 6

Balanced margins

average slack $14/3 \approx 4.66$

← $W = 68$ → slack

→ Suppose there ten tourist guides, of which, six can speak French and six can speak German (two can speak both French and German). Everyone comes with their own charges. We want to select five. 0 ←
8 ←
6 ←

→ Suppose there ten tourist guides, of which, six can speak French and six can speak German (two can speak both French and German). Everyone comes with their own charges. We want to select five. 4 ←
4 ←
6 ←

We are given a sequence of words of lengths 21
21
20
21
→ $w_1, w_2, w_3 \dots w_n$

Each line can have at most W characters.

w_i w_{i+1} ... w_j

If a line has from the i -th to j -th word then its slack is defined to be

$$s = W - \left[(w_i + 1) + (w_{i+1} + 1) + \dots + (w_{j-1} + 1) + w_j \right]$$

$l_1, l_2, \dots, l_k$

$$\text{Variance} = \underbrace{\text{avg}(l_1^2, l_2^2, \dots, l_k^2)}_{\text{fixed.}} - (\underbrace{\text{avg}(l_1, l_2, \dots, l_k)}_{\text{fixed.}})^2$$

Arrange the words in lines to minimize the **sum of squares of the slacks** of all lines.

Puzzle find integers a, b, c such that

$$a + b + c = 14$$

and $a^2 + b^2 + c^2$ is minimized.

Answer:

What do you observe?

Balanced Margins

Idea 1: fit as many words as you can

→ can be very unbalanced

Greedy Idea:

compute the average slack per line. Go line by line and try to keep the slack for each line as close as possible to the average slack.

doesn't give an optimal solution.

is

$\overline{5}$	5	2
$\overline{1}$	1	4
$\overline{5}$	5	5

51, 45.

$\frac{11}{3}$
 $= 3.66$

Dynamic Programming.

- Try to categorize the set of all possible solutions

Each solution is a partition of words into k lines.

$$n = n_1 + n_2 + n_3 + \dots + n_k$$

Categories:

$n_k = 1$	last line has 1 word.
$n_k = 2$	last line has 2 words.
$\vdots$	
$n_k = n-1$	last line has $n-1$ words.

Assuming last line has words $w_{p+1}, \dots, w_n$, (i.e. $n-p$ words)
can you compute the optimal solution via a subproblem?

$$\text{OPT}(w_1, w_2, \dots, w_p) + \text{slack}(w_{p+1}, w_{p+2}, \dots, w_n)^2$$

$$\text{OPT}(n) = \min \left\{ \begin{array}{l} \text{OPT}(n-1) + [W - w_n]^2 \\ \text{OPT}(n-2) + [W - w_n - w_{n-1} - 1]^2 \\ \text{OPT}(n-3) + [W - w_n - w_{n-1} - w_{n-2} - 2]^2 \\ \vdots \end{array} \right.$$

Running time $O(n^2)$

Pseudocode for computing the optimal value and the optimal solution.

$S \leftarrow$ array of length $n+1$

// $S[j]$ denotes the minimum sum of squares of slacks for first j words, for $1 \leq j \leq n$

$N \leftarrow$ array of length $n+1$

// $N[j]$ denotes the index of the first word in the last line in the optimal arrangement of first j words.

$s[0] \leftarrow 0$; $S[j] \leftarrow \infty$ for $j > 0$; $N[0] \leftarrow 0$

for ($j = 1$ to n)

for ($r = j$ to 1)

slack $\leftarrow W - (w_r + w_{r+1} + \dots + w_j + j - r)$

if (slack ≥ 0 and $S[j] > S[r-1] + (\text{slack})^2$)

$S[j] \leftarrow S[r-1] + (\text{slack})^2$

$N[j] \leftarrow r$

Optimal Arrangement:
Run Arrange(n).

Arrange(j) {

Arrange($N(j) - 1$)

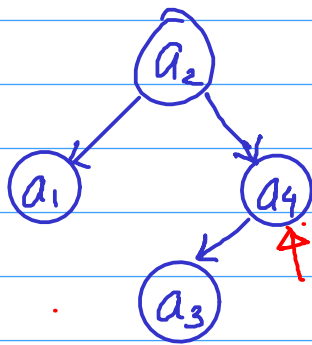
print words from $N(j)$ to j
in a new line.

}

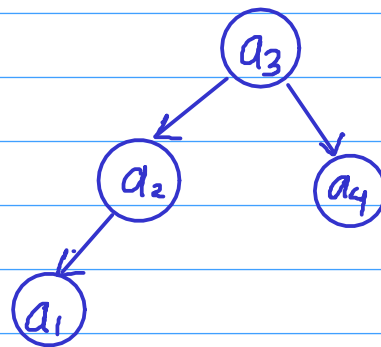
Optimal Binary Search Tree

Sep 7

$$a_1 \leq a_2 \leq a_3 \leq a_4$$



$$\begin{array}{r}
 7 \times 2 \\
 5 \times 1 \\
 4 \times 3 \\
 6 \times 2 \\
 \hline
 43
 \end{array}$$



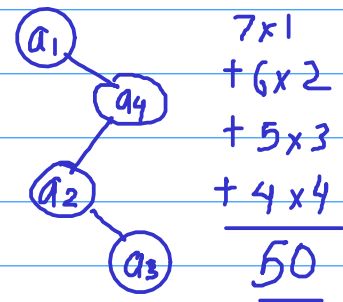
$$\begin{array}{r}
 7 \times 3 \\
 5 \times 2 \\
 4 \times 1 \\
 6 \times 2 \\
 \hline
 \end{array}$$

suppose we know how frequent are the search queries for each of the elements.

Say

frequencies

a_1	$\rightarrow$	7
a_2	$\rightarrow$	5
a_3	$\rightarrow$	4
a_4	$\rightarrow$	6



$$\begin{array}{r}
 7 \times 1 \\
 + 6 \times 2 \\
 + 5 \times 3 \\
 + 4 \times 4 \\
 \hline
 50
 \end{array}$$

Total search cost

First binary tree

43

Second binary tree

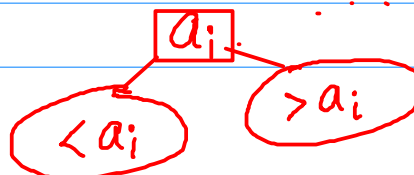
47

Compute the binary search tree with minimum total search cost $\sum_i f_i h_i$

where $\underline{f_i} \rightarrow$ frequency (a_i) $\underline{h_i} \rightarrow$ depth (a_i)

Greedy Idea 1:

maximum frequency element $\rightarrow$ root

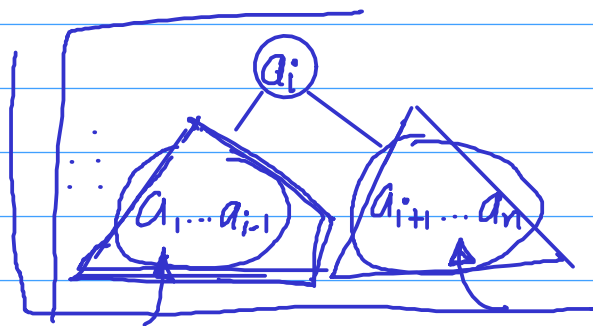


Dynamic Programming

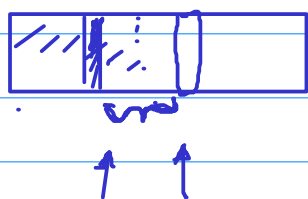
$$a_1 < a_2 < a_3 \dots < a_n$$

Possible solutions : every valid binary search tree.

Categories :
 Root $\leftarrow a_1$
 Root $\leftarrow a_2$
 $\vdots$
 Root $\leftarrow a_n$



$$\text{Tree}(a_1, \dots, a_n) = \min_{1 \leq i \leq n}$$



$$\text{Tree}(a_1, \dots, a_{i-1}) + \text{Tree}(a_{i+1}, \dots, a_n) + \sum_{j=1}^n f_j \cdot 1$$

How many distinct subproblems = $\binom{n}{2}$

$a_1 a_2 a_3 a_4 a_5$

$a_1 a_2 a_3$

a_5

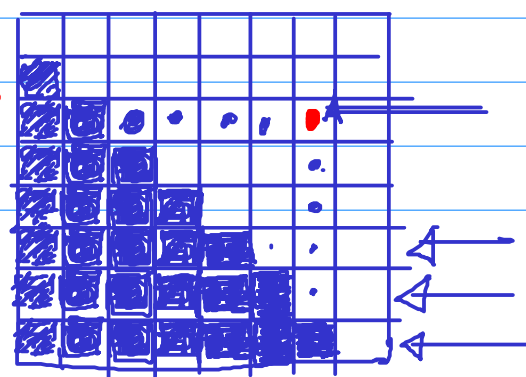
$a_2 a_3$

$a_i a_{i+1} a_j$

$$\text{Tree}(i, j) = \min_{\substack{k \\ i \leq k \leq j}} \{ \text{Tree}(i, k-1) + \text{Tree}(k+1, j) \} + \sum_{k=i}^j f_k$$

$j > i$

$i \downarrow$



$OPT \leftarrow 2 \text{ dim array.}$

$O(n^3)$

$OPT[i, i] \leftarrow f_i \quad \text{for each } i$

for ($i = n-1$ to 1)

for ($j = i+1$ to n)

$$OPT[i, j] = \sum_{k=i}^j f_k + \min_{\substack{k: \\ i \leq k \leq j}} \{OPT[i, k-1] + OPT[k+1, j]\}$$

↑
Optimal cost
for $(a_i a_{i+1} \dots a_j)$

Add/modify code to compute the optimal solution.

Sequence Alignment

Computational biology, Spell checking

Similarity between two strings

Needleman and Wunsch defined a notion of similarity

Example

$$\begin{array}{c} \downarrow \\ \text{UGCTGACU} \\ \rightarrow \text{GAATGCA} \end{array}$$

Given

Gap Penalty δ

Mismatch cost (for each pair) α_{xy}

$$\begin{array}{ccccccc} \text{U} & \text{G} & \text{C} & \text{---} & \text{TG} & \text{A} & \text{C} & \text{U} \\ \uparrow & \uparrow & \uparrow & & \uparrow & & \uparrow & \uparrow \\ \text{---} & \text{G} & \text{A} & \text{A} & \text{TG} & \text{---} & \text{C} & \text{A} \end{array}$$

$$\begin{array}{l} \delta \\ \delta \\ \delta \\ \delta \\ \delta \\ \delta \\ \delta \\ \delta \end{array}$$

$$\begin{array}{l} \alpha_{CA} \\ \alpha_{UA} \end{array}$$

$$\begin{array}{l} 3\delta \\ + \alpha_{CA} \\ + \alpha_{UA} \end{array}$$

$$\begin{array}{c} \text{UGC---TG---ACU} \\ \text{---GAATGCA---} \\ 5\delta + \alpha_{CA} \end{array}$$

Total cost = sum of the gap and mismatch costs.

Find an alignment of the two strings with minimum total cost.

Input: $x_1, x_2, \dots, x_m$ and $y_1, y_2, \dots, y_n$
 $\delta, \{\alpha_{x_i y_j}\}$

Categories of solutions.

①

$$\begin{array}{c} \text{---} \\ \text{---} \end{array}$$

②

$$\begin{array}{c} \text{---} \\ \text{---} \end{array}$$

③

$$\begin{array}{c} \text{---} \\ \text{---} \end{array}$$

$$OPT(m, n) = \min \begin{cases} \alpha_{x_m y_n} + OPT(m-1, n-1) \\ \delta + OPT(m-1, n) \\ \delta + OPT(m, n-1) \end{cases}$$

$x_1 \dots x_{m-1}$
 $\nearrow y_1 \dots y_{n-1}$

$OPT(i, j)$ no. of distinct subproblems $m \times n$

Implementation

$A \leftarrow$ 2D array $m \times n$

// $A[i, j]$ denotes the minimum cost of alignment for $x_1 x_2 \dots x_i$ and $y_1 y_2 \dots y_j$

$A[i, 0] \leftarrow \delta i$ for each i

$A[0, j] \leftarrow \delta j$ for each j

for ($i = 1$ to n)

for ($j = 1$ to n)

$$A[i, j] = \min \left\{ \begin{aligned} &\alpha_{x_i y_j} + A[i-1, j-1], \\ &\delta + A[i-1, j], \\ &\delta + A[i, j-1] \end{aligned} \right\}$$

HW Space $O(mn)$

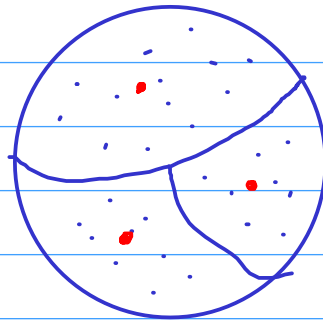
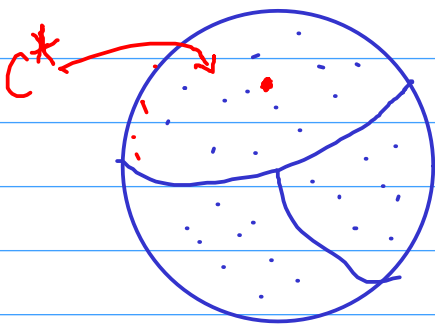
Can you get space $O(m+n)$ and time $O(mn)$

Divide and Conquer

Sep 20

Summarizing Greedy and Dynamic Programming

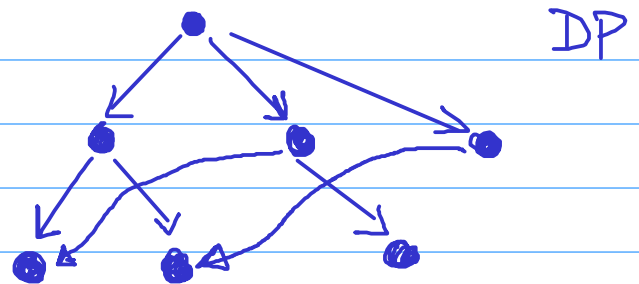
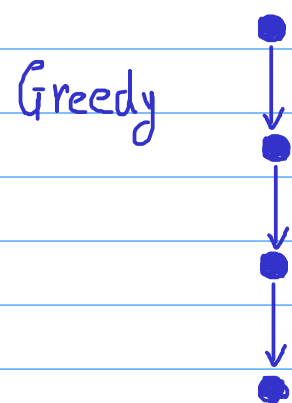
- Dividing the set of possible solutions into multiple categories.



Greedy: there must be an optimal solution in a certain category C^* (greedy choice)

DP: Will take best of the optimal solutions from each category.

- To compute the optimal solution from a chosen category $\rightarrow$ smaller subproblem (Recursion)



No. of distinct subproblems should be small.

Data Compression: Coding

assign a fixed length 0-1 string to each character.

a $\rightarrow$ 00001
b $\rightarrow$ 00010
⋮

5 bit encoding can work for up to 32 characters.

Is a smaller length encoding possible?

With fixed length - not possible

Variable length encoding

- can be more efficient when no. of characters is not 2^k .
- can use smaller length codes for more frequent characters.

Example: Morse Code (dots and dashes and spaces)

e • Z — — • •
t — q — — • —
a • —
⋮

Problem

• — • — $\rightarrow$ aa
 $\rightarrow$ eta
 $\rightarrow$ aet
 $\rightarrow$ etet

Solution: Gap after every character

Prefix Code

Def: For any two different characters x and y
 $c(x)$ should not be a prefix of $c(y)$.

Ex $\left. \begin{array}{l} x \rightarrow 00 \\ y \rightarrow 001 \end{array} \right\}$ Not a prefix code

Ex $\left[\begin{array}{l} a \rightarrow 0 \\ b \rightarrow 10 \\ c \rightarrow 11 \end{array} \right\}$ Prefix code.

$abaca \rightarrow \underbrace{0100110}$

Claim: For a prefix code, any 0-1 string is unambiguously decodable.

Just scan left to right, as soon as the current substring matches one of the codewords, output the corresponding character.

Optimal Prefix Codes

	Freq		Code 1	Code 2
	0.4	A	00	0
	0.4	T	01	10
→	0.1	P	10	110 ←
→	0.1	G	11	111 ←

100
char

2
200
bits

$$\begin{aligned} & 0.4 \times 1 \\ & + 0.4 \times 2 \\ & + 0.1 \times 3 + 0.1 \times 3 \\ & = 1.8 \end{aligned}$$

Avg
Bit
length
 $\frac{180}{100}$
bits.

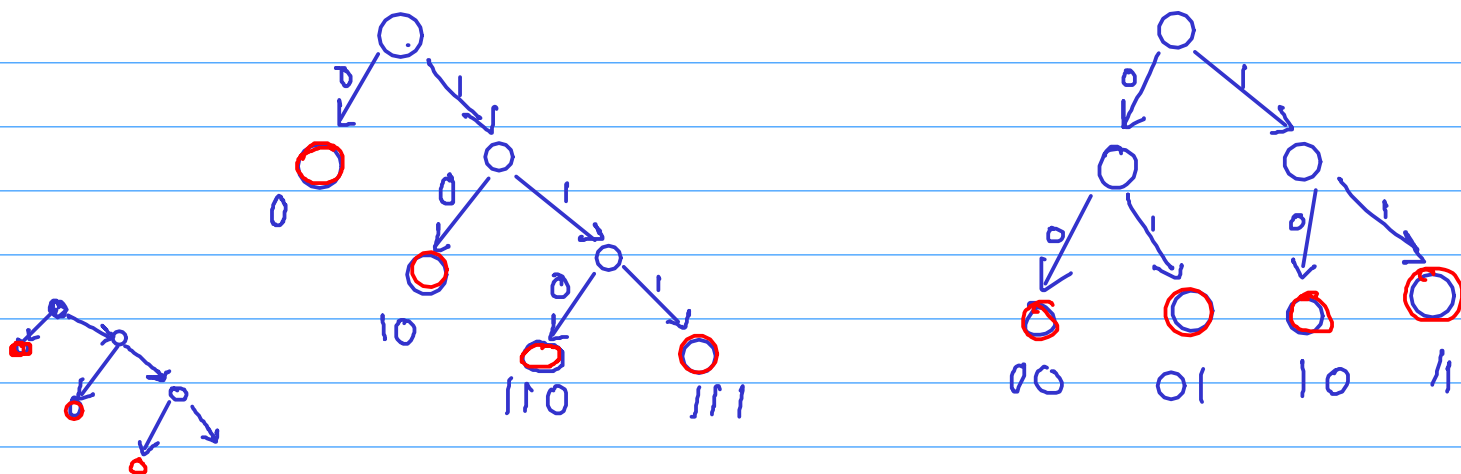
Prob Given frequencies $f_1, f_2, \dots, f_n$ for n characters find a prefix code that minimizes $\sum f_i l_i$.

$$(\sum f_i = 1)$$

avg encoding length for i -th character. $\uparrow$ length of the encoding

Observation: Each prefix code corresponds to a binary tree.

codewords correspond to leaves of the tree.



Approach 1: assign 0 to highest frequency.

Approach 2: assign 0 to highest freq if it is above some threshold.

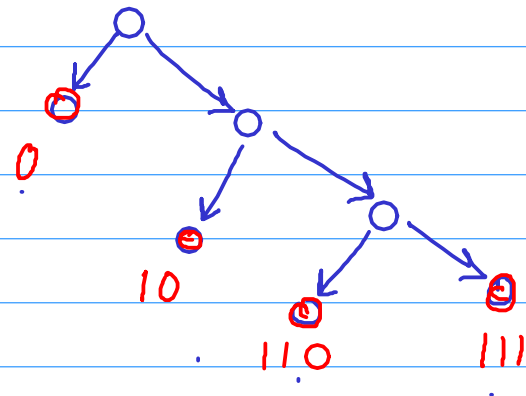
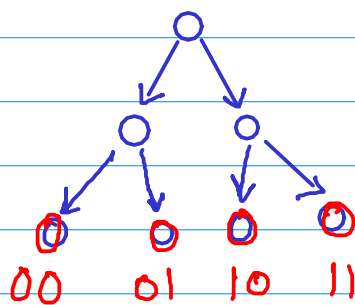
Approach 3: Do a balanced division of frequencies into two parts.

0.32, 0.25, 0.18, 0.25

Observations:

low frequency $\rightarrow$ higher length
high frequency $\rightarrow$ lower length

Approach 1: for highest frequency character, assign '0' i.e. length one codeword.



$(0.25, 0.25, 0.25, 0.25)$

$(0.28, 0.24, 0.24, 0.24)$

2

$$0.28 \times 1 + 0.24 \times 2 + 0.24 \times 3 + 0.24 \times 3 = 2.2$$

$(\underline{0.37}, 0.21, 0.21, 0.21)$
2

2

$$= 2.05$$

$(\underline{0.35}, 0.35, 0.15, 0.15)$
1

2

$$= 1.95$$

Assign '0' if frequency higher than certain threshold.

Doesn't work.

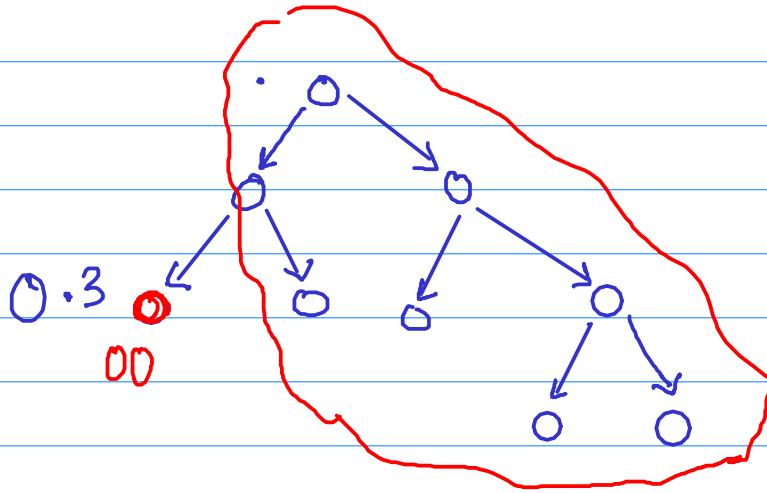
Cannot decide 1, just by looking at f_1 .

Suppose there is some way to decide the encoding length for highest frequency character.

say length 2. '00'

0.3, 0.25, 0.2, 0.15, 0.1

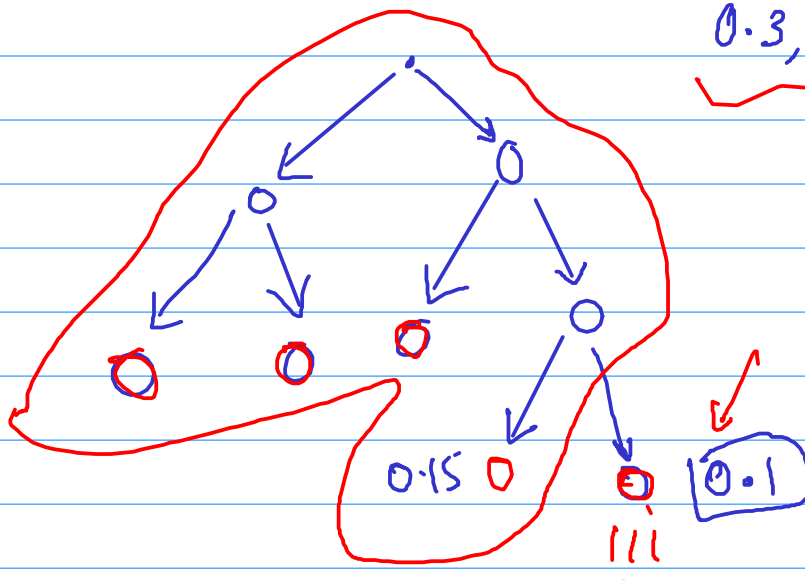
can we reduce the rest of the encodings to a subproblem?



Not able to frame it as a smaller instance of the same problem.

→ Suppose we can fix the length for the least frequent character.

0.3, 0.25, 0.2, 0.15, 0.1

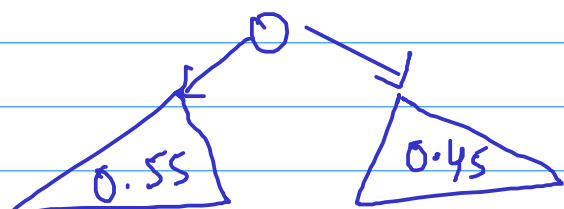
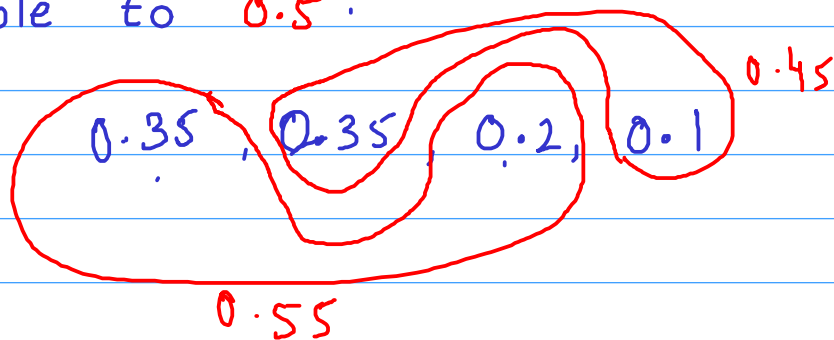


same issue.

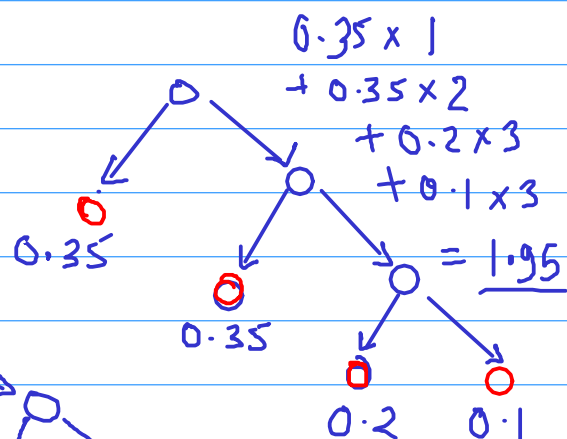
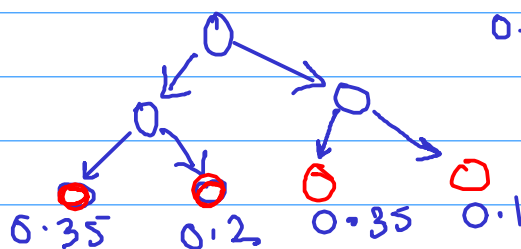
Shannon and Fano (1940's)

Balanced Partition

Divide the list into two parts such that total frequency of each part is as close as possible to 0.5.

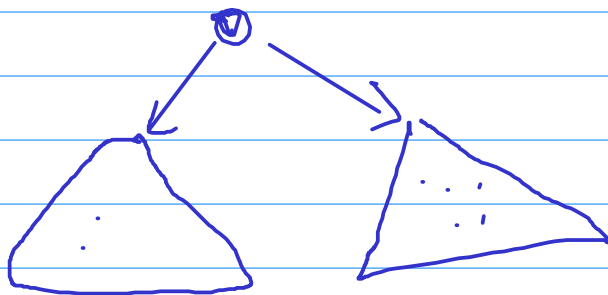


2



By balanced partition approach we are getting average encoding length = 2. But there is a better solution with 1.95.

→ Try various possibilities for the partition?

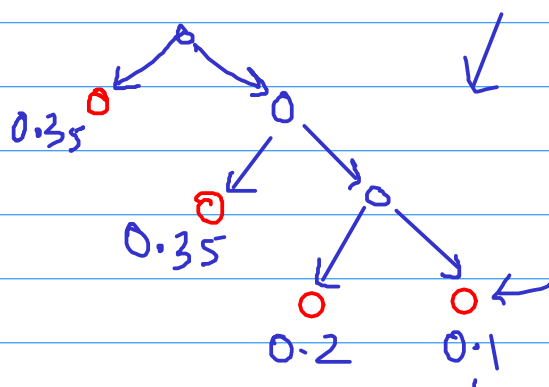
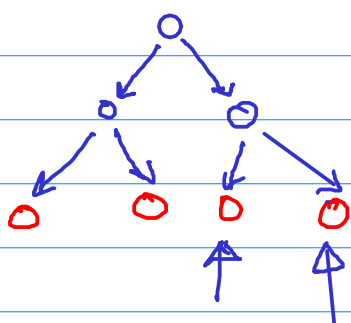


Exponentially many possibilities

Huffman [1952]

Obs 1: If you fix a binary tree, then there is a natural way to map characters to leaves

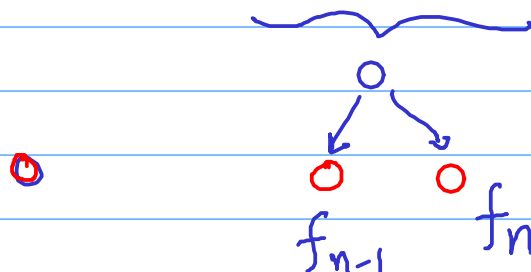
0.35, 0.35, 0.2, 0.1



Lowest frequency character has the largest depth.

Obs 2: The leaf with the largest depth must have a sibling which is a leaf.

$$f_1 \geq f_2 \geq \dots \geq f_{n-1} \geq f_n$$

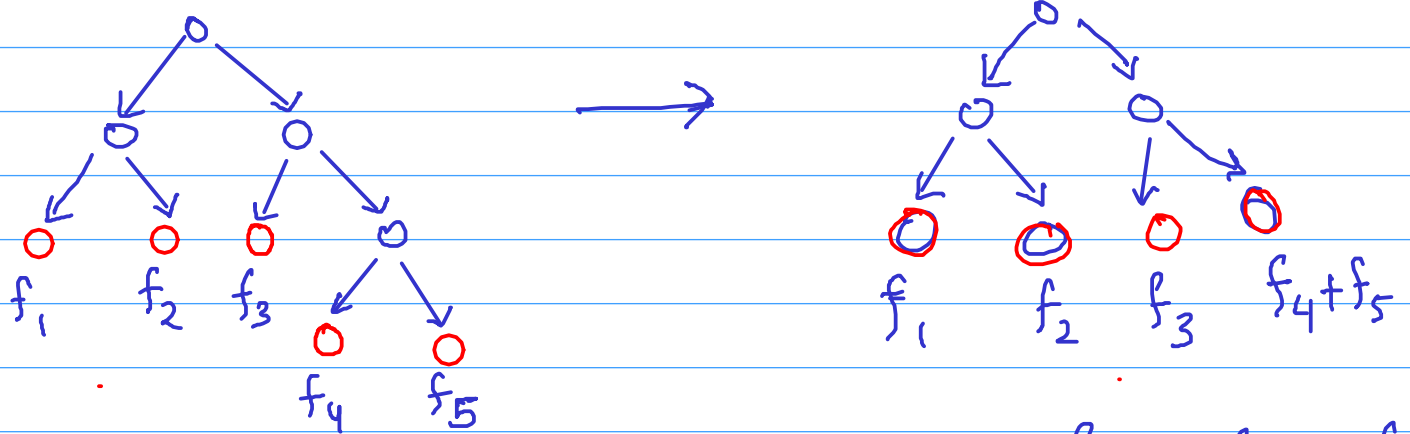


Claim: $\downarrow$

Lowest and second lowest frequency characters can be mapped to two largest depth siblings.

Now, the rest of the tree can found recursively.

f_1, f_2, f_3, f_4, f_5 (in decreasing order)

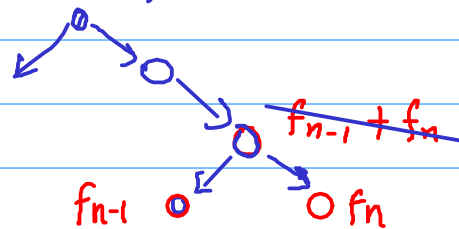


$$\text{cost}_0: 2f_1 + 2f_2 + 2f_3 + 3f_4 + 3f_5 \quad \text{cost}_1: 2f_1 + 2f_2 + 2f_3 + 2f_4 + 2f_5$$

$$\boxed{\text{cost}_0 = \text{cost}_1 + f_4 + f_5}$$

→ for any binary tree with n leaves where f_{n-1} and f_n are siblings, there is a corresponding binary tree with $n-1$ leaves with labelings

$f_1, f_2, f_3, \dots, f_{n-2}, f_{n-1} + f_n$



Algorithm:

Input: $f_1 \geq f_2 \geq \dots \geq f_{n-1} \geq f_n$ (n char)

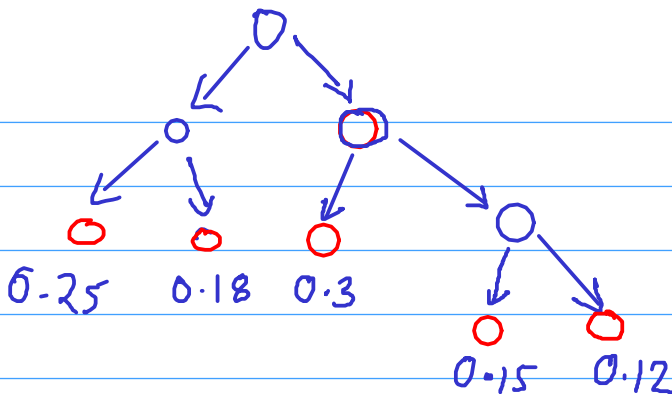
Recursively compute optimal binary tree for

→ $f_1, f_2, \dots, f_{n-2}, f_{n-1} + f_n$ ($n-1$ char)

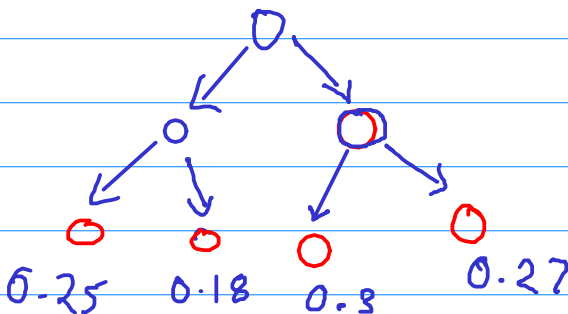
For the leaf labeled $f_{n-1} + f_n$, add two children with label f_{n-1}, f_n .

Example

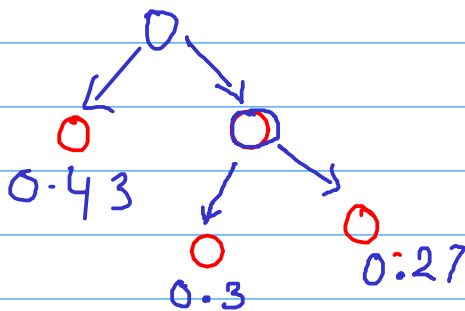
0.3, 0.25, 0.18, 0.15, 0.12



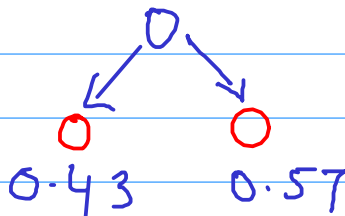
0.3, 0.25, 0.18, 0.27



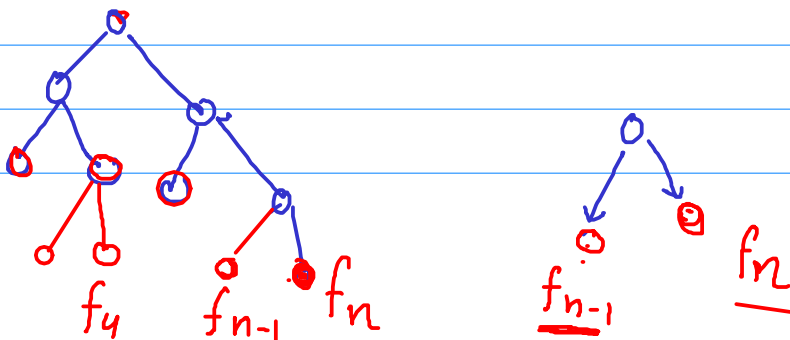
0.3, 0.27, 0.43



0.43, 0.57



$$\sum f_i l_i$$



Proof of Correctness

① There is an optimal solution where f_{n-1} and f_n are siblings.

② There is a one-to-one correspondence between

$A = \{ \text{full binary trees with } n \text{ leaves labeled } f_1, f_2, \dots, f_n \mid \text{where } f_n \text{ and } f_{n-1} \text{ are siblings} \}$

$B = \{ \text{full binary trees with } n-1 \text{ leaves labeled } f_1, f_2, f_3, \dots, f_{n-2}, \underline{f_{n-1} + f_n} \}$

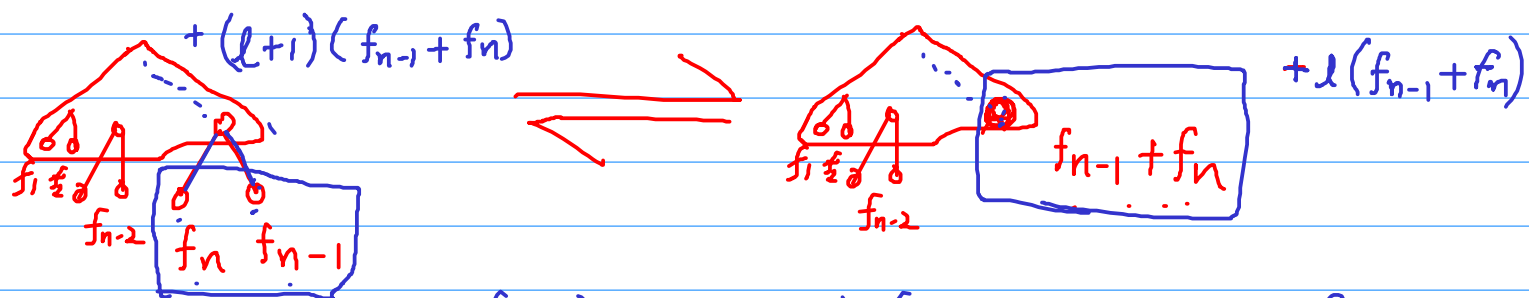
Say, $A = \{ T_1, T_2, T_3, \dots, T_N \}$
 $B = \{ R_1, R_2, R_3, \dots, R_N \}$

Remove leaves labeled f_{n-1}, f_n .

label their parent $f_{n-1} + f_n$

$T_j \longleftrightarrow R_j$

For the leaf labeled $f_{n-1} + f_n$,
add two children labeled f_{n-1}, f_n



Moreover $\text{cost}(T_j) = \text{cost}(R_j) + f_{n-1} + f_n$

Thus, R_j is optimal in $B \iff T_j$ is optimal in A .

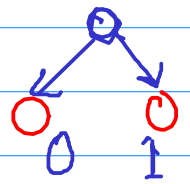
Huffman Codes:

Text over some large alphabet size $\longrightarrow$ space efficient bit representation

- Que: Can we apply this technique when the data is already in 0/1 bit representation?

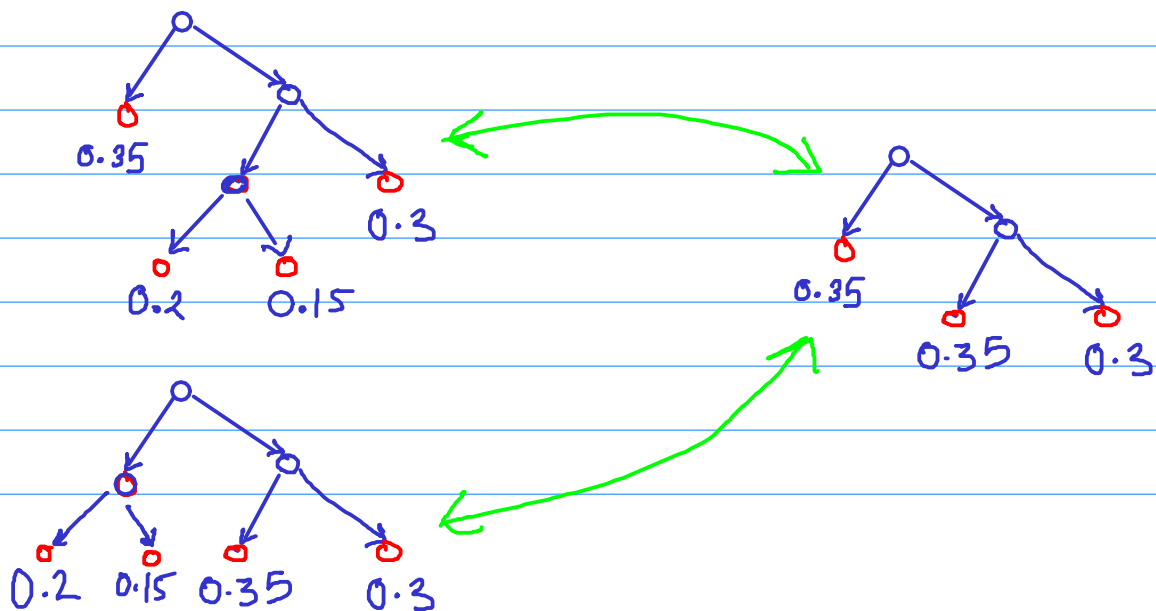
Suppose there is data where 0 is much more frequent than 1.

b a g ... a a
001000110101000011100110000010



$\rightarrow$ a { 000 ... 2/8 $\longrightarrow$ 00
b { 001 3/16 $\longrightarrow$ 100
c { $\vdots$ $\vdots$ $\vdots$
h { 111 1/16 $\longrightarrow$ 1101

- There are many other data compression techniques
 - adaptive
 - Algebraic



- Input:
 - a directed graph $G(V, E)$
 - edge capacities
 $\{c_e \in \mathbb{N} : e \in E\}$
 - a source vertex s
 - a sink vertex t

Assumption: No incoming edge to s
No outgoing edge from t .

→ Output: An s - t flow which maximizes the total flow out of s .

Def: An s - t flow is a function

$$f: E \rightarrow \mathbb{R}_{\geq 0}$$

which satisfies

- Capacity constraints $0 \leq f(e) \leq c_e$
- Flow conservation

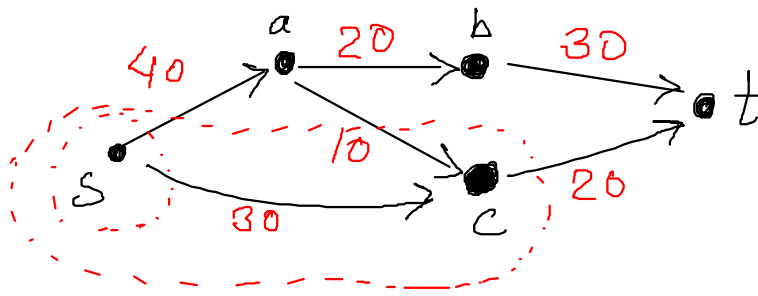
$$\text{for any } v \neq s, t \quad f^{\text{out}}(v) = f^{\text{in}}(v)$$

$$\sum_{e \text{ out of } v} f(e) = \sum_{\substack{e \text{ into } \\ v}} f(e)$$

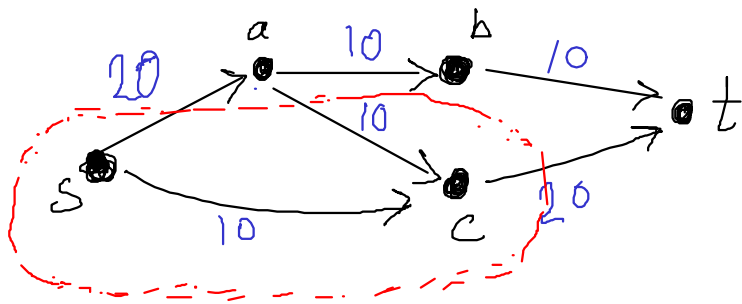
Capacities in red.

Flow values in blue.

Input

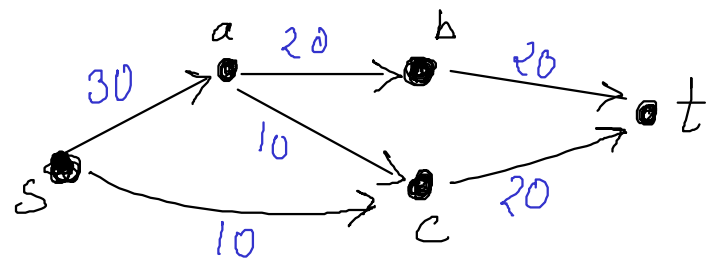


capacities



A flow

Flow value 30



another flow

Flow value 40

Flow value $\sum_{e \text{ out of } s} f(e) = \sum_{e \text{ into } t} f(e)$

Want to maximize flow value.

Claim: Consider a subset $U \subseteq V$
s.t. $s \in U$, but $t \notin U$

Then the net flow out of U is same as
the net flow out of s .

$$\sum_{e \text{ out of } U} f(e) - \sum_{e \text{ into } U} f(e) = f^{\text{out}}(s)$$

This number is the s - t flow value.

Proof Consider

$$\sum_{u \in U} (f^{\text{out}}(u) - f^{\text{in}}(u)) = f^{\text{out}}(s) \quad (\text{Conservation})$$

$$= \sum_{u \in U} \left(\sum_{e \text{ out of } u} f(e) - \sum_{e \text{ into } u} f(e) \right)$$

$$= \sum_{e \text{ out of } U} f(e) - \sum_{e \text{ into } U} f(e)$$

Natural upper bound on maximum s-t flow ?

$$\bullet \sum_{\substack{e \text{ out} \\ \text{of } s}} c_e$$

$$\bullet \sum_{\substack{e \text{ into} \\ t}} c_e$$

Def: $U \subseteq V$ is an s-t cut if
 $s \in U$ but $t \notin U$.

Def $\text{Cap}(U) := \sum_{\substack{e \text{ out of} \\ U}} c_e$

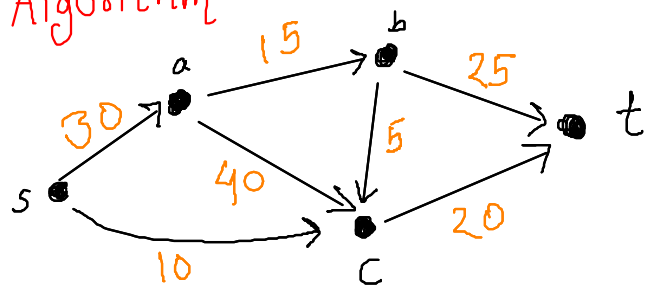
For an s-t cut U , $\text{cap}(U)$ is an upper bound
on the flow value.

Theorem Maximum s-t flow $\leq$ minimum capacity of
an s-t cut

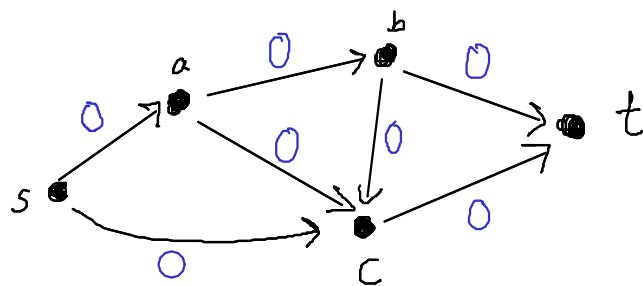
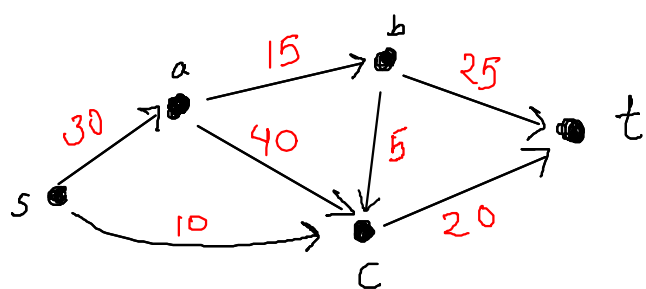
Does the equality always hold?

Amazingly, yes!

Algorithm



Idea: Start with zero flow on all the edges.



Find an s - t path and push as large flow as possible

Paths

Bottleneck

$s - a - b - t$

15

$s - a - c - t$

20

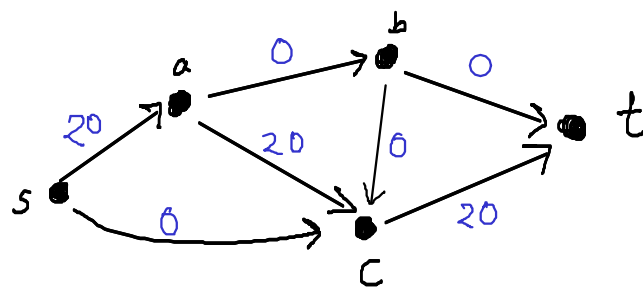
$s - c - t$

10

$s - a - b - c - t$

5

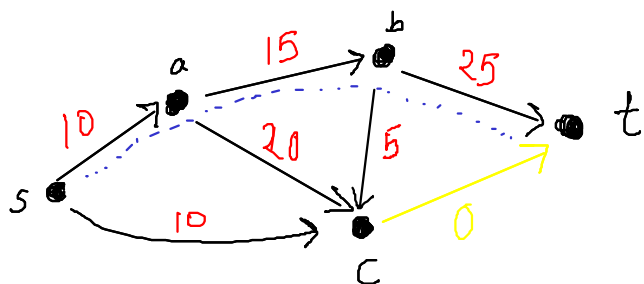
Push a flow of 20
along $s - a - c - t$



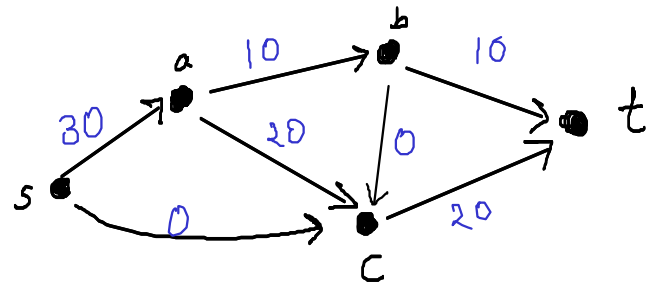
Remove saturated edges

Update capacities.

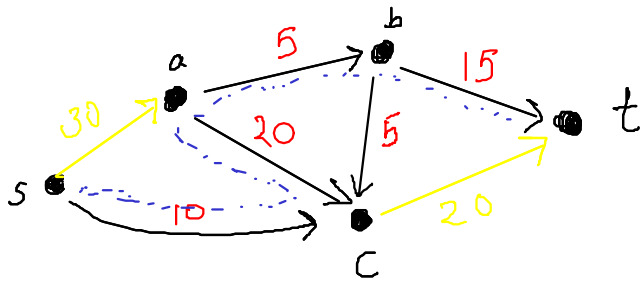
And then find an s - t path.



push a flow of 10
along path $s - a - b - t$



Remove saturated edges
And then find an s - t path.



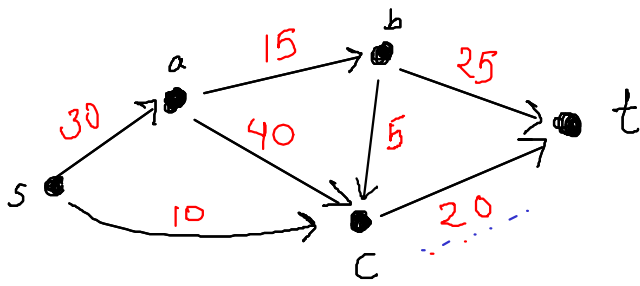
No s - t path !!

How can we push more flow? Push back along (a, c)

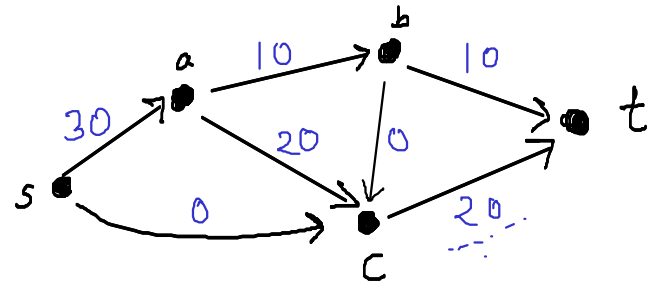
Def Residual graph w.r.t. a flow f

- Vertex set same.
- For each edge $e = (u, v)$
 - Forward edge (u, v) $c_e \leftarrow c_e - f(e)$
keep this edge only if positive residual capacity
 - Backward edge $e' = (v, u)$ $c_{e'} \leftarrow f(e)$
keep this edge only if $f(e) > 0$

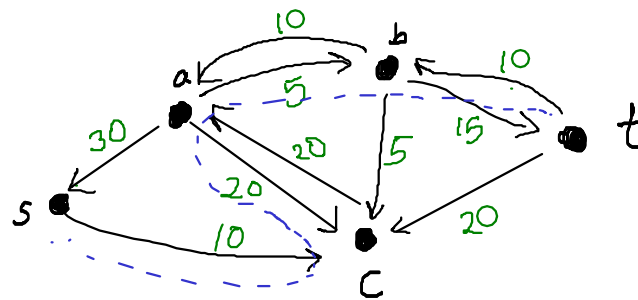
Find a s - t path in the residual graph.



Original capacities



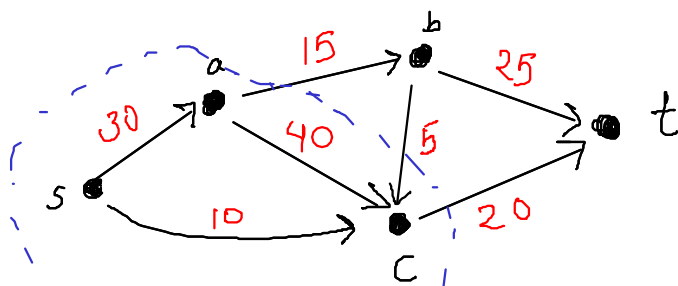
flow f



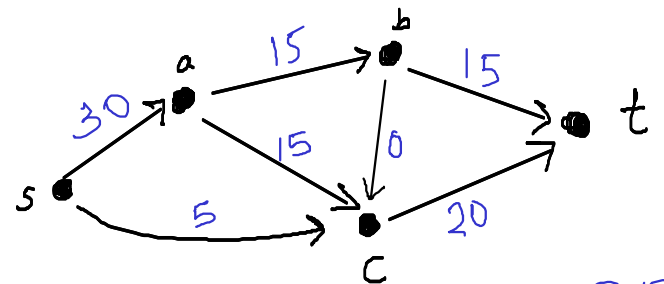
Residual Graph G_f

Path in the residual Graph

$s-c-a-b-t$ push 5

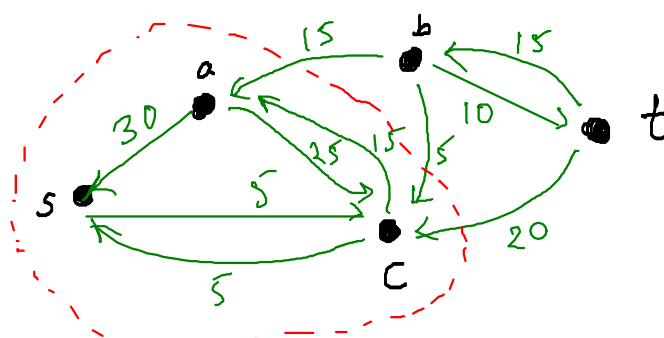


Original capacities



flow f

35



Residual Graph G_f

No outgoing
edges
from
 $\{s, a, c\}$

Algorithm

Initialize $f = 0$

While you can {

find an s - t path P in the residual graph G_f

$b \leftarrow$ min capacity of an edge on P .

for each edge $e = (u, v) \in P$ {

if (u, v) is a forward edge

$$f(u, v) \leftarrow \underline{f(u, v) + b}$$

if (u, v) is a backward edge

$$f(v, u) \leftarrow \underline{f(v, u) - b}$$

}

Update the residual graph G_f .

}

Claim 1: After each iteration of the while loop,
 f remains a valid flow.

Claim 2: If we cannot find an s - t path then
the flow is maximum.

Claim 3: Algorithm always terminates.

Proof of Claim 1:

Conservation of flow at any vertex v

Case 1

$$f^{\text{out}}(v) - f^{\text{in}}(v)$$

Case 2

Capacity constraints.

→ Forward edge

→ Backward edge

Proof of Claim 2

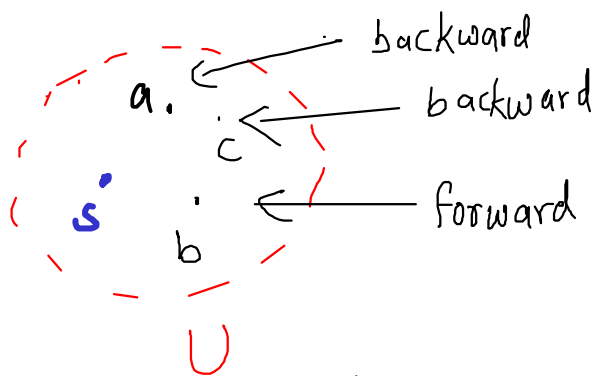
Idea is to show an s - t cut in original graph whose capacity is equal to current flow value.

Residual graph:

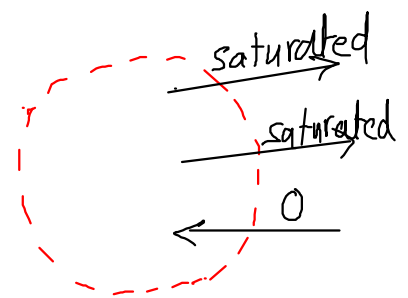
We can't find an s - t path.

$\Rightarrow \exists$ subset $U \subseteq V$ s.t. $s \in U, t \notin U$ and there are no outgoing edges from U .

$U \leftarrow$ reachable vertices from s .



Residual Graph.



Flow graph

Obs

In the flow graph
outgoing edges ^{from U} are saturated
edge into U have zero flow.

$$\rightarrow \text{flow value} = f^{\text{out}}(U) - f^{\text{in}}(U)$$

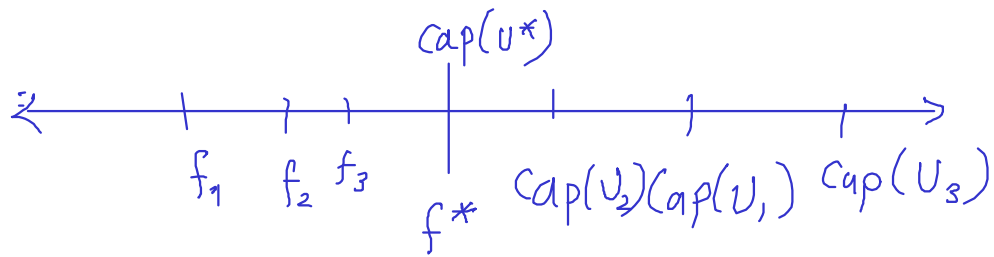
$$= \sum_{\substack{e \text{ out} \\ \text{of } U}} \text{cap}(e) - 0$$

$$= \text{Cap}(U)$$

But we know
 $\text{flow value} \leq \text{cap}(U)$

Hence the flow is maximum.

Hence $\text{Cap}(U) = \min$ s-t cut capacity.



Thm $\text{Max}^{s-t} \text{flow} = \text{Min s-t cut}$

Terminate

Flow value increase is always integral.

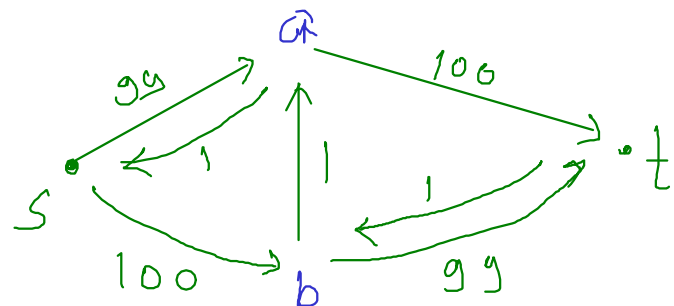
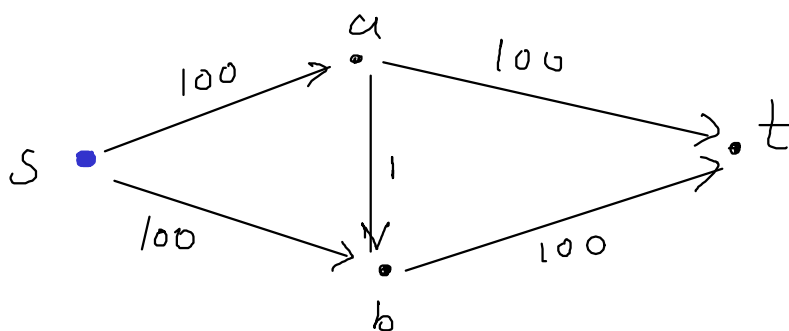
Flow value increases by at least 1.

No. of iteration $\leq$ sum of edge capacities.

Time $O(|E| \cdot C)$

Pseudopolynomial

exponential in the input size



Find s-t paths in a clever way.

- Shortest length path (Edmonds Karp)
 $O(VE^2)$ Strongly Polynomial time
- Max bottleneck ~~$O(E \log C)$~~

$$O(E^2 \log C \cdot E)$$

↑ Not strongly polynomial.

Augmenting Path

Reduction :

The following statements mean the same

→ Problem A reduces to Problem B

→ One can design an algorithm for problem A using a subroutine for problem B.

→ Notation $A \leq B$

Example: Multiplication $\leq$ Squaring

TA allocation $\leq$ Network flow

Network flow $\leq$ Linear programming

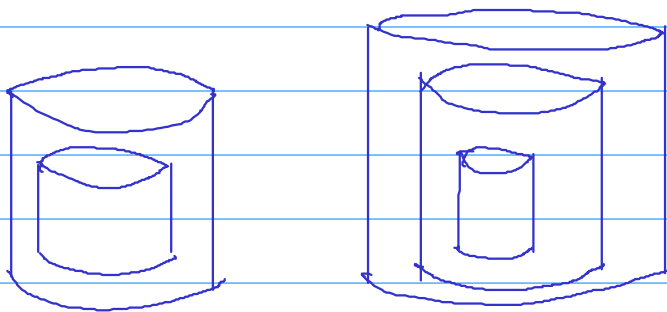
Often we need to use the reduction only once.
as in the second example.

These are called many-one reductions.

Applications of Network flow.

1. Network flow variants
 - Multiple sources and sinks with demands
 - Undirected Network
2. Bipartite Matching
3. TA allocation
4. Maximum number of edge disjoint paths from s to t .
5. Airline Scheduling / Container arrangement
6. Project selection

Container arrangement

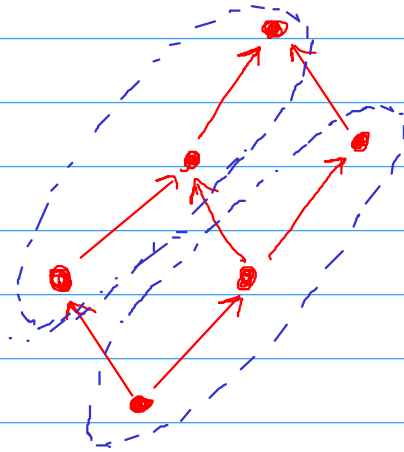


Exam: greedy algorithm worked in the setting of 2 parameters.

More general version: containers with 3 parameters
height, width, length

Abstract version

Given a partial order on n elements,
partition them into minimum number of chains



Lower bound : any set of mutually incomparable elements.

Amazingly, if the minimum number of chains is k , then there is a set of k mutually incomparable elements

Another example :

Trains arriving/departing at a station, schedule them
using minimum number of platforms.

Taxi Scheduling

A taxi company gets a list of bookings for the next day.

Want to minimize the number of taxis required.

Bookings

B_1 : 9:00 Chembur $\rightarrow$ Airport 10:00

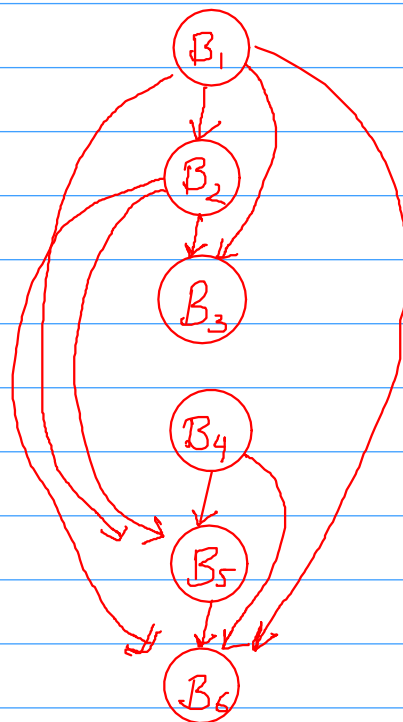
B_2 : 10:30 Andheri $\rightarrow$ IITB 11:15

B_3 : 11:30 IITB $\rightarrow$ TIFR 1:30

B_4 : 10:15 Dadar $\rightarrow$ Airport 11:15

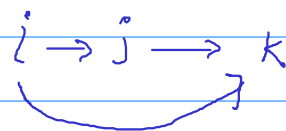
B_5 : 12:00 Powai $\rightarrow$ LTT 12:45

B_6 : 13:00 Chembur $\rightarrow$ Sion 13:30

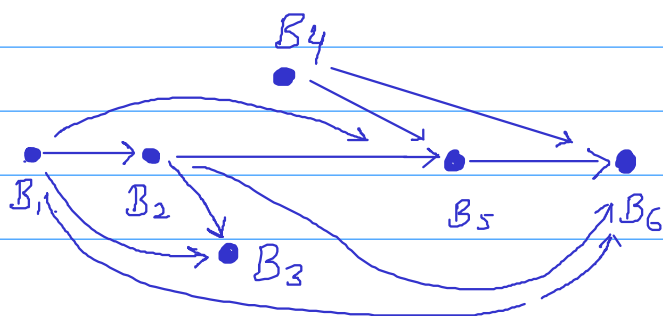


Input: Directed graph — acyclic
— transitively closed

(Partial Order on a set of elements)



Output: Partition of vertices into minimum number of paths.



Algorithm via Network Flow

Given a partial order (directed graph)

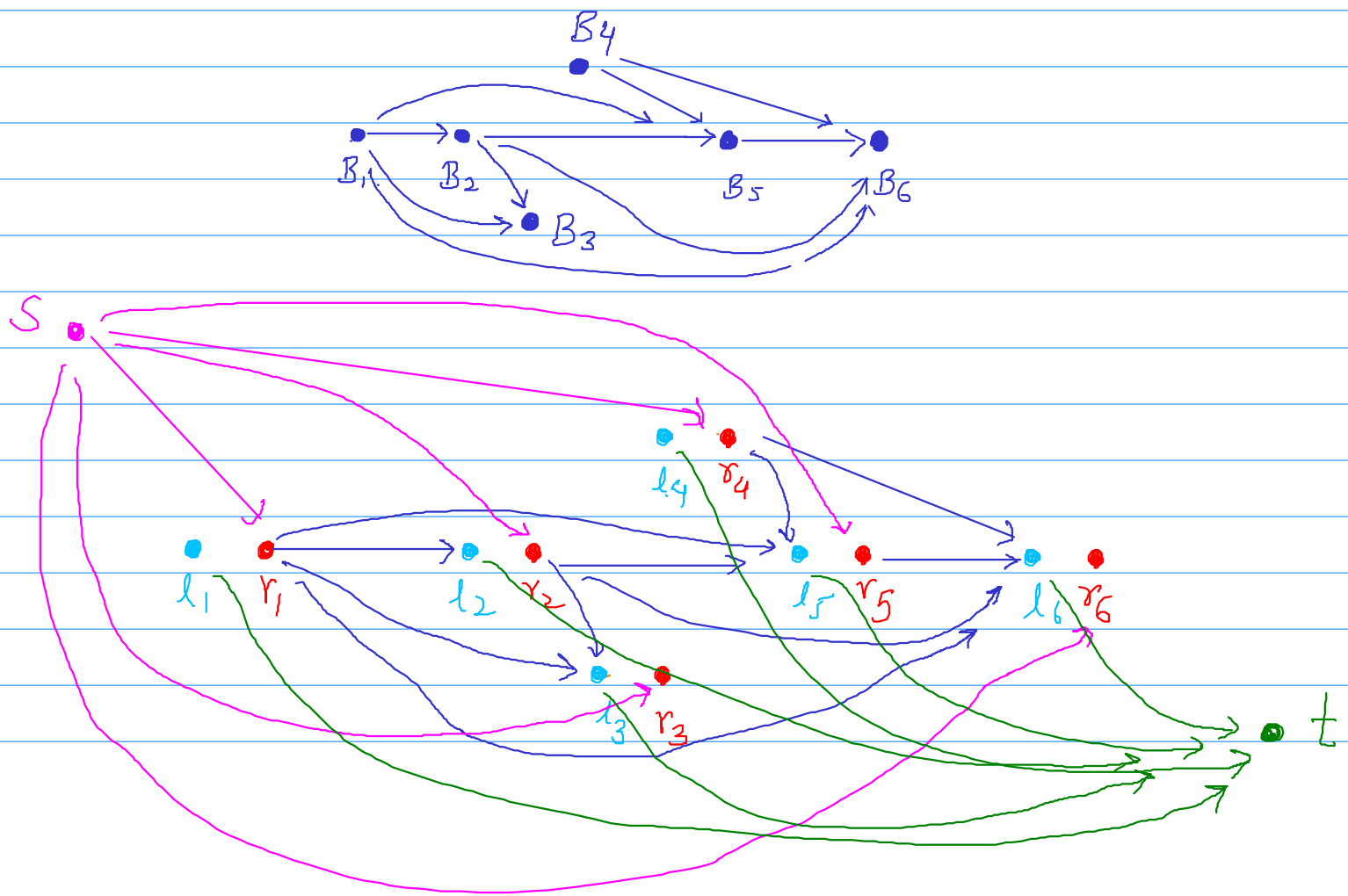
For every element B_i , create two nodes

l_i, r_i

If $B_i \rightarrow B_j$ then add an edge
from r_i to l_j of capacity 1.

From source s to all r_i 's capacity 1

from each l_i to sink t capacity 1



Claim 1 :

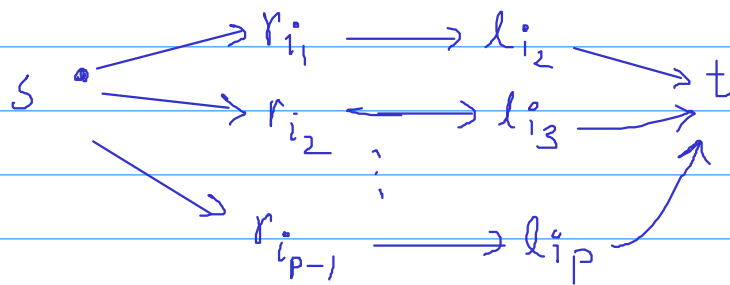
If there is a partition of the partially ordered set into k chains

then there is a flow of $n-k$ units in the network.

Proof: for any chain with p elements, say

$$B_{i_1} \rightarrow B_{i_2} \rightarrow B_{i_3} \dots \rightarrow B_{i_p}$$

We will have $p-1$ units of flow along the following paths



Note that any vertex r_i or l_j will be used in only one of the paths because any B_i appears in exactly one chain.

Clearly adding the flows corresponding to all the k chains, we have $n-k$ units of flow.

Claim 2: If there is l units of flow in the network ^(integral)

then there is a partition of the given partially ordered set into $n-l$ chains

Proof: If there is flow along $r_i \rightarrow l_j$ then we will put B_j as a successor of B_i in one of the chains.

Since r_i can have at most one unit of outgoing flow we get that any B_i will get at most one successor.

Similarly any B_j will get at most one predecessor.

Hence the result will be a collection of chains.

How many chains are there?

We start with n elements as n different chains.

For every one unit of flow two chains get combined into one.

Hence $n-l$ chains.

Note: A flow is not necessarily integral.

But recall that the max flow algorithm gives an integral flow whenever the capacities are integer.

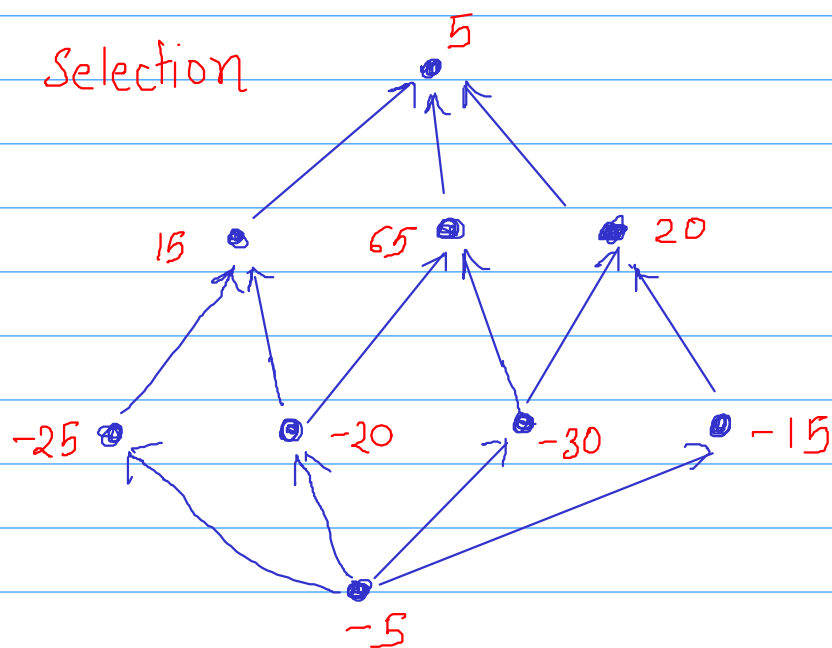
Algorithm ① from given partial ordered set, construct the flow network

② Compute a maximum flow in the network (which is integral)

③ Use claim 2 to obtain a collection of chains from the flow.

HW Claim 1 implies that the collection of chains we get is optimal.

Project Selection



Downward closed subset : If we take an element j we must take everything below it.

Find the downward closed subset which maximizes the total sum.

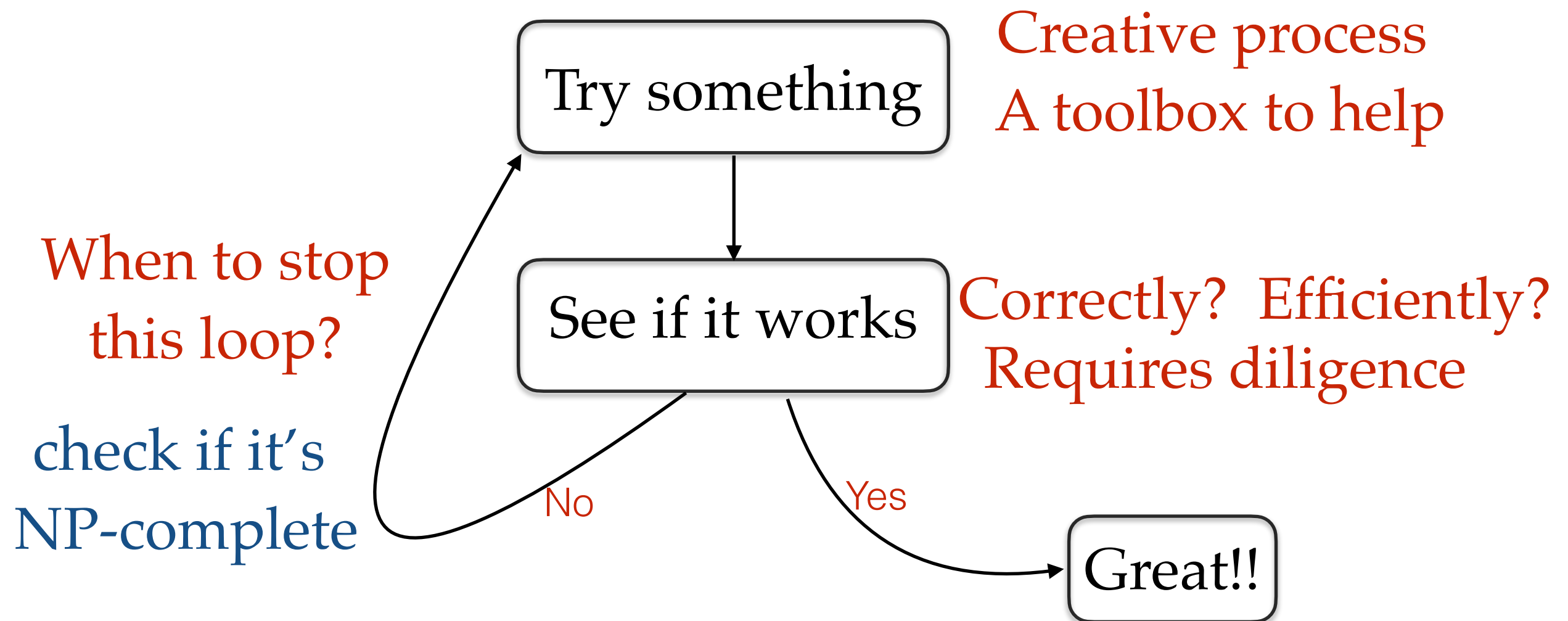
Reduce this problem to the minimum cut problem.

NP, NP-completeness and a Million Dollar Question

March 18, 2025

Objectives

- How to design algorithms.



History of P

- Notion of Efficient Algorithms there since ancient times
- Addition, Multiplication, GCD, Repeated squaring (Pingala), Astronomical calculations.
- [1950s] Dynamic Programming, Shortest Path, Simplex algorithm, Minimum spanning tree
- [1960s] FFT, Scheduling, Network flow, bipartite matching, and related combinatorial problems
- Doing better than brute force search

P (Polynomial time solvable)

- [Edmonds \[1965\]](#) proposed polynomial time as a characterization of efficient computation
- “It is by no means obvious whether or not there exists an algorithm whose difficulty increases only **algebraically** with the size of the graph”
- “For practical purposes the difference between algebraic and exponential is more crucial than between finite and non-finite.”

P (Polynomial time solvable)

- Why polynomial time?
 - if a procedure is considered efficient, running it n times might also be considered efficient.
 - Polynomial time remains independent of computation model.
 - Another perspective: if you double the input size, the running time gets multiplied by a constant.
 - If the running time is n^{100} , is it still efficient?
 - In reality, we never get such running times.

Not in P

- [1960s] For many problems, people could not find better than exponential time algorithms.
- There was no clear explanation why some problems are in P, while others are not.
- Try to guess, whether a polynomial time algorithm is known or not.

In P or not in P ?

- Roommate Allocation:
 - n students, some like each other, some don't.
 - Allocate rooms s.t. roommates like each other.
 - Polynomial time algorithm known or not?
 - Yes [Edmonds 1965]

In P or not in P ?

- Triple Roommate Allocation:
 - n students, some like each other, some don't.
 - Allocate rooms s.t. all 3 roommates like each other.
 - Polynomial time algorithm known or not?
 - No

In P or not in P ?

- Given a graph and a number k , is there a path of length k ?
 - Not known to be in P
- Given a graph with s and t vertices, is there a path from s to t with even length?
 - In P
- Same problem in directed graphs?
 - Not known to be in P

In P or not in P ?

- Given a graph with edges colored red or blue, is there an s-t path with alternating red and blue edges?
 - In P
- Same problem in directed graphs?
 - Not known to be in P

In P or not in P ?

- Given a number (in binary), is it factorizable?
 - In P (only in 2002)
- Given a number (in binary), find its factors?
 - Not known to be in P

In P or not in P ?

- Given a set of intervals, largest subset of disjoint intervals
 - In P
- Given a graph, find the largest independent set (vertices sharing no edges).
 - Not known to be in P
- Given a graph, find the largest set of edges not sharing any vertex
 - In P
- Given a graph, find the largest set of triangles not sharing any vertex
 - Not known to be in P

In P or not in P ?

- Set of trains arriving / departing at a station, can we schedule using k platforms ?
 - In P
- Given a list of courses, and pairs which should avoid a clash, can we schedule using k time slots?
 - Not known to be in P
 - Also known as graph coloring (easy for 2 colors)
- n Jobs, m processors, not every processor can handle every job. Processors can work in parallel. Can we finish in k units of time?
 - In P (via Network Flow)

In P or not in P ?

- Minimum weight spanning tree
 - In P
- **Steiner Tree**: Given a subset of vertices (terminals), find the minimum weight tree that connects the terminals
 - Not known to be in P
- **Traveling Salesperson problem**: given a list of cities, you have to visit every city and come back with minimum cost
 - Not known to be in P

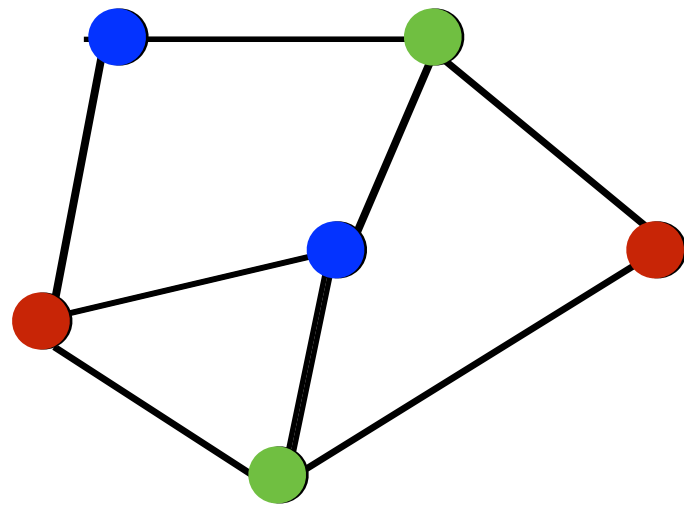
In P or not in P ?

- **Satisfiability**: given a Boolean formula, is it satisfiable?
 - Not known to be in P
- **Minimum circuit size**: Given a Boolean function, is there a circuit for it with at most k Boolean operations?
 - Not known to be in P

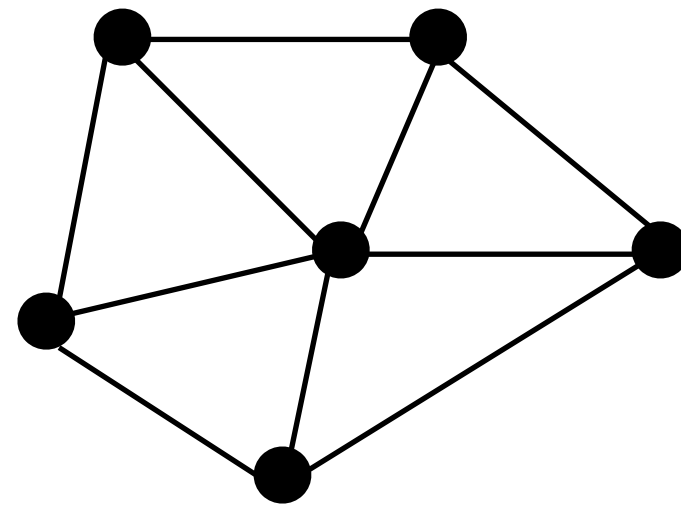
Towards NP

- [1960s] For many problems, people could not find better than exponential time algorithms.
 - Subset sum, Load balancing, Traveling Salesperson, Graphs Isomorphism, Primality, Linear programming, Minimum Circuit Size, Satisfiability, 3-colorability
- People observed some of these can be reduced to others.
- For example, **3-colorability $\leq$ SAT**
- In fact, all of these problems reduce to SAT.
- The key common property of these problems was having “easily verifiable proofs” for the ‘yes’ instances.

3-colorability



A 3-colorable graph



A graph
not 3-colorable

Satisfiability

- $(x \vee y \vee z) \text{ AND } (\neg x \vee y) \text{ AND } (x \vee y) \text{ AND } (x \vee \neg y)$
- Satisfiable
- $(\neg x \vee y) \text{ AND } (\neg y \vee z) \text{ AND } (\neg z \vee \neg x) \text{ AND } (x)$
- Unsatisfiable

3-colorability reduces to SAT

- Given a graph, can we color vertices with 3 colors?
 - create Boolean variables to represent the coloring
 - 3 Boolean variables for each vertex - x_i, y_i, z_i
 - encode the verification procedure as Boolean constraints
 - each vertex has a color — $(x_i \vee y_i \vee z_i)$ for each i
 - adjacent vertices have different colors
 - for every edge (i,j) : $\neg(x_i \wedge x_j), \neg(y_i \wedge y_j), \neg(z_i \wedge z_j)$
 - Boolean formula = AND of all the constraints.
 - Graph is 3-colorable if and only if there is an satisfying assignment for the above Boolean formula
 - An algorithm for SAT will give an algorithm for 3-colorability
- [Cook, Levin 1971] All of the problems mentioned reduce to SAT

Reductions

- Problem A reduces to problem B ($A \leq B$)
 - if A can be solved in polynomial time using a given **subroutine** that solves B.
 - task of solving A reduces to task of solving B
- **Example:** Taxi scheduling reduces to bipartite matching
- **Example:** Multiplication reduces to squaring
 - Multiplication is as easy as squaring
 - Squaring is as hard as Multiplication

Reductions

- Problem A reduces to Problem B:
 - (1) convert input φ_A for A to input φ_B for B (or set of inputs $\varphi_{B1}, \varphi_{B2}, \varphi_{B3}$)
 - (2) $\text{Solution}(\varphi_B)$ should be converted to $\text{solution}(\varphi_A)$.
- Conclusion:
 - A is as easy as B.
 - B is as hard as A.
- $A \leq B$ and $B \leq C$ **implies** $A \leq C$

Reductions (decision problems)

- A problem is said to be a decision problem, if for every input the answer is 'yes' or 'no'.
- **Karp-reduction:** Problem A reduces to Problem B:
 - If we have a polynomial time algorithm which takes an input φ_A for problem A and outputs an input φ_B for B such that
 - answer for φ_A is 'yes' **if and only if** answer for φ_B is 'yes'.
- $A \leq B$ and $B \leq C$ **implies** $A \leq C$
- Every computational problem has a natural decision version, hence we will be focusing on decision problems.

NP or Easily Verifiable Proofs

- Many problem which seemed hard have **easily verifiable proofs** for 'yes' inputs.
- Load Balancing: is there a load allocation with makespan at most k ?
 - **Proof:** an allocation with makespan $k' \leq k$
 - **Verifier:** check if the proposed allocation is valid and its makespan
- Factorize Numbers: is a given number factorizable?
 - **Proof:** two factors
 - **Verifier:** multiply the proposed two numbers and check if you get the input number.
- Not clear if 'no' inputs have easily verifiable proofs.

Easily Verifiable Proofs

- SAT: given a Boolean formula (CNF - AND of ORs), is there an assignment of variables, which makes it true?

Example. $(\neg x \vee y) \wedge (\neg y \vee x)$

- Proof: an satisfying assignment to the variables (example: True, False)
- Verifier: check if the proposed assignment makes the formula true.
- Graph Isomorphism: given two graphs, are they isomorphic?

- Proof: a mapping between two sets of vertices
- Verifier: check if the given mapping preserves edges and non-edges

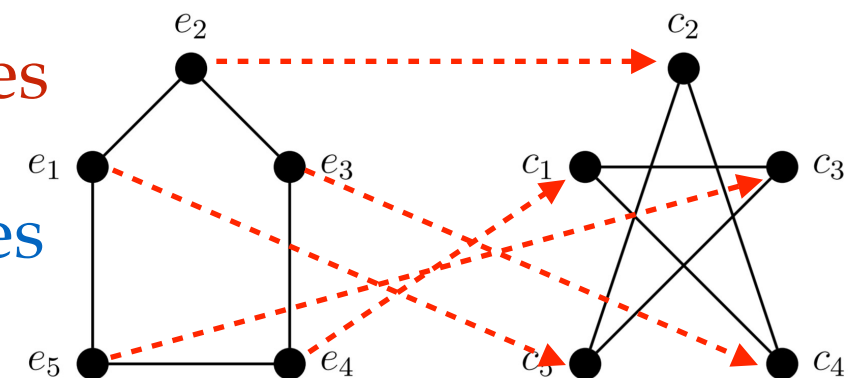


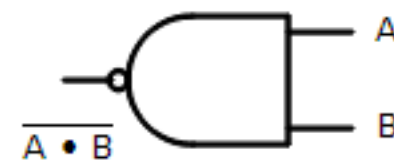
image source: [1]

Easily Verifiable Proofs

- Subset Sum: given numbers $a_1, a_2, a_3, \dots, a_n$ and a number b , is there a subset of a_i 's that sum up to b ?
 - Proof: a subset of numbers
 - Verifier: check if the proposed subset has sum equal to b
- Circuit Size: given a Boolean function f truth table, is there a circuit with at most s gates that computes f ?

- Proof: a circuit

- Verifier: verify if the circuit output matches with the truth table for every input



(b)



$$Z = \overline{A \bullet B} = \overline{A} + \overline{B}$$

A	B	Z
0	0	1
1	0	1
1	1	0

The Class NP

- **Definition:** a ‘yes or no’ decision problem is in NP if there is an easily verifiable proof for each ‘yes’ input.
- a ‘yes or no’ decision problem is in NP if there is a polynomial time Algorithm V (verifier), such that for any input x ,
 - if x is a ‘yes’ input then there **exists** c s.t. $\text{size}(c) \leq \text{poly}(\text{size}(x))$ and $V(x, c) = \text{True}$.
 - if x is a ‘no’ input then **for any** c , $V(x, c) = \text{False}$

The Class NP

- P - can **find** the solution in polynomial time
- NP - can **verify** a proposed solution in polynomial time

$$P \subseteq NP$$

- if a problem is in P, it is also in NP.
- A problem falling into NP is a positive thing.
- NP **does not mean** Non-Polynomial time.
- NP stands for Non-deterministic polynomial time.

Problems in NP?

- Given an integer n , are there integers x, y, z such that

$$x^3 + y^3 + z^3 = n ?$$

e.g. for $n = 39$: $134476^3 - 159380^3 + 117367^3 = 39$.

- **Not clear**, because x, y, z can be much larger than n . Efficient verification may not be possible.
- Given a flow network, is there a flow with $f^{out}(s) = k$?
 - Yes.
 - Proof: for every edge a flow value.
 - Verifier checks flow conservation, capacity constraints, and computes $f^{out}(s)$
 - Proof: 0
 - Verifier runs the max flow algorithm and checks whether $max\ flow \geq k$

Problems in NP?

- Given two Polynomial expressions, are they equal?
 - e.g., $(a^2 + n b^2) (c^2 + n d^2) = (ac - n bd)^2 + n(ad + bc)^2$
- Not clear if it is in NP
- Given two Polynomial expressions, are they different?
 - It is in NP
 - Proof: a substitution of variables with numbers
 - Verifier: evaluates both the expressions on this substitution and verifies if evaluations are different.

Problems in NP?

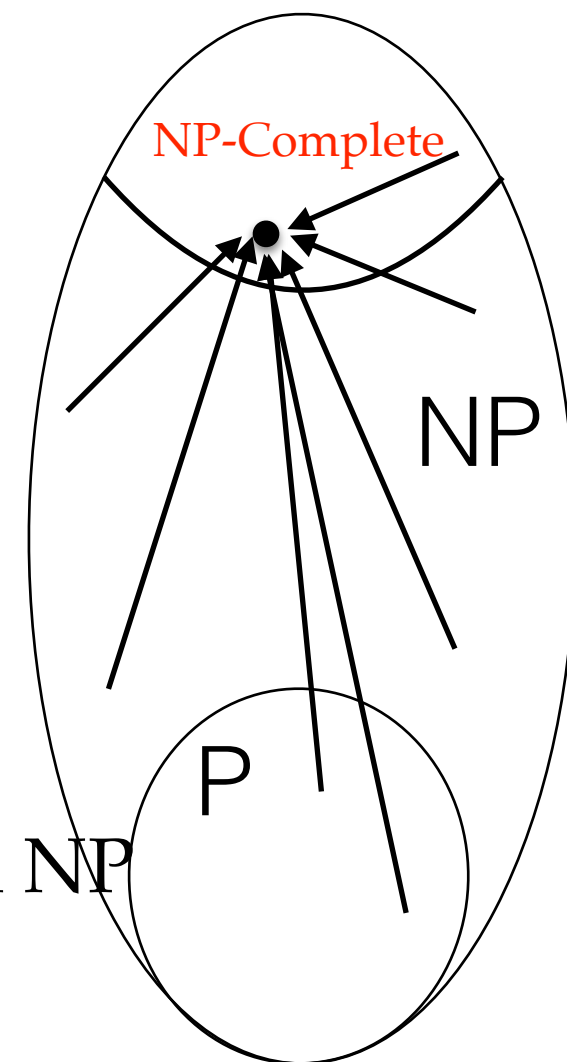
- Given a graph G and another graph H , is H a subgraph of G ?
- It is NP
- Proof: A subset of vertices of G , together with a mapping from vertices of H .
- Verifier: checks whether any pair has an edge in H if and only if the corresponding pair in G has an edge.

Problems in NP?

- Given a graph G and a number k ,
the maximum size of any clique in G is k ?
- Not clear if it is in NP
- Given a graph G and a number k ,
the maximum size of any clique in G is **not** k ?
- Not clear if it is in NP

NP-completeness

- [Cook-Levin \[1971\]](#): If we have a subroutine for SAT problem, we can design a polynomial time algorithm for **every problem in NP**
 - for any problem A in NP, $A \leq \text{SAT}$
 - SAT is '**NP-complete**'.
- [Karp \[1972\]](#): 21 other problems are NP-complete.
 - TSP, Subset Sum, Integer Programming, Graph Coloring, Job Sequencing, Independent Set, 3D-matching etc.
 - They are all equivalent and are hardest problems in NP



The tree of Reductions

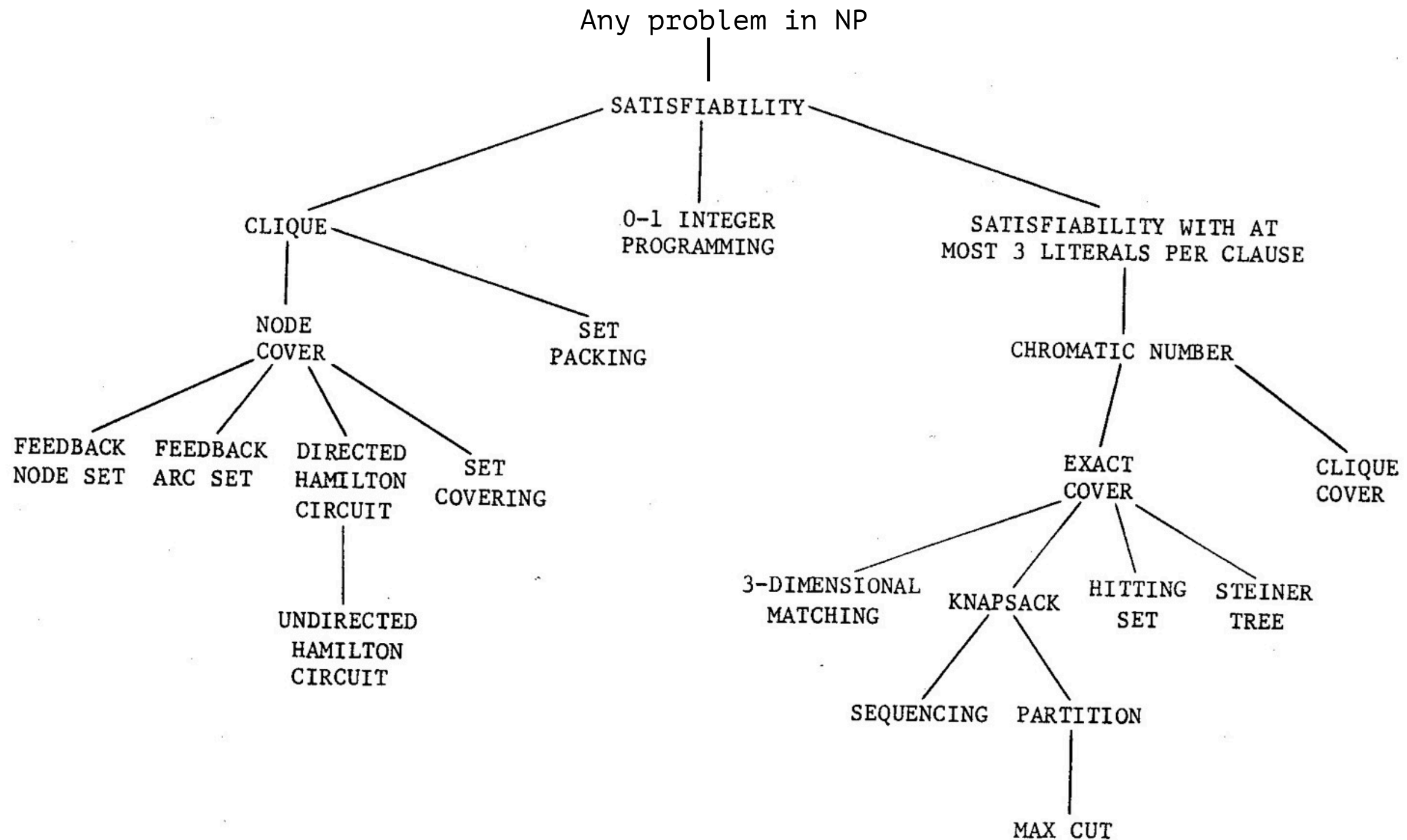


FIGURE 1 - Complete Problems

P vs NP

- Thousands of problems have been shown to be NP-complete.
- If you solve any of them, all of them get solved.
- People have not been able to give an efficient algorithm in last 50 years, for any of these.
- One can say, there is just one NP-complete problem.
- $P = NP \iff$
SAT (and every other problem in NP) has a polynomial time algorithm

Philosophically...

- Problems intuitively / philosophically in class NP
 - is a given mathematical statement (provably) true?
(a proposed proof can be verified)
 - is there a cure for a mentioned disease?
(a proposed cure can be verified)
 - given the public key, can you find the private key?
(a private-public key pair can be verified)
 - Given a partially made movie, can it be completed to make a great movie?
(given a movie, you can critic it)

Philosophically...

- P vs NP = Mechanical vs Creativity
- $P = NP$ would mean
 - all diseases can be cured,
 - all mathematical conjectures can be resolved,
 - crypto systems can be broken
 - All film critics can make great films

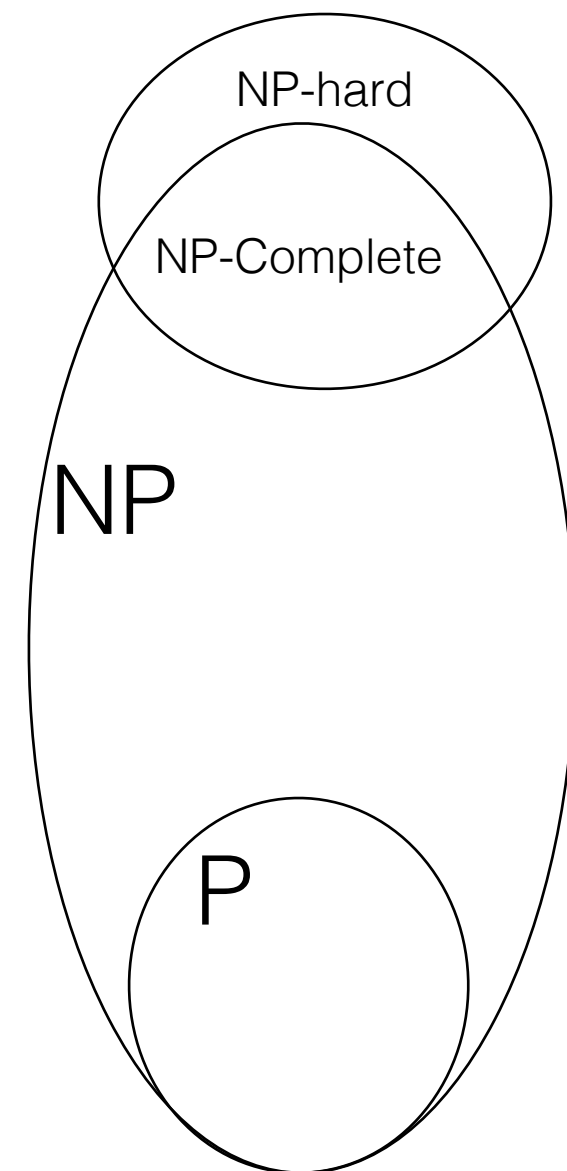
How is it useful?

- Widely believed $P \neq NP$, but no proof for it.
Million Dollars for a proof either way.
- You encounter a new problem X
and can't find a Polynomial time algorithm for it
try to prove that it is NP-hard.
 - Choose a suitable NP-complete / NP-hard problem H and
reduce H to your problem X .
 - I.e., H can be solved using a subroutine for X .
 - “I am not able to design an algorithm for it, but nobody could in last 50 years 😊”
- Amazingly, most problems turn out to be either in P or NP-complete.
 - **Exceptions:** Graph Isomorphism, Minimum circuit Size, Factoring

Babai 2015,
quasi-poly

NP-complete and NP-hard

- Problem X is said to be NP-complete if
 1. X is in NP
 2. Every problem in NP reduces to X
- Problem Y is said to be NP-hard if
 - Every problem in NP reduces to Y



Any problem in NP reduces to SAT

[Section 8.4 in Kleinberg Tardos]

- There is a verifier algorithm V such that for any input x ,
 - if x is a 'yes' input then there exists y s.t. $V(x,y) = \text{True}$.
 - if x is a 'no' input then for all y , $V(x,y) = \text{False}$
- **Reduction:** Given x , output a boolean formula $f(x)$ such that
 - if x is a 'yes' input then $f(x)$ has a satisfying assignment
 - if x is a 'no' input then $f(x)$ does not have a satisfying assignment
- Proof y encoded as Boolean variables.
- Each step of algorithm V will be converted to a Boolean constraint.

Any problem Q in NP reduces to SAT

- Algorithm V : Input $(1,0,1,0,0,\dots, y_1, y_2, \dots, y_m)$
- Say it uses p bits memory and time T .
- Create another pT Boolean variables.
- At time t , an instruction will apply AND/OR/NOT on some memory locations and store it in another location
- $$Z_{t+1,5} = Z_{t,3} \vee Z_{t,9}$$
- $f(x) =$ AND of all such Boolean constraints.
- $f(x)$ has a satisfying assignment (y,z)
if and only if algorithm V outputs True on input $(x, y_1, y_2, \dots, y_m)$
if and only if x is a yes input.

IND-SET decision

- **IND-SET (OPT)**: given a graph G , find the largest independent set of vertices.
- **IND-SET (decision)**: given a graph G , and a number k , is there an independent set of size k ?
- Reduction from **IND-SET (OPT)** to **IND-SET (decision)**
 - first find the largest size of an independent set
 - start with $k \leftarrow n$
 - keep decreasing k till G has no independent set of size $k-1$

IND-SET optimization and decision

- Reduction from **IND-SET (OPT)** to **IND-SET (decision)**
 - first find the largest size of an independent set, say it is k
 - check whether $G - v_1$ has an independent set of size k
 - if yes, recursively find an independent set of size k in $G - v_1$
 - if no, then recursively find an independent set of size $k-1$ in $(G - \text{neighbors}(v_1))$ and include v_1 with it.

IND-SET is NP-complete

- (1) IND-SET is in NP
 - the proof will be an independent set of size k
 - the verifier will check if it is indeed an independent set and has size k
- (2) IND-SET is in NP-hard
 - That is, any problem in NP reduces to IND-SET
 - We already know that any problem in NP reduces to SAT
 - We will show that SAT reduces to IND-SET
 - From transitivity of reductions, it will follow that IND-SET is NP-hard
 - Step 1 (homework): SAT reduces to 3-SAT (given formula is 3-CNF)

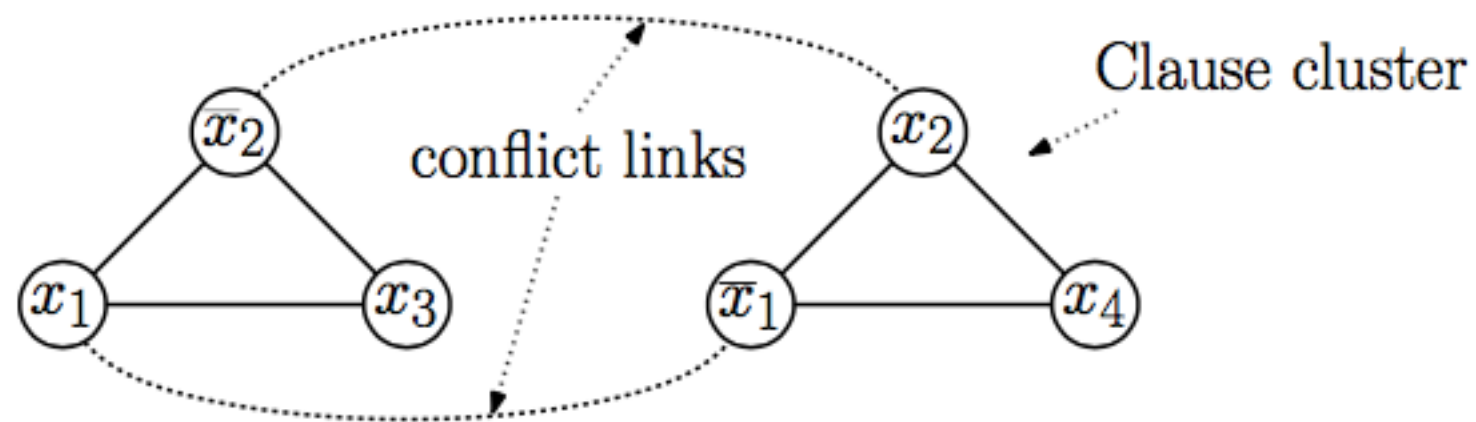
3-SAT to IND-SET Reduction

- IND-SET: given a graph G and a number k , is there an independent set of size k ?
- **Reduction:** Given a 3-CNF formula ϕ , we want to construct a graph $G(\phi)$ and a number $k(\phi)$ such that
 - if ϕ has a **satisfying assignment**, then $G(\phi)$ **has an independent set** of size $k(\phi)$
 - if ϕ does not have a **satisfying assignment**, then $G(\phi)$ **does not have any independent set** of size $k(\phi)$
 - Construction of $G(\phi)$:
 - for every clause, introduce a triangle with one vertex corresponding to each literal in the clause
 - add an edge between two vertices across triangles if one corresponds to x_i and the other corresponds to $\neg x_i$
 - $k(\phi) = \text{number of clauses} = \text{number of triangles}$

SAT to IND-SET Reduction

- Example:
- $\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$

$G(\phi) =$



- $k(\phi) = 2$

Proof of correctness ($\Rightarrow$)

- Suppose ϕ is satisfiable, choose one satisfying assignment.
- From each clause, pick an arbitrary literal which is set to true.
- Pick vertices corresponding to the picked literals.
- The number of vertices picked is equal to the number of clauses.
- The picked vertices form an independent set because
 - 1) we pick one vertex from each triangle
 - 2) if vertex corresponding to literal x_i is picked then its neighbors correspond to $\neg x_i$, and hence will not be picked.

Proof of correctness ($\Leftarrow$)

- Suppose $G(\phi)$ has an independent set of size $k(\phi)$.
- Since $k(\phi)$ = number of triangles, the independent set must have one vertex from each triangle.
- For each vertex in the independent set, set corresponding literal true.
- This is possible because the independent set cannot have vertices corresponding to both x_i and $\neg x_i$
- Set the remaining variables (if any) arbitrarily.
- This is a satisfying assignment ϕ for because for every clause at least one literal is set true.

Homework

- Easy reductions:
 - IND-SET reduces to VERTEX-COVER
 - IND-SET reduces to CLIQUE
 - VERTEX-COVER reduces to SET-COVER
 - SAT reduces to INTEGER LINEAR PROG
- Not so easy:
 - k-CLIQUE reduces to k-COLORING
 - SAT reduces to DIRECTED HAMILTONIAN CYCLE

Thank you

References

- [1] <https://math.stackexchange.com/questions/3141500/are-these-two-graphs-isomorphic-why-why-not>
- [2] <http://electronics-course.com/logic-gates>

CS261: A Second Course in Algorithms

Lecture #7: Linear Programming: Introduction and Applications*

Tim Roughgarden[†]

January 26, 2016

1 Preamble

With this lecture we commence the second part of the course, on *linear programming*, with an emphasis on applications on duality theory.¹ We'll spend a fair amount of quality time with linear programs for two reasons.

First, linear programming is very useful algorithmically, both for proving theorems and for solving real-world problems.

Linear programming is a remarkable sweet spot between power/generality and computational efficiency.

For example, all of the problems studied in previous lectures can be viewed as special cases of linear programming, and there are also zillions of other examples. Despite this generality, linear programs can be solved efficiently, both in theory (meaning in polynomial time) and in practice (with input sizes up into the millions).

Even when a computational problem that you care about does not reduce directly to solving a linear program, linear programming is an extremely helpful subroutine to have in your pocket. For example, in the fourth and last part of the course, we'll design approximation algorithms for *NP*-hard problems that use linear programming in the algorithm and/or analysis. In practice, probably most of the cycles spent on solving linear programs is in service of solving integer programs (which are generally *NP*-hard). State-of-the-art

*©2016, Tim Roughgarden.

[†]Department of Computer Science, Stanford University, 474 Gates Building, 353 Serra Mall, Stanford, CA 94305. Email: tim@cs.stanford.edu.

¹The term “programming” here is not meant in the same sense as computer programming (linear programming pre-dates modern computers). It's in the same spirit as “television programming,” meaning assembling a scheduled of planned activities. (See also “dynamic programming”.)

algorithms for the latter problem invoke a linear programming solver over and over again to make consistent progress.

Second, linear programming is conceptually useful — understanding it, and especially LP duality, gives you the “right way” to think about a host of different problems in a simple and consistent way. For example, the optimality conditions we’ve studied in past lectures (like the max-flow/min-cut theorem and Hall’s theorem) can be viewed as special cases of linear programming duality. LP duality is more or less the ultimate answer to the question “how do we know when we’re done?” As such, it’s extremely useful for proving that an algorithm is correct (or approximately correct).

We’ll talk about both these aspects of linear programming at length.

2 How to Think About Linear Programming

2.1 Comparison to Systems of Linear Equations

Once upon a time, in some course you may have forgotten, you learned about linear systems of equations. Such a system consists of m linear equations in real-valued variables $x_1, \dots, x_n$:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m. \end{aligned}$$

The a_{ij} ’s and the b_i ’s are given; the goal is to check whether or not there are values for the x_j ’s such that all m constraints are satisfied. You learned at some point that this problem can be solved efficiently, for example by Gaussian elimination. By “solved” we mean that the algorithm returns a feasible solution, or correctly reports that no feasible solution exists.

Here’s an issue, though: what about inequalities? For example, recall the maximum flow problem. There are conservation constraints, which are equations and hence OK. But the capacity constraints are fundamentally inequalities. (There is also the constraint that flow values should be nonnegative.) Inequalities are part of the problem description of many other problems that we’d like to solve. The point of linear programming is to solve systems of linear equations *and inequalities*. Moreover, when there are multiple feasible solutions, we would like to compute the “best” one.

2.2 Ingredients of a Linear Program

There is a convenient and flexible language for specifying linear programs, and we’ll get lots of practice using it during this lecture. Sometimes it’s easy to translate a computational problem into this language, sometimes it takes some tricks (we’ll see examples of both).

To specify a linear program, you need to declare what’s allowed and what you want.

Ingredients of a Linear Program

1. *Decision variables* $x_1, \dots, x_n \in \mathbb{R}$.
2. *Linear constraints*, each of the form

$$\sum_{j=1}^n a_j x_j \quad (*) \quad b_i,$$

where $(*)$ could be $\leq$, $\geq$, or $=$.

3. A *linear objective function*, of the form

$$\max \sum_{j=1}^n c_j x_j$$

or

$$\min \sum_{j=1}^n c_j x_j.$$

Several comments. First, the a_{ij} 's, b_i 's, and c_j 's are *constants*, meaning they are part of the input, numbers hard-wired into the linear program (like 5, -1, 10, etc.). The x_j 's are free, and it is the job of a linear programming algorithm to figure out the best values for them. Second, when specifying constraints, there is no need to make use of both " $\leq$ " and " $\geq$ " inequalities — one can be transformed into the other just by multiplying all the coefficients by -1 (the a_{ij} 's and b_i 's are allowed to be positive or negative). Similarly, equality constraints are superfluous, in that the constraint that a quantity equals b_i is equivalent to the pair of inequality constraints stating that the quantity is both at least b_i and at most b_i . Finally, there is also no difference between the "min" and "max" cases for the objective function — one is easily converted into the other just by multiplying all the c_j 's by -1 (the c_j 's are allowed to be positive or negative).

So what's not allowed in a linear program? Terms like x_j^2 , $x_j x_k$, $\log(1 + x_j)$, etc. So whenever a decision variable appears in an expression, it is alone, possibly multiplied by a constant (and then summed with other such terms). While these linearity requirements may seem restrictive, we'll see that many real-world problems can be formulated as or well approximated by linear programs.

2.3 A Simple Example

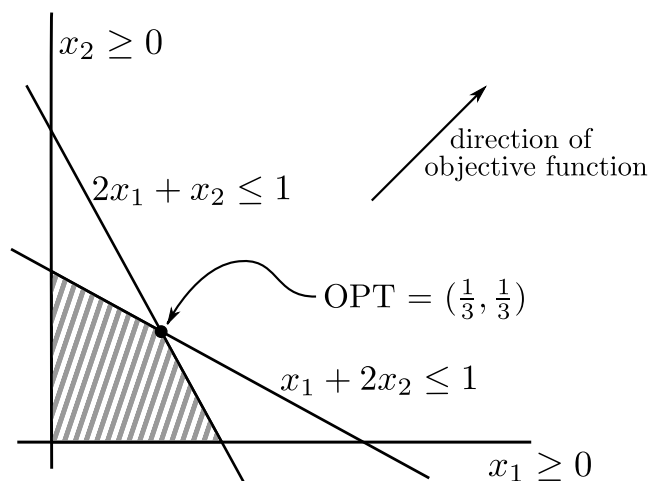


Figure 1: a toy example of linear program.

To make linear programs more concrete and develop your geometric intuition about them, let's look at a toy example. (Many “real” examples of linear programs are coming shortly.) Suppose there are two decision variables x_1 and x_2 — so we can visualize solutions as points (x_1, x_2) in the plane. See Figure 2.3. Let's consider the (linear) objective function of maximizing the sum of the decision variables:

$$\max x_1 + x_2.$$

We'll look at four (linear) constraints:

$$\begin{aligned} x_1 &\geq 0 \\ x_2 &\geq 0 \\ 2x_1 + x_2 &\leq 1 \\ x_1 + 2x_2 &\leq 1. \end{aligned}$$

The first two inequalities restrict feasible solutions to the non-negative quadrant of the plane. The second two inequalities further restrict feasible solutions to lie in the shaded region depicted in Figure 2.3. Geometrically, the objective function asks for the feasible point furthest in the direction of the coefficient vector $(1, 1)$ — the “most northeastern” feasible point. Put differently, the level sets of the objective function are parallel lines running northwest to southeast.² Eyeballing the feasible region, this point is $(\frac{1}{3}, \frac{1}{3})$, for an optimal objective function value of $\frac{2}{3}$. This is the “last point of intersection” between a level set of the objective function and the feasible region (as one sweeps from southwest to northeast).

²Recall that a *level set* of a function g has the form $\{\mathbf{x} : g(\mathbf{x}) = c\}$, for some constant c . That is, all points in a level set have equal objective function value.

2.4 Geometric Intuition

While it's always dangerous to extrapolate from two or three dimensions to an arbitrary number, the geometric intuition above remains valid for general linear programs, with an arbitrary number of dimensions (i.e., decision variables) and constraints. Even though we can't draw pictures when there are many dimensions, the relevant algebra carries over without any difficulties. Specifically:

1. A linear constraint in n dimensions corresponds to a halfspace in $\mathbb{R}^n$. Thus a feasible region is an intersection of halfspaces, the higher-dimensional analog of a polygon.³
2. The level sets of the objective function are parallel $(n - 1)$ -dimensional hyperplanes in $\mathbb{R}^n$, each orthogonal to the coefficient vector $\mathbf{c}$ of the objective function.
3. The optimal solution is the feasible point furthest in the direction of $\mathbf{c}$ (for a maximization problem) or $-\mathbf{c}$ (for a minimization problem). Equivalently, it is the last point of intersection (traveling in the direction $\mathbf{c}$ or $-\mathbf{c}$) of a level set of the objective function and the feasible region.
4. When there is a unique optimal solution, it is a vertex (i.e., “corner”) of the feasible region.

There are a few edge cases which can occur but are not especially important in CS261.

1. There might be no feasible solutions at all. For example, if we add the constraint $x_1 + x_2 \geq 1$ to our toy example, then there are no longer any feasible solutions. Linear programming algorithms correctly detect when this case occurs.
2. The optimal objective function value is unbounded ($+\infty$ for a maximization problem, $-\infty$ for a minimization problem). Note a necessary but not sufficient condition for this case is that the feasible region is unbounded. For example, if we dropped the constraints $2x_1 + x_2 \leq 1$ and $x_1 + 2x_2 \leq 1$ from our toy example, then it would have unbounded objective function value. Again, linear programming algorithms correctly detect when this case occurs.
3. The optimal solution need not be unique, as a “side” of the feasible region might be parallel to the levels sets of the objective function. Whenever the feasible region is bounded, however, there always exists an optimal solution that is a vertex of the feasible region.⁴

³A finite intersection of halfspaces is also called a “polyhedron;” in the common special case where the feasible region is bounded, it is called a “polytope.”

⁴There are some annoying edge cases for unbounded feasible regions, for example the linear program $\max(x_1 + x_2)$ subject to $x_1 + x_2 = 1$.

3 Some Applications of Linear Programming

Zillions of problems reduce to linear programming. It would take an entire course to cover even just its most famous applications. Some of these applications are conceptually a bit boring but still very important — as early as the 1940s, the military was using linear programming to figure out the most efficient way to ship supplies from factories to where they were needed.⁵ Several central problems in computer science reduce to linear programming, and we describe some of these in detail in this section. Throughout, keep in mind that all of these linear programs can be solved efficiently, both in theory and in practice. We'll say more about algorithms for linear programming in a later lecture.

3.1 Maximum Flow

If we return to the definition of the maximum flow problem in Lecture #1, we see that it translates quite directly to a linear program.

1. *Decision variables:* what are we try to solve for? A flow, of course, Specifically, the amount f_e of flow on each edge e . So our variables are just $\{f_e\}_{e \in E}$.
2. *Constraints:* Recall we have conservation constraints and capacity constraints. We can write the former as

$$\underbrace{\sum_{e \in \delta^-(v)} f_e}_{\text{flow in}} - \underbrace{\sum_{e \in \delta^+(v)} f_e}_{\text{flow out}} = 0$$

for every vertex $v \neq s, t$.⁶ We can write the latter as

$$f_e \leq u_e$$

for every edge $e \in E$. Since decision variables of linear programs are by default allowed to take on arbitrary real values (positive or negative), we also need to remember to add nonnegativity constraints:

$$f_e \geq 0$$

for every edge $e \in E$. Observe that every one of these $2m + n - 2$ constraints (where $m = |E|$ and $n = |V|$) is linear — each decision variable f_e only appears by itself (with a coefficient of 1 or -1).

3. *Objective function:* We just copy the same one we used in Lecture #1:

$$\max \sum_{e \in \delta^+(s)} f_e.$$

Note that this is again a linear function.

⁵Note this is well before computer science was field; for example, Stanford's Computer Science Department was founded only in 1965.

⁶Recall that δ^- and δ^+ denote the edges incoming to and outgoing from v , respectively.

3.2 Minimum-Cost Flow

In Lecture #6 we introduced the minimum-cost flow problem. Extending specialized algorithms for maximum flow to generalized algorithms takes non-trivial work (see Problem Set #2 for starters). If we're just using linear programming, however, the generalization is immediate.⁷ The main change is in the objective function. As defined last lecture, it is simply

$$\min \sum_{e \in E} c_e f_e,$$

where c_e is the cost of edge e . Since the c_e 's are fixed numbers (i.e., part of the input), this is a linear objective function.

For the version of the minimum-cost flow problem defined last lecture, we should also add the constraint

$$\sum_{e \in \delta^+(s)} f_e = d,$$

where d is the target flow value. (One can also add the analogous constraint for t , but this is already implied by the other constraints.)

To further highlight how flexible linear programs can be, suppose we want to impose a lower bound ℓ_e (other than 0) on the amount of flow on each edge e , in addition to the usual upper bound u_e . This is trivial to accommodate in our linear program — just replace “ $f_e \geq 0$ ” by $f_e \geq \ell_e$.⁸

3.3 Fitting a Line

We now consider two less obvious applications of linear programming, to basic problems in machine learning. We first consider the problem of fitting a line to data points (i.e., linear regression), perhaps the simplest non-trivial machine learning problem.

Formally, the input consists of m data points $\mathbf{p}^1, \dots, \mathbf{p}^m \in \mathbb{R}^d$, each with d real-valued “features” (i.e., coordinates).⁹ For example, perhaps $d = 3$, and each data point corresponds to a 3rd-grader, listing the household income, number of owned books, and number of years of parental education. Also part of the input is a “label” $\ell_i \in \mathbb{R}$ for each point $\mathbf{p}^i$.¹⁰ For example, ℓ_i could be the score earned by the 3rd-grader in question on a standardized test. We reiterate that the $\mathbf{p}^i$'s and ℓ_i 's are fixed (part of the input), not decision variables.

⁷While linear programming is a reasonable way to solve the maximum flow and minimum-cost flow problems, especially if the goal is to have a “quick and dirty” solution, but the best specialized algorithms for these problems are generally faster.

⁸If you prefer to use flow algorithms, there is a simple reduction from this problem to the special case with $\ell_e = 0$ for all $e \in E$ (do you see it?).

⁹Feel free to take $d = 1$ throughout the rest of the lecture, which is already a practically relevant and computationally interesting case.

¹⁰This is a canonical “supervised learning” problem, meaning that the algorithm is provided with labeled data.

Informally, the goal is to express the ℓ_i as well as possible as a linear function of the $\mathbf{p}_i$'s. That is, the goal is to compute a linear function $h : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $h(\mathbf{p}^i) \approx \ell_i$ for every data point i .

The two most common motivations for computing a “best-fit” linear function are prediction and data analysis. In the first scenario, one uses labeled data to identify a linear function h that, at least for these data points, does a good job of predicting the label ℓ_i from the feature values $\mathbf{p}^i$. The hope is that this linear function “generalizes,” meaning that it also makes accurate predictions for other data points for which the label is not already known. There is a lot of beautiful and useful theory in statistics and machine learning about when one can and cannot expect a hypothesis to generalize, which you’ll learn about if you take courses in those areas. In the second scenario, the goal is to understand the relationship between each feature of the data points and the labels, and also the relationships between the different features. As a simple example, it’s clearly interesting to know when one of the d features is much more strongly correlated with the label ℓ^i than any of the others.

We now show that computing the best line, for one definition of “best,” reduces to linear programming. Recall that every linear function $h : \mathbb{R}^d \rightarrow \mathbb{R}$ has the form

$$h(\mathbf{z}) = \sum_{j=1}^d a_j z_j + b$$

for some coefficients $a_1, \dots, a_d$ and intercept b . (This is one of several equivalent definitions of a linear function.¹¹ So it’s natural to take $a_1, \dots, a_d, b$ as our decision variables.

What’s our objective function? Clearly if the data points are colinear we want to compute the line that passes through all of them. But this will never happen, so we must compromise between how well we approximate different points.

For a given choice of $a_1, \dots, a_d, b$, define the error on point i as

$$E_i(\mathbf{a}, b) = \left| \underbrace{\left(\sum_{j=1}^d a_j p_j^i - b \right)}_{\text{prediction}} - \underbrace{\ell^i}_{\text{“ground truth”}} \right|. \quad (1)$$

Geometrically, when $d = 1$, we can think of each $(\mathbf{p}^i, \ell^i)$ as a point in the plane and (1) is just the vertical distance between this point and the computed line.

In this lecture, we consider the objective function of minimizing the sum of errors:

$$\min_{\mathbf{a}, b} \sum_{i=1}^m E_i(\mathbf{a}, b). \quad (2)$$

This is not the most common objective for linear regression; more standard is minimizing the squared error $\sum_{i=1}^m E_i^2(\mathbf{a}, b)$. While our motivation for choosing (2) is primarily pedagogical,

¹¹Sometimes people use “linear function” to mean the special case where $b = 0$, and “affine function” for the case of arbitrary b .

this objective is reasonable and is sometimes used in practice. The advantage over squared error is that it is more robust to outliers. Squaring the error of an outlier makes it a squeakier wheel. That is, a stray point (e.g., a faulty sensor or data entry error) will influence the line chosen under (2) less than it would with the squared error objective (Figure 2).¹²

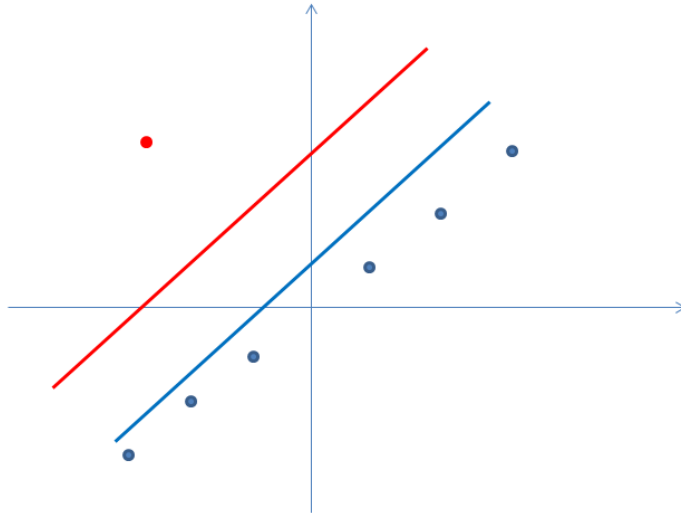


Figure 2: When there exists an outlier (red point), using the objective function defined in (2) causes the best-fit line not to "stray" as far away from the non-outliers (blue line) as when using the squared error objective (red line), because the squared error objective would penalize more greatly when the chosen line is far from the outlier.

Consider the problem of choosing $\mathbf{a}, b$ to minimize (2). (Since the a_j 's and b can be anything, there are no constraints.) The problem: *this is not a linear program*. The source of nonlinearity is the absolute value sign $|\cdot|$ in (1). Happily, in this case and many others, absolute values can be made linear with a simple trick.

The trick is to introduce extra variables $e_1, \dots, e_m$, one per data point. The intent is for e_i to take on the value $E_i(\mathbf{a}, b)$. Motivated by the identity $|x| = \max\{x, -x\}$, we add two constraints for each data point:

$$e_i \geq \left(\sum_{j=1}^d a_j p_j^i - b \right) - \ell^i \quad (3)$$

and

$$e_i \geq - \left[\left(\sum_{j=1}^d a_j p_j^i - b \right) - \ell^i \right]. \quad (4)$$

¹²Squared error can be minimized efficiently using an extension of linear programming known as *convex programming*. (For the present "ordinary least squares" version of the problem, it can even be solved analytically, in closed form.) We may discuss convex programming in a future lecture.

We change the objective function to

$$\min \sum_{i=1}^m e_i. \quad (5)$$

Note that optimizing (5) subject to all constraints of the form (3) and (4) is a linear program, with decision variables $e_1, \dots, e_m, a_1, \dots, a_d, b$.

The key point is: at an optimal solution to this linear program, it must be that $e_i = E_i(\mathbf{a}, b)$ for every data point i . Feasibility of the solution already implies that $e_i \geq E_i(\mathbf{a}, b)$ for every i . And if $e_i > E_i(\mathbf{a}, b)$ for some i , then we can decrease e_i slightly, so that (3) and (4) still hold, to obtain a superior feasible solution. We conclude that an optimal solution to this linear program represents the line minimizing the sum of errors (2).

3.4 Computing a Linear Classifier

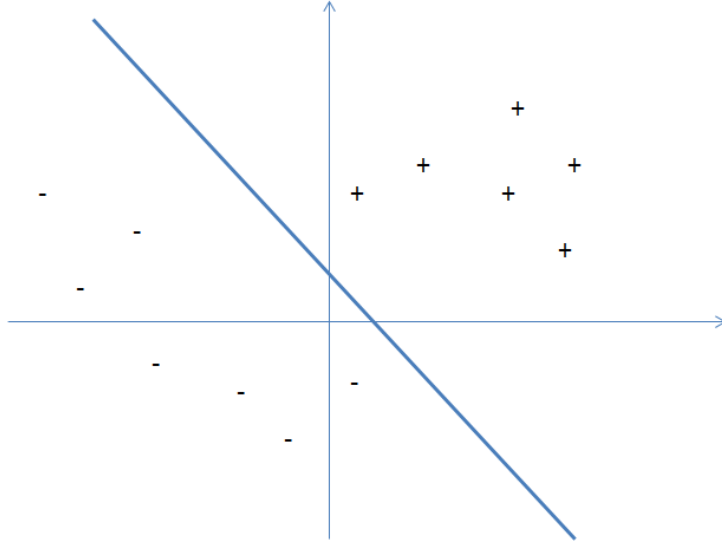


Figure 3: We want to find a linear function that separates the positive points (plus signs) from the negative points (minus signs)

Next we consider a second fundamental problem in machine learning, that of learning a linear classifier.¹³ While in Section 3.3 we sought a real-valued function (from $\mathbb{R}^d$ to $\mathbb{R}$), here we're looking for a binary function (from $\mathbb{R}^d$ to $\{0, 1\}$). For example, data points could represent images, and we want to know which ones contain a cat and which ones don't.

Formally, the input consists of m “positive” data points $\mathbf{p}^1, \dots, \mathbf{p}^m \in \mathbb{R}^d$ and m' “negative” data points $\mathbf{q}^1, \dots, \mathbf{q}^{m'} \in \mathbb{R}^d$. In the terminology of the previous section, all of the labels

¹³Also called halfspaces, perceptrons, linear threshold functions, etc.

are “1” or “0,” and we have partitioned the data accordingly. (So this is again a supervised learning problem.)

The goal is to compute a linear function $h(\mathbf{z}) = \sum_{j=1} a_j z_j + b$ (from $\mathbb{R}^d$ to $\mathbb{R}$) such that

$$h(\mathbf{p}^i) > 0 \quad (6)$$

for all positive points and

$$h(\mathbf{q}^i) < 0 \quad (7)$$

for all negative points. Geometrically, we are looking for a hyperplane in $\mathbb{R}^d$ such all positive points are on one side and all negative points on the other; the coefficients $\mathbf{a}$ specify the normal vector of the hyperplane and the intercept b specifies its shift. See Figure 3. Such a hyperplane can be used for predicting the labels of other, unlabeled points (check which side of the hyperplane it is on and predict that it is positive or negative, accordingly). If there is no such hyperplane, an algorithm should correctly report this fact.

This problem almost looks like a linear program by definition. The only issue is that the constraints (6) and (7) are strict inequalities, which are not allowed in linear programs. Again, the simple trick of adding an extra decision variable solves the problem. The new decision variable δ represents the “margin” by which the hyperplane satisfies (6) and (7). So we

$$\max \delta$$

subject to

$$\begin{aligned} \sum_{j=1}^d a_j p_j^i + b - \delta &\geq 0 && \text{for all positive points } \mathbf{p}^i \\ \sum_{j=1}^d a_j q_j^i + b + \delta &\leq 0 && \text{for all negative points } \mathbf{q}^i, \end{aligned}$$

which is a linear program with decision variables $\delta, a_1, \dots, a_d, b$. If the optimal solution to this linear program has strictly positive objective function value, then the values of the variables $a_1, \dots, a_d, b$ define the desired separating hyperplane. If not, then there is no such hyperplane. We conclude that computing a linear classifier reduces to linear programming.

3.5 Extension: Minimizing Hinge Loss

There is an obvious issue with the problem setup in Section 3.4: what if the data set is not as nice as the picture in Figure 3, and there is no separating hyperplane? This is usually the case in practice, for example if the data is noisy (as it always is). Even if there’s no perfect hyperplane, we’d still like to compute something that we can use to predict the labels of unlabeled points.

We outline two ways to extend the linear programming approach in Section 3.4 to handle non-separable data.¹⁴ The first idea is to compute the hyperplane that minimizes some notion

¹⁴In practice, these two approaches are often combined.

of “classification error.” After all, this is what we did in Section 3.3, where we computed the line minimizing the sum of the errors.

Probably the most natural plan would be to compute the hyperplane that puts the fewest number of points on the wrong side of the hyperplane — to minimize the number of inequalities of the form (6) or (7) that are violated. Unfortunately, this is an *NP*-hard problem, and one typically uses notions of error that are more computationally tractable. Here, we’ll discuss the widely used notion of *hinge loss*.

Let’s say that in a perfect world, we would like a linear function h such that

$$h(\mathbf{p}^i) \geq 1 \tag{8}$$

for all positive points $\mathbf{p}^i$ and

$$h(\mathbf{q}^i) \leq -1 \tag{9}$$

for all negative points $\mathbf{q}^i$; the “1” here is somewhat arbitrary, but we need to pick some constant for the purposes of normalization. The *hinge loss* incurred by a linear function h on a point is just the extent to which the corresponding inequality (8) or (9) fails to hold. For a positive point $\mathbf{p}^i$, this is $\max\{1 - h(\mathbf{p}^i), 0\}$; for a negative point $\mathbf{q}^i$, this is $\max\{1 + h(\mathbf{q}^i), 0\}$. Note that taking the maximum with zero ensures that we don’t reward a linear function for classifying a point “extra-correctly.” Geometrically, when $d = 1$, the hinge loss is the vertical distance that a data point would have to travel to be on the correct side of the hyperplane, with a “buffer” of 1 between the point and the hyperplane.

Computing the linear function that minimizes the total hinge loss can be formulated as a linear program. While hinge loss is not linear, it is just the maximum of two linear functions. So by introducing one extra variable and two extra constraints per data point, just like in Section 3.3, we obtain the linear program

$$\min \sum_{i=1}^m e_i$$

subject to:

$$\begin{aligned} e_i &\geq 1 - \left(\sum_{j=1}^d a_j p_j^i + b \right) && \text{for every positive point } \mathbf{p}^i \\ e_i &\geq 1 + \left(\sum_{j=1}^d a_j q_j^i + b \right) && \text{for every negative point } \mathbf{q}^i \\ e_i &\geq 0 && \text{for every point} \end{aligned}$$

in the decision variables $e_1, \dots, e_m, a_1, \dots, a_d, b$.

3.6 Extension: Increasing the Dimension

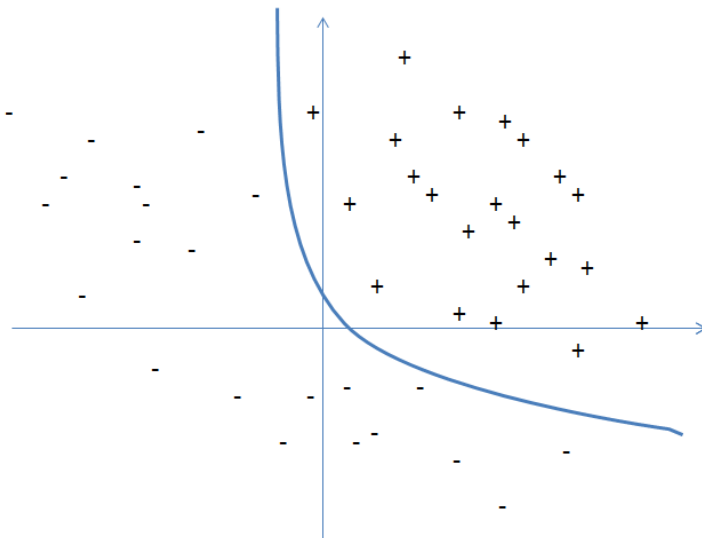


Figure 4: The points are not linearly separable, but they can be separated by a quadratic line.

A second approach to dealing with non-linearly-separable data is to use nonlinear boundaries. E.g., in Figure 4, the positive and negative points cannot be separated perfectly by any line, but they can be separated by a relatively simple boundary (e.g., of a quadratic function). But how we can allow nonlinear boundaries while retaining the computational tractability of our previous solutions?

The key idea is to generate extra features (i.e., dimensions) for each data point. That is, for some dimension $d' \geq d$ and some function $\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$, we map each $\mathbf{p}^i$ to $\varphi(\mathbf{p}^i)$ and each $\mathbf{q}_i$ to $\varphi(\mathbf{q}^i)$. We'll then try to separate the images of these points in d' -dimensional space using a linear function.¹⁵

A concrete example of such a function φ is the map

$$(z_1, \dots, z_d) \mapsto (z_1, \dots, z_d, z_1^2, \dots, z_d^2, z_1 z_2, z_1 z_3, \dots, z_{d-1} z_d); \quad (10)$$

that is, each data point is expanded with all of the pairwise products of its features. This map is interesting even when $d = 1$:

$$z \mapsto (z, z^2). \quad (11)$$

Our goal is now to compute a linear function in the expanded space, meaning coefficients

¹⁵This is the basic idea behind “support vector machines;” see CS229 for much more on the topic.

$a_1, \dots, a_{d'}$ and an intercept b , that separates the positive and negative points:

$$\sum_{i=1}^{d'} a_j \cdot \varphi(\mathbf{p}^i)_j + b > 0 \quad (12)$$

for all positive points and

$$\sum_{i=1}^{d'} a_j \cdot \varphi(\mathbf{q}^i)_j + b < 0 \quad (13)$$

for all negative points. Note that if the new feature set includes all of the original features, as in (10), then every hyperplane in the original d -dimensional space remains available in the expanded space (just set $a_{d+1}, a_{d+2}, \dots, a_{d'} = 0$). But there are also many new options, and hence it is more likely that there is way to perfectly separate the (images under φ of the) data points. For example, even with $d = 1$ and the map (11), linear functions in the expanded space have the form $h(z) = a_1 z^2 + a_2 z + b$, which is a quadratic function in the original space.

We can think of the map φ as being applied in a preprocessing step. Then, the resulting problem of meeting all the constraints (12) and (13) is exactly the problem that we already solved in Section 3.4. The resulting linear program has decision variables $\delta, a_1, \dots, a_{d'}, b$ ($d' + 2$ in all, up from $d + 2$ in the original space).¹⁶

¹⁶The magic of support vector machines is that, for many maps φ including (10) and (11), and for many methods of computing a separating hyperplane, the computation required scales only with the original dimension d , even if the expanded dimension d' is radically larger. This is known as the “kernel trick;” see CS229 for more details.

Many problems take the form of maximizing or minimizing an objective, given limited resources and competing constraints. If we can specify the objective as a linear function of certain variables, and if we can specify the constraints on resources as equalities or inequalities on those variables, then we have a ***linear-programming problem***. Linear programs arise in a variety of practical applications. We begin by studying an application in electoral politics.

A political problem

Suppose that you are a politician trying to win an election. Your district has three different types of areas—urban, suburban, and rural. These areas have, respectively, 100,000, 200,000, and 50,000 registered voters. Although not all the registered voters actually go to the polls, you decide that to govern effectively, you would like at least half the registered voters in each of the three regions to vote for you. You are honorable and would never consider supporting policies in which you do not believe. You realize, however, that certain issues may be more effective in winning votes in certain places. Your primary issues are building more roads, gun control, farm subsidies, and a gasoline tax dedicated to improved public transit. According to your campaign staff's research, you can estimate how many votes you win or lose from each population segment by spending \$1,000 on advertising on each issue. This information appears in the table of Figure 29.1. In this table, each entry indicates the number of thousands of either urban, suburban, or rural voters who would be won over by spending \$1,000 on advertising in support of a particular issue. Negative entries denote votes that would be lost. Your task is to figure out the minimum amount of money that you need to spend in order to win 50,000 urban votes, 100,000 suburban votes, and 25,000 rural votes.

You could, by trial and error, devise a strategy that wins the required number of votes, but the strategy you come up with might not be the least expensive one. For example, you could devote \$20,000 of advertising to building roads, \$0 to gun control, \$4,000 to farm subsidies, and \$9,000 to a gasoline tax. In this case, you

policy	urban	suburban	rural
build roads	-2	5	3
gun control	8	2	-5
farm subsidies	0	0	10
gasoline tax	10	0	-2

Figure 29.1 The effects of policies on voters. Each entry describes the number of thousands of urban, suburban, or rural voters who could be won over by spending \$1,000 on advertising support of a policy on a particular issue. Negative entries denote votes that would be lost.

would win $20(-2) + 0(8) + 4(0) + 9(10) = 50$ thousand urban votes, $20(5) + 0(2) + 4(0) + 9(0) = 100$ thousand suburban votes, and $20(3) + 0(-5) + 4(10) + 9(-2) = 82$ thousand rural votes. You would win the exact number of votes desired in the urban and suburban areas and more than enough votes in the rural area. (In fact, in the rural area, you would receive more votes than there are voters.) In order to garner these votes, you would have paid for $20 + 0 + 4 + 9 = 33$ thousand dollars of advertising.

Naturally, you may wonder whether this strategy is the best possible. That is, could you achieve your goals while spending less on advertising? Additional trial and error might help you to answer this question, but wouldn't you rather have a systematic method for answering such questions? In order to develop one, we shall formulate this question mathematically. We introduce 4 variables:

- x_1 is the number of thousands of dollars spent on advertising on building roads,
- x_2 is the number of thousands of dollars spent on advertising on gun control,
- x_3 is the number of thousands of dollars spent on advertising on farm subsidies, and
- x_4 is the number of thousands of dollars spent on advertising on a gasoline tax.

We can write the requirement that we win at least 50,000 urban votes as

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50. \quad (29.1)$$

Similarly, we can write the requirements that we win at least 100,000 suburban votes and 25,000 rural votes as

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.2)$$

and

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25. \quad (29.3)$$

Any setting of the variables x_1, x_2, x_3, x_4 that satisfies inequalities (29.1)–(29.3) yields a strategy that wins a sufficient number of each type of vote. In order to

keep costs as small as possible, you would like to minimize the amount spent on advertising. That is, you want to minimize the expression

$$x_1 + x_2 + x_3 + x_4 . \quad (29.4)$$

Although negative advertising often occurs in political campaigns, there is no such thing as negative-cost advertising. Consequently, we require that

$$x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, \text{ and } x_4 \geq 0 . \quad (29.5)$$

Combining inequalities (29.1)–(29.3) and (29.5) with the objective of minimizing (29.4), we obtain what is known as a “linear program.” We format this problem as

$$\text{minimize} \quad x_1 + x_2 + x_3 + x_4 \quad (29.6)$$

subject to

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 \quad (29.7)$$

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.8)$$

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 \quad (29.9)$$

$$x_1, x_2, x_3, x_4 \geq 0 . \quad (29.10)$$

The solution of this linear program yields your optimal strategy.

General linear programs

In the general linear-programming problem, we wish to optimize a linear function subject to a set of linear inequalities. Given a set of real numbers $a_1, a_2, \dots, a_n$ and a set of variables $x_1, x_2, \dots, x_n$, we define a **linear function** f on those variables by

$$f(x_1, x_2, \dots, x_n) = a_1x_1 + a_2x_2 + \cdots + a_nx_n = \sum_{j=1}^n a_jx_j .$$

If b is a real number and f is a linear function, then the equation

$$f(x_1, x_2, \dots, x_n) = b$$

is a **linear equality** and the inequalities

$$f(x_1, x_2, \dots, x_n) \leq b$$

and

$$f(x_1, x_2, \dots, x_n) \geq b$$

are **linear inequalities**. We use the general term **linear constraints** to denote either linear equalities or linear inequalities. In linear programming, we do not allow strict inequalities. Formally, a **linear-programming problem** is the problem of either minimizing or maximizing a linear function subject to a finite set of linear constraints. If we are to minimize, then we call the linear program a **minimization linear program**, and if we are to maximize, then we call the linear program a **maximization linear program**.

The remainder of this chapter covers how to formulate and solve linear programs. Although several polynomial-time algorithms for linear programming have been developed, we will not study them in this chapter. Instead, we shall study the simplex algorithm, which is the oldest linear-programming algorithm. The simplex algorithm does not run in polynomial time in the worst case, but it is fairly efficient and widely used in practice.

An overview of linear programming

In order to describe properties of and algorithms for linear programs, we find it convenient to express them in canonical forms. We shall use two forms, **standard** and **slack**, in this chapter. We will define them precisely in Section 29.1. Informally, a linear program in standard form is the maximization of a linear function subject to linear *inequalities*, whereas a linear program in slack form is the maximization of a linear function subject to linear *equalities*. We shall typically use standard form for expressing linear programs, but we find it more convenient to use slack form when we describe the details of the simplex algorithm. For now, we restrict our attention to maximizing a linear function on n variables subject to a set of m linear inequalities.

Let us first consider the following linear program with two variables:

$$\text{maximize} \quad x_1 + x_2 \tag{29.11}$$

subject to

$$4x_1 - x_2 \leq 8 \tag{29.12}$$

$$2x_1 + x_2 \leq 10 \tag{29.13}$$

$$5x_1 - 2x_2 \geq -2 \tag{29.14}$$

$$x_1, x_2 \geq 0. \tag{29.15}$$

We call any setting of the variables x_1 and x_2 that satisfies all the constraints (29.12)–(29.15) a **feasible solution** to the linear program. If we graph the constraints in the (x_1, x_2) -Cartesian coordinate system, as in Figure 29.2(a), we see

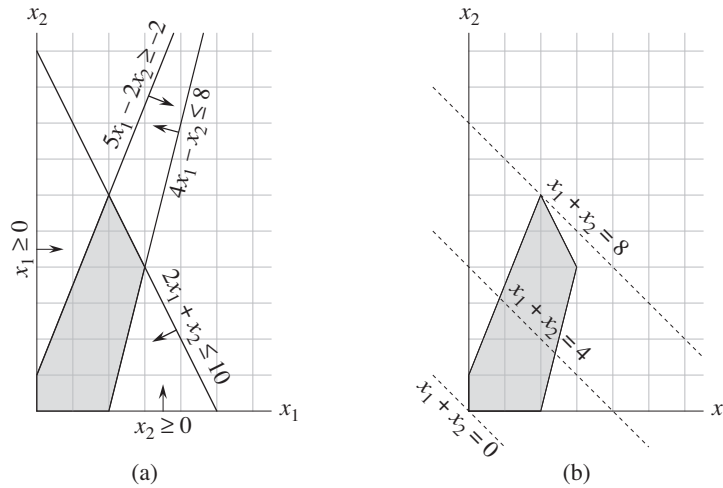


Figure 29.2 (a) The linear program given in (29.12)–(29.15). Each constraint is represented by a line and a direction. The intersection of the constraints, which is the feasible region, is shaded. (b) The dotted lines show, respectively, the points for which the objective value is 0, 4, and 8. The optimal solution to the linear program is $x_1 = 2$ and $x_2 = 6$ with objective value 8.

that the set of feasible solutions (shaded in the figure) forms a convex region¹ in the two-dimensional space. We call this convex region the **feasible region** and the function we wish to maximize the **objective function**. Conceptually, we could evaluate the objective function $x_1 + x_2$ at each point in the feasible region; we call the value of the objective function at a particular point the **objective value**. We could then identify a point that has the maximum objective value as an optimal solution. For this example (and for most linear programs), the feasible region contains an infinite number of points, and so we need to determine an efficient way to find a point that achieves the maximum objective value without explicitly evaluating the objective function at every point in the feasible region.

In two dimensions, we can optimize via a graphical procedure. The set of points for which $x_1 + x_2 = z$, for any z , is a line with a slope of -1 . If we plot $x_1 + x_2 = 0$, we obtain the line with slope -1 through the origin, as in Figure 29.2(b). The intersection of this line and the feasible region is the set of feasible solutions that have an objective value of 0. In this case, that intersection of the line with the feasible region is the single point $(0, 0)$. More generally, for any z , the intersection

¹An intuitive definition of a convex region is that it fulfills the requirement that for any two points in the region, all points on a line segment between them are also in the region.

of the line $x_1 + x_2 = z$ and the feasible region is the set of feasible solutions that have objective value z . Figure 29.2(b) shows the lines $x_1 + x_2 = 0$, $x_1 + x_2 = 4$, and $x_1 + x_2 = 8$. Because the feasible region in Figure 29.2 is bounded, there must be some maximum value z for which the intersection of the line $x_1 + x_2 = z$ and the feasible region is nonempty. Any point at which this occurs is an optimal solution to the linear program, which in this case is the point $x_1 = 2$ and $x_2 = 6$ with objective value 8.

It is no accident that an optimal solution to the linear program occurs at a vertex of the feasible region. The maximum value of z for which the line $x_1 + x_2 = z$ intersects the feasible region must be on the boundary of the feasible region, and thus the intersection of this line with the boundary of the feasible region is either a single vertex or a line segment. If the intersection is a single vertex, then there is just one optimal solution, and it is that vertex. If the intersection is a line segment, every point on that line segment must have the same objective value; in particular, both endpoints of the line segment are optimal solutions. Since each endpoint of a line segment is a vertex, there is an optimal solution at a vertex in this case as well.

Although we cannot easily graph linear programs with more than two variables, the same intuition holds. If we have three variables, then each constraint corresponds to a half-space in three-dimensional space. The intersection of these half-spaces forms the feasible region. The set of points for which the objective function obtains a given value z is now a plane (assuming no degenerate conditions). If all coefficients of the objective function are nonnegative, and if the origin is a feasible solution to the linear program, then as we move this plane away from the origin, in a direction normal to the objective function, we find points of increasing objective value. (If the origin is not feasible or if some coefficients in the objective function are negative, the intuitive picture becomes slightly more complicated.) As in two dimensions, because the feasible region is convex, the set of points that achieve the optimal objective value must include a vertex of the feasible region. Similarly, if we have n variables, each constraint defines a half-space in n -dimensional space. We call the feasible region formed by the intersection of these half-spaces a **simplex**. The objective function is now a hyperplane and, because of convexity, an optimal solution still occurs at a vertex of the simplex.

The **simplex algorithm** takes as input a linear program and returns an optimal solution. It starts at some vertex of the simplex and performs a sequence of iterations. In each iteration, it moves along an edge of the simplex from a current vertex to a neighboring vertex whose objective value is no smaller than that of the current vertex (and usually is larger.) The simplex algorithm terminates when it reaches a local maximum, which is a vertex from which all neighboring vertices have a smaller objective value. Because the feasible region is convex and the objective function is linear, this local optimum is actually a global optimum. In Section 29.4,

we shall use a concept called “duality” to show that the solution returned by the simplex algorithm is indeed optimal.

Although the geometric view gives a good intuitive view of the operations of the simplex algorithm, we shall not refer to it explicitly when developing the details of the simplex algorithm in Section 29.3. Instead, we take an algebraic view. We first write the given linear program in slack form, which is a set of linear equalities. These linear equalities express some of the variables, called “basic variables,” in terms of other variables, called “nonbasic variables.” We move from one vertex to another by making a basic variable become nonbasic and making a nonbasic variable become basic. We call this operation a “pivot” and, viewed algebraically, it is nothing more than rewriting the linear program in an equivalent slack form.

The two-variable example described above was particularly simple. We shall need to address several more details in this chapter. These issues include identifying linear programs that have no solutions, linear programs that have no finite optimal solution, and linear programs for which the origin is not a feasible solution.

Applications of linear programming

Linear programming has a large number of applications. Any textbook on operations research is filled with examples of linear programming, and linear programming has become a standard tool taught to students in most business schools. The election scenario is one typical example. Two more examples of linear programming are the following:

- An airline wishes to schedule its flight crews. The Federal Aviation Administration imposes many constraints, such as limiting the number of consecutive hours that each crew member can work and insisting that a particular crew work only on one model of aircraft during each month. The airline wants to schedule crews on all of its flights using as few crew members as possible.
- An oil company wants to decide where to drill for oil. Siting a drill at a particular location has an associated cost and, based on geological surveys, an expected payoff of some number of barrels of oil. The company has a limited budget for locating new drills and wants to maximize the amount of oil it expects to find, given this budget.

With linear programs, we also model and solve graph and combinatorial problems, such as those appearing in this textbook. We have already seen a special case of linear programming used to solve systems of difference constraints in Section 24.4. In Section 29.2, we shall study how to formulate several graph and network-flow problems as linear programs. In Section 35.4, we shall use linear programming as a tool to find an approximate solution to another graph problem.

Algorithms for linear programming

This chapter studies the simplex algorithm. This algorithm, when implemented carefully, often solves general linear programs quickly in practice. With some carefully contrived inputs, however, the simplex algorithm can require exponential time. The first polynomial-time algorithm for linear programming was the ***ellipsoid algorithm***, which runs slowly in practice. A second class of polynomial-time algorithms are known as ***interior-point methods***. In contrast to the simplex algorithm, which moves along the exterior of the feasible region and maintains a feasible solution that is a vertex of the simplex at each iteration, these algorithms move through the interior of the feasible region. The intermediate solutions, while feasible, are not necessarily vertices of the simplex, but the final solution is a vertex. For large inputs, interior-point algorithms can run as fast as, and sometimes faster than, the simplex algorithm. The chapter notes point you to more information about these algorithms.

If we add to a linear program the additional requirement that all variables take on integer values, we have an ***integer linear program***. Exercise 34.5-3 asks you to show that just finding a feasible solution to this problem is NP-hard; since no polynomial-time algorithms are known for any NP-hard problems, there is no known polynomial-time algorithm for integer linear programming. In contrast, we can solve a general linear-programming problem in polynomial time.

In this chapter, if we have a linear program with variables $x = (x_1, x_2, \dots, x_n)$ and wish to refer to a particular setting of the variables, we shall use the notation $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$.

29.1 Standard and slack forms

This section describes two formats, standard form and slack form, that are useful when we specify and work with linear programs. In standard form, all the constraints are inequalities, whereas in slack form, all constraints are equalities (except for those that require the variables to be nonnegative).

Standard form

In ***standard form***, we are given n real numbers $c_1, c_2, \dots, c_n$; m real numbers $b_1, b_2, \dots, b_m$; and mn real numbers a_{ij} for $i = 1, 2, \dots, m$ and $j = 1, 2, \dots, n$. We wish to find n real numbers $x_1, x_2, \dots, x_n$ that

$$\text{maximize} \quad \sum_{j=1}^n c_j x_j \quad (29.16)$$

subject to

$$\sum_{j=1}^n a_{ij} x_j \leq b_i \quad \text{for } i = 1, 2, \dots, m \quad (29.17)$$

$$x_j \geq 0 \quad \text{for } j = 1, 2, \dots, n. \quad (29.18)$$

Generalizing the terminology we introduced for the two-variable linear program, we call expression (29.16) the **objective function** and the $n + m$ inequalities in lines (29.17) and (29.18) the **constraints**. The n constraints in line (29.18) are the **nonnegativity constraints**. An arbitrary linear program need not have nonnegativity constraints, but standard form requires them. Sometimes we find it convenient to express a linear program in a more compact form. If we create an $m \times n$ matrix $A = (a_{ij})$, an m -vector $b = (b_i)$, an n -vector $c = (c_j)$, and an n -vector $x = (x_j)$, then we can rewrite the linear program defined in (29.16)–(29.18) as

$$\text{maximize} \quad c^T x \quad (29.19)$$

subject to

$$Ax \leq b \quad (29.20)$$

$$x \geq 0. \quad (29.21)$$

In line (29.19), $c^T x$ is the inner product of two vectors. In inequality (29.20), Ax is a matrix-vector product, and in inequality (29.21), $x \geq 0$ means that each entry of the vector x must be nonnegative. We see that we can specify a linear program in standard form by a tuple (A, b, c) , and we shall adopt the convention that A , b , and c always have the dimensions given above.

We now introduce terminology to describe solutions to linear programs. We used some of this terminology in the earlier example of a two-variable linear program. We call a setting of the variables $\bar{x}$ that satisfies all the constraints a **feasible solution**, whereas a setting of the variables $\bar{x}$ that fails to satisfy at least one constraint is an **infeasible solution**. We say that a solution $\bar{x}$ has **objective value** $c^T \bar{x}$. A feasible solution $\bar{x}$ whose objective value is maximum over all feasible solutions is an **optimal solution**, and we call its objective value $c^T \bar{x}$ the **optimal objective value**. If a linear program has no feasible solutions, we say that the linear program is **infeasible**; otherwise it is **feasible**. If a linear program has some feasible solutions but does not have a finite optimal objective value, we say that the linear program is **unbounded**. Exercise 29.1-9 asks you to show that a linear program can have a finite optimal objective value even if the feasible region is not bounded.

Converting linear programs into standard form

It is always possible to convert a linear program, given as minimizing or maximizing a linear function subject to linear constraints, into standard form. A linear program might not be in standard form for any of four possible reasons:

1. The objective function might be a minimization rather than a maximization.
2. There might be variables without nonnegativity constraints.
3. There might be **equality constraints**, which have an equal sign rather than a less-than-or-equal-to sign.
4. There might be **inequality constraints**, but instead of having a less-than-or-equal-to sign, they have a greater-than-or-equal-to sign.

When converting one linear program L into another linear program L' , we would like the property that an optimal solution to L' yields an optimal solution to L . To capture this idea, we say that two maximization linear programs L and L' are **equivalent** if for each feasible solution $\bar{x}$ to L with objective value z , there is a corresponding feasible solution $\bar{x}'$ to L' with objective value z , and for each feasible solution $\bar{x}'$ to L' with objective value z , there is a corresponding feasible solution $\bar{x}$ to L with objective value z . (This definition does not imply a one-to-one correspondence between feasible solutions.) A minimization linear program L and a maximization linear program L' are equivalent if for each feasible solution $\bar{x}$ to L with objective value z , there is a corresponding feasible solution $\bar{x}'$ to L' with objective value $-z$, and for each feasible solution $\bar{x}'$ to L' with objective value z , there is a corresponding feasible solution $\bar{x}$ to L with objective value $-z$.

We now show how to remove, one by one, each of the possible problems in the list above. After removing each one, we shall argue that the new linear program is equivalent to the old one.

To convert a minimization linear program L into an equivalent maximization linear program L' , we simply negate the coefficients in the objective function. Since L and L' have identical sets of feasible solutions and, for any feasible solution, the objective value in L is the negative of the objective value in L' , these two linear programs are equivalent. For example, if we have the linear program

$$\begin{array}{ll} \text{minimize} & -2x_1 + 3x_2 \\ \text{subject to} & \\ & x_1 + x_2 = 7 \\ & x_1 - 2x_2 \leq 4 \\ & x_1 \geq 0, \end{array}$$

and we negate the coefficients of the objective function, we obtain

$$\begin{array}{llll}
\text{maximize} & 2x_1 & - & 3x_2 \\
\text{subject to} & & & \\
& x_1 & + & x_2 = 7 \\
& x_1 & - & 2x_2 \leq 4 \\
& x_1 & & \geq 0 .
\end{array}$$

Next, we show how to convert a linear program in which some of the variables do not have nonnegativity constraints into one in which each variable has a nonnegativity constraint. Suppose that some variable x_j does not have a nonnegativity constraint. Then, we replace each occurrence of x_j by $x'_j - x''_j$, and add the nonnegativity constraints $x'_j \geq 0$ and $x''_j \geq 0$. Thus, if the objective function has a term $c_j x_j$, we replace it by $c_j x'_j - c_j x''_j$, and if constraint i has a term $a_{ij} x_j$, we replace it by $a_{ij} x'_j - a_{ij} x''_j$. Any feasible solution $\hat{x}$ to the new linear program corresponds to a feasible solution $\bar{x}$ to the original linear program with $\bar{x}_j = \hat{x}'_j - \hat{x}''_j$ and with the same objective value. Also, any feasible solution $\bar{x}$ to the original linear program corresponds to a feasible solution $\hat{x}$ to the new linear program with $\hat{x}'_j = \bar{x}_j$ and $\hat{x}''_j = 0$ if $\bar{x}_j \geq 0$, or with $\hat{x}'_j = \bar{x}_j$ and $\hat{x}''_j = 0$ if $\bar{x}_j < 0$. The two linear programs have the same objective value regardless of the sign of $\bar{x}_j$. Thus, the two linear programs are equivalent. We apply this conversion scheme to each variable that does not have a nonnegativity constraint to yield an equivalent linear program in which all variables have nonnegativity constraints.

Continuing the example, we want to ensure that each variable has a corresponding nonnegativity constraint. Variable x_1 has such a constraint, but variable x_2 does not. Therefore, we replace x_2 by two variables x'_2 and x''_2 , and we modify the linear program to obtain

$$\begin{array}{llllll}
\text{maximize} & 2x_1 & - & 3x'_2 & + & 3x''_2 \\
\text{subject to} & & & & & \\
& x_1 & + & x'_2 & - & x''_2 = 7 \\
& x_1 & - & 2x'_2 & + & 2x''_2 \leq 4 \\
& x_1, x'_2, x''_2 & & & & \geq 0 .
\end{array} \tag{29.22}$$

Next, we convert equality constraints into inequality constraints. Suppose that a linear program has an equality constraint $f(x_1, x_2, \dots, x_n) = b$. Since $x = y$ if and only if both $x \geq y$ and $x \leq y$, we can replace this equality constraint by the pair of inequality constraints $f(x_1, x_2, \dots, x_n) \leq b$ and $f(x_1, x_2, \dots, x_n) \geq b$. Repeating this conversion for each equality constraint yields a linear program in which all constraints are inequalities.

Finally, we can convert the greater-than-or-equal-to constraints to less-than-or-equal-to constraints by multiplying these constraints through by -1 . That is, any inequality of the form

$$\sum_{j=1}^n a_{ij}x_j \geq b_i$$

is equivalent to

$$\sum_{j=1}^n -a_{ij}x_j \leq -b_i .$$

Thus, by replacing each coefficient a_{ij} by $-a_{ij}$ and each value b_i by $-b_i$, we obtain an equivalent less-than-or-equal-to constraint.

Finishing our example, we replace the equality in constraint (29.22) by two inequalities, obtaining

$$\begin{aligned} &\text{maximize} && 2x_1 &-& 3x'_2 &+& 3x''_2 \\ &\text{subject to} && x_1 &+& x'_2 &-& x''_2 &\leq & 7 \\ & && x_1 &+& x'_2 &-& x''_2 &\geq & 7 \\ & && x_1 &-& 2x'_2 &+& 2x''_2 &\leq & 4 \\ & && x_1, x'_2, x''_2 &&&&&\geq & 0 . \end{aligned} \tag{29.23}$$

Finally, we negate constraint (29.23). For consistency in variable names, we rename x'_2 to x_2 and x''_2 to x_3 , obtaining the standard form

$$\text{maximize} \quad 2x_1 - 3x_2 + 3x_3 \tag{29.24}$$

subject to

$$x_1 + x_2 - x_3 \leq 7 \tag{29.25}$$

$$-x_1 - x_2 + x_3 \leq -7 \tag{29.26}$$

$$x_1 - 2x_2 + 2x_3 \leq 4 \tag{29.27}$$

$$x_1, x_2, x_3 \geq 0 . \tag{29.28}$$

Converting linear programs into slack form

To efficiently solve a linear program with the simplex algorithm, we prefer to express it in a form in which some of the constraints are equality constraints. More precisely, we shall convert it into a form in which the nonnegativity constraints are the only inequality constraints, and the remaining constraints are equalities. Let

$$\sum_{j=1}^n a_{ij}x_j \leq b_i \tag{29.29}$$

be an inequality constraint. We introduce a new variable s and rewrite inequality (29.29) as the two constraints

$$s = b_i - \sum_{j=1}^n a_{ij} x_j, \quad (29.30)$$

$$s \geq 0. \quad (29.31)$$

We call s a **slack variable** because it measures the **slack**, or difference, between the left-hand and right-hand sides of equation (29.29). (We shall soon see why we find it convenient to write the constraint with only the slack variable on the left-hand side.) Because inequality (29.29) is true if and only if both equation (29.30) and inequality (29.31) are true, we can convert each inequality constraint of a linear program in this way to obtain an equivalent linear program in which the only inequality constraints are the nonnegativity constraints. When converting from standard to slack form, we shall use x_{n+i} (instead of s) to denote the slack variable associated with the i th inequality. The i th constraint is therefore

$$x_{n+i} = b_i - \sum_{j=1}^n a_{ij} x_j, \quad (29.32)$$

along with the nonnegativity constraint $x_{n+i} \geq 0$.

By converting each constraint of a linear program in standard form, we obtain a linear program in a different form. For example, for the linear program described in (29.24)–(29.28), we introduce slack variables x_4 , x_5 , and x_6 , obtaining

$$\text{maximize} \quad 2x_1 - 3x_2 + 3x_3 \quad (29.33)$$

subject to

$$x_4 = 7 - x_1 - x_2 + x_3 \quad (29.34)$$

$$x_5 = -7 + x_1 + x_2 - x_3 \quad (29.35)$$

$$x_6 = 4 - x_1 + 2x_2 - 2x_3 \quad (29.36)$$

$$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0. \quad (29.37)$$

In this linear program, all the constraints except for the nonnegativity constraints are equalities, and each variable is subject to a nonnegativity constraint. We write each equality constraint with one of the variables on the left-hand side of the equality and all others on the right-hand side. Furthermore, each equation has the same set of variables on the right-hand side, and these variables are also the only ones that appear in the objective function. We call the variables on the left-hand side of the equalities **basic variables** and those on the right-hand side **nonbasic variables**.

For linear programs that satisfy these conditions, we shall sometimes omit the words “maximize” and “subject to,” as well as the explicit nonnegativity constraints. We shall also use the variable z to denote the value of the objective func-

tion. We call the resulting format **slack form**. If we write the linear program given in (29.33)–(29.37) in slack form, we obtain

$$z = 2x_1 - 3x_2 + 3x_3 \quad (29.38)$$

$$x_4 = 7 - x_1 - x_2 + x_3 \quad (29.39)$$

$$x_5 = -7 + x_1 + x_2 - x_3 \quad (29.40)$$

$$x_6 = 4 - x_1 + 2x_2 - 2x_3. \quad (29.41)$$

As with standard form, we find it convenient to have a more concise notation for describing a slack form. As we shall see in Section 29.3, the sets of basic and nonbasic variables will change as the simplex algorithm runs. We use N to denote the set of indices of the nonbasic variables and B to denote the set of indices of the basic variables. We always have that $|N| = n$, $|B| = m$, and $N \cup B = \{1, 2, \dots, n + m\}$. The equations are indexed by the entries of B , and the variables on the right-hand sides are indexed by the entries of N . As in standard form, we use b_i , c_j , and a_{ij} to denote constant terms and coefficients. We also use v to denote an optional constant term in the objective function. (We shall see a little later that including the constant term in the objective function makes it easy to determine the value of the objective function.) Thus we can concisely define a slack form by a tuple (N, B, A, b, c, v) , denoting the slack form

$$z = v + \sum_{j \in N} c_j x_j \quad (29.42)$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad \text{for } i \in B, \quad (29.43)$$

in which all variables x are constrained to be nonnegative. Because we subtract the sum $\sum_{j \in N} a_{ij} x_j$ in (29.43), the values a_{ij} are actually the negatives of the coefficients as they “appear” in the slack form.

For example, in the slack form

$$\begin{aligned} z &= 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \\ x_1 &= 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \\ x_2 &= 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \\ x_4 &= 18 - \frac{x_3}{2} + \frac{x_5}{2}, \end{aligned}$$

we have $B = \{1, 2, 4\}$, $N = \{3, 5, 6\}$,

$$A = \begin{pmatrix} a_{13} & a_{15} & a_{16} \\ a_{23} & a_{25} & a_{26} \\ a_{43} & a_{45} & a_{46} \end{pmatrix} = \begin{pmatrix} -1/6 & -1/6 & 1/3 \\ 8/3 & 2/3 & -1/3 \\ 1/2 & -1/2 & 0 \end{pmatrix},$$

$$b = \begin{pmatrix} b_1 \\ b_2 \\ b_4 \end{pmatrix} = \begin{pmatrix} 8 \\ 4 \\ 18 \end{pmatrix},$$

$c = (c_3 \ c_5 \ c_6)^T = (-1/6 \ -1/6 \ -2/3)^T$, and $v = 28$. Note that the indices into A , b , and c are not necessarily sets of contiguous integers; they depend on the index sets B and N . As an example of the entries of A being the negatives of the coefficients as they appear in the slack form, observe that the equation for x_1 includes the term $x_3/6$, yet the coefficient a_{13} is actually $-1/6$ rather than $+1/6$.

Exercises

29.1-1

If we express the linear program in (29.24)–(29.28) in the compact notation of (29.19)–(29.21), what are n , m , A , b , and c ?

29.1-2

Give three feasible solutions to the linear program in (29.24)–(29.28). What is the objective value of each one?

29.1-3

For the slack form in (29.38)–(29.41), what are N , B , A , b , c , and v ?

29.1-4

Convert the following linear program into standard form:

$$\begin{array}{llllll} \text{minimize} & 2x_1 & + & 7x_2 & + & x_3 \\ \text{subject to} & & & & & \\ & x_1 & & & - & x_3 & = & 7 \\ & 3x_1 & + & x_2 & & & \geq & 24 \\ & & & x_2 & & & \geq & 0 \\ & & & & & x_3 & \leq & 0 \end{array}.$$

29.1-5

Convert the following linear program into slack form:

$$\begin{array}{llllll}
 \text{maximize} & 2x_1 & & - & 6x_3 & \\
 \text{subject to} & & & & & \\
 & x_1 & + & x_2 & - & x_3 \leq 7 \\
 & 3x_1 & - & x_2 & & \geq 8 \\
 & -x_1 & + & 2x_2 & + & 2x_3 \geq 0 \\
 & x_1, x_2, x_3 & & & & \geq 0 .
 \end{array}$$

What are the basic and nonbasic variables?

29.1-6

Show that the following linear program is infeasible:

$$\begin{array}{llllll}
 \text{maximize} & 3x_1 & - & 2x_2 & & \\
 \text{subject to} & & & & & \\
 & x_1 & + & x_2 & \leq & 2 \\
 & -2x_1 & - & 2x_2 & \leq & -10 \\
 & x_1, x_2 & & & \geq & 0 .
 \end{array}$$

29.1-7

Show that the following linear program is unbounded:

$$\begin{array}{llllll}
 \text{maximize} & x_1 & - & x_2 & & \\
 \text{subject to} & & & & & \\
 & -2x_1 & + & x_2 & \leq & -1 \\
 & -x_1 & - & 2x_2 & \leq & -2 \\
 & x_1, x_2 & & & \geq & 0 .
 \end{array}$$

29.1-8

Suppose that we have a general linear program with n variables and m constraints, and suppose that we convert it into standard form. Give an upper bound on the number of variables and constraints in the resulting linear program.

29.1-9

Give an example of a linear program for which the feasible region is not bounded, but the optimal objective value is finite.

29.2 Formulating problems as linear programs

Although we shall focus on the simplex algorithm in this chapter, it is also important to be able to recognize when we can formulate a problem as a linear program. Once we cast a problem as a polynomial-sized linear program, we can solve it in polynomial time by the ellipsoid algorithm or interior-point methods. Several linear-programming software packages can solve problems efficiently, so that once the problem is in the form of a linear program, such a package can solve it.

We shall look at several concrete examples of linear-programming problems. We start with two problems that we have already studied: the single-source shortest-paths problem (see Chapter 24) and the maximum-flow problem (see Chapter 26). We then describe the minimum-cost-flow problem. Although the minimum-cost-flow problem has a polynomial-time algorithm that is not based on linear programming, we won't describe the algorithm. Finally, we describe the multicommodity-flow problem, for which the only known polynomial-time algorithm is based on linear programming.

When we solved graph problems in Part VI, we used attribute notation, such as $v.d$ and $(u, v).f$. Linear programs typically use subscripted variables rather than objects with attached attributes, however. Therefore, when we express variables in linear programs, we shall indicate vertices and edges through subscripts. For example, we denote the shortest-path weight for vertex v not by $v.d$ but by d_v . Similarly, we denote the flow from vertex u to vertex v not by $(u, v).f$ but by f_{uv} . For quantities that are given as inputs to problems, such as edge weights or capacities, we shall continue to use notations such as $w(u, v)$ and $c(u, v)$.

Shortest paths

We can formulate the single-source shortest-paths problem as a linear program. In this section, we shall focus on how to formulate the single-pair shortest-path problem, leaving the extension to the more general single-source shortest-paths problem as Exercise 29.2-3.

In the single-pair shortest-path problem, we are given a weighted, directed graph $G = (V, E)$, with weight function $w : E \rightarrow \mathbb{R}$ mapping edges to real-valued weights, a source vertex s , and destination vertex t . We wish to compute the value d_t , which is the weight of a shortest path from s to t . To express this problem as a linear program, we need to determine a set of variables and constraints that define when we have a shortest path from s to t . Fortunately, the Bellman-Ford algorithm does exactly this. When the Bellman-Ford algorithm terminates, it has computed, for each vertex v , a value d_v (using subscript notation here rather than attribute notation) such that for each edge $(u, v) \in E$, we have $d_v \leq d_u + w(u, v)$.

The source vertex initially receives a value $d_s = 0$, which never changes. Thus we obtain the following linear program to compute the shortest-path weight from s to t :

$$\text{maximize } d_t \quad (29.44)$$

subject to

$$d_v \leq d_u + w(u, v) \quad \text{for each edge } (u, v) \in E, \quad (29.45)$$

$$d_s = 0. \quad (29.46)$$

You might be surprised that this linear program maximizes an objective function when it is supposed to compute shortest paths. We do not want to minimize the objective function, since then setting $\bar{d}_v = 0$ for all $v \in V$ would yield an optimal solution to the linear program without solving the shortest-paths problem. We maximize because an optimal solution to the shortest-paths problem sets each $\bar{d}_v$ to $\min_{u:(u,v) \in E} \{\bar{d}_u + w(u, v)\}$, so that $\bar{d}_v$ is the largest value that is less than or equal to all of the values in the set $\{\bar{d}_u + w(u, v)\}$. We want to maximize d_v for all vertices v on a shortest path from s to t subject to these constraints on all vertices v , and maximizing d_t achieves this goal.

This linear program has $|V|$ variables d_v , one for each vertex $v \in V$. It also has $|E| + 1$ constraints: one for each edge, plus the additional constraint that the source vertex's shortest-path weight always has the value 0.

Maximum flow

Next, we express the maximum-flow problem as a linear program. Recall that we are given a directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$, and two distinguished vertices: a source s and a sink t . As defined in Section 26.1, a flow is a nonnegative real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies the capacity constraint and flow conservation. A maximum flow is a flow that satisfies these constraints and maximizes the flow value, which is the total flow coming out of the source minus the total flow into the source. A flow, therefore, satisfies linear constraints, and the value of a flow is a linear function. Recalling also that we assume that $c(u, v) = 0$ if $(u, v) \notin E$ and that there are no antiparallel edges, we can express the maximum-flow problem as a linear program:

$$\text{maximize } \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} \quad (29.47)$$

subject to

$$f_{uv} \leq c(u, v) \quad \text{for each } u, v \in V, \quad (29.48)$$

$$\sum_{v \in V} f_{vu} = \sum_{v \in V} f_{uv} \quad \text{for each } u \in V - \{s, t\}, \quad (29.49)$$

$$f_{uv} \geq 0 \quad \text{for each } u, v \in V. \quad (29.50)$$

This linear program has $|V|^2$ variables, corresponding to the flow between each pair of vertices, and it has $2|V|^2 + |V| - 2$ constraints.

It is usually more efficient to solve a smaller-sized linear program. The linear program in (29.47)–(29.50) has, for ease of notation, a flow and capacity of 0 for each pair of vertices u, v with $(u, v) \notin E$. It would be more efficient to rewrite the linear program so that it has $O(V + E)$ constraints. Exercise 29.2-5 asks you to do so.

Minimum-cost flow

In this section, we have used linear programming to solve problems for which we already knew efficient algorithms. In fact, an efficient algorithm designed specifically for a problem, such as Dijkstra's algorithm for the single-source shortest-paths problem, or the push-relabel method for maximum flow, will often be more efficient than linear programming, both in theory and in practice.

The real power of linear programming comes from the ability to solve new problems. Recall the problem faced by the politician in the beginning of this chapter. The problem of obtaining a sufficient number of votes, while not spending too much money, is not solved by any of the algorithms that we have studied in this book, yet we can solve it by linear programming. Books abound with such real-world problems that linear programming can solve. Linear programming is also particularly useful for solving variants of problems for which we may not already know of an efficient algorithm.

Consider, for example, the following generalization of the maximum-flow problem. Suppose that, in addition to a capacity $c(u, v)$ for each edge (u, v) , we are given a real-valued cost $a(u, v)$. As in the maximum-flow problem, we assume that $c(u, v) = 0$ if $(u, v) \notin E$, and that there are no antiparallel edges. If we send f_{uv} units of flow over edge (u, v) , we incur a cost of $a(u, v)f_{uv}$. We are also given a flow demand d . We wish to send d units of flow from s to t while minimizing the total cost $\sum_{(u,v) \in E} a(u, v)f_{uv}$ incurred by the flow. This problem is known as the **minimum-cost-flow problem**.

Figure 29.3(a) shows an example of the minimum-cost-flow problem. We wish to send 4 units of flow from s to t while incurring the minimum total cost. Any particular legal flow, that is, a function f satisfying constraints (29.48)–(29.49), incurs a total cost of $\sum_{(u,v) \in E} a(u, v)f_{uv}$. We wish to find the particular 4-unit flow that minimizes this cost. Figure 29.3(b) shows an optimal solution, with total cost $\sum_{(u,v) \in E} a(u, v)f_{uv} = (2 \cdot 2) + (5 \cdot 2) + (3 \cdot 1) + (7 \cdot 1) + (1 \cdot 3) = 27$.

There are polynomial-time algorithms specifically designed for the minimum-cost-flow problem, but they are beyond the scope of this book. We can, however, express the minimum-cost-flow problem as a linear program. The linear program looks similar to the one for the maximum-flow problem with the additional con-

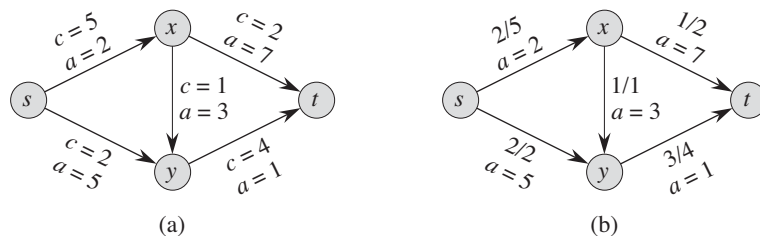


Figure 29.3 (a) An example of a minimum-cost-flow problem. We denote the capacities by c and the costs by a . Vertex s is the source and vertex t is the sink, and we wish to send 4 units of flow from s to t . (b) A solution to the minimum-cost flow problem in which 4 units of flow are sent from s to t . For each edge, the flow and capacity are written as flow/capacity.

straint that the value of the flow be exactly d units, and with the new objective function of minimizing the cost:

$$\text{minimize} \quad \sum_{(u,v) \in E} a(u,v) f_{uv} \quad (29.51)$$

subject to

$$\begin{aligned} f_{uv} &\leq c(u,v) && \text{for each } u, v \in V, \\ \sum_{v \in V} f_{vu} - \sum_{v \in V} f_{uv} &= 0 && \text{for each } u \in V - \{s, t\}, \\ \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} &= d, \\ f_{uv} &\geq 0 && \text{for each } u, v \in V. \end{aligned} \quad (29.52)$$

Multicommodity flow

As a final example, we consider another flow problem. Suppose that the Lucky Puck company from Section 26.1 decides to diversify its product line and ship not only hockey pucks, but also hockey sticks and hockey helmets. Each piece of equipment is manufactured in its own factory, has its own warehouse, and must be shipped, each day, from factory to warehouse. The sticks are manufactured in Vancouver and must be shipped to Saskatoon, and the helmets are manufactured in Edmonton and must be shipped to Regina. The capacity of the shipping network does not change, however, and the different items, or **commodities**, must share the same network.

This example is an instance of a **multicommodity-flow problem**. In this problem, we are again given a directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$. As in the maximum-flow problem, we implicitly assume that $c(u, v) = 0$ for $(u, v) \notin E$, and that the graph has no antipar-

allel edges. In addition, we are given k different commodities, $K_1, K_2, \dots, K_k$, where we specify commodity i by the triple $K_i = (s_i, t_i, d_i)$. Here, vertex s_i is the source of commodity i , vertex t_i is the sink of commodity i , and d_i is the demand for commodity i , which is the desired flow value for the commodity from s_i to t_i . We define a flow for commodity i , denoted by f_i , (so that f_{iuv} is the flow of commodity i from vertex u to vertex v) to be a real-valued function that satisfies the flow-conservation and capacity constraints. We now define f_{uv} , the **aggregate flow**, to be the sum of the various commodity flows, so that $f_{uv} = \sum_{i=1}^k f_{iuv}$. The aggregate flow on edge (u, v) must be no more than the capacity of edge (u, v) . We are not trying to minimize any objective function in this problem; we need only determine whether such a flow exists. Thus, we write a linear program with a “null” objective function:

$$\begin{aligned}
 & \text{minimize} && 0 \\
 & \text{subject to} && \\
 & && \sum_{i=1}^k f_{iuv} \leq c(u, v) \quad \text{for each } u, v \in V, \\
 & && \sum_{v \in V} f_{iuv} - \sum_{v \in V} f_{ivu} = 0 \quad \text{for each } i = 1, 2, \dots, k \text{ and} \\
 & && \quad \text{for each } u \in V - \{s_i, t_i\}, \\
 & && \sum_{v \in V} f_{i, s_i, v} - \sum_{v \in V} f_{i, v, s_i} = d_i \quad \text{for each } i = 1, 2, \dots, k, \\
 & && f_{iuv} \geq 0 \quad \text{for each } u, v \in V \text{ and} \\
 & && \quad \text{for each } i = 1, 2, \dots, k.
 \end{aligned}$$

The only known polynomial-time algorithm for this problem expresses it as a linear program and then solves it with a polynomial-time linear-programming algorithm.

Exercises

29.2-1

Put the single-pair shortest-path linear program from (29.44)–(29.46) into standard form.

29.2-2

Write out explicitly the linear program corresponding to finding the shortest path from node s to node y in Figure 24.2(a).

29.2-3

In the single-source shortest-paths problem, we want to find the shortest-path weights from a source vertex s to all vertices $v \in V$. Given a graph G , write a

linear program for which the solution has the property that d_v is the shortest-path weight from s to v for each vertex $v \in V$.

29.2-4

Write out explicitly the linear program corresponding to finding the maximum flow in Figure 26.1(a).

29.2-5

Rewrite the linear program for maximum flow (29.47)–(29.50) so that it uses only $O(V + E)$ constraints.

29.2-6

Write a linear program that, given a bipartite graph $G = (V, E)$, solves the maximum-bipartite-matching problem.

29.2-7

In the **minimum-cost multicommodity-flow problem**, we are given directed graph $G = (V, E)$ in which each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$ and a cost $a(u, v)$. As in the multicommodity-flow problem, we are given k different commodities, $K_1, K_2, \dots, K_k$, where we specify commodity i by the triple $K_i = (s_i, t_i, d_i)$. We define the flow f_i for commodity i and the aggregate flow f_{uv} on edge (u, v) as in the multicommodity-flow problem. A feasible flow is one in which the aggregate flow on each edge (u, v) is no more than the capacity of edge (u, v) . The cost of a flow is $\sum_{u,v \in V} a(u, v) f_{uv}$, and the goal is to find the feasible flow of minimum cost. Express this problem as a linear program.

29.3 The simplex algorithm

The simplex algorithm is the classical method for solving linear programs. In contrast to most of the other algorithms in this book, its running time is not polynomial in the worst case. It does yield insight into linear programs, however, and is often remarkably fast in practice.

In addition to having a geometric interpretation, described earlier in this chapter, the simplex algorithm bears some similarity to Gaussian elimination, discussed in Section 28.1. Gaussian elimination begins with a system of linear equalities whose solution is unknown. In each iteration, we rewrite this system in an equivalent form that has some additional structure. After some number of iterations, we have rewritten the system so that the solution is simple to obtain. The simplex algorithm proceeds in a similar manner, and we can view it as Gaussian elimination for inequalities.

We now describe the main idea behind an iteration of the simplex algorithm. Associated with each iteration will be a “basic solution” that we can easily obtain from the slack form of the linear program: set each nonbasic variable to 0 and compute the values of the basic variables from the equality constraints. An iteration converts one slack form into an equivalent slack form. The objective value of the associated basic feasible solution will be no less than that at the previous iteration, and usually greater. To achieve this increase in the objective value, we choose a nonbasic variable such that if we were to increase that variable’s value from 0, then the objective value would increase, too. The amount by which we can increase the variable is limited by the other constraints. In particular, we raise it until some basic variable becomes 0. We then rewrite the slack form, exchanging the roles of that basic variable and the chosen nonbasic variable. Although we have used a particular setting of the variables to guide the algorithm, and we shall use it in our proofs, the algorithm does not explicitly maintain this solution. It simply rewrites the linear program until an optimal solution becomes “obvious.”

An example of the simplex algorithm

We begin with an extended example. Consider the following linear program in standard form:

$$\text{maximize } 3x_1 + x_2 + 2x_3 \quad (29.53)$$

subject to

$$x_1 + x_2 + 3x_3 \leq 30 \quad (29.54)$$

$$2x_1 + 2x_2 + 5x_3 \leq 24 \quad (29.55)$$

$$4x_1 + x_2 + 2x_3 \leq 36 \quad (29.56)$$

$$x_1, x_2, x_3 \geq 0. \quad (29.57)$$

In order to use the simplex algorithm, we must convert the linear program into slack form; we saw how to do so in Section 29.1. In addition to being an algebraic manipulation, slack is a useful algorithmic concept. Recalling from Section 29.1 that each variable has a corresponding nonnegativity constraint, we say that an equality constraint is *tight* for a particular setting of its nonbasic variables if they cause the constraint’s basic variable to become 0. Similarly, a setting of the nonbasic variables that would make a basic variable become negative *violates* that constraint. Thus, the slack variables explicitly maintain how far each constraint is from being tight, and so they help to determine how much we can increase values of nonbasic variables without violating any constraints.

Associating the slack variables x_4 , x_5 , and x_6 with inequalities (29.54)–(29.56), respectively, and putting the linear program into slack form, we obtain

$$z = 3x_1 + x_2 + 2x_3 \quad (29.58)$$

$$x_4 = 30 - x_1 - x_2 - 3x_3 \quad (29.59)$$

$$x_5 = 24 - 2x_1 - 2x_2 - 5x_3 \quad (29.60)$$

$$x_6 = 36 - 4x_1 - x_2 - 2x_3 . \quad (29.61)$$

The system of constraints (29.59)–(29.61) has 3 equations and 6 variables. Any setting of the variables x_1 , x_2 , and x_3 defines values for x_4 , x_5 , and x_6 ; therefore, we have an infinite number of solutions to this system of equations. A solution is feasible if all of $x_1, x_2, \dots, x_6$ are nonnegative, and there can be an infinite number of feasible solutions as well. The infinite number of possible solutions to a system such as this one will be useful in later proofs. We focus on the **basic solution**: set all the (nonbasic) variables on the right-hand side to 0 and then compute the values of the (basic) variables on the left-hand side. In this example, the basic solution is $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_6) = (0, 0, 0, 30, 24, 36)$ and it has objective value $z = (3 \cdot 0) + (1 \cdot 0) + (2 \cdot 0) = 0$. Observe that this basic solution sets $\bar{x}_i = b_i$ for each $i \in B$. An iteration of the simplex algorithm rewrites the set of equations and the objective function so as to put a different set of variables on the right-hand side. Thus, a different basic solution is associated with the rewritten problem. We emphasize that the rewrite does not in any way change the underlying linear-programming problem; the problem at one iteration has the identical set of feasible solutions as the problem at the previous iteration. The problem does, however, have a different basic solution than that of the previous iteration.

If a basic solution is also feasible, we call it a **basic feasible solution**. As we run the simplex algorithm, the basic solution is almost always a basic feasible solution. We shall see in Section 29.5, however, that for the first few iterations of the simplex algorithm, the basic solution might not be feasible.

Our goal, in each iteration, is to reformulate the linear program so that the basic solution has a greater objective value. We select a nonbasic variable x_e whose coefficient in the objective function is positive, and we increase the value of x_e as much as possible without violating any of the constraints. The variable x_e becomes basic, and some other variable x_l becomes nonbasic. The values of other basic variables and of the objective function may also change.

To continue the example, let's think about increasing the value of x_1 . As we increase x_1 , the values of x_4 , x_5 , and x_6 all decrease. Because we have a nonnegativity constraint for each variable, we cannot allow any of them to become negative. If x_1 increases above 30, then x_4 becomes negative, and x_5 and x_6 become negative when x_1 increases above 12 and 9, respectively. The third constraint (29.61) is the tightest constraint, and it limits how much we can increase x_1 . Therefore, we switch the roles of x_1 and x_6 . We solve equation (29.61) for x_1 and obtain

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} . \quad (29.62)$$

To rewrite the other equations with x_6 on the right-hand side, we substitute for x_1 using equation (29.62). Doing so for equation (29.59), we obtain

$$\begin{aligned} x_4 &= 30 - x_1 - x_2 - 3x_3 \\ &= 30 - \left(9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}\right) - x_2 - 3x_3 \\ &= 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4}. \end{aligned} \quad (29.63)$$

Similarly, we combine equation (29.62) with constraint (29.60) and with objective function (29.58) to rewrite our linear program in the following form:

$$z = 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4} \quad (29.64)$$

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} \quad (29.65)$$

$$x_4 = 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4} \quad (29.66)$$

$$x_5 = 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2}. \quad (29.67)$$

We call this operation a *pivot*. As demonstrated above, a pivot chooses a nonbasic variable x_e , called the *entering variable*, and a basic variable x_l , called the *leaving variable*, and exchanges their roles.

The linear program described in equations (29.64)–(29.67) is equivalent to the linear program described in equations (29.58)–(29.61). We perform two operations in the simplex algorithm: rewrite equations so that variables move between the left-hand side and the right-hand side, and substitute one equation into another. The first operation trivially creates an equivalent problem, and the second, by elementary linear algebra, also creates an equivalent problem. (See Exercise 29.3-3.)

To demonstrate this equivalence, observe that our original basic solution $(0, 0, 0, 30, 24, 36)$ satisfies the new equations (29.65)–(29.67) and has objective value $27 + (1/4) \cdot 0 + (1/2) \cdot 0 - (3/4) \cdot 36 = 0$. The basic solution associated with the new linear program sets the nonbasic values to 0 and is $(9, 0, 0, 21, 6, 0)$, with objective value $z = 27$. Simple arithmetic verifies that this solution also satisfies equations (29.59)–(29.61) and, when plugged into objective function (29.58), has objective value $(3 \cdot 9) + (1 \cdot 0) + (2 \cdot 0) = 27$.

Continuing the example, we wish to find a new variable whose value we wish to increase. We do not want to increase x_6 , since as its value increases, the objective value decreases. We can attempt to increase either x_2 or x_3 ; let us choose x_3 . How far can we increase x_3 without violating any of the constraints? Constraint (29.65) limits it to 18, constraint (29.66) limits it to $42/5$, and constraint (29.67) limits it to $3/2$. The third constraint is again the tightest one, and therefore we rewrite the third constraint so that x_3 is on the left-hand side and x_5 is on the right-hand

side. We then substitute this new equation, $x_3 = 3/2 - 3x_2/8 - x_5/4 + x_6/8$, into equations (29.64)–(29.66) and obtain the new, but equivalent, system

$$z = \frac{111}{4} + \frac{x_2}{16} - \frac{x_5}{8} - \frac{11x_6}{16} \quad (29.68)$$

$$x_1 = \frac{33}{4} - \frac{x_2}{16} + \frac{x_5}{8} - \frac{5x_6}{16} \quad (29.69)$$

$$x_3 = \frac{3}{2} - \frac{3x_2}{8} - \frac{x_5}{4} + \frac{x_6}{8} \quad (29.70)$$

$$x_4 = \frac{69}{4} + \frac{3x_2}{16} + \frac{5x_5}{8} - \frac{x_6}{16} . \quad (29.71)$$

This system has the associated basic solution $(33/4, 0, 3/2, 69/4, 0, 0)$, with objective value $111/4$. Now the only way to increase the objective value is to increase x_2 . The three constraints give upper bounds of 132, 4, and ∞ , respectively. (We get an upper bound of ∞ from constraint (29.71) because, as we increase x_2 , the value of the basic variable x_4 increases also. This constraint, therefore, places no restriction on how much we can increase x_2 .) We increase x_2 to 4, and it becomes nonbasic. Then we solve equation (29.70) for x_2 and substitute in the other equations to obtain

$$z = 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \quad (29.72)$$

$$x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \quad (29.73)$$

$$x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \quad (29.74)$$

$$x_4 = 18 - \frac{x_3}{2} + \frac{x_5}{2} . \quad (29.75)$$

At this point, all coefficients in the objective function are negative. As we shall see later in this chapter, this situation occurs only when we have rewritten the linear program so that the basic solution is an optimal solution. Thus, for this problem, the solution $(8, 4, 0, 18, 0, 0)$, with objective value 28, is optimal. We can now return to our original linear program given in (29.53)–(29.57). The only variables in the original linear program are x_1 , x_2 , and x_3 , and so our solution is $x_1 = 8$, $x_2 = 4$, and $x_3 = 0$, with objective value $(3 \cdot 8) + (1 \cdot 4) + (2 \cdot 0) = 28$. Note that the values of the slack variables in the final solution measure how much slack remains in each inequality. Slack variable x_4 is 18, and in inequality (29.54), the left-hand side, with value $8 + 4 + 0 = 12$, is 18 less than the right-hand side of 30. Slack variables x_5 and x_6 are 0 and indeed, in inequalities (29.55) and (29.56), the left-hand and right-hand sides are equal. Observe also that even though the coefficients in the original slack form are integral, the coefficients in the other linear programs are not necessarily integral, and the intermediate solutions are not

necessarily integral. Furthermore, the final solution to a linear program need not be integral; it is purely coincidental that this example has an integral solution.

Pivoting

We now formalize the procedure for pivoting. The procedure PIVOT takes as input a slack form, given by the tuple (N, B, A, b, c, v) , the index l of the leaving variable x_l , and the index e of the entering variable x_e . It returns the tuple $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$ describing the new slack form. (Recall again that the entries of the $m \times n$ matrices A and $\hat{A}$ are actually the negatives of the coefficients that appear in the slack form.)

```

PIVOT( $N, B, A, b, c, v, l, e$ )
1  // Compute the coefficients of the equation for new basic variable  $x_e$ .
2  let  $\hat{A}$  be a new  $m \times n$  matrix
3   $\hat{b}_e = b_l / a_{le}$ 
4  for each  $j \in N - \{e\}$ 
5       $\hat{a}_{ej} = a_{lj} / a_{le}$ 
6   $\hat{a}_{el} = 1 / a_{le}$ 
7  // Compute the coefficients of the remaining constraints.
8  for each  $i \in B - \{l\}$ 
9       $\hat{b}_i = b_i - a_{ie} \hat{b}_e$ 
10     for each  $j \in N - \{e\}$ 
11          $\hat{a}_{ij} = a_{ij} - a_{ie} \hat{a}_{ej}$ 
12      $\hat{a}_{il} = -a_{ie} \hat{a}_{el}$ 
13  // Compute the objective function.
14   $\hat{v} = v + c_e \hat{b}_e$ 
15  for each  $j \in N - \{e\}$ 
16       $\hat{c}_j = c_j - c_e \hat{a}_{ej}$ 
17   $\hat{c}_l = -c_e \hat{a}_{el}$ 
18  // Compute new sets of basic and nonbasic variables.
19   $\hat{N} = N - \{e\} \cup \{l\}$ 
20   $\hat{B} = B - \{l\} \cup \{e\}$ 
21  return  $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$ 

```

PIVOT works as follows. Lines 3–6 compute the coefficients in the new equation for x_e by rewriting the equation that has x_l on the left-hand side to instead have x_e on the left-hand side. Lines 8–12 update the remaining equations by substituting the right-hand side of this new equation for each occurrence of x_e . Lines 14–17 do the same substitution for the objective function, and lines 19 and 20 update the

sets of nonbasic and basic variables. Line 21 returns the new slack form. As given, if $a_{le} = 0$, PIVOT would cause an error by dividing by 0, but as we shall see in the proofs of Lemmas 29.2 and 29.12, we call PIVOT only when $a_{le} \neq 0$.

We now summarize the effect that PIVOT has on the values of the variables in the basic solution.

Lemma 29.1

Consider a call to PIVOT(N, B, A, b, c, v, l, e) in which $a_{le} \neq 0$. Let the values returned from the call be $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$, and let $\bar{x}$ denote the basic solution after the call. Then

1. $\bar{x}_j = 0$ for each $j \in \hat{N}$.
2. $\bar{x}_e = b_l / a_{le}$.
3. $\bar{x}_i = b_i - a_{ie} \hat{b}_e$ for each $i \in \hat{B} - \{e\}$.

Proof The first statement is true because the basic solution always sets all nonbasic variables to 0. When we set each nonbasic variable to 0 in a constraint

$$x_i = \hat{b}_i - \sum_{j \in \hat{N}} \hat{a}_{ij} x_j ,$$

we have that $\bar{x}_i = \hat{b}_i$ for each $i \in \hat{B}$. Since $e \in \hat{B}$, line 3 of PIVOT gives

$$\bar{x}_e = \hat{b}_e = b_l / a_{le} ,$$

which proves the second statement. Similarly, using line 9 for each $i \in \hat{B} - \{e\}$, we have

$$\bar{x}_i = \hat{b}_i = b_i - a_{ie} \hat{b}_e ,$$

which proves the third statement. ■

The formal simplex algorithm

We are now ready to formalize the simplex algorithm, which we demonstrated by example. That example was a particularly nice one, and we could have had several other issues to address:

- How do we determine whether a linear program is feasible?
- What do we do if the linear program is feasible, but the initial basic solution is not feasible?
- How do we determine whether a linear program is unbounded?
- How do we choose the entering and leaving variables?

In Section 29.5, we shall show how to determine whether a problem is feasible, and if so, how to find a slack form in which the initial basic solution is feasible. Therefore, let us assume that we have a procedure INITIALIZE-SIMPLEX(A, b, c) that takes as input a linear program in standard form, that is, an $m \times n$ matrix $A = (a_{ij})$, an m -vector $b = (b_i)$, and an n -vector $c = (c_j)$. If the problem is infeasible, the procedure returns a message that the program is infeasible and then terminates. Otherwise, the procedure returns a slack form for which the initial basic solution is feasible.

The procedure SIMPLEX takes as input a linear program in standard form, as just described. It returns an n -vector $\bar{x} = (\bar{x}_j)$ that is an optimal solution to the linear program described in (29.19)–(29.21).

SIMPLEX(A, b, c)

```

1  ( $N, B, A, b, c, v$ ) = INITIALIZE-SIMPLEX( $A, b, c$ )
2  let  $\Delta$  be a new vector of length  $n$ 
3  while some index  $j \in N$  has  $c_j > 0$ 
4      choose an index  $e \in N$  for which  $c_e > 0$ 
5      for each index  $i \in B$ 
6          if  $a_{ie} > 0$ 
7               $\Delta_i = b_i / a_{ie}$ 
8          else  $\Delta_i = \infty$ 
9      choose an index  $l \in B$  that minimizes  $\Delta_i$ 
10     if  $\Delta_l == \infty$ 
11         return “unbounded”
12     else ( $N, B, A, b, c, v$ ) = PIVOT( $N, B, A, b, c, v, l, e$ )
13 for  $i = 1$  to  $n$ 
14     if  $i \in B$ 
15          $\bar{x}_i = b_i$ 
16     else  $\bar{x}_i = 0$ 
17 return ( $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n$ )

```

The SIMPLEX procedure works as follows. In line 1, it calls the procedure INITIALIZE-SIMPLEX(A, b, c), described above, which either determines that the linear program is infeasible or returns a slack form for which the basic solution is feasible. The **while** loop of lines 3–12 forms the main part of the algorithm. If all coefficients in the objective function are negative, then the **while** loop terminates. Otherwise, line 4 selects a variable x_e , whose coefficient in the objective function is positive, as the entering variable. Although we may choose any such variable as the entering variable, we assume that we use some prespecified deterministic rule. Next, lines 5–9 check each constraint and pick the one that most severely limits the amount by which we can increase x_e without violating any of the nonnegativ-

ity constraints; the basic variable associated with this constraint is x_l . Again, we are free to choose one of several variables as the leaving variable, but we assume that we use some prespecified deterministic rule. If none of the constraints limits the amount by which the entering variable can increase, the algorithm returns “unbounded” in line 11. Otherwise, line 12 exchanges the roles of the entering and leaving variables by calling $\text{PIVOT}(N, B, A, b, c, v, l, e)$, as described above. Lines 13–16 compute a solution $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n$ for the original linear-programming variables by setting all the nonbasic variables to 0 and each basic variable $\bar{x}_i$ to b_i , and line 17 returns these values.

To show that **SIMPLEX** is correct, we first show that if **SIMPLEX** has an initial feasible solution and eventually terminates, then it either returns a feasible solution or determines that the linear program is unbounded. Then, we show that **SIMPLEX** terminates. Finally, in Section 29.4 (Theorem 29.10) we show that the solution returned is optimal.

Lemma 29.2

Given a linear program (A, b, c) , suppose that the call to **INITIALIZE-SIMPLEX** in line 1 of **SIMPLEX** returns a slack form for which the basic solution is feasible. Then if **SIMPLEX** returns a solution in line 17, that solution is a feasible solution to the linear program. If **SIMPLEX** returns “unbounded” in line 11, the linear program is unbounded.

Proof We use the following three-part loop invariant:

At the start of each iteration of the **while** loop of lines 3–12,

1. the slack form is equivalent to the slack form returned by the call of **INITIALIZE-SIMPLEX**,
2. for each $i \in B$, we have $b_i \geq 0$, and
3. the basic solution associated with the slack form is feasible.

Initialization: The equivalence of the slack forms is trivial for the first iteration. We assume, in the statement of the lemma, that the call to **INITIALIZE-SIMPLEX** in line 1 of **SIMPLEX** returns a slack form for which the basic solution is feasible. Thus, the third part of the invariant is true. Because the basic solution is feasible, each basic variable x_i is nonnegative. Furthermore, since the basic solution sets each basic variable x_i to b_i , we have that $b_i \geq 0$ for all $i \in B$. Thus, the second part of the invariant holds.

Maintenance: We shall show that each iteration of the **while** loop maintains the loop invariant, assuming that the **return** statement in line 11 does not execute. We shall handle the case in which line 11 executes when we discuss termination.

An iteration of the **while** loop exchanges the role of a basic and a nonbasic variable by calling the PIVOT procedure. By Exercise 29.3-3, the slack form is equivalent to the one from the previous iteration which, by the loop invariant, is equivalent to the initial slack form.

We now demonstrate the second part of the loop invariant. We assume that at the start of each iteration of the **while** loop, $b_i \geq 0$ for each $i \in B$, and we shall show that these inequalities remain true after the call to PIVOT in line 12. Since the only changes to the variables b_i and the set B of basic variables occur in this assignment, it suffices to show that line 12 maintains this part of the invariant. We let b_i , a_{ij} , and B refer to values before the call of PIVOT, and $\hat{b}_i$ refer to values returned from PIVOT.

First, we observe that $\hat{b}_e \geq 0$ because $b_l \geq 0$ by the loop invariant, $a_{le} > 0$ by lines 6 and 9 of SIMPLEX, and $\hat{b}_e = b_l/a_{le}$ by line 3 of PIVOT.

For the remaining indices $i \in B - \{l\}$, we have that

$$\begin{aligned}\hat{b}_i &= b_i - a_{ie}\hat{b}_e && \text{(by line 9 of PIVOT)} \\ &= b_i - a_{ie}(b_l/a_{le}) && \text{(by line 3 of PIVOT)} .\end{aligned}\tag{29.76}$$

We have two cases to consider, depending on whether $a_{ie} > 0$ or $a_{ie} \leq 0$. If $a_{ie} > 0$, then since we chose l such that

$$b_l/a_{le} \leq b_i/a_{ie} \quad \text{for all } i \in B ,\tag{29.77}$$

we have

$$\begin{aligned}\hat{b}_i &= b_i - a_{ie}(b_l/a_{le}) && \text{(by equation (29.76))} \\ &\geq b_i - a_{ie}(b_i/a_{ie}) && \text{(by inequality (29.77))} \\ &= b_i - b_i \\ &= 0 ,\end{aligned}$$

and thus $\hat{b}_i \geq 0$. If $a_{ie} \leq 0$, then because a_{le} , b_i , and b_l are all nonnegative, equation (29.76) implies that $\hat{b}_i$ must be nonnegative, too.

We now argue that the basic solution is feasible, i.e., that all variables have nonnegative values. The nonbasic variables are set to 0 and thus are nonnegative. Each basic variable x_i is defined by the equation

$$x_i = b_i - \sum_{j \in N} a_{ij}x_j .$$

The basic solution sets $\bar{x}_i = b_i$. Using the second part of the loop invariant, we conclude that each basic variable $\bar{x}_i$ is nonnegative.

Termination: The **while** loop can terminate in one of two ways. If it terminates because of the condition in line 3, then the current basic solution is feasible and line 17 returns this solution. The other way it terminates is by returning “unbounded” in line 11. In this case, for each iteration of the **for** loop in lines 5–8, when line 6 is executed, we find that $a_{ie} \leq 0$. Consider the solution $\bar{x}$ defined as

$$\bar{x}_i = \begin{cases} \infty & \text{if } i = e, \\ 0 & \text{if } i \in N - \{e\}, \\ b_i - \sum_{j \in N} a_{ij} \bar{x}_j & \text{if } i \in B. \end{cases}$$

We now show that this solution is feasible, i.e., that all variables are nonnegative. The nonbasic variables other than $\bar{x}_e$ are 0, and $\bar{x}_e = \infty > 0$; thus all nonbasic variables are nonnegative. For each basic variable $\bar{x}_i$, we have

$$\begin{aligned} \bar{x}_i &= b_i - \sum_{j \in N} a_{ij} \bar{x}_j \\ &= b_i - a_{ie} \bar{x}_e. \end{aligned}$$

The loop invariant implies that $b_i \geq 0$, and we have $a_{ie} \leq 0$ and $\bar{x}_e = \infty > 0$. Thus, $\bar{x}_i \geq 0$.

Now we show that the objective value for the solution $\bar{x}$ is unbounded. From equation (29.42), the objective value is

$$\begin{aligned} z &= v + \sum_{j \in N} c_j \bar{x}_j \\ &= v + c_e \bar{x}_e. \end{aligned}$$

Since $c_e > 0$ (by line 4 of SIMPLEX) and $\bar{x}_e = \infty$, the objective value is ∞ , and thus the linear program is unbounded. ■

It remains to show that SIMPLEX terminates, and when it does terminate, the solution it returns is optimal. Section 29.4 will address optimality. We now discuss termination.

Termination

In the example given in the beginning of this section, each iteration of the simplex algorithm increased the objective value associated with the basic solution. As Exercise 29.3-2 asks you to show, no iteration of SIMPLEX can decrease the objective value associated with the basic solution. Unfortunately, it is possible that an iteration leaves the objective value unchanged. This phenomenon is called *degeneracy*, and we shall now study it in greater detail.

The assignment in line 14 of PIVOT, $\hat{v} = v + c_e \hat{b}_e$, changes the objective value. Since SIMPLEX calls PIVOT only when $c_e > 0$, the only way for the objective value to remain unchanged (i.e., $\hat{v} = v$) is for $\hat{b}_e$ to be 0. This value is assigned as $\hat{b}_e = b_l / a_{le}$ in line 3 of PIVOT. Since we always call PIVOT with $a_{le} \neq 0$, we see that for $\hat{b}_e$ to equal 0, and hence the objective value to be unchanged, we must have $b_l = 0$.

Indeed, this situation can occur. Consider the linear program

$$\begin{aligned} z &= & x_1 &+& x_2 &+& x_3 \\ x_4 &= 8 &-& x_1 &-& x_2 \\ x_5 &= &&& x_2 &-& x_3 . \end{aligned}$$

Suppose that we choose x_1 as the entering variable and x_4 as the leaving variable. After pivoting, we obtain

$$\begin{aligned} z &= 8 &&& +& x_3 &-& x_4 \\ x_1 &= 8 &-& x_2 &&& -& x_4 \\ x_5 &= && x_2 &-& x_3 . \end{aligned}$$

At this point, our only choice is to pivot with x_3 entering and x_5 leaving. Since $b_5 = 0$, the objective value of 8 remains unchanged after pivoting:

$$\begin{aligned} z &= 8 &+& x_2 &-& x_4 &-& x_5 \\ x_1 &= 8 &-& x_2 &-& x_4 \\ x_3 &= && x_2 &&& -& x_5 . \end{aligned}$$

The objective value has not changed, but our slack form has. Fortunately, if we pivot again, with x_2 entering and x_1 leaving, the objective value increases (to 16), and the simplex algorithm can continue.

Degeneracy can prevent the simplex algorithm from terminating, because it can lead to a phenomenon known as **cycling**: the slack forms at two different iterations of SIMPLEX are identical. Because of degeneracy, SIMPLEX could choose a sequence of pivot operations that leave the objective value unchanged but repeat a slack form within the sequence. Since SIMPLEX is a deterministic algorithm, if it cycles, then it will cycle through the same series of slack forms forever, never terminating.

Cycling is the only reason that SIMPLEX might not terminate. To show this fact, we must first develop some additional machinery.

At each iteration, SIMPLEX maintains A , b , c , and v in addition to the sets N and B . Although we need to explicitly maintain A , b , c , and v in order to implement the simplex algorithm efficiently, we can get by without maintaining them. In other words, the sets of basic and nonbasic variables suffice to uniquely determine the slack form. Before proving this fact, we prove a useful algebraic lemma.

Lemma 29.3

Let I be a set of indices. For each $j \in I$, let α_j and β_j be real numbers, and let x_j be a real-valued variable. Let γ be any real number. Suppose that for any settings of the x_j , we have

$$\sum_{j \in I} \alpha_j x_j = \gamma + \sum_{j \in I} \beta_j x_j . \quad (29.78)$$

Then $\alpha_j = \beta_j$ for each $j \in I$, and $\gamma = 0$.

Proof Since equation (29.78) holds for any values of the x_j , we can use particular values to draw conclusions about α , β , and γ . If we let $x_j = 0$ for each $j \in I$, we conclude that $\gamma = 0$. Now pick an arbitrary index $j \in I$, and set $x_j = 1$ and $x_k = 0$ for all $k \neq j$. Then we must have $\alpha_j = \beta_j$. Since we picked j as any index in I , we conclude that $\alpha_j = \beta_j$ for each $j \in I$. ■

A particular linear program has many different slack forms; recall that each slack form has the same set of feasible and optimal solutions as the original linear program. We now show that the slack form of a linear program is uniquely determined by the set of basic variables. That is, given the set of basic variables, a unique slack form (unique set of coefficients and right-hand sides) is associated with those basic variables.

Lemma 29.4

Let (A, b, c) be a linear program in standard form. Given a set B of basic variables, the associated slack form is uniquely determined.

Proof Assume for the purpose of contradiction that there are two different slack forms with the same set B of basic variables. The slack forms must also have identical sets $N = \{1, 2, \dots, n + m\} - B$ of nonbasic variables. We write the first slack form as

$$z = v + \sum_{j \in N} c_j x_j \quad (29.79)$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \text{ for } i \in B , \quad (29.80)$$

and the second as

$$z = v' + \sum_{j \in N} c'_j x_j \quad (29.81)$$

$$x_i = b'_i - \sum_{j \in N} a'_{ij} x_j \text{ for } i \in B . \quad (29.82)$$

Consider the system of equations formed by subtracting each equation in line (29.82) from the corresponding equation in line (29.80). The resulting system is

$$0 = (b_i - b'_i) - \sum_{j \in N} (a_{ij} - a'_{ij})x_j \quad \text{for } i \in B$$

or, equivalently,

$$\sum_{j \in N} a_{ij}x_j = (b_i - b'_i) + \sum_{j \in N} a'_{ij}x_j \quad \text{for } i \in B.$$

Now, for each $i \in B$, apply Lemma 29.3 with $\alpha_j = a_{ij}$, $\beta_j = a'_{ij}$, $\gamma = b_i - b'_i$, and $I = N$. Since $\alpha_i = \beta_i$, we have that $a_{ij} = a'_{ij}$ for each $j \in N$, and since $\gamma = 0$, we have that $b_i = b'_i$. Thus, for the two slack forms, A and b are identical to A' and b' . Using a similar argument, Exercise 29.3-1 shows that it must also be the case that $c = c'$ and $v = v'$, and hence that the slack forms must be identical. ■

We can now show that cycling is the only possible reason that SIMPLEX might not terminate.

Lemma 29.5

If SIMPLEX fails to terminate in at most $\binom{n+m}{m}$ iterations, then it cycles.

Proof By Lemma 29.4, the set B of basic variables uniquely determines a slack form. There are $n + m$ variables and $|B| = m$, and therefore, there are at most $\binom{n+m}{m}$ ways to choose B . Thus, there are only at most $\binom{n+m}{m}$ unique slack forms. Therefore, if SIMPLEX runs for more than $\binom{n+m}{m}$ iterations, it must cycle. ■

Cycling is theoretically possible, but extremely rare. We can prevent it by choosing the entering and leaving variables somewhat more carefully. One option is to perturb the input slightly so that it is impossible to have two solutions with the same objective value. Another option is to break ties by always choosing the variable with the smallest index, a strategy known as **Bland's rule**. We omit the proof that these strategies avoid cycling.

Lemma 29.6

If lines 4 and 9 of SIMPLEX always break ties by choosing the variable with the smallest index, then SIMPLEX must terminate. ■

We conclude this section with the following lemma.

Lemma 29.7

Assuming that INITIALIZE-SIMPLEX returns a slack form for which the basic solution is feasible, SIMPLEX either reports that a linear program is unbounded, or it terminates with a feasible solution in at most $\binom{n+m}{m}$ iterations.

Proof Lemmas 29.2 and 29.6 show that if INITIALIZE-SIMPLEX returns a slack form for which the basic solution is feasible, SIMPLEX either reports that a linear program is unbounded, or it terminates with a feasible solution. By the contrapositive of Lemma 29.5, if SIMPLEX terminates with a feasible solution, then it terminates in at most $\binom{n+m}{m}$ iterations. ■

Exercises**29.3-1**

Complete the proof of Lemma 29.4 by showing that it must be the case that $c = c'$ and $v = v'$.

29.3-2

Show that the call to PIVOT in line 12 of SIMPLEX never decreases the value of v .

29.3-3

Prove that the slack form given to the PIVOT procedure and the slack form that the procedure returns are equivalent.

29.3-4

Suppose we convert a linear program (A, b, c) in standard form to slack form. Show that the basic solution is feasible if and only if $b_i \geq 0$ for $i = 1, 2, \dots, m$.

29.3-5

Solve the following linear program using SIMPLEX:

$$\begin{array}{llll}
 \text{maximize} & 18x_1 & + & 12.5x_2 \\
 \text{subject to} & & & \\
 & x_1 & + & x_2 \leq 20 \\
 & x_1 & & \leq 12 \\
 & & x_2 & \leq 16 \\
 & x_1, x_2 & & \geq 0 .
 \end{array}$$

29.3-6

Solve the following linear program using SIMPLEX:

$$\begin{array}{llll}
 \text{maximize} & 5x_1 & - & 3x_2 \\
 \text{subject to} & & & \\
 & x_1 & - & x_2 \leq 1 \\
 & 2x_1 & + & x_2 \leq 2 \\
 & x_1, x_2 & & \geq 0 .
 \end{array}$$

29.3-7

Solve the following linear program using SIMPLEX:

$$\begin{array}{llllll}
 \text{minimize} & x_1 & + & x_2 & + & x_3 \\
 \text{subject to} & & & & & \\
 & 2x_1 & + & 7.5x_2 & + & 3x_3 \geq 10000 \\
 & 20x_1 & + & 5x_2 & + & 10x_3 \geq 30000 \\
 & x_1, x_2, x_3 & & & & \geq 0 .
 \end{array}$$

29.3-8

In the proof of Lemma 29.5, we argued that there are at most $\binom{m+n}{n}$ ways to choose a set B of basic variables. Give an example of a linear program in which there are strictly fewer than $\binom{m+n}{n}$ ways to choose the set B .

29.4 Duality

We have proven that, under certain assumptions, SIMPLEX terminates. We have not yet shown that it actually finds an optimal solution to a linear program, however. In order to do so, we introduce a powerful concept called **linear-programming duality**.

Duality enables us to prove that a solution is indeed optimal. We saw an example of duality in Chapter 26 with Theorem 26.6, the max-flow min-cut theorem. Suppose that, given an instance of a maximum-flow problem, we find a flow f with value $|f|$. How do we know whether f is a maximum flow? By the max-flow min-cut theorem, if we can find a cut whose value is also $|f|$, then we have verified that f is indeed a maximum flow. This relationship provides an example of duality: given a maximization problem, we define a related minimization problem such that the two problems have the same optimal objective values.

Given a linear program in which the objective is to maximize, we shall describe how to formulate a **dual** linear program in which the objective is to minimize and

whose optimal value is identical to that of the original linear program. When referring to dual linear programs, we call the original linear program the *primal*.

Given a primal linear program in standard form, as in (29.16)–(29.18), we define the dual linear program as

$$\text{minimize} \quad \sum_{i=1}^m b_i y_i \quad (29.83)$$

subject to

$$\sum_{i=1}^m a_{ij} y_i \geq c_j \quad \text{for } j = 1, 2, \dots, n, \quad (29.84)$$

$$y_i \geq 0 \quad \text{for } i = 1, 2, \dots, m. \quad (29.85)$$

To form the dual, we change the maximization to a minimization, exchange the roles of coefficients on the right-hand sides and the objective function, and replace each less-than-or-equal-to by a greater-than-or-equal-to. Each of the m constraints in the primal has an associated variable y_i in the dual, and each of the n constraints in the dual has an associated variable x_j in the primal. For example, consider the linear program given in (29.53)–(29.57). The dual of this linear program is

$$\text{minimize} \quad 30y_1 + 24y_2 + 36y_3 \quad (29.86)$$

subject to

$$y_1 + 2y_2 + 4y_3 \geq 3 \quad (29.87)$$

$$y_1 + 2y_2 + y_3 \geq 1 \quad (29.88)$$

$$3y_1 + 5y_2 + 2y_3 \geq 2 \quad (29.89)$$

$$y_1, y_2, y_3 \geq 0. \quad (29.90)$$

We shall show in Theorem 29.10 that the optimal value of the dual linear program is always equal to the optimal value of the primal linear program. Furthermore, the simplex algorithm actually implicitly solves both the primal and the dual linear programs simultaneously, thereby providing a proof of optimality.

We begin by demonstrating *weak duality*, which states that any feasible solution to the primal linear program has a value no greater than that of any feasible solution to the dual linear program.

Lemma 29.8 (Weak linear-programming duality)

Let $\bar{x}$ be any feasible solution to the primal linear program in (29.16)–(29.18) and let $\bar{y}$ be any feasible solution to the dual linear program in (29.83)–(29.85). Then, we have

$$\sum_{j=1}^n c_j \bar{x}_j \leq \sum_{i=1}^m b_i \bar{y}_i.$$

Proof We have

$$\begin{aligned}
 \sum_{j=1}^n c_j \bar{x}_j &\leq \sum_{j=1}^n \left(\sum_{i=1}^m a_{ij} \bar{y}_i \right) \bar{x}_j \quad (\text{by inequalities (29.84)}) \\
 &= \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} \bar{x}_j \right) \bar{y}_i \\
 &\leq \sum_{i=1}^m b_i \bar{y}_i \quad (\text{by inequalities (29.17)}) . \quad \blacksquare
 \end{aligned}$$

Corollary 29.9

Let $\bar{x}$ be a feasible solution to a primal linear program (A, b, c) , and let $\bar{y}$ be a feasible solution to the corresponding dual linear program. If

$$\sum_{j=1}^n c_j \bar{x}_j = \sum_{i=1}^m b_i \bar{y}_i ,$$

then $\bar{x}$ and $\bar{y}$ are optimal solutions to the primal and dual linear programs, respectively.

Proof By Lemma 29.8, the objective value of a feasible solution to the primal cannot exceed that of a feasible solution to the dual. The primal linear program is a maximization problem and the dual is a minimization problem. Thus, if feasible solutions $\bar{x}$ and $\bar{y}$ have the same objective value, neither can be improved. $\blacksquare$

Before proving that there always is a dual solution whose value is equal to that of an optimal primal solution, we describe how to find such a solution. When we ran the simplex algorithm on the linear program in (29.53)–(29.57), the final iteration yielded the slack form (29.72)–(29.75) with objective $z = 28 - x_3/6 - x_5/6 - 2x_6/3$, $B = \{1, 2, 4\}$, and $N = \{3, 5, 6\}$. As we shall show below, the basic solution associated with the final slack form is indeed an optimal solution to the linear program; an optimal solution to linear program (29.53)–(29.57) is therefore $(\bar{x}_1, \bar{x}_2, \bar{x}_3) = (8, 4, 0)$, with objective value $(3 \cdot 8) + (1 \cdot 4) + (2 \cdot 0) = 28$. As we also show below, we can read off an optimal dual solution: the negatives of the coefficients of the primal objective function are the values of the dual variables. More precisely, suppose that the last slack form of the primal is

$$\begin{aligned}
 z &= v' + \sum_{j \in N} c'_j x_j \\
 x_i &= b'_i - \sum_{j \in N} a'_{ij} x_j \quad \text{for } i \in B .
 \end{aligned}$$

Then, to produce an optimal dual solution, we set

$$\bar{y}_i = \begin{cases} -c'_{n+i} & \text{if } (n+i) \in N, \\ 0 & \text{otherwise.} \end{cases} \quad (29.91)$$

Thus, an optimal solution to the dual linear program defined in (29.86)–(29.90) is $\bar{y}_1 = 0$ (since $n+1 = 4 \in B$), $\bar{y}_2 = -c'_5 = 1/6$, and $\bar{y}_3 = -c'_6 = 2/3$. Evaluating the dual objective function (29.86), we obtain an objective value of $(30 \cdot 0) + (24 \cdot (1/6)) + (36 \cdot (2/3)) = 28$, which confirms that the objective value of the primal is indeed equal to the objective value of the dual. Combining these calculations with Lemma 29.8 yields a proof that the optimal objective value of the primal linear program is 28. We now show that this approach applies in general: we can find an optimal solution to the dual and simultaneously prove that a solution to the primal is optimal.

Theorem 29.10 (Linear-programming duality)

Suppose that SIMPLEX returns values $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ for the primal linear program (A, b, c) . Let N and B denote the nonbasic and basic variables for the final slack form, let c' denote the coefficients in the final slack form, and let $\bar{y} = (\bar{y}_1, \bar{y}_2, \dots, \bar{y}_m)$ be defined by equation (29.91). Then $\bar{x}$ is an optimal solution to the primal linear program, $\bar{y}$ is an optimal solution to the dual linear program, and

$$\sum_{j=1}^n c_j \bar{x}_j = \sum_{i=1}^m b_i \bar{y}_i. \quad (29.92)$$

Proof By Corollary 29.9, if we can find feasible solutions $\bar{x}$ and $\bar{y}$ that satisfy equation (29.92), then $\bar{x}$ and $\bar{y}$ must be optimal primal and dual solutions. We shall now show that the solutions $\bar{x}$ and $\bar{y}$ described in the statement of the theorem satisfy equation (29.92).

Suppose that we run SIMPLEX on a primal linear program, as given in lines (29.16)–(29.18). The algorithm proceeds through a series of slack forms until it terminates with a final slack form with objective function

$$z = v' + \sum_{j \in N} c'_j x_j. \quad (29.93)$$

Since SIMPLEX terminated with a solution, by the condition in line 3 we know that

$$c'_j \leq 0 \quad \text{for all } j \in N. \quad (29.94)$$

If we define

$$c'_j = 0 \quad \text{for all } j \in B, \quad (29.95)$$

we can rewrite equation (29.93) as

$$\begin{aligned} z &= v' + \sum_{j \in N} c'_j x_j \\ &= v' + \sum_{j \in N} c'_j x_j + \sum_{j \in B} c'_j x_j \quad (\text{because } c'_j = 0 \text{ if } j \in B) \\ &= v' + \sum_{j=1}^{n+m} c'_j x_j \quad (\text{because } N \cup B = \{1, 2, \dots, n+m\}). \end{aligned} \quad (29.96)$$

For the basic solution $\bar{x}$ associated with this final slack form, $\bar{x}_j = 0$ for all $j \in N$, and $z = v'$. Since all slack forms are equivalent, if we evaluate the original objective function on $\bar{x}$, we must obtain the same objective value:

$$\sum_{j=1}^n c_j \bar{x}_j = v' + \sum_{j=1}^{n+m} c'_j \bar{x}_j \quad (29.97)$$

$$\begin{aligned} &= v' + \sum_{j \in N} c'_j \bar{x}_j + \sum_{j \in B} c'_j \bar{x}_j \\ &= v' + \sum_{j \in N} (c'_j \cdot 0) + \sum_{j \in B} (0 \cdot \bar{x}_j) \\ &= v'. \end{aligned} \quad (29.98)$$

We shall now show that $\bar{y}$, defined by equation (29.91), is feasible for the dual linear program and that its objective value $\sum_{i=1}^m b_i \bar{y}_i$ equals $\sum_{j=1}^n c_j \bar{x}_j$. Equation (29.97) says that the first and last slack forms, evaluated at $\bar{x}$, are equal. More generally, the equivalence of all slack forms implies that for *any* set of values $x = (x_1, x_2, \dots, x_n)$, we have

$$\sum_{j=1}^n c_j x_j = v' + \sum_{j=1}^{n+m} c'_j x_j.$$

Therefore, for any particular set of values $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$, we have

$$\begin{aligned}
& \sum_{j=1}^n c_j \bar{x}_j \\
&= v' + \sum_{j=1}^{n+m} c'_j \bar{x}_j \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{j=n+1}^{n+m} c'_j \bar{x}_j \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{i=1}^m c'_{n+i} \bar{x}_{n+i} \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{i=1}^m (-\bar{y}_i) \bar{x}_{n+i} \quad (\text{by equations (29.91) and (29.95)}) \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{i=1}^m (-\bar{y}_i) \left(b_i - \sum_{j=1}^n a_{ij} \bar{x}_j \right) \quad (\text{by equation (29.32)}) \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j - \sum_{i=1}^m b_i \bar{y}_i + \sum_{i=1}^m \sum_{j=1}^n (a_{ij} \bar{x}_j) \bar{y}_i \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j - \sum_{i=1}^m b_i \bar{y}_i + \sum_{j=1}^n \sum_{i=1}^m (a_{ij} \bar{y}_i) \bar{x}_j \\
&= \left(v' - \sum_{i=1}^m b_i \bar{y}_i \right) + \sum_{j=1}^n \left(c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i \right) \bar{x}_j ,
\end{aligned}$$

so that

$$\sum_{j=1}^n c_j \bar{x}_j = \left(v' - \sum_{i=1}^m b_i \bar{y}_i \right) + \sum_{j=1}^n \left(c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i \right) \bar{x}_j . \quad (29.99)$$

Applying Lemma 29.3 to equation (29.99), we obtain

$$v' - \sum_{i=1}^m b_i \bar{y}_i = 0 , \quad (29.100)$$

$$c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i = c_j \quad \text{for } j = 1, 2, \dots, n . \quad (29.101)$$

By equation (29.100), we have that $\sum_{i=1}^m b_i \bar{y}_i = v'$, and hence the objective value of the dual $\left(\sum_{i=1}^m b_i \bar{y}_i \right)$ is equal to that of the primal (v'). It remains to show

that the solution $\bar{y}$ is feasible for the dual problem. From inequalities (29.94) and equations (29.95), we have that $c'_j \leq 0$ for all $j = 1, 2, \dots, n + m$. Hence, for any $j = 1, 2, \dots, n$, equations (29.101) imply that

$$\begin{aligned} c_j &= c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i \\ &\leq \sum_{i=1}^m a_{ij} \bar{y}_i, \end{aligned}$$

which satisfies the constraints (29.84) of the dual. Finally, since $c'_j \leq 0$ for each $j \in N \cup B$, when we set $\bar{y}$ according to equation (29.91), we have that each $\bar{y}_i \geq 0$, and so the nonnegativity constraints are satisfied as well. ■

We have shown that, given a feasible linear program, if INITIALIZE-SIMPLEX returns a feasible solution, and if SIMPLEX terminates without returning “unbounded,” then the solution returned is indeed an optimal solution. We have also shown how to construct an optimal solution to the dual linear program.

Exercises

29.4-1

Formulate the dual of the linear program given in Exercise 29.3-5.

29.4-2

Suppose that we have a linear program that is not in standard form. We could produce the dual by first converting it to standard form, and then taking the dual. It would be more convenient, however, to be able to produce the dual directly. Explain how we can directly take the dual of an arbitrary linear program.

29.4-3

Write down the dual of the maximum-flow linear program, as given in lines (29.47)–(29.50) on page 860. Explain how to interpret this formulation as a minimum-cut problem.

29.4-4

Write down the dual of the minimum-cost-flow linear program, as given in lines (29.51)–(29.52) on page 862. Explain how to interpret this problem in terms of graphs and flows.

29.4-5

Show that the dual of the dual of a linear program is the primal linear program.

29.4-6

Which result from Chapter 26 can be interpreted as weak duality for the maximum-flow problem?

29.5 The initial basic feasible solution

In this section, we first describe how to test whether a linear program is feasible, and if it is, how to produce a slack form for which the basic solution is feasible. We conclude by proving the fundamental theorem of linear programming, which says that the SIMPLEX procedure always produces the correct result.

Finding an initial solution

In Section 29.3, we assumed that we had a procedure INITIALIZE-SIMPLEX that determines whether a linear program has any feasible solutions, and if it does, gives a slack form for which the basic solution is feasible. We describe this procedure here.

A linear program can be feasible, yet the initial basic solution might not be feasible. Consider, for example, the following linear program:

$$\text{maximize} \quad 2x_1 - x_2 \quad (29.102)$$

subject to

$$2x_1 - x_2 \leq 2 \quad (29.103)$$

$$x_1 - 5x_2 \leq -4 \quad (29.104)$$

$$x_1, x_2 \geq 0. \quad (29.105)$$

If we were to convert this linear program to slack form, the basic solution would set $x_1 = 0$ and $x_2 = 0$. This solution violates constraint (29.104), and so it is not a feasible solution. Thus, INITIALIZE-SIMPLEX cannot just return the obvious slack form. In order to determine whether a linear program has any feasible solutions, we will formulate an *auxiliary linear program*. For this auxiliary linear program, we can find (with a little work) a slack form for which the basic solution is feasible. Furthermore, the solution of this auxiliary linear program determines whether the initial linear program is feasible and if so, it provides a feasible solution with which we can initialize SIMPLEX.

Lemma 29.11

Let L be a linear program in standard form, given as in (29.16)–(29.18). Let x_0 be a new variable, and let L_{aux} be the following linear program with $n + 1$ variables:

$$\text{maximize} \quad -x_0 \quad (29.106)$$

subject to

$$\sum_{j=1}^n a_{ij}x_j - x_0 \leq b_i \quad \text{for } i = 1, 2, \dots, m, \quad (29.107)$$

$$x_j \geq 0 \quad \text{for } j = 0, 1, \dots, n. \quad (29.108)$$

Then L is feasible if and only if the optimal objective value of L_{aux} is 0.

Proof Suppose that L has a feasible solution $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$. Then the solution $\bar{x}_0 = 0$ combined with $\bar{x}$ is a feasible solution to L_{aux} with objective value 0. Since $x_0 \geq 0$ is a constraint of L_{aux} and the objective function is to maximize $-x_0$, this solution must be optimal for L_{aux} .

Conversely, suppose that the optimal objective value of L_{aux} is 0. Then $\bar{x}_0 = 0$, and the remaining solution values of $\bar{x}$ satisfy the constraints of L . ■

We now describe our strategy to find an initial basic feasible solution for a linear program L in standard form:

INITIALIZE-SIMPLEX(A, b, c)

- 1 let k be the index of the minimum b_i
- 2 **if** $b_k \geq 0$ // is the initial basic solution feasible?
- 3 **return** ($\{1, 2, \dots, n\}, \{n+1, n+2, \dots, n+m\}, A, b, c, 0$)
- 4 form L_{aux} by adding $-x_0$ to the left-hand side of each constraint
and setting the objective function to $-x_0$
- 5 let (N, B, A, b, c, v) be the resulting slack form for L_{aux}
- 6 $l = n + k$
- 7 // L_{aux} has $n + 1$ nonbasic variables and m basic variables.
- 8 $(N, B, A, b, c, v) = \text{PIVOT}(N, B, A, b, c, v, l, 0)$
- 9 // The basic solution is now feasible for L_{aux} .
- 10 iterate the **while** loop of lines 3–12 of SIMPLEX until an optimal solution
to L_{aux} is found
- 11 **if** the optimal solution to L_{aux} sets $\bar{x}_0$ to 0
- 12 **if** $\bar{x}_0$ is basic
- 13 perform one (degenerate) pivot to make it nonbasic
- 14 from the final slack form of L_{aux} , remove x_0 from the constraints and
restore the original objective function of L , but replace each basic
variable in this objective function by the right-hand side of its
associated constraint
- 15 **return** the modified final slack form
- 16 **else return** “infeasible”

INITIALIZE-SIMPLEX works as follows. In lines 1–3, we implicitly test the basic solution to the initial slack form for L given by $N = \{1, 2, \dots, n\}$, $B = \{n + 1, n + 2, \dots, n + m\}$, $\bar{x}_i = b_i$ for all $i \in B$, and $\bar{x}_j = 0$ for all $j \in N$. (Creating the slack form requires no explicit effort, as the values of A , b , and c are the same in both slack and standard forms.) If line 2 finds this basic solution to be feasible—that is, $\bar{x}_i \geq 0$ for all $i \in N \cup B$ —then line 3 returns the slack form. Otherwise, in line 4, we form the auxiliary linear program L_{aux} as in Lemma 29.11. Since the initial basic solution to L is not feasible, the initial basic solution to the slack form for L_{aux} cannot be feasible either. To find a basic feasible solution, we perform a single pivot operation. Line 6 selects $l = n + k$ as the index of the basic variable that will be the leaving variable in the upcoming pivot operation. Since the basic variables are $x_{n+1}, x_{n+2}, \dots, x_{n+m}$, the leaving variable x_l will be the one with the most negative value. Line 8 performs that call of PIVOT, with x_0 entering and x_l leaving. We shall see shortly that the basic solution resulting from this call of PIVOT will be feasible. Now that we have a slack form for which the basic solution is feasible, we can, in line 10, repeatedly call PIVOT to fully solve the auxiliary linear program. As the test in line 11 demonstrates, if we find an optimal solution to L_{aux} with objective value 0, then in lines 12–14, we create a slack form for L for which the basic solution is feasible. To do so, we first, in lines 12–13, handle the degenerate case in which x_0 may still be basic with value $\bar{x}_0 = 0$. In this case, we perform a pivot step to remove x_0 from the basis, using any $e \in N$ such that $a_{0e} \neq 0$ as the entering variable. The new basic solution remains feasible; the degenerate pivot does not change the value of any variable. Next we delete all x_0 terms from the constraints and restore the original objective function for L . The original objective function may contain both basic and nonbasic variables. Therefore, in the objective function we replace each basic variable by the right-hand side of its associated constraint. Line 15 then returns this modified slack form. If, on the other hand, line 11 discovers that the original linear program L is infeasible, then line 16 returns this information.

We now demonstrate the operation of INITIALIZE-SIMPLEX on the linear program (29.102)–(29.105). This linear program is feasible if we can find nonnegative values for x_1 and x_2 that satisfy inequalities (29.103) and (29.104). Using Lemma 29.11, we formulate the auxiliary linear program

$$\text{maximize} \quad -x_0 \quad (29.109)$$

subject to

$$2x_1 - x_2 - x_0 \leq 2 \quad (29.110)$$

$$x_1 - 5x_2 - x_0 \leq -4 \quad (29.111)$$

$$x_1, x_2, x_0 \geq 0.$$

By Lemma 29.11, if the optimal objective value of this auxiliary linear program is 0, then the original linear program has a feasible solution. If the optimal objective

value of this auxiliary linear program is negative, then the original linear program does not have a feasible solution.

We write this linear program in slack form, obtaining

$$\begin{aligned} z &= & -x_0 \\ x_3 &= 2 - 2x_1 + x_2 + x_0 \\ x_4 &= -4 - x_1 + 5x_2 + x_0 . \end{aligned}$$

We are not out of the woods yet, because the basic solution, which would set $x_4 = -4$, is not feasible for this auxiliary linear program. We can, however, with one call to PIVOT, convert this slack form into one in which the basic solution is feasible. As line 8 indicates, we choose x_0 to be the entering variable. In line 6, we choose as the leaving variable x_4 , which is the basic variable whose value in the basic solution is most negative. After pivoting, we have the slack form

$$\begin{aligned} z &= -4 - x_1 + 5x_2 - x_4 \\ x_0 &= 4 + x_1 - 5x_2 + x_4 \\ x_3 &= 6 - x_1 - 4x_2 + x_4 . \end{aligned}$$

The associated basic solution is $(\bar{x}_0, \bar{x}_1, \bar{x}_2, \bar{x}_3, \bar{x}_4) = (4, 0, 0, 6, 0)$, which is feasible. We now repeatedly call PIVOT until we obtain an optimal solution to L_{aux} . In this case, one call to PIVOT with x_2 entering and x_0 leaving yields

$$\begin{aligned} z &= & -x_0 \\ x_2 &= \frac{4}{5} - \frac{x_0}{5} + \frac{x_1}{5} + \frac{x_4}{5} \\ x_3 &= \frac{14}{5} + \frac{4x_0}{5} - \frac{9x_1}{5} + \frac{x_4}{5} . \end{aligned}$$

This slack form is the final solution to the auxiliary problem. Since this solution has $x_0 = 0$, we know that our initial problem was feasible. Furthermore, since $x_0 = 0$, we can just remove it from the set of constraints. We then restore the original objective function, with appropriate substitutions made to include only nonbasic variables. In our example, we get the objective function

$$2x_1 - x_2 = 2x_1 - \left(\frac{4}{5} - \frac{x_0}{5} + \frac{x_1}{5} + \frac{x_4}{5} \right) .$$

Setting $x_0 = 0$ and simplifying, we get the objective function

$$-\frac{4}{5} + \frac{9x_1}{5} - \frac{x_4}{5} ,$$

and the slack form

$$\begin{aligned}
z &= -\frac{4}{5} + \frac{9x_1}{5} - \frac{x_4}{5} \\
x_2 &= \frac{4}{5} + \frac{x_1}{5} + \frac{x_4}{5} \\
x_3 &= \frac{14}{5} - \frac{9x_1}{5} + \frac{x_4}{5} .
\end{aligned}$$

This slack form has a feasible basic solution, and we can return it to procedure SIMPLEX.

We now formally show the correctness of INITIALIZE-SIMPLEX.

Lemma 29.12

If a linear program L has no feasible solution, then INITIALIZE-SIMPLEX returns “infeasible.” Otherwise, it returns a valid slack form for which the basic solution is feasible.

Proof First suppose that the linear program L has no feasible solution. Then by Lemma 29.11, the optimal objective value of L_{aux} , defined in (29.106)–(29.108), is nonzero, and by the nonnegativity constraint on x_0 , the optimal objective value must be negative. Furthermore, this objective value must be finite, since setting $x_i = 0$, for $i = 1, 2, \dots, n$, and $x_0 = |\min_{i=1}^m \{b_i\}|$ is feasible, and this solution has objective value $-|\min_{i=1}^m \{b_i\}|$. Therefore, line 10 of INITIALIZE-SIMPLEX finds a solution with a nonpositive objective value. Let $\bar{x}$ be the basic solution associated with the final slack form. We cannot have $\bar{x}_0 = 0$, because then L_{aux} would have objective value 0, which contradicts that the objective value is negative. Thus the test in line 11 results in line 16 returning “infeasible.”

Suppose now that the linear program L does have a feasible solution. From Exercise 29.3-4, we know that if $b_i \geq 0$ for $i = 1, 2, \dots, m$, then the basic solution associated with the initial slack form is feasible. In this case, lines 2–3 return the slack form associated with the input. (Converting the standard form to slack form is easy, since A , b , and c are the same in both.)

In the remainder of the proof, we handle the case in which the linear program is feasible but we do not return in line 3. We argue that in this case, lines 4–10 find a feasible solution to L_{aux} with objective value 0. First, by lines 1–2, we must have

$$b_k < 0 ,$$

and

$$b_k \leq b_i \quad \text{for each } i \in B . \tag{29.112}$$

In line 8, we perform one pivot operation in which the leaving variable x_l (recall that $l = n + k$, so that $b_l < 0$) is the left-hand side of the equation with minimum b_i , and the entering variable is x_0 , the extra added variable. We now show

that after this pivot, all entries of b are nonnegative, and hence the basic solution to L_{aux} is feasible. Letting $\bar{x}$ be the basic solution after the call to PIVOT, and letting $\hat{b}$ and $\hat{B}$ be values returned by PIVOT, Lemma 29.1 implies that

$$\bar{x}_i = \begin{cases} b_i - a_{ie}\hat{b}_e & \text{if } i \in \hat{B} - \{e\}, \\ b_l/a_{le} & \text{if } i = e. \end{cases} \quad (29.113)$$

The call to PIVOT in line 8 has $e = 0$. If we rewrite inequalities (29.107), to include coefficients a_{i0} ,

$$\sum_{j=0}^n a_{ij}x_j \leq b_i \quad \text{for } i = 1, 2, \dots, m, \quad (29.114)$$

then

$$a_{i0} = a_{ie} = -1 \quad \text{for each } i \in B. \quad (29.115)$$

(Note that a_{i0} is the coefficient of x_0 as it appears in inequalities (29.114), not the negation of the coefficient, because L_{aux} is in standard rather than slack form.) Since $l \in B$, we also have that $a_{le} = -1$. Thus, $b_l/a_{le} > 0$, and so $\bar{x}_e > 0$. For the remaining basic variables, we have

$$\begin{aligned} \bar{x}_i &= b_i - a_{ie}\hat{b}_e && \text{(by equation (29.113))} \\ &= b_i - a_{ie}(b_l/a_{le}) && \text{(by line 3 of PIVOT)} \\ &= b_i - b_l && \text{(by equation (29.115) and } a_{le} = -1) \\ &\geq 0 && \text{(by inequality (29.112)) ,} \end{aligned}$$

which implies that each basic variable is now nonnegative. Hence the basic solution after the call to PIVOT in line 8 is feasible. We next execute line 10, which solves L_{aux} . Since we have assumed that L has a feasible solution, Lemma 29.11 implies that L_{aux} has an optimal solution with objective value 0. Since all the slack forms are equivalent, the final basic solution to L_{aux} must have $\bar{x}_0 = 0$, and after removing x_0 from the linear program, we obtain a slack form that is feasible for L . Line 15 then returns this slack form. ■

Fundamental theorem of linear programming

We conclude this chapter by showing that the SIMPLEX procedure works. In particular, any linear program either is infeasible, is unbounded, or has an optimal solution with a finite objective value. In each case, SIMPLEX acts appropriately.

Theorem 29.13 (Fundamental theorem of linear programming)

Any linear program L , given in standard form, either

1. has an optimal solution with a finite objective value,
2. is infeasible, or
3. is unbounded.

If L is infeasible, SIMPLEX returns “infeasible.” If L is unbounded, SIMPLEX returns “unbounded.” Otherwise, SIMPLEX returns an optimal solution with a finite objective value.

Proof By Lemma 29.12, if linear program L is infeasible, then SIMPLEX returns “infeasible.” Now suppose that the linear program L is feasible. By Lemma 29.12, INITIALIZE-SIMPLEX returns a slack form for which the basic solution is feasible. By Lemma 29.7, therefore, SIMPLEX either returns “unbounded” or terminates with a feasible solution. If it terminates with a finite solution, then Theorem 29.10 tells us that this solution is optimal. On the other hand, if SIMPLEX returns “unbounded,” Lemma 29.2 tells us the linear program L is indeed unbounded. Since SIMPLEX always terminates in one of these ways, the proof is complete. ■

Exercises**29.5-1**

Give detailed pseudocode to implement lines 5 and 14 of INITIALIZE-SIMPLEX.

29.5-2

Show that when the main loop of SIMPLEX is run by INITIALIZE-SIMPLEX, it can never return “unbounded.”

29.5-3

Suppose that we are given a linear program L in standard form, and suppose that for both L and the dual of L , the basic solutions associated with the initial slack forms are feasible. Show that the optimal objective value of L is 0.

29.5-4

Suppose that we allow strict inequalities in a linear program. Show that in this case, the fundamental theorem of linear programming does not hold.

29.5-5

Solve the following linear program using SIMPLEX:

$$\begin{array}{ll}
\text{maximize} & x_1 + 3x_2 \\
\text{subject to} & \\
& x_1 - x_2 \leq 8 \\
& -x_1 - x_2 \leq -3 \\
& -x_1 + 4x_2 \leq 2 \\
& x_1, x_2 \geq 0 .
\end{array}$$

29.5-6

Solve the following linear program using SIMPLEX:

$$\begin{array}{ll}
\text{maximize} & x_1 - 2x_2 \\
\text{subject to} & \\
& x_1 + 2x_2 \leq 4 \\
& -2x_1 - 6x_2 \leq -12 \\
& x_2 \leq 1 \\
& x_1, x_2 \geq 0 .
\end{array}$$

29.5-7

Solve the following linear program using SIMPLEX:

$$\begin{array}{ll}
\text{maximize} & x_1 + 3x_2 \\
\text{subject to} & \\
& -x_1 + x_2 \leq -1 \\
& -x_1 - x_2 \leq -3 \\
& -x_1 + 4x_2 \leq 2 \\
& x_1, x_2 \geq 0 .
\end{array}$$

29.5-8

Solve the linear program given in (29.6)–(29.10).

29.5-9Consider the following 1-variable linear program, which we call P :

$$\begin{array}{ll}
\text{maximize} & tx \\
\text{subject to} & \\
& rx \leq s \\
& x \geq 0 ,
\end{array}$$

where r , s , and t are arbitrary real numbers. Let D be the dual of P .

State for which values of r , s , and t you can assert that

1. Both P and D have optimal solutions with finite objective values.
2. P is feasible, but D is infeasible.
3. D is feasible, but P is infeasible.
4. Neither P nor D is feasible.

Problems

29-1 Linear-inequality feasibility

Given a set of m linear inequalities on n variables $x_1, x_2, \dots, x_n$, the **linear-inequality feasibility problem** asks whether there is a setting of the variables that simultaneously satisfies each of the inequalities.

- a. Show that if we have an algorithm for linear programming, we can use it to solve a linear-inequality feasibility problem. The number of variables and constraints that you use in the linear-programming problem should be polynomial in n and m .
- b. Show that if we have an algorithm for the linear-inequality feasibility problem, we can use it to solve a linear-programming problem. The number of variables and linear inequalities that you use in the linear-inequality feasibility problem should be polynomial in n and m , the number of variables and constraints in the linear program.

29-2 Complementary slackness

Complementary slackness describes a relationship between the values of primal variables and dual constraints and between the values of dual variables and primal constraints. Let $\bar{x}$ be a feasible solution to the primal linear program given in (29.16)–(29.18), and let $\bar{y}$ be a feasible solution to the dual linear program given in (29.83)–(29.85). Complementary slackness states that the following conditions are necessary and sufficient for $\bar{x}$ and $\bar{y}$ to be optimal:

$$\sum_{i=1}^m a_{ij} \bar{y}_i = c_j \text{ or } \bar{x}_j = 0 \quad \text{for } j = 1, 2, \dots, n$$

and

$$\sum_{j=1}^n a_{ij} \bar{x}_j = b_i \text{ or } \bar{y}_i = 0 \quad \text{for } i = 1, 2, \dots, m.$$

- a. Verify that complementary slackness holds for the linear program in lines (29.53)–(29.57).
- b. Prove that complementary slackness holds for any primal linear program and its corresponding dual.
- c. Prove that a feasible solution $\bar{x}$ to a primal linear program given in lines (29.16)–(29.18) is optimal if and only if there exist values $\bar{y} = (\bar{y}_1, \bar{y}_2, \dots, \bar{y}_m)$ such that
 1. $\bar{y}$ is a feasible solution to the dual linear program given in (29.83)–(29.85),
 2. $\sum_{i=1}^m a_{ij} \bar{y}_i = c_j$ for all j such that $\bar{x}_j > 0$, and
 3. $\bar{y}_i = 0$ for all i such that $\sum_{j=1}^n a_{ij} \bar{x}_j < b_i$.

29-3 Integer linear programming

An **integer linear-programming problem** is a linear-programming problem with the additional constraint that the variables x must take on integral values. Exercise 34.5-3 shows that just determining whether an integer linear program has a feasible solution is NP-hard, which means that there is no known polynomial-time algorithm for this problem.

- a. Show that weak duality (Lemma 29.8) holds for an integer linear program.
- b. Show that duality (Theorem 29.10) does not always hold for an integer linear program.
- c. Given a primal linear program in standard form, let us define P to be the optimal objective value for the primal linear program, D to be the optimal objective value for its dual, IP to be the optimal objective value for the integer version of the primal (that is, the primal with the added constraint that the variables take on integer values), and ID to be the optimal objective value for the integer version of the dual. Assuming that both the primal integer program and the dual integer program are feasible and bounded, show that

$$IP \leq P = D \leq ID.$$

29-4 Farkas's lemma

Let A be an $m \times n$ matrix and c be an n -vector. Then Farkas's lemma states that exactly one of the systems

$$Ax \leq 0,$$

$$c^T x > 0$$

and

$$A^T y = c,$$

$$y \geq 0$$

is solvable, where x is an n -vector and y is an m -vector. Prove Farkas's lemma.

29-5 Minimum-cost circulation

In this problem, we consider a variant of the minimum-cost-flow problem from Section 29.2 in which we are not given a demand, a source, or a sink. Instead, we are given, as before, a flow network and edge costs $a(u, v)$. A flow is feasible if it satisfies the capacity constraint on every edge and flow conservation at *every* vertex. The goal is to find, among all feasible flows, the one of minimum cost. We call this problem the **minimum-cost-circulation problem**.

- a. Formulate the minimum-cost-circulation problem as a linear program.
- b. Suppose that for all edges $(u, v) \in E$, we have $a(u, v) > 0$. Characterize an optimal solution to the minimum-cost-circulation problem.
- c. Formulate the maximum-flow problem as a minimum-cost-circulation problem linear program. That is given a maximum-flow problem instance $G = (V, E)$ with source s , sink t and edge capacities c , create a minimum-cost-circulation problem by giving a (possibly different) network $G' = (V', E')$ with edge capacities c' and edge costs a' such that you can discern a solution to the maximum-flow problem from a solution to the minimum-cost-circulation problem.
- d. Formulate the single-source shortest-path problem as a minimum-cost-circulation problem linear program.

Chapter notes

This chapter only begins to study the wide field of linear programming. A number of books are devoted exclusively to linear programming, including those by Chvátal [69], Gass [130], Karloff [197], Schrijver [303], and Vanderbei [344]. Many other books give a good coverage of linear programming, including those by Papadimitriou and Steiglitz [271] and Ahuja, Magnanti, and Orlin [7]. The coverage in this chapter draws on the approach taken by Chvátal.

The simplex algorithm for linear programming was invented by G. Dantzig in 1947. Shortly after, researchers discovered how to formulate a number of problems in a variety of fields as linear programs and solve them with the simplex algorithm. As a result, applications of linear programming flourished, along with several algorithms. Variants of the simplex algorithm remain the most popular methods for solving linear-programming problems. This history appears in a number of places, including the notes in [69] and [197].

The ellipsoid algorithm was the first polynomial-time algorithm for linear programming and is due to L. G. Khachian in 1979; it was based on earlier work by N. Z. Shor, D. B. Judin, and A. S. Nemirovskii. Grötschel, Lovász, and Schrijver [154] describe how to use the ellipsoid algorithm to solve a variety of problems in combinatorial optimization. To date, the ellipsoid algorithm does not appear to be competitive with the simplex algorithm in practice.

Karmarkar's paper [198] includes a description of the first interior-point algorithm. Many subsequent researchers designed interior-point algorithms. Good surveys appear in the article of Goldfarb and Todd [141] and the book by Ye [361].

Analysis of the simplex algorithm remains an active area of research. V. Klee and G. J. Minty constructed an example on which the simplex algorithm runs through $2^n - 1$ iterations. The simplex algorithm usually performs very well in practice and many researchers have tried to give theoretical justification for this empirical observation. A line of research begun by K. H. Borgwardt, and carried on by many others, shows that under certain probabilistic assumptions on the input, the simplex algorithm converges in expected polynomial time. Spielman and Teng [322] made progress in this area, introducing the “smoothed analysis of algorithms” and applying it to the simplex algorithm.

The simplex algorithm is known to run efficiently in certain special cases. Particularly noteworthy is the network-simplex algorithm, which is the simplex algorithm, specialized to network-flow problems. For certain network problems, including the shortest-paths, maximum-flow, and minimum-cost-flow problems, variants of the network-simplex algorithm run in polynomial time. See, for example, the article by Orlin [268] and the citations therein.

Approximation algorithms

Oct 18

If not able to compute an optimal solution, go for an approximately optimal solution.

Minimization problem : your solution cost $\leq \alpha \cdot$ optimal cost
Maximization problem : your solution value $\geq \beta \cdot$ optimal value

$\nearrow 2, 1.5, 1+\epsilon$

$\searrow 0.5, 0.9, 1-\epsilon$

Load balancing

m processors (identical)

n jobs with processing times $t_1, t_2, t_3, \dots, t_n$

Assign jobs to processors. (not splittable)
Want to finish all the jobs as soon as possible.

Minimize makespan

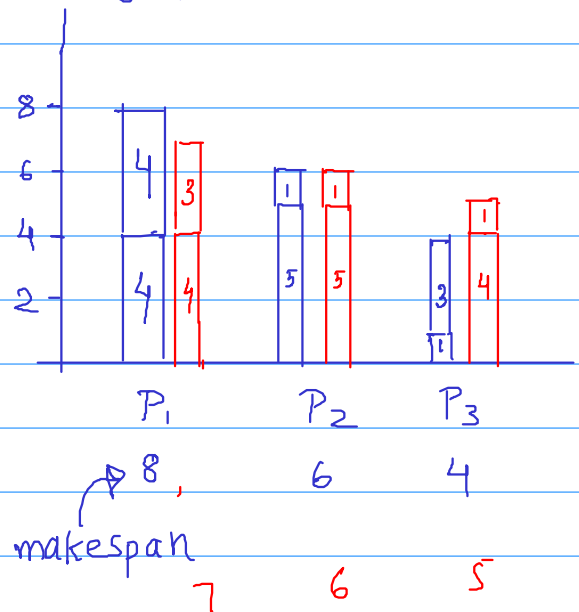


maximum total load of any processor.

Example

$m = 3$ processors

jobs $\rightarrow 4, 4, 5, 1, 1, 3$



Greedy algorithm

For $i = 1$ to n

assign job i to the processor which has the minimum load currently.

Does greedy always give an optimal solution?

4, 4, 5, 1, 1, 3

P_1	P_2	P_3
4	4	5
1	1	3
5	5	8

doesn't give optimal

Is there a better order for the greedy algorithm.

sort in decreasing order

5, 4, 4, 3, 1, 1

P_1	P_2	P_3
5	4	4
	3	1
		1
5	7	6

HW find an example where this algorithm is not optimal

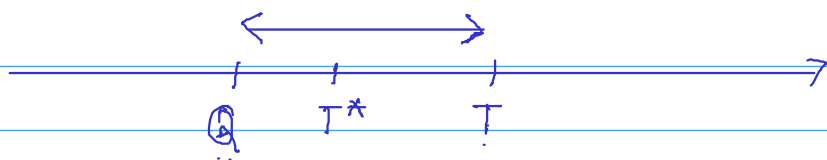
→ There is no polynomial time algorithm known for minimizing makespan.

Suppose T^* is the optimal makespan.

Claim: Greedy algorithm (arbitrary order) gives a solution with makespan $T \leq 2T^*$ [Graham 1966]

2-approximation algorithm.

How can we prove such a bound when we don't have any idea about T^*



Find some natural lower bounds for T^* and relate them with T .

Lower bounds on T^*

$$\textcircled{1} \quad T^* \geq \text{average load of a processor } (Q_1) \\ = \sum_{i=1}^n t_i / m$$

$$\textcircled{2} \quad T^* \geq \max \{t_1, t_2, \dots, t_n\} \quad (Q_2)$$

~~$$\text{Greedy} \leq 2 \cdot \max \{t_1, \dots, t_n\} \leq 2 T^*$$~~

counter example 2 processors, Jobs - 6, 7, 5, 5, 6, 8

~~$$\text{Greedy} \leq 2 \cdot \text{average load of a processor}$$~~

counter example 3 processors, Jobs - 4, 4, 4, 30

Claim: Load on any processor by Greedy algorithm

$$\leq \begin{array}{c} \text{Max processing time} \\ \text{Of any job} \end{array} + \begin{array}{c} \text{Average load} \\ \text{of a processor} \end{array} \\ (Q_2 + Q_1)$$

Claim implies

$$T \leq T^* + T^* = 2 T^*$$

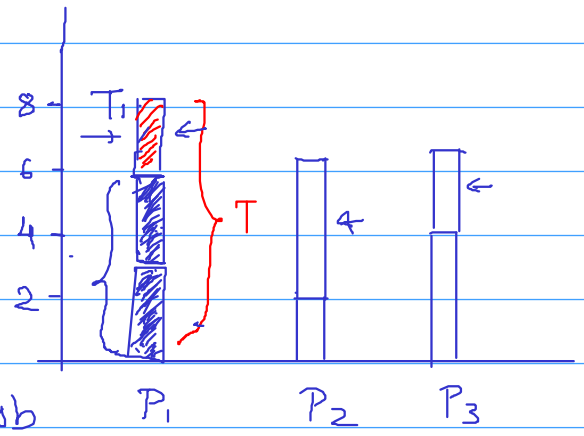
$\Rightarrow$ 2-approximation.

Proof the Claim:

$T =$ the last load assigned (T_1)

+

Total load before the last load (T_2)



$T_1 \leq \max$ processing time of a job

$T_2 =$ minimum total load of any processor at that time

$\leq$ avg total load among all processors at that time

$\leq Q_1$

When we assign a job to a processor its current total load is minimum among all processors.

$$\Rightarrow T \leq Q_1 + Q_2 \leq 2T^*$$

Que: Is our analysis tight?

It is possible that the greedy algorithm always gives, say 1.5 approximation, but our analysis is weak.

Try to construct an example where greedy (arbitrary order) gives a solution with makespan $\approx 2T^*$

HW

2 processors 4, 4, 4, 30

$T^* = 30$

$T = 34$

What about the greedy algorithm with decreasing order of processing times.

Sort the jobs in decreasing order of processing times
For $i = 1$ to n

assign job i to the processor which has the minimum load currently.

Claim: Greedy algorithm with decreasing order of processing times is a $3/2$ -approximation algorithm
 $T \leq 3/2 T^*$

Where can we improve the previous argument.

Can we show

$$T_i = \text{last load assigned} \leq \frac{1}{2} T^*$$

No, if only one job assigned.

Yes if there are at least two jobs assigned on the processor

Final analysis: Two cases

① only one job assigned to the processor

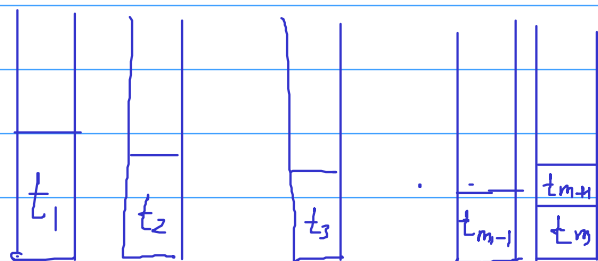
$$T \leq \text{max processing time} \leq T^*$$

② At least two jobs assigned to the processor

Then there must be at least $m+1$ jobs in total because greedy will assign first m jobs to m different processors

jobs processing times

$$\rightarrow t_1 \geq t_2 \geq t_3 \dots \geq t_m \geq t_{m+1} \geq \dots$$



Last job assigned $\leq t_{m+1}$

$$T^* \geq 2 t_{m+1} \Rightarrow \text{last job assigned} \leq t_{m+1} \leq \frac{T^*}{2}$$

because in the optimal solution, there must be some processor that takes at least two jobs from first $m+1$

$T =$ last job assigned + total load before the last job

$$\leq \frac{T^*}{2} + T^*$$

$$\leq \frac{3T^*}{2}$$

Further Improvements:

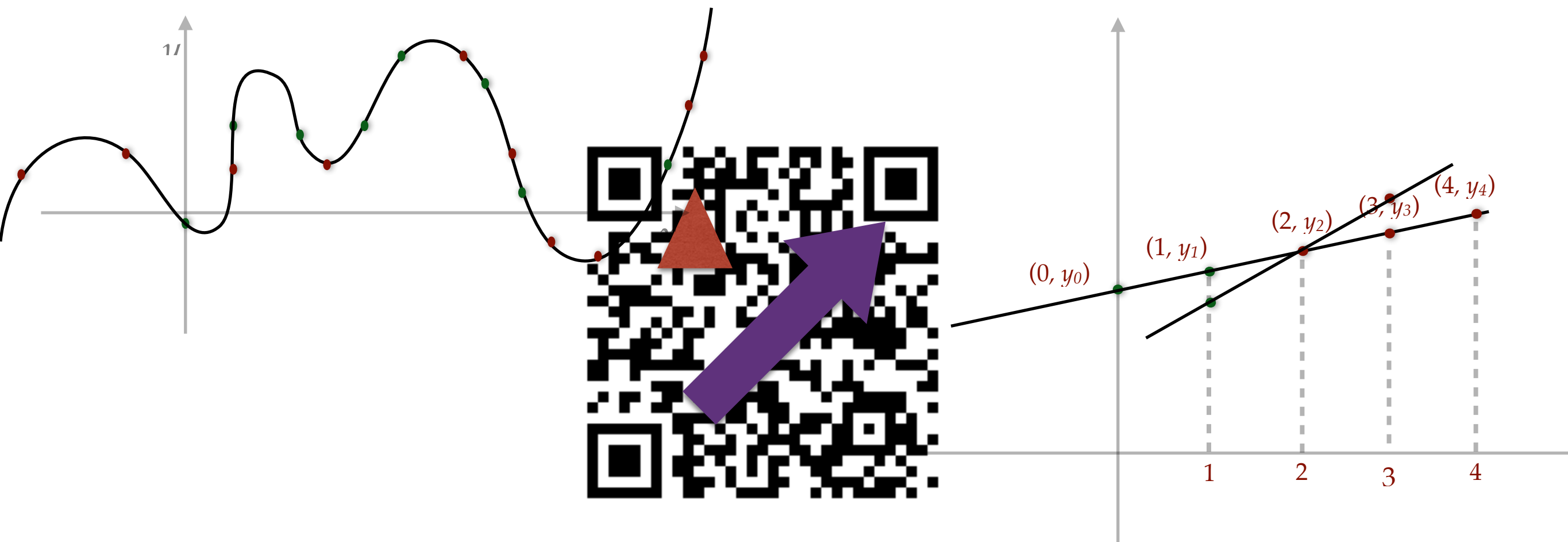
→ Greedy with decreasing order can be actually shown to be $4/3$ approximation.

→ More sophisticated techniques give arbitrarily small approximation factor

$1 + \varepsilon$ for any $\varepsilon > 0$

but running time $O(n^{1/\varepsilon^2})$

Algebra in Computer Science: QR codes

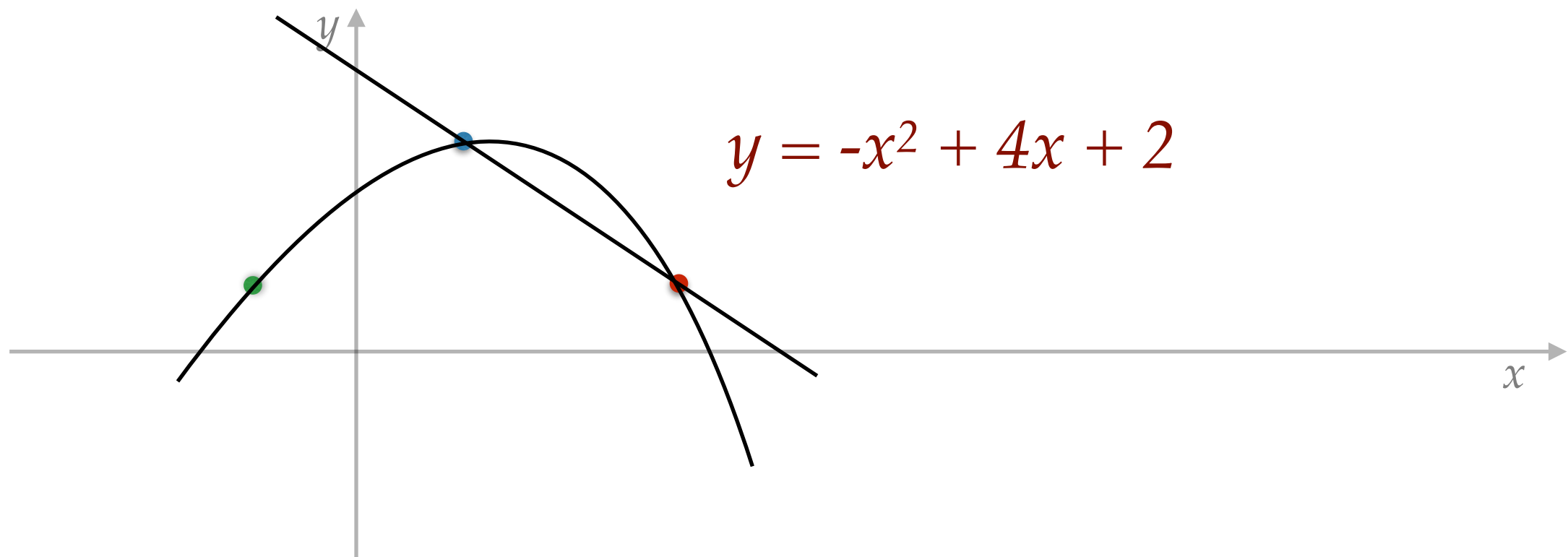


Algebra

- Addition, multiplication, division
- Polynomials, roots, evaluations
- Modular arithmetic
 - $9 + 4 \times 7 \equiv 13 \pmod{24}$
 - $5 \times 4 \equiv -1 \pmod{7}$
- Algebra has wide range of applications in computer science
 - Data compression
 - Reliable and secure communication
 - Efficient verification of computation
 - Software verification

Basic fact from Algebra

- A polynomial $f(x)$ of degree d has at most d roots.
- Equivalently,
- Given $d+1$ points in the plane, there is a unique degree d curve passing through them.



Basic facts from Linear Algebra

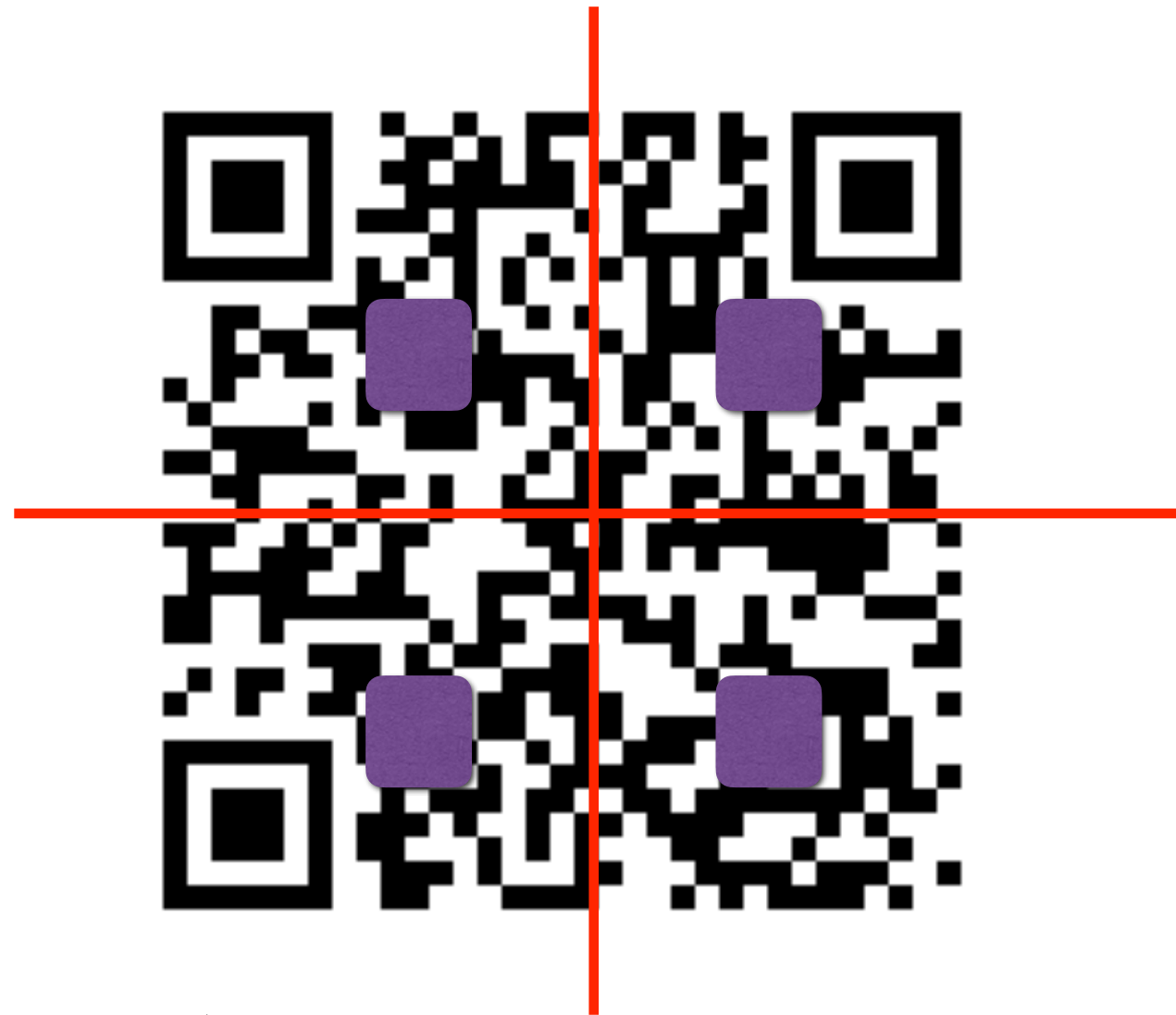
- n linear equations in n variables
- Unique solution if they are linearly independent
- If the RHS is zero, then there is no nonzero solution.
- The facts about roots of a polynomial and solutions for system of linear equations hold true over modular arithmetic (modulo a prime).
- More generally over Galois Fields.

QR codes



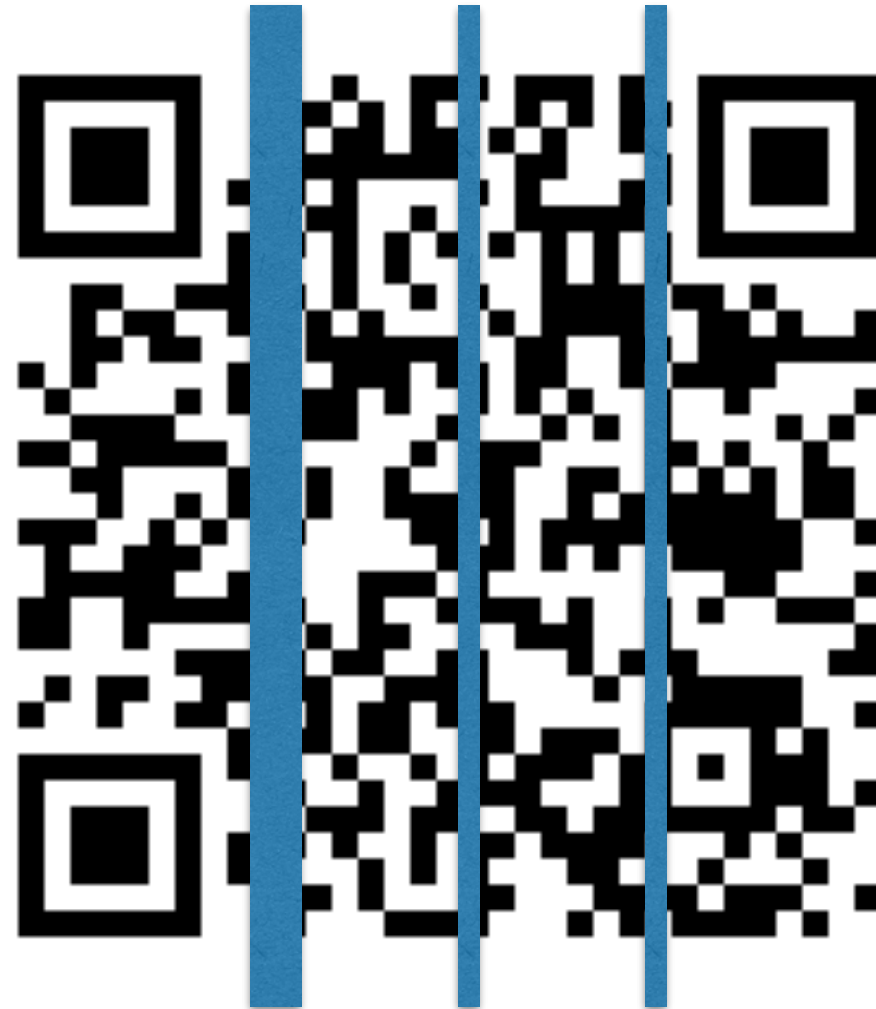
- Can be read, even when partially occluded / erased

Redundancy in QR codes



- Can be read, even when partially erased or modified
- **Guess:** the information is copied multiple times
- Possibly, the same bit gets erased from each copy

Redundancy in QR codes



- Can be read, even when 30% portion from anywhere is erased or modified (Level H)

Redundancy in QR codes



- Can be read, even when 30% portion from anywhere is erased or modified (Level H)

Redundancy in QR codes



- Can be read, even when 30% portion from anywhere is erased or modified (Level H)

Redundancy in QR codes



- Too much erased, cannot be read

Pixel pattern



- 33×33 grid of pixels
- 1089 bits or ~136 bytes
- Some pixel patterns are used for position and alignment detection
- Some pixels encode format information, like error correction level
- 100 bytes of data can be stored.

Information redundancy

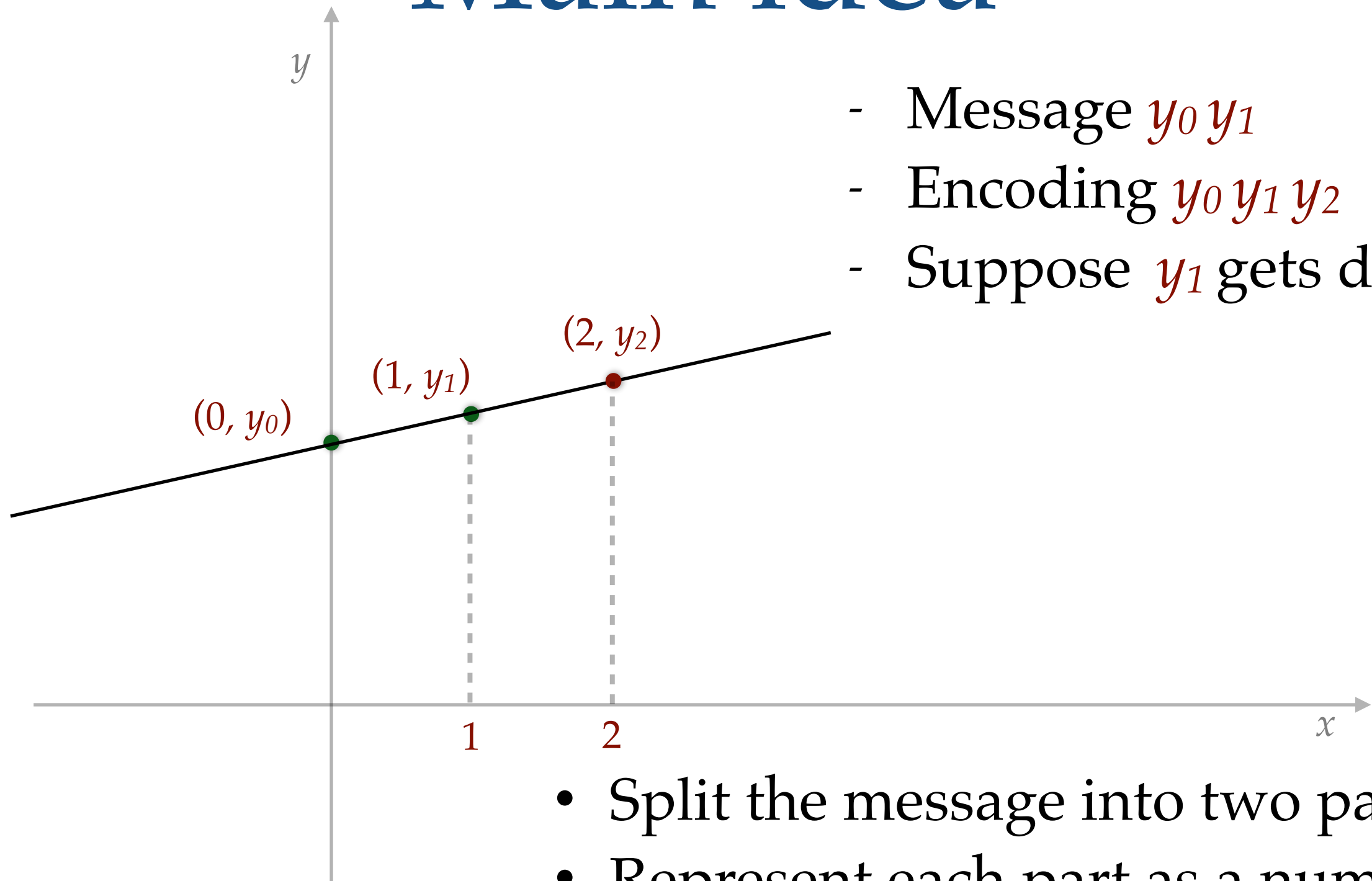


- 100 bytes of data can be stored.
- www.cse.iitb.ac.in/~risc2024/
29 characters
- Could have stored at most 3 copies
- Level H error correction: guaranteed to work even if any 32 bytes are deleted or modified

Challenge

- 36 bytes of information.
- Store it using 100 bytes.
- Recover after any 32 bytes are deleted or modified.
- Simply duplicating the data will not work
- Coding theory: algebra and geometry

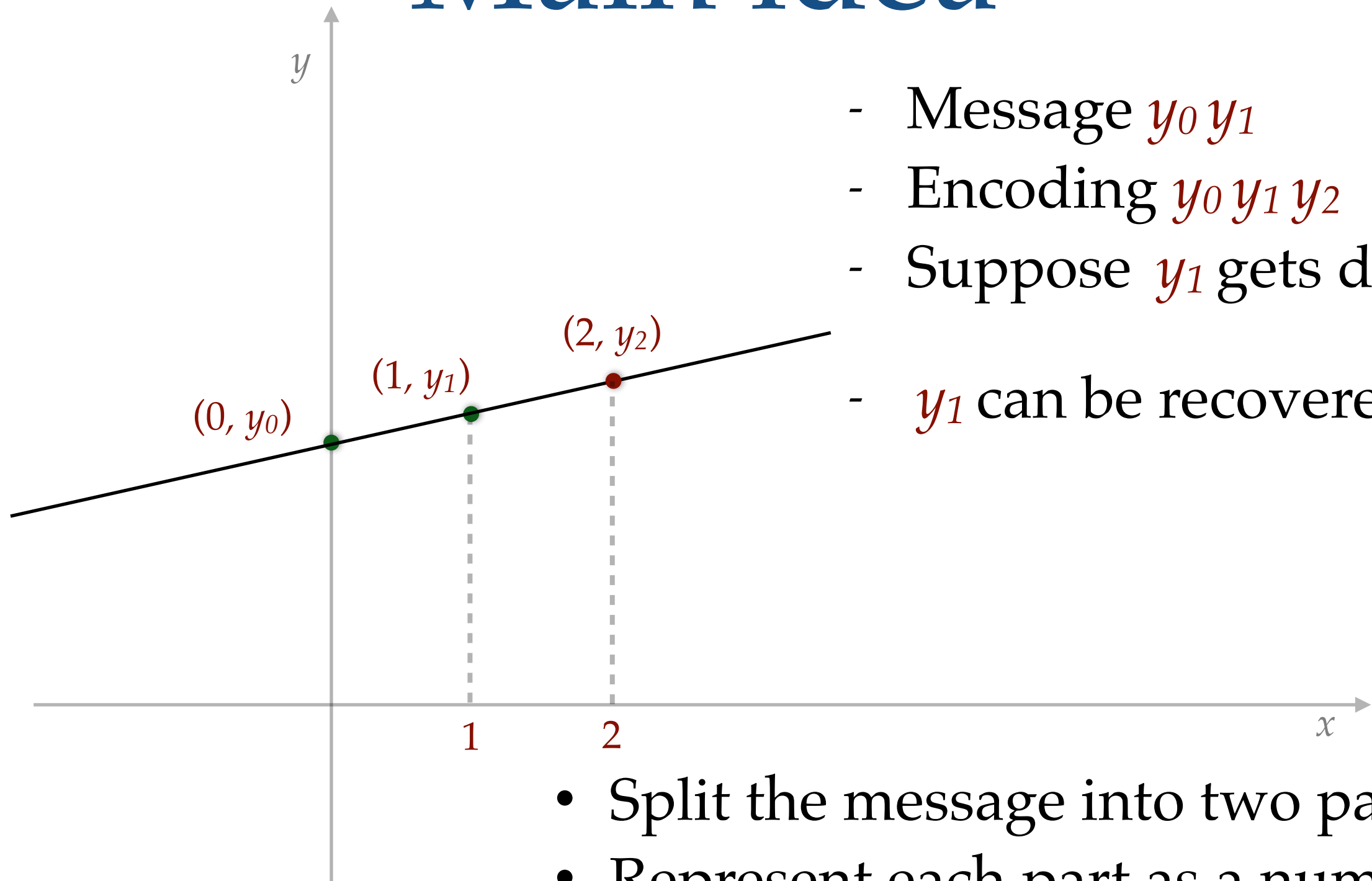
Main idea



- Message $y_0 y_1$
- Encoding $y_0 y_1 y_2$
- Suppose y_1 gets deleted

- Split the message into two parts
- Represent each part as a number, say y_0 and y_1

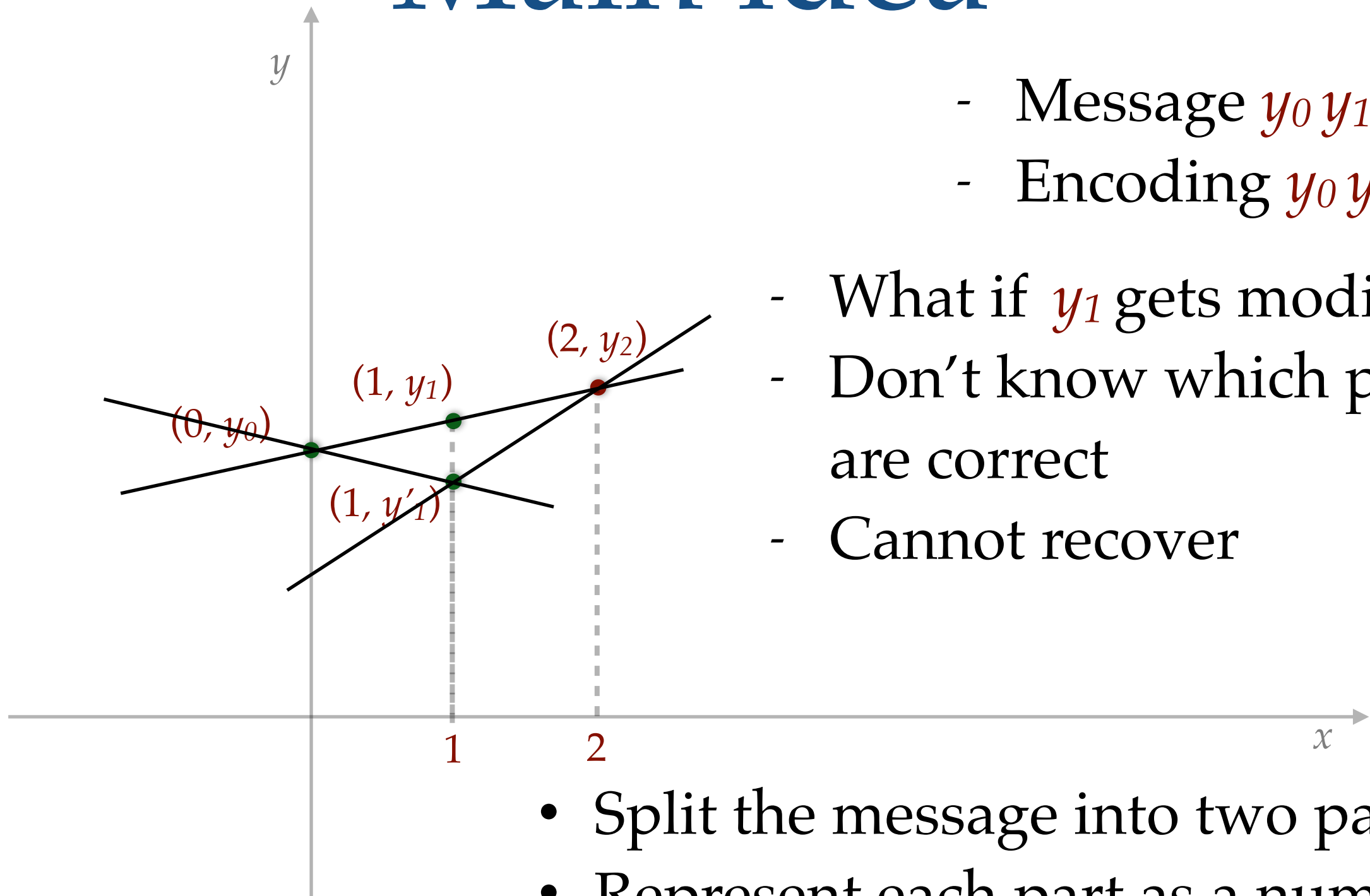
Main idea



- Message $y_0 y_1$
- Encoding $y_0 y_1 y_2$
- Suppose y_1 gets deleted
- y_1 can be recovered

- Split the message into two parts
- Represent each part as a number, say y_0 and y_1

Main idea



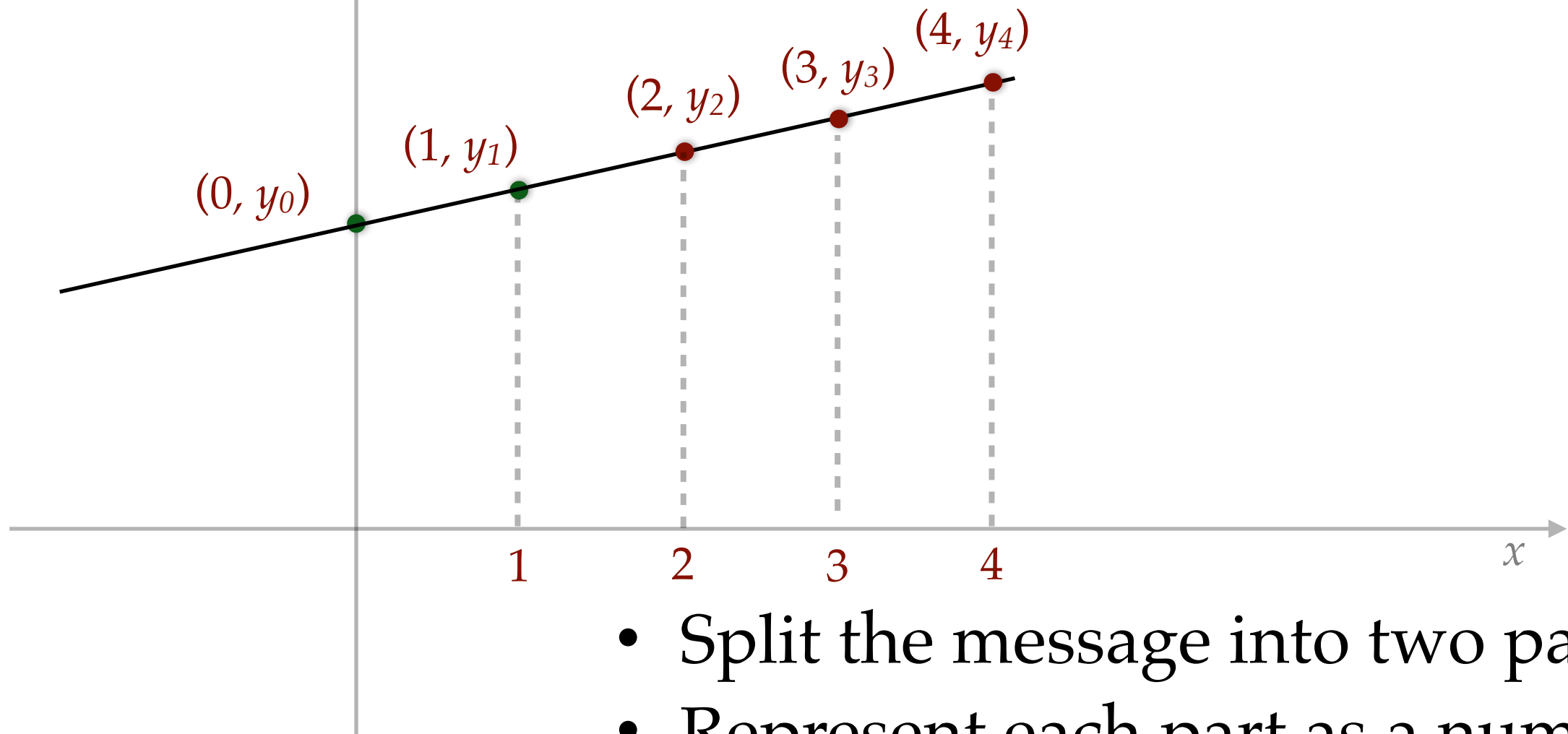
- Message $y_0 y_1$
- Encoding $y_0 y_1 y_2$

- What if y_1 gets modified?
- Don't know which parts are correct
- Cannot recover

- Split the message into two parts
- Represent each part as a number, say y_0 and y_1

Error correction

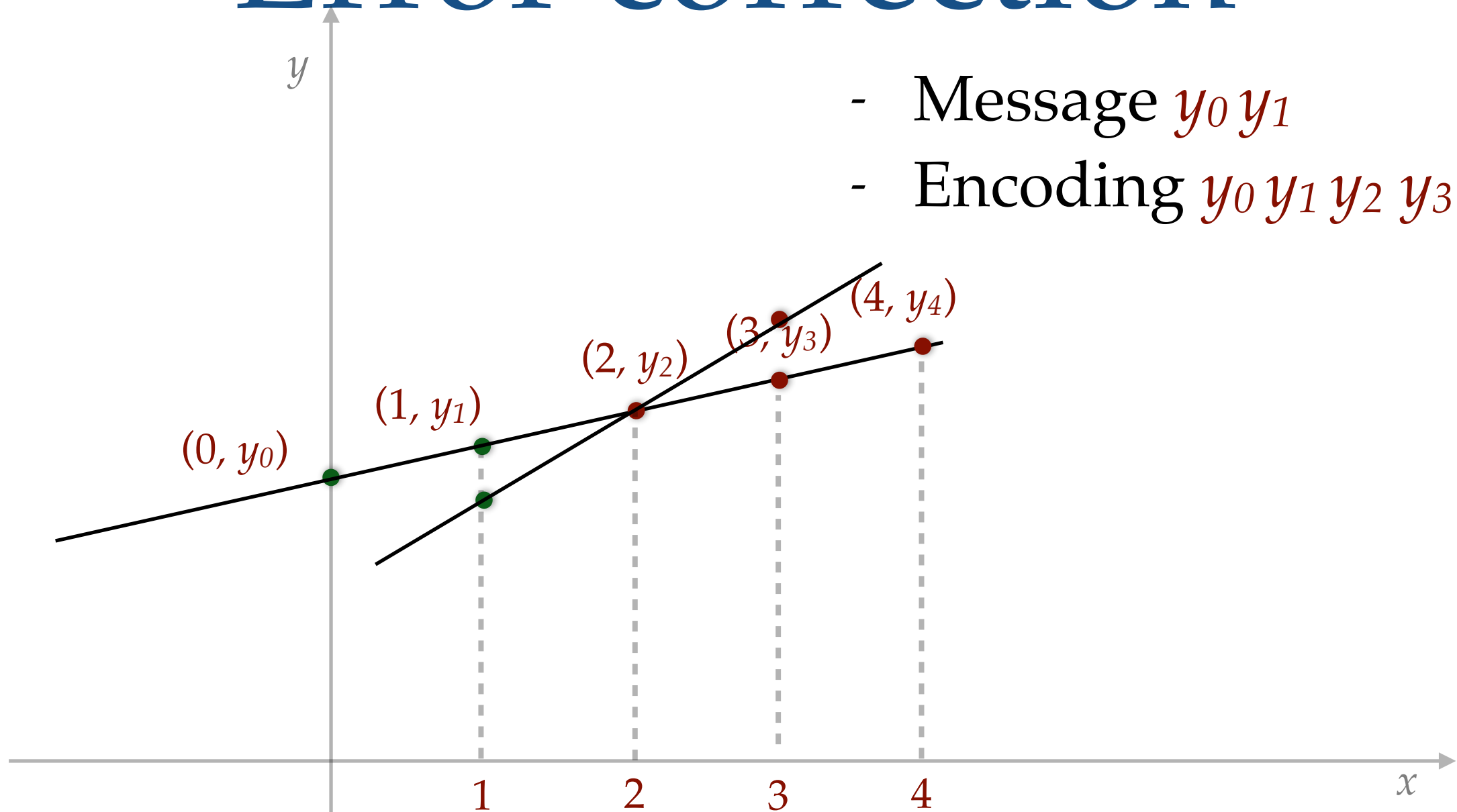
- Message $y_0 y_1$
- Encoding $y_0 y_1 y_2 y_3 y_4$



- Split the message into two parts
- Represent each part as a number, say y_0 and y_1

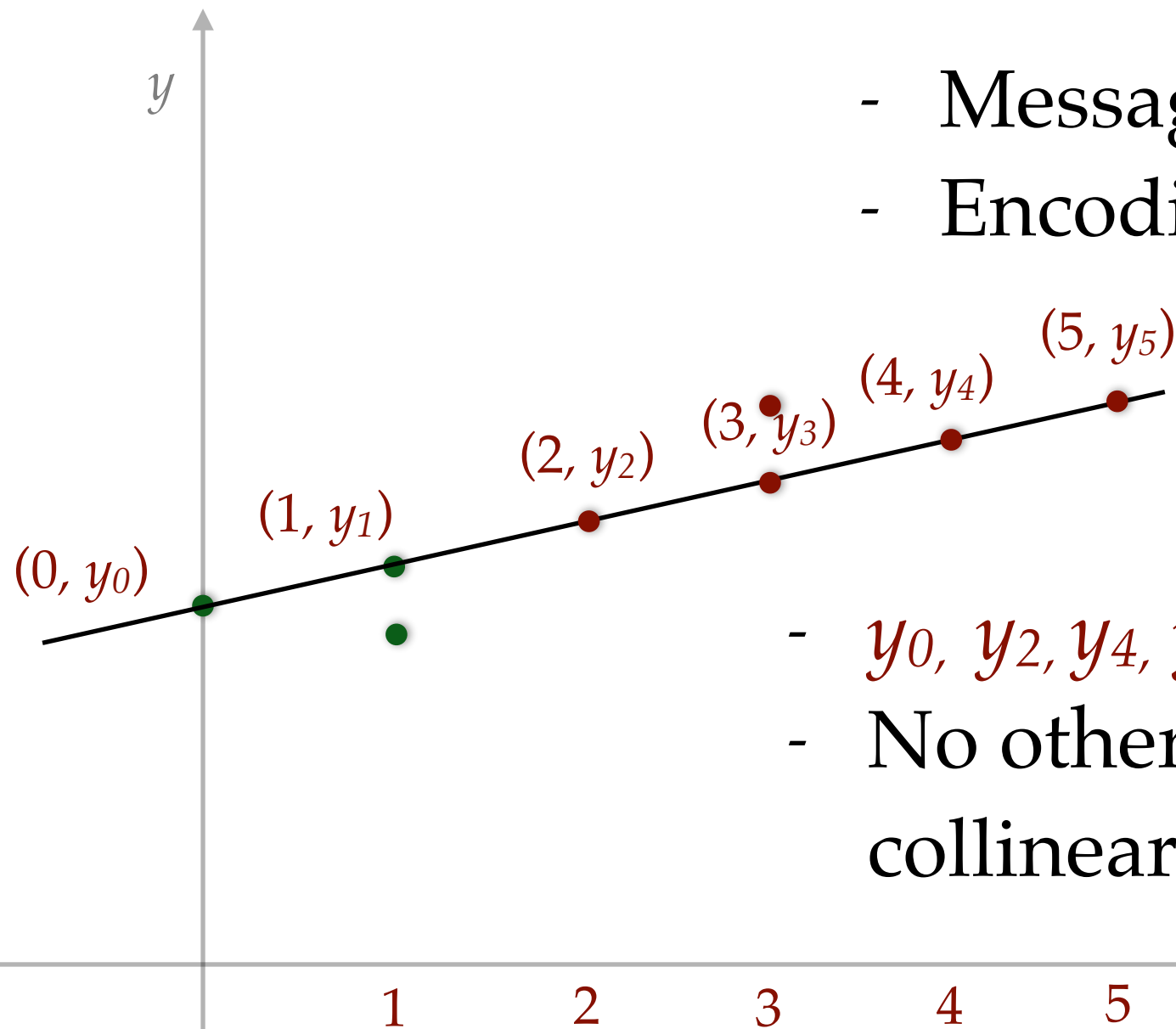
Error correction

- Message $y_0 y_1$
- Encoding $y_0 y_1 y_2 y_3 y_4$



- What if y_1 and y_3 get modified?
- We know **three** points are correct, but don't know which three.

Error correction



- Message $y_0 y_1$
- Encoding $y_0 y_1 y_2 y_3 y_4 y_5$

- y_0, y_2, y_4, y_5 are collinear
- No other 4 points are collinear

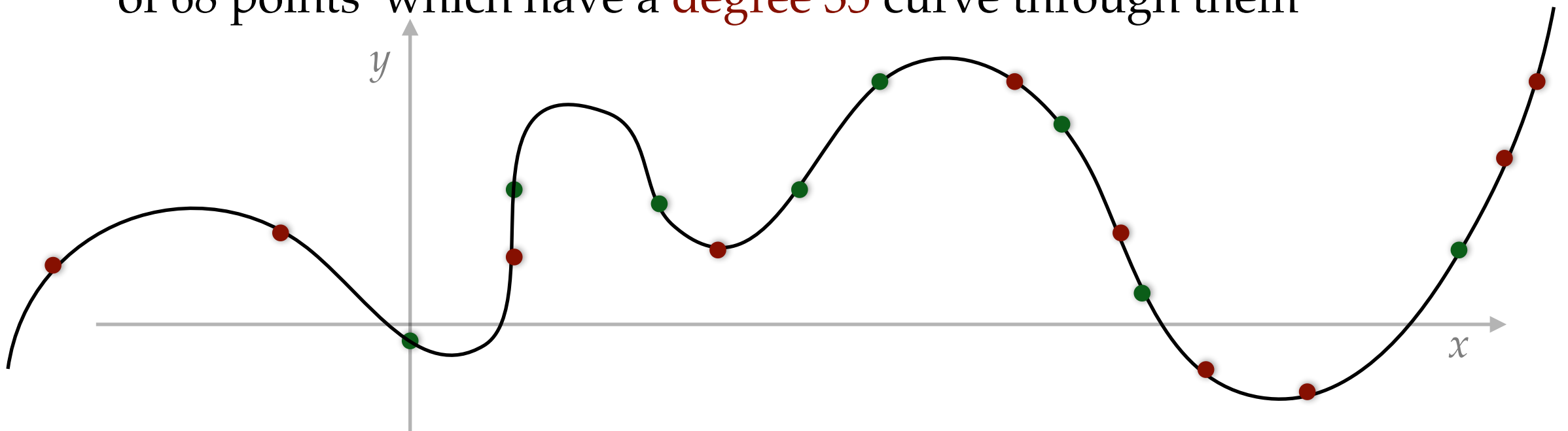
- What if y_1 and y_3 get modified?
- We know **four** points are correct, but don't know which four.

~~Challenge solved~~

- Message split into 2 blocks.
- converted into 6 blocks.
- Can recover after any 2 blocks are deleted or modified.
- We can handle 33% errors in our data.
- Does it solve what we wanted?
- Not really.
What if a small portion from every block is modified?

Towards challenge

- Split the 36 byte message into many blocks, say **36 blocks**
- Each block is one byte
- Visualize them as **36 points** in the plane.
- Pass a unique **degree 35** curve through them
- Take **64 other points** on the curve (total 100 points)
- **Homework:** Even if **any 32 points** are modified, there is only one set of 68 points which have a **degree 35** curve through them



Coding theory

- This construction is called
 - Reed Solomon (RS) codes [1960]
 - Bose–Chaudhuri–Hocquenghem (BCH) codes [1959 / 60]
- Need modular arithmetic, so that numbers don't blow up.
- Used in all kinds of communications, data storage
 - wired, wireless, satellite, CD, hard disks, servers
- Other questions people study:
 - Fast error correction / Local error correction

Coding theory

- QR codes use “dual” of Reed Solomon Code.
- Which is a special case of Bose–Chaudhuri–Hocquenghem (BCH) codes

Galois Field

- QR codes: each byte (8 pixels) is viewed as a number in $\{0,1, \dots, 255\}$.
- GF(256)
 - Addition: bitwise XOR
 - Multiplication: a multiplication table (256×256) such that
 - Identity. For any $a \in \{0,1,\dots, 255\}$, $a \times 1 = a$
 - Commutative. $a \times b = b \times a$
 - Associative. $a(bc) = (ab)c$
 - Distributive. $a(b+c) = ab + ac$
 - Inverse. For any $a \in \{0,1,\dots, 255\}$, there exists a^{-1} s.t. $a^{-1} a = 1$

Error correction bytes

- Suppose the message is 2 bytes: m_0, m_1
- Suppose we want to add 4 error correction bytes: m_2, m_3, m_4, m_5
- Typically, the error correction bytes are a linear combination of the message bytes
- Example 1: parity checksum is used for data transmission
- Example 2: debit card's last digit is some kind of linear combination of remaining digits ($\text{mod } 10$)

Error correction bytes

- Suppose the message is 2 bytes: m_0, m_1
- Suppose we want to add 4 error correction bytes: m_2, m_3, m_4, m_5
- Error correction bytes defined via 4 linear equations, for example:
 - $m_0 + 2m_1 + m_2 + m_3 - 5m_4 + 7m_5 = 0$
 - $m_0 - 3m_1 - m_2 - m_3 + 2m_4 + 2m_5 = 0$
 - $m_0 + 5m_1 + 3m_2 + m_3 - m_4 - 3m_5 = 0$
 - $2m_0 - m_1 + 4m_2 - 3m_3 - 6m_4 + m_5 = 0$
- Given m_0, m_1 , such four linear equations will uniquely determine m_2, m_3, m_4, m_5
- Encoded message will be $m_0 m_1 m_2 m_3 m_4 m_5$

Deletion

- $m_0 + 2m_1 + m_2 + m_3 - 5m_4 + 7m_5 = 0$
- $m_0 - 3m_1 - m_2 - m_3 + 2m_4 + 2m_5 = 0$
- $m_0 + 5m_1 + 3m_2 + m_3 - m_4 - 3m_5 = 0$
- $2m_0 - m_1 + 4m_2 - 3m_3 - 6m_4 + m_5 = 0$
- Suppose 2 bytes get deleted.
- Say, m_1, m_3 .
- We can solve these equations to recover m_1, m_3 .
- 4 equations, 2 unknowns.
- In fact, we can recover four deleted bytes.

Corruption / Modification

- $m_0 + 2m_1 + m_2 + m_3 - 5m_4 + 7m_5 = 0$
- $m_0 - 3m_1 - m_2 - m_3 + 2m_4 + 2m_5 = 0$
- $m_0 + 5m_1 + 3m_2 + m_3 - m_4 - 3m_5 = 0$
- $2m_0 - m_1 + 4m_2 - 3m_3 - 6m_4 + m_5 = 0$
- Suppose 2 bytes get corrupted.
- Say, m_1, m_3 .
- Then the right hand side will not be zero.
- That will tell us something is wrong.
- If we know which bytes are correct and which are corrupted, then we can find the correct values by solving this system.

Corruption / Modification

- $m_0 + 2m_1 + m_2 + m_3 - 5m_4 + 7m_5 = 0$
- $m_0 - 3m_1 - m_2 - m_3 + 2m_4 + 2m_5 = 0$
- $m_0 + 5m_1 + 3m_2 + m_3 - m_4 - 3m_5 = 0$
- $2m_0 - m_1 + 4m_2 - 3m_3 - 6m_4 + m_5 = 0$
- Suppose 2 bytes get corrupted. Say, m_1, m_3 .
- Then the right hand side will not be zero.
- But, we don't know which 2 bytes are corrupted.
- We can try (6 choose 2) possibilities, and see for which possibility gives a solvable system of equation.
- In general, that is exponential. Also, what if multiple possibilities are solvable.

BCH Codes

- We choose equations in a clever way, so that we are able to recover the corrupted bytes.
- We see the encoded message as coefficients of a polynomial with 4 specified roots.
- $1 m_0 + 1 m_1 + 1 m_2 + 1 m_3 + 1 m_4 + 1 m_5 = 0$
- $\alpha^5 m_0 + \alpha^4 m_1 + \alpha^3 m_2 + \alpha^2 m_3 + \alpha m_4 + 1 m_5 = 0$
- $\alpha^{10} m_0 + \alpha^8 m_1 + \alpha^6 m_2 + \alpha^4 m_3 + \alpha^2 m_4 + 1 m_5 = 0$
- $\alpha^{15} m_0 + \alpha^{12} m_1 + \alpha^9 m_2 + \alpha^6 m_3 + \alpha^3 m_4 + 1 m_5 = 0$

BCH Codes

- Suppose 2 bytes get corrupted.
- Intended message $m_0 m_1 m_2 m_3 m_4 m_5$
- Received message $c_0 c_1 c_2 c_3 c_4 c_5$
 - $1 c_0 + 1 c_1 + 1 c_2 + 1 c_3 + 1 c_4 + 1 c_5 = 2$
 - $\alpha^5 c_0 + \alpha^4 c_1 + \alpha^3 c_2 + \alpha^2 c_3 + \alpha c_4 + 1 c_5 = 5$
 - $\alpha^{10} c_0 + \alpha^8 c_1 + \alpha^6 c_2 + \alpha^4 c_3 + \alpha^2 c_4 + 1 c_5 = 6$
 - $\alpha^{15} c_0 + \alpha^{12} c_1 + \alpha^9 c_2 + \alpha^6 c_3 + \alpha^3 c_4 + 1 c_5 = 11$

BCH Codes

- Intended message $m_0 m_1 m_2 m_3 m_4 m_5$
- Received message $c_0 c_1 c_2 c_3 c_4 c_5$
- Let us subtract the two set of equations
 - $1(c_0-m_0)+1(c_1-m_1)+1(c_2-m_2)+1(c_3-m_3)+1(c_4-m_4)+1(c_5-m_5)=2$
 - $\alpha^5(c_0-m_0)+\alpha^4(c_1-m_1)+\alpha^3(c_2-m_2)+\alpha^2(c_3-m_3)+\alpha(c_4-m_4)+1(c_5-m_5)=5$
 - $\alpha^{10}(c_0-m_0)+\alpha^8(c_1-m_1)+\alpha^6(c_2-m_2)+\alpha^4(c_3-m_3)+\alpha^2(c_4-m_4)+1(c_5-m_5)=6$
 - $\alpha^{15}(c_0-m_0)+\alpha^{12}(c_1-m_1)+\alpha^9(c_2-m_2)+\alpha^6(c_3-m_3)+\alpha^3(c_4-m_4)+1(c_5-m_5)=11$
- Define $e_0 = c_0 - m_0$,
 $e_1 = c_1 - m_1$,
 $e_2 = c_2 - m_2$,....

BCH Codes

- Intended message $m_0 m_1 m_2 m_3 m_4 m_5$
- Received message $c_0 c_1 c_2 c_3 c_4 c_5$
- Differences $e_0 e_1 e_2 e_3 e_4 e_5$
 - $1 e_0 + 1 e_1 + 1 e_2 + 1 e_3 + 1 e_4 + 1 e_5 = 2$
 - $\alpha^5 e_0 + \alpha^4 e_1 + \alpha^3 e_2 + \alpha^2 e_3 + \alpha e_4 + 1 e_5 = 5$
 - $\alpha^{10} e_0 + \alpha^8 e_1 + \alpha^6 e_2 + \alpha^4 e_3 + \alpha^2 e_4 + 1 e_5 = 6$
 - $\alpha^{15} e_0 + \alpha^{12} e_1 + \alpha^9 e_2 + \alpha^6 e_3 + \alpha^3 e_4 + 1 e_5 = 11$
- Suppose e_{5-i}, e_{5-j} are nonzero. Rest are zero.

BCH Codes

- Intended message $m_0 m_1 m_2 m_3 m_4 m_5$
- Received message $c_0 c_1 c_2 c_3 c_4 c_5$
- Differences $e_0 e_1 e_2 e_3 e_4 e_5$
 - $1 e_{5-i} + 1 e_{5-j} = 2$
 - $\alpha^i e_{5-i} + \alpha^j e_{5-j} = 5$
 - $\alpha^{2i} e_{5-i} + \alpha^{2j} e_{5-j} = 6$
 - $\alpha^{3i} e_{5-i} + \alpha^{3j} e_{5-j} = 11$
- Suppose e_{5-i}, e_{5-j} are nonzero. Rest are zero.

BCH Codes

- $1 e_{5-i} + 1 e_{5-j} = 2$
- $\alpha^i e_{5-i} + \alpha^j e_{5-j} = 5$
- $\alpha^{2i} e_{5-i} + \alpha^{2j} e_{5-j} = 6$
- $\alpha^{3i} e_{5-i} + \alpha^{3j} e_{5-j} = 11$
- We do not know i, j . We do not know $e_0 e_1 e_2 e_3 e_4 e_5$
- We will first find α^i and α^j . Then we will solve the system of equations.
- To find α^i and α^j find the coefficients of the polynomial
 - $(y - \alpha^i)(y - \alpha^j) = y^2 + a y + b$

BCH Codes

- $1 e_{5-i} + 1 e_{5-j} = 2$
- $\alpha^i e_{5-i} + \alpha^j e_{5-j} = 5$
- $\alpha^{2i} e_{5-i} + \alpha^{2j} e_{5-j} = 6$
- $\alpha^{3i} e_{5-i} + \alpha^{3j} e_{5-j} = 11$
- $(y - \alpha^i)(y - \alpha^j) = y^2 + a y + b$
- To find a, b , multiply first three equations by $b, a, 1$, respectively and add.
 - $(b + a \alpha^i + \alpha^{2i}) e_{5-i} + (b + a \alpha^j + \alpha^{2j}) e_{5-j} = 2b + 5a + 6$
- Note that α^i and α^j are roots of $y^2 + a y + b$.
- Thus above equation becomes
 - $(0) e_{5-i} + (0) e_{5-j} = 2b + 5a + 6$

BCH Codes

- $1 e_{5-i} + 1 e_{5-j} = 2$
- $\alpha^i e_{5-i} + \alpha^j e_{5-j} = 5$
- $\alpha^{2i} e_{5-i} + \alpha^{2j} e_{5-j} = 6$
- $\alpha^{3i} e_{5-i} + \alpha^{3j} e_{5-j} = 11$
- $0 = 2b + 5a + 6$
- Similarly, multiply last three equations by $b, a, 1$, respectively and add.
 - $(b \alpha^i + a \alpha^{2i} + \alpha^{3i}) e_{5-i} + (b \alpha^j + a \alpha^{2j} + \alpha^{3j}) e_{5-j} = 5b + 6a + 11$
 - $\alpha^i (b + a \alpha^i + \alpha^{2i}) e_{5-i} + \alpha^j (b + a \alpha^j + \alpha^{2j}) e_{5-j} = 5b + 6a + 11$
- Note that α^i and α^j are roots of $y^2 + a y + b$.
- Thus above equation becomes
 - $(0) e_{5-i} + (0) e_{5-j} = 5b + 6a + 11$

BCH Codes

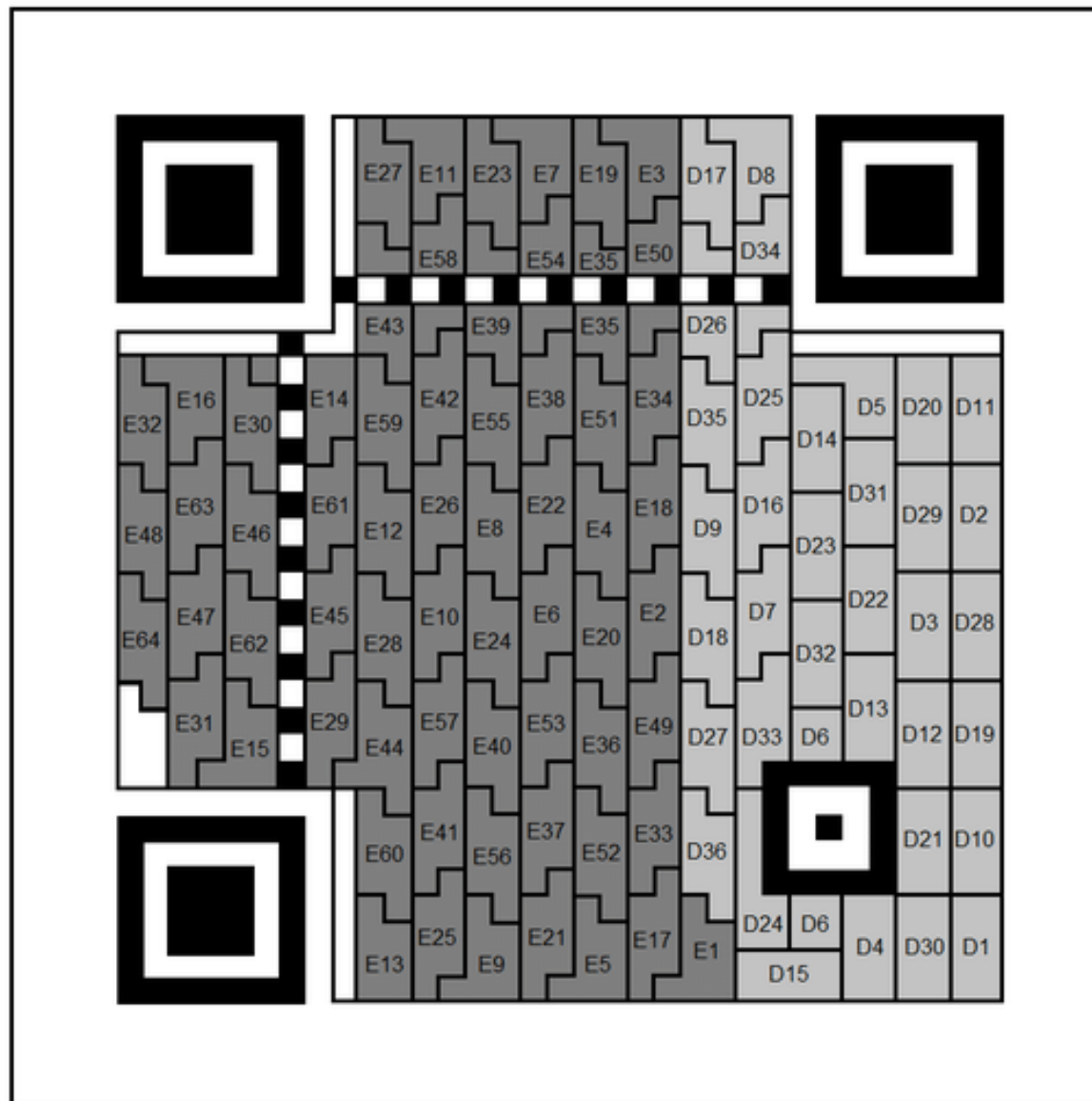
- We get two equations
 - $0 = 2b + 5a + 6$
 - $0 = 5b + 6a + 11$
- Solving these will give us a, b .
- Recall that α^i and α^j are roots of $y^2 + a y + b$.
- Check which of $\alpha^0, \alpha^1, \alpha^2, \alpha^3, \alpha^4, \alpha^5$ are roots of $y^2 + a y + b$.
- Once we know α^i and α^j , we can solve the earlier system of equations to find e_{5-i}, e_{5-j} . Remaining four e_k 's are zero.
- Thus, we know all $e_0 = c_0 - m_0, e_1 = c_1 - m_1, e_2 = c_2 - m_2, \dots$
- And we can get the actual message $m_0 m_1 m_2 m_3 m_4 m_5$

QR code generation

- For version 4 QR code, with error level H (30%),
- 36 bytes of message and 64 additional bytes generated for error correction.
- The encoded message will have 100 bytes.
- These 100 bytes are the coefficients of a degree 99 polynomial with 64 specified roots $\alpha^0, \alpha^1, \dots, \alpha^{63}$
- Where the highest 36 coefficients are the given message
- Guarantee: recover after any 32 bytes out of 100 get corrupted.

Data arrangement

- Version 4 QR code, with error level H (30%)
- From DOI:10.1007 / 978-3-319-72359-4_42

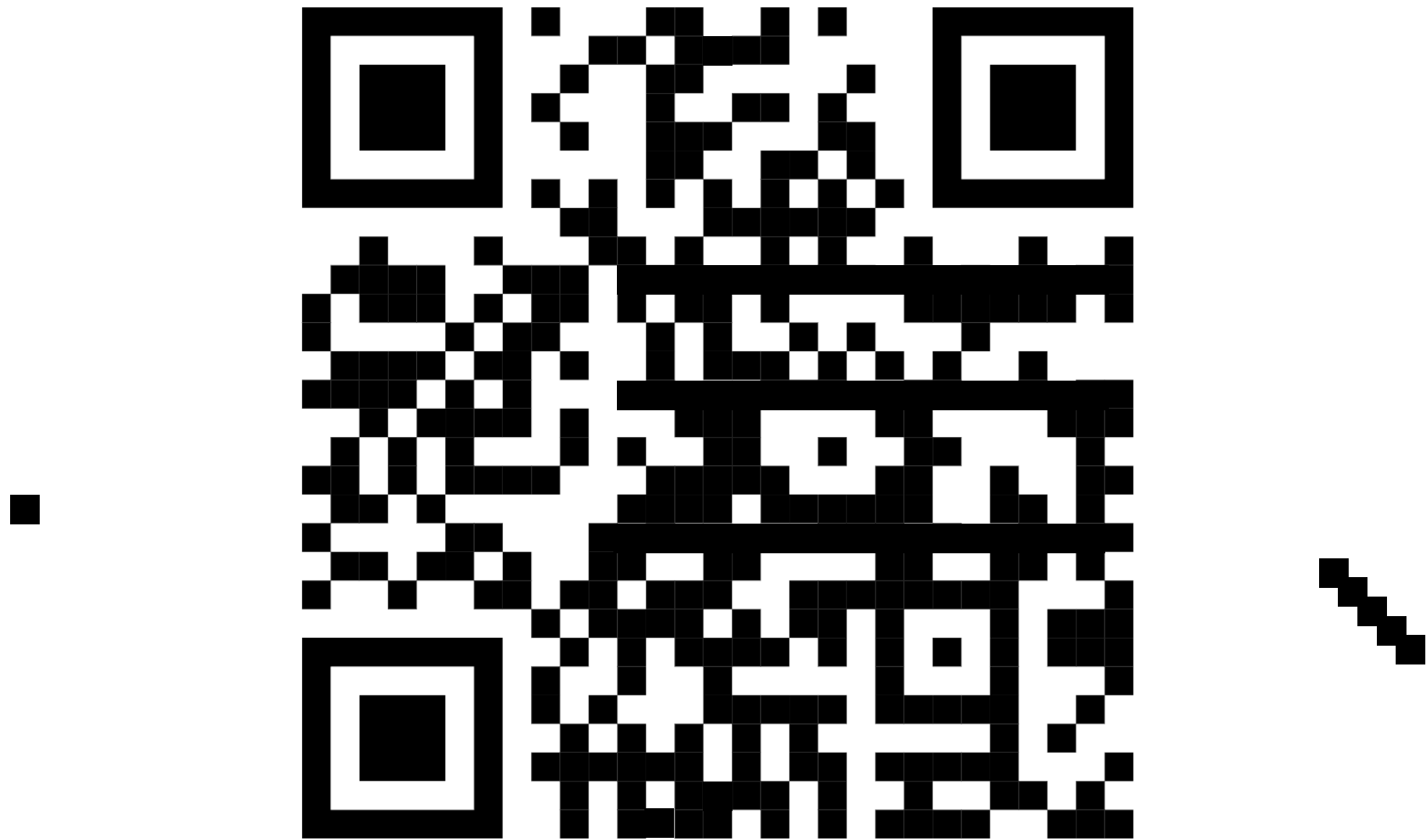


D1 – D9	Data Block 1
D10 – D18	Data Block 2
D19 – D27	Data Block 3
D28 – D36	Data Block 4
E1 – E16	Error Correction Block 1
E17 – E32	Error Correction Block 2
E33 – E48	Error Correction Block 3
E49 – E64	Error Correction Block 4

Thanks

- If you are interested in theory talks,
- subscribe to mailing list
<https://www.cse.iitb.ac.in/~theory/seminar.html>
-

Redundancy in QR codes



- Too much erased, cannot be read